# AoPS

Art of Problem Solving

## Computer Science Course

Review Edition — Volume I
Levels 0 – 12 • Complete Course Content

**DRAFT REVIEW EDITION**

Generated September 2026 • Internal Review Only

## About This Review Document

This is a draft review edition of the Art of Problem Solving (AoPS) Computer Science Course, covering all 13 levels from Level 0 (Before Computers) through Level 12 (Advanced AI). It is produced for internal quality review, structural assessment, and content completeness checking.

### Document Structure

Each level opens with a separator page. Each module contains: the lesson narrative, hands-on simulation summary, illustration figures, and a challenge question. Illustrations are embedded as figure descriptions referencing the SVG source files. Interactive simulations are summarised with a description box.

Placeholders indicate sections that exist as interactive HTML content (JavaScript simulations, animations) which cannot be rendered in static PDF. These are expected and do not indicate missing content.

### Figure Numbering

Figures are numbered sequentially across the entire document. Each figure references its source illustration file for the full interactive version.

# Table of Contents


## Level 00 -- Before Computers

- Module 0.1 Why Did Humans Need to Compute?
- Module 0.2 Early Mechanical Computation
- Module 0.3 The Loom That Taught Computers
- Module 0.4 Babbage and Ada: The First Programmers
- Module 0.5 Hollerith and the Punch Card Era

## Level 01 -- Physical Computing

- Module 1.1 What is Electricity? (Only What We Need)
- Module 1.2 Switches: The Atom of Computing
- Module 1.3 Relay Logic: Building Decisions from Metal
- Module 1.4 Vacuum Tubes: Faster Switching
- Module 1.5 The Transistor: Revolution in a Grain of Sand
- Module 1.6 From Transistors to Chips: Moore's Law

## Level 02 -- Digital Foundations

- Module 2.1 Why Binary? The Physics of Two States
- Module 2.2 Number Systems: Binary, Octal, Hex, Two's Complement
- Module 2.3 Boolean Algebra and Logic Gates
- Module 2.4 Combinational Circuits
- Module 2.5 Sequential Circuits: Memory from Gates
- Module 2.6 From Gates to a Simple ALU

## Level 03 -- Machine Architecture

- Module 3 1 Memory
- Module 3 2 Fetch Decode Execute
- Module 3 3 Isa
- Module 3 4 Io
- Module 3 5 Bus

## Level 04 -- Software Foundations

- Module 4 1 Machine Code
- Module 4 2 Stack Calling
- Module 4 3 Os Concepts
- Module 4 4 Memory Management
- Module 4 5 File Systems

## Level 05 -- Programming

- Module 5 1 C Language
- Immutable: creates a NEW object
- Module 5 3 Algorithms Intro
- Module 5 4 Recursion
- Module 5 5 Sorting
- Module 5 6 Searching

## Level 06 -- Data Structures & Algorithms

... append until capacity full

Module 6 2 Stacks Queues

Extremely simplified no collision handling

In-order: left root right sorted output for BST

Build heap in O(n):

nn matrix where matrix[i][j] = 1 if edge exists

Module 6 7 Sorting Advanced

Or manually:

## Level 07 -- Systems Programming

For CPU-bound parallel work:

Module 7 2 Sync Primitives

1. Create socket IPv4, TCP (stream)

Module 7 4 Shell

Apply to all routes:

## Level 08 -- Networks & Protocols

Module 8 1 Physical Datalink

Module 8 2 Network Layer

Module 8 3 Transport Layer

Module 8 4 Application Layer

Module 8 5 Tls Ssl

Raw socket requires root/admin privileges

## Level 09 -- Databases

Module 9 1 Relational Model

Module 9 2 Btrees

Module 9 3 Transactions

[('KEYWORD','SELECT'), ('IDENT','name'), ('OP',','), ('IDENT','age'),

## Level 10 -- Web & Application Dev

Module 10 1 Html Css Js

Module 10 2 How Browsers Render

Parse JSON string Python dict

app.py

## Level 11 -- AI & Machine Learning

Module 11 1 Search Intelligence

3D vector

= (XX) Xy works for small datasets

Module 11 4 Neural Networks

L/z2 gradient of MSE

## Level 12 -- Advanced AI

Input: [The, cat, sat, on]

Text embeddings

Cosine similarity between first two (semantically similar)
Module 12 4 Ai Agents
Simplified example

LEVEL 00

# Before Computers

The Human Problem that Forced Computation into Existence

*Before there was a machine, there was a desperate need.*

## What This Level Is About

Computing didn't emerge from curiosity alone. It was forced into existence by real human problems that were simply too large, too repetitive, and too error-prone for unaided human minds to handle.

This level traces that journey — from counting beans on an abacus to punching holes in cards to tell a machine what to do. Every invention you'll meet here was born from frustration with the previous solution.

By the end of Level 0, you will understand:

• **Why** computation had to be mechanized

• **How** mechanical and electromechanical devices encoded and processed information

• **What** a "stored instruction" means and why it's the idea that made modern computers possible

• **How** punch cards bridged the gap between mechanical and digital computing

## Modules

| Module | Title | Core Idea | |--------|-------|-----------| | [0.1](./module-0-1-why-compute/lesson.md) | Why Did Humans Need to Compute? | Scale breaks human calculation | | [0.2](./module-0-2-mechanical/lesson.md) | Early Mechanical Computation | Gears and levers as arithmetic | | [0.3](./module-0-3-loom/lesson.md) | The Lo

# Module 0.1 Why Did Humans Need to Compute?

## The Problem

*Imagine you are the United States government in 1880. You need to count 50 million people, record their ages, occupations, birthplaces, and family sizes — and you need to do it accurately enough to redraw congressional districts and allocate federal funding.*

>

*You have pencils, paper, and humans.*

>

*It takes **8 years**.*

>

*By the time you finish, the next census is already 2 years away.*

This wasn't a failure of effort. It was a failure of **scale**. Human brains are remarkable, but they break down when asked to do the same arithmetic operation millions of times without error.

That breakdown is exactly why computing had to be invented.

---

## The Story

### Problem 1: The Census Crisis

The 1880 US Census employed hundreds of clerks who hand-tallied results on paper. The final count was published in 1888 — 8 years after the census was taken. The US Constitution requires a census every 10 years. The math was terrifying: **the 1890 census was going to be even bigger**, and at the current rate, it might not be finished before the 1900 census began.

Congress was alarmed. A competition was held to find a faster method. A young engineer named Herman Hollerith won it — but that's Module 0.5's story.

The point here is simpler: **large-scale counting broke human capacity**.

---

### Problem 2: Navigation and the Cost of Arithmetic Errors

In the 17th and 18th centuries, a ship's navigator used printed mathematical tables to find their position at sea. These tables — logarithms, trigonometric values, astronomical positions — were computed by hand and typeset by hand.

Hand computation means human error. Typesetting means more human error.

In 1628, the Swedish warship *Vasa* sank on its maiden voyage partly due to a design calculation error. Navigation errors sent ships onto rocks. Artillery fired at incorrect trajectories killed friendly troops.

The British government commissioned **Mathematical Tables** projects employing teams of human "computers" — people whose job title was literally *computer* — to calculate and cross-check these

values. Errors still crept in.

**The problem**: arithmetic at the scale required for navigation and engineering was too error-prone for humans alone.

---

### Problem 3: Artillery and Wartime Calculation

During World War 1 and World War 2, artillery crews needed **firing tables** — books of pre-calculated values telling gunners how to aim based on distance, wind speed, shell weight, and air pressure. Each table required thousands of calculations.

At Aberdeen Proving Ground in the US, teams of (human) computers — many of them women, including those who would later program ENIAC — spent days computing a single trajectory by hand. During WW2, the demand for new firing tables outpaced what humans could calculate.

This bottleneck directly funded the construction of ENIAC, the first general-purpose electronic computer, completed in 1945.

**The problem**: the speed of human calculation was a strategic military liability.

---

## The Pattern

Look at these three problems together:

| Problem | Scale | Consequence of slowness | |---------|-------|-------------------------| | Census counting | 50 million people | Government dysfunction | | Navigation tables | Millions of arithmetic ops | Ships lost, lives lost | | Artillery trajectories | Thousands of ballistic calc | Military disadvantage |

In every case, the bottleneck was the same: **humans doing repetitive arithmetic at scale**.

The solution was always the same direction: **offload repetitive computation to a machine**.

---

## The Concept: What is Computation?

Computation, at its core, is the manipulation of symbols according to rules.

• **Symbol**: any representation of information (a number, a letter, a hole in a card)

• **Rule**: a defined operation (add these two numbers, shift this value left)

• **Manipulation**: applying the rule to the symbol to produce a new symbol

That's it. A human doing long division is computing. A gear turning in a calculating machine is computing. A transistor switching in a chip is computing.

The difference between a human computer and an electronic computer is:

• **Speed**: milliseconds vs nanoseconds

• **Reliability**: humans make errors; circuits don't (unless broken)

• **Tirelessness**: humans get tired; machines don't

---

## What "Stored Instructions" Means (Preview)

The next major leap wasn't just making machines faster — it was making them **programmable**. A machine that can only add is limited. A machine that can follow a *sequence of instructions* — and change what it does based on previous results — is general-purpose.

The idea of storing instructions (a program) alongside data is the single most important idea in computing. You'll see it born in Module 0.3 with Jacquard's loom, formalized in Module 0.4 with Babbage, and industrialized in Module 0.5 with Hollerith.

---

## Hands-On Exercise

**Open the simulation:** [`simulation.html`](./simulation.html)

The simulation puts you in the role of a human "computer" from 1880. You are given a census tally sheet and must manually count and sum demographic data across multiple districts.

**What to do:**

Open `simulation.html` in your browser

You'll see a table of raw census records — names, ages, occupations

Your task: tally the totals for each category by clicking to count

Notice how errors accumulate as the dataset grows

Toggle to "machine mode" to see the same task done instantly

**What to observe:**

• How long does the human tally take vs the machine?

• Where do errors appear in your manual count?

• What would this feel like at 50 million records?

---

## Going Deeper

The humans employed as "computers" before electronic machines were a real profession. Between 1930 and 1945, the US government's Mathematical Tables Project employed over 450 people as computers. Many were women with mathematics degrees who couldn't get other professional employment due to discri

### Illustrations

**Figure 1 • FIG 0.1-A**

### The Scale Problem — Human Computation at Its Limits



*Illustration: FIG 0.1-A | The Scale Problem — Human Computation at Its Limits [SVG diagram — see module-0-1-why-compute.html]*

The three historical crises that demanded mechanical computation. Left: a census clerk's tally desk overwhelmed with paper. Centre: an 8-year completion timeline — already overlapping the next mandatory census. Right: the three independent problems sharing a single root cause — repetitive arithmetic at human scale.

**Figure 2 • FIG 0.1-B**

### What Is Computation? — Symbol, Rule, Manipulation



*Illustration: FIG 0.1-B | What Is Computation? — Symbol, Rule, Manipulation [SVG diagram — see module-0-1-why-compute.html]*

Computation is substrate-independent. The medium changes — beads, gears, voltage — but the abstract structure is identical: a symbol has a state, a rule specifies a transformation, a new symbol emerges.

**Figure 3 • FIG 0.1-C**

### Evolution of Human Computation — From Tally Sticks to Transistors



*Illustration: FIG 0.1-C | Evolution of Human Computation — From Tally Sticks to Transistors [SVG diagram — see module-0-1-why-compute.html]*

Computation timeline from 2400 BC to 1947. Five eras of substrate: manual aids, mechanical calculators, analytical engines, electro-mechanical tabulators, and electronic computers. The abstract concept of symbol manipulation is constant throughout.

**Figure 4 • FIG 0.1-D**

### Human vs Machine — Census Tally Speed Comparison



*Illustration: FIG 0.1-D | Human vs Machine — Census Tally Speed Comparison [SVG diagram — see module-0-1-why-compute.html]*

Static rendering of the Module 0.1 simulation. Left: a partial human census tally with an error highlighted in row 3. Right: the Hollerith tabulator output — complete, error-free, in 6 weeks. The 78× speed improvement was not incremental; it was civilizational.

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 0.1 The 1880 Census Simulator: Human computation vs. mechanical tabulation [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 0.1 The Cost of Human Computation **Type:** Calculation + reasoning **Estimated time:** 2030 minutes **Tools needed:** Pen and paper (or a calculator ironic, we know) --- Setup The 1880 US Census counted approximately **50,000,000 people**. Each person's record included 5 data fields: name, age, occupation, birthplace, and family size. A trained human "computer" in 1880 could process (read, tally, and record) roughly **500 records per day**, working an 8-hour day. The Hollerith tabulating machine (1890) could process roughly **65 cards per minute**, running 8 hours a day.

# Module 0.2 Early Mechanical Computation

## The Problem

*You are a 17th-century merchant calculating compound interest on a loan. The calculation requires multiplying large numbers together — numbers with 6, 8, even 10 digits. There are no calculators. There are no computers. There is only you, a quill, and arithmetic rules you learned as a child.*

>

*A single multiplication can take **30 minutes**. A mistake means starting over. You need to do dozens of these calculations a day.*

>

*What if a machine could do it for you?*

Arithmetic at scale was exhausting, slow, and error-prone. Long before electronics, inventors tried to solve this with **gears, levers, and rotating cylinders** — encoding numbers in physical positions and letting mechanical motion do the work.

## The Story

### The Abacus — The First Computational Tool (~2400 BC)

The abacus wasn't a calculator — it was a **memory aid**. By moving beads into positions, an operator could hold intermediate results without relying on mental memory alone. The abacus offloaded working memory to a physical object.

Key insight: **externalizing state**. Instead of keeping numbers in your head, you encode them in something physical you can see and manipulate. This idea — state stored in a physical medium — is the seed of all computing.

A skilled abacus user can beat a calculator on simple additions. But an abacus can't carry its own operations. A human must still perform each step.

---

### Napier's Bones — 1617

John Napier, a Scottish mathematician, invented a set of **multiplication rods** in 1617. Each rod was engraved with a multiplication table for a single digit. Laid side by side, they turned multi-digit multiplication into a series of additions that even a non-mathematician could perform.

**How they work:**

Pick the rods corresponding to each digit of your multiplicand

Lay them side by side

Read off the row corresponding to your multiplier

Add the diagonal entries to get the product

For `357 × 8`:

• Place rods for 3, 5, 7

• Read row 8: `2/4 | 4/0 | 5/6`

• Add diagonals: `2 | (4+4) | (0+5) | 6` = `2 | 8 | 5 | 6` = 2856

The machine didn't compute — it **encoded multiplication tables** so a human could assemble the answer from simpler lookups. This is a precursor to lookup tables in software.

---

### Schickard's Calculating Clock — 1623

Wilhelm Schickard built the first known mechanical calculator in 1623 — it was called the "calculating clock." It used a set of Napier-like rods for multiplication and a separate set of wheels for addition and subtraction, with an automatic carry mechanism.

The **carry mechanism** was the key innovation: when a digit wheel rolled from 9 to 0, it automatically advanced the next wheel by 1. This is the same principle as the carry in binary addition — the single most important operation in arithmetic.

Schickard's machine was destroyed in a fire and largely forgotten. But the idea survived.

---

### Pascal's Pascaline — 1642

Blaise Pascal, at age 18, built the **Pascaline** to help his father — a tax collector — with tedious addition and subtraction. It was the first mass-produced mechanical calculator (around 50 were built).

**How it works:**

• A series of interlocked gear wheels, each representing one decimal digit (0–9)

• To add 1, you turn the rightmost wheel one notch

• When a wheel completes a full rotation (9 → 0), it advances the next wheel by one notch — the **carry**

• Subtraction was done by a complementary method (adding the 9's complement)

The Pascaline could only add and subtract. It could not multiply or divide directly. Its greatest contribution was the **mechanical carry** — proving that the most difficult part of arithmetic (propagating carries) could be done by gears without human intervention.

**Why it matters:** Pascal proved that **arithmetic rules could be embedded in physical mechanisms**. The law of adding carries isn't just a mental rule — it can be a gear meshing with another gear.

---

### Leibniz's Stepped Reckoner — 1694

Gottfried Wilhelm Leibniz improved on Pascal by adding a **stepped drum** (Leibniz wheel) — a cylinder with 9 teeth of varying length. By sliding the drum, you could engage 1 to 9 teeth, effectively multiplying by any single digit.

This allowed the Stepped Reckoner to **multiply and divide** via repeated addition/subtraction with a crank-operated register — the first calculator to support all four arithmetic operations.

Leibniz was also the inventor of binary arithmetic. He wrote in 1679 about a system using only 0s and 1s, inspired by the Chinese I Ching. He never connected binary to circuits — that took another 258 years — but the mathematics was there.

---

### The Pattern Across All These Machines

| Machine | Year | Operation | Key Innovation | |---------|------|-----------|---------------| | Abacus | ~2400 BC | Add/Subtract | External state storage | | Napier's Bones | 1617 | Multiply | Encoded lookup tables | | Schickard's Clock | 1623 | Add/Multiply | Mechanical carry | | Pascaline | 1642 | Add/Subtract | Reliable mechanical carry | | Stepped Reckoner | 1694 | All four ops | Variable-engagement gear |

Each machine solved a real bottleneck. Each one embedded a rule of arithmetic into a physical mechanism, offloading cognitive work onto metal and motion.

---

## The Concept: Encoding Information in Physical State

Every one of these machines shares a single deep idea: **numbers are encoded as physical positions**.

• A bead's position on a wire = a digit's value

• A gear wheel's rotational position = a digit (0–9)

• A rod's alignment = a multiplication table entry

This is the essence of computing: **representing information as physical state** and **manipulating that state according to rules**.

The shift from mechanical gears to electrical circuits is a shift in *medium*, not in *concept*. A transistor's on/off state is just a gear wheel's position, but a billion times faster and a billion times smaller.

### What "Carrying" Really Means

The mecha

## Illustrations

**Figure 5 • FIG 0.2-A**

### The Mechanical Calculator — Arithmetic Encoded in Metal and Motion

*Illustration: FIG 0.2-A | The Mechanical Calculator — Arithmetic Encoded in Metal and Motion [SVG diagram — see module-0-2-mechanical.html]*

The two great mechanical calculation inventions side by side. Left: the Pascaline (1642) — six interlocked decimal wheels. Wheel 3 is at digit 9, highlighted in red, about to trigger a carry cascade to wheel 4 (gold). Right: Napier's Bones (1617) — multiplication encoded as rod engravings. Row 6 is selected; diagonal addition of the highlighted cells yields 357 × 6 = 2142.

**Figure 6 • FIG 0.2-B**

### The Mechanical Carry — How 9 + 1 Advances the Next Wheel

*Illustration: FIG 0.2-B | The Mechanical Carry — How 9 + 1 Advances the Next Wheel [SVG diagram — see module-0-2-mechanical.html]*

The Pascaline carry mechanism in three phases. Step 1: units wheel is at 1, tens wheel is at 9. Step 2: adding 1 causes the units wheel to advance to 2 and overflow, sending a carry pulse to the tens wheel (red). Step 3: the tens wheel has advanced from 9 to 0 (gold), the carry is resolved. This same logical structure — overflow produces a carry to the next column — appears in every binary adder ever built.

**Figure 7 • FIG 0.2-C**

### Napier's Bones — How 357 × 6 Works Step by Step

*Illustration: FIG 0.2-C | Napier's Bones — How 357 × 6 Works Step by Step [SVG diagram — see module-0-2-mechanical.html]*

Napier's Bones: 357 × 6 in three steps. Place rods 3, 5, 7. Read row 6: cells show 1/8, 3/0, 4/2. Add diagonals left-to-right: 1 | (8+3) | (0+4) | 2 → carry the 1 from 11 → result 2142. The multiplication problem has been reduced to a sequence of additions — a lookup strategy still used in CPU microcode tables.

**Figure 8 • FIG 0.2-D**

**Mechanical Calculator Lineage — Key Innovations Across Five Machines**

*Illustration: FIG 0.2-D | Mechanical Calculator Lineage — Key Innovations Across Five Machines [SVG diagram — see module-0-2-mechanical.html]*

The mechanical calculator lineage. Each machine contributed exactly one critical innovation that appears in modern computers: external state storage (registers), lookup encoding (ROM), automatic carry (ALU carry flag), variable gear engagement (barrel shifter), and programmability (stored-program architecture).

**Figure 9 • FIG 0.2-E**

**Virtual Pascaline — Accumulator State: 999 + 1 (Multi-Digit Carry)**

*Illustration: FIG 0.2-E | Virtual Pascaline — Accumulator State: 999 + 1 (Multi-Digit Carry) [SVG diagram — see module-0-2-mechanical.html]*

The Pascaline's most revealing edge case: adding 1 to 999. All three wheels are at 9 (highlighted red) before. Each carry must wait for the previous wheel to complete its rotation before advancing the next. After: all three wheels have settled to 0 and the thousands wheel has advanced to 1 (gold). This sequential carry delay — O(n) for n carry wheels — motivated the carry lookahead adder designs of the 1950s, still present in every modern CPU.

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 0.2 Mechanical Computation Simulator: Napier's Bones (1617) & The Pascaline (1642) [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 0.2 Gears and Carries **Type:** Simulation use + written mechanical trace **Estimated time:** 2535 minutes **Tools needed:** `simulation.html` open in your browser + pen and paper --- Part A Napier's Bones (Simulation) Use the **Napier's Bones tool** in the simulation to solve these multiplications. For each one, write down: 1. Which rods you selected 2. Which row you read 3. The diagonal values you added 4. The final answer Problems 1. `347 6` 2. `829 7` 3. `1284 9` **After completing each:** verify your answer using regular multiplication. They must match. --- Par

# Module 0.3 The Loom That Taught Computers

## The Problem

*You own a silk weaving factory in Lyon, France, in 1800. Your most valuable product is fabric woven in complex patterns — flowers, birds, geometric designs that wealthy clients pay a fortune for.*

>

*Each pattern requires a master weaver to manually control which threads go up and which go down on every single pass of the shuttle. One wrong move and the pattern breaks. A skilled weaver takes **years** to learn. They are expensive, they tire, they make mistakes, and they can only work on one pattern at a time.*

>

*If a pattern is successful, you want to make thousands of metres of it. But the weaver must essentially re-learn the pattern for every metre — holding the entire sequence in their head.*

>

*How do you separate the knowledge of the pattern from the act of weaving?*

This is not a question about fabric. It is the question that gave birth to the concept of **stored instructions** — the idea that a procedure can be written down and executed by a machine without a human in the loop.

---

## The Story

### Weaving Before Jacquard

A loom works by lifting certain warp threads (running lengthwise) while passing a shuttle carrying the weft thread (running crosswise) through the gap. By changing which warp threads are raised on each pass, you create patterns.

In a simple loom, this was done by hand — the weaver manually selected threads. For complex patterns, a **drawboy** sat beside the loom, pulling cords to raise groups of threads according to the weaver's shouted instructions. The drawboy had to memorize or follow a complex sequence. Errors were common. The drawboy had to be trained for each new pattern.

The bottleneck: **the pattern existed only in human memory and voice**.

---

### Basile Bouchon's Paper Roll — 1725

The first attempt to automate pattern control came from Basile Bouchon in 1725. He created a mechanism where a roll of paper with holes punched in it was pressed against a row of needles. Where there was a hole, a needle passed through and lifted a thread hook. Where there was no hole, the needle was blocked.

**The profound shift**: the pattern is no longer in a person's head. It is encoded as **holes in paper** — a physical, durable, reproducible medium.

This is the first time in history that **data was stored externally to a human mind and read by a machine**.

The limitation: Bouchon's roll only controlled a portion of the threads. A human operator still controlled the rest.

### Falcon's Punched Cards — 1728

Jean-Baptiste Falcon improved on Bouchon three years later by replacing the paper roll with a **chain of stiff punched cards**, linked together in sequence. Each card controlled one row of the pattern. Cards could be detached and rearranged.

Two key advances:

**Discrete units**: each card was a separate instruction (not a continuous roll)

**Reordering**: patterns could be modified by rearranging cards

This is the birth of **sequential, discrete instructions** — the ancestor of every program ever written.

### Jacquard's Loom — 1804

Joseph Marie Jacquard synthesized and perfected these ideas into a complete automated system in 1804. His attachment sat on top of a standard loom and completely automated the thread selection process.

**How it works:**

A chain of stiff punched cards passes over a square cylinder (prism)

On each pass of the shuttle, the next card is pressed against a row of spring-loaded needles

**Where there is a hole**: the needle passes through, hooks its corresponding thread hook, raises the warp thread → thread is UP

**Where there is no hole**: the needle is blocked, the thread is NOT lifted → thread is DOWN

The shuttle passes through the gap created by the raised threads

The next card advances — the next row of the pattern is set

The complete pattern — potentially thousands of rows — is stored in the card chain. The loom executes it automatically, row by row, without any human decision-making during operation.

**One key number**: A Jacquard loom could execute a pattern requiring 24,000 cards in sequence. A single weaver could then produce complex silk damask at the rate a simple loom produces plain fabric.

### The Reaction

The impact was immediate and violent. Lyon's weavers rioted. Jacquard was attacked in the street. His looms were burned. The city council confiscated the machines.

Then economics won. By 1812, Napoleon had ordered 11,000 Jacquard looms installed across France. By 1833, there were 30,000 in Lyon alone. The weavers who once opposed the machine now operated it.

### Babbage Sees It

Charles Babbage visited a weaving exhibition in London in the 1830s and saw a portrait of Jacquard himself — **woven in silk on a Jacquard loom**, using 24,000 punch cards. The portrait had the photographic detail of an engraving.

Babbage later wrote: *"The Jacquard loom is capable of weaving any design which the imagination of man may conceive."*

He immediately grasped that the same principle — **instructions stored on cards, read sequentially by a machine** — could control a calculating engine. Not just weaving, but any computation.

This insight is the direct intellectual link between the Jacquard loom and the Analytical Engine (Module 0.4) — and by extension, to every computer ever built.

---

## The Concept: What a Punch Card Actually Is

A punch card is a **binary encoding of instructions**.

Each position on the card can be either:

• **Hole** → 1 (do this: lift this thread / read this data)

• **No hole** → 0 (don't do this: leave thread down / skip)

A row of 8 positions gives you 8 binary decisions. That's 1 byte. A card with 24 rows of 80 positions is 1,920 bits = 240 bytes.

This is not a metaphor. IBM's 80-column punch cards, used until the 1980s, encoded exactly 80 characters of data per card using a binary hole/no-hole scheme — the direct industrial descendant of Jacquard's system.

### The Three Revolutionary Ideas

| Idea | What it meant | |------|

### Illustrations

**Figure 10 • FIG 0.3-A**

**The Jacquard Loom — Instructions Stored in Holes**

*Illustration: FIG 0.3-A | The Jacquard Loom — Instructions Stored in Holes [SVG diagram — see module-0-3-loom.html]*

The Jacquard loom (1804) — the first machine to execute stored instructions. Left: the loom frame with vertical warp threads interlaced with horizontal weft threads producing a woven pattern. The Jacquard head at the top reads the active punch card. Right: the card encoding — a hole allows a needle to pass through, raising the corresponding heddle and lifting that warp thread; a blank blocks the needle, leaving the thread down. Each row of holes encodes one weft pick. The sequence of cards is a program.

**Figure 11 • FIG 0.3-B**

### Punch Card Encoding — Pattern to Holes to Binary

*Illustration: FIG 0.3-B | Punch Card Encoding — Pattern to Holes to Binary [SVG diagram — see module-0-3-loom.html]*

The encoding pipeline. Stage 1: a 5-row weave pattern where dark cells indicate the weft thread passing over the warp. Stage 2: the corresponding punch card — a hole appears wherever the weft must cross over (dark cell = 1 = hole). Stage 3: each row reads as a binary byte. Row 1 (1 0 1 0 1 0 1 0) produces a plain alternating twill; Row 3 (1 1 0 0 1 1 0 0) produces a wider herringbone. The sequence of cards is the complete fabric program.

**Figure 12 • FIG 0.3-C**

### Stored Instructions — From Card Chain to Stored Program

*Illustration: FIG 0.3-C | Stored Instructions — From Card Chain to Stored Program [SVG diagram — see module-0-3-loom.html]*

The intellectual lineage of stored instructions. Jacquard (1804): instructions as a physical card chain — sequential, unmodifiable. Hollerith (1890): cards encode data records, separating program logic from data content. Von Neumann (1945): instructions and data share the same memory — the program can now inspect and modify itself. Modern CPUs (2024): the identical architecture, shrunk to nanometre scale. The Jacquard loom's core insight — that instructions can be stored externally and executed in order — flows unbroken through 220 years of computing.

**Figure 13 • FIG 0.3-D**

### Jacquard's Workshop — Lyon, 1804

*Illustration: FIG 0.3-D | Jacquard's Workshop — Lyon, 1804 [SVG diagram — see module-0-3-loom.html]*

Historical reconstruction of a Jacquard loom workshop, Lyon 1804. An operator feeds the active punch card to the Jacquard head, which reads each hole and controls the heddle harness below. A card library on the right holds multiple pattern programs — different cards produce different fabrics on the same loom. This is the first practical separation of hardware (loom) from software (card programs), a concept Babbage explicitly borrowed when designing the Analytical Engine.

**Figure 14 • FIG 0.3-E**

**Card Pattern Simulator — 8×8 Weave Grid with Binary Output**

*Illustration: FIG 0.3-E | Card Pattern Simulator — 8×8 Weave Grid with Binary Output [SVG diagram — see module-0-3-loom.html]*

Static view of the Module 0.3 simulation. Left: the 8×8 pattern editor with a diamond motif drawn — dark cells indicate where the weft thread passes over the warp. Centre: the automatically generated punch card with holes punched at the corresponding positions. Right: the binary encoding — each row produces one byte of pattern data, with 1s (red) corresponding to holes and thread lifts. The complete 8-row card is the program that produces the diamond weave.

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 0.3 Jacquard Loom Simulator: Program a punch card chain. The loom executes it hole by hole, row by row. [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 0.3 Program the Loom **Type:** Simulation design + binary encoding + pattern decoding **Estimated time:** 3040 minutes **Tools needed:** `simulation.html` open in your browser + pen and paper --- Part A Design and Weave (Simulation) Using the **Punch Card Designer** in the simulation, design card chains to produce the following patterns. For each, take note of your card design (which holes you punched). The loom has **8 threads** (columns) and you can use up to **8 cards** (rows). Pattern 1 Checkerboard Row 1: threads 1,3,5,7 UP Row 2: threads 2,4,6,8 UP Row 3: threa

# Module 0.4 Babbage and Ada: The First Programmers

## The Problem

*You are an astronomer in 1820. You need a table of logarithms accurate to 20 decimal places. The only way to produce it: hire a room full of human "computers" to calculate each value by hand and have another room check their work.*

>

*Charles Babbage receives such a table, sits down to check it, and finds **error after error**. He slams it on the desk and says to his colleague:*

>

*"I wish to God these calculations had been executed by steam."*

That frustration launched 40 years of obsessive work — and produced the first complete design of a general-purpose programmable computer, a century before electronics made it practical.

## The Story

### The Difference Engine — 1822

Babbage's first machine, the **Difference Engine**, was designed to compute polynomial functions using the **method of finite differences**.

**Why finite differences?** Almost every useful mathematical table — logarithms, trigonometric functions, astronomical positions — can be *approximated* by a polynomial. And polynomials have a beautiful property: if you take successive differences of their values, you eventually reach a constant.

For example, the polynomial $f(n) = n^2 + n + 41$ at n = 0,1,2,3...:

```
n:        0    1    2    3    4    5
f(n):    41   43   47   53   61   71
1st diff:   2    4    6    8   10
2nd diff:     2    2    2    2
```

The second difference is always 2. So to compute the *next* value of `f(n)`, you only ever need to **add** — no multiplication required.

Babbage's Difference Engine encoded this: it had columns of gear wheels, each representing one "order" of difference. By cranking a handle, it automatically computed the next value by sequential addition — mechanically stepping through differences.

**What it was:** A single-purpose calculator for polynomial tabulation. It could not be reprogrammed. You set up the initial values on the columns, cranked the handle, and it printed the results automatically.

**What happened:** Babbage began construction in 1823 with British government funding. After a decade and £17,000 (millions in today's money), the machine was abandoned. The manufacturing precision required was beyond what 1830s toolmaking could deliver. Parts didn't fit. The project collapsed.

A complete Difference Engine No. 2 was finally built by the Science Museum in London in **1991** using Babbage's original plans — and it worked perfectly.

### The Analytical Engine — 1837 onward

Before the Difference Engine was even finished, Babbage's mind had already moved on to something far more ambitious.

The **Analytical Engine** was designed to be a **general-purpose mechanical computer**. Its design included, in 1837:

| Component | Babbage's name | Modern equivalent | |---|---|---| | Memory for 1,000 numbers of 50 digits | **The Store** | RAM | | The computing unit | **The Mill** | CPU / ALU | | Input via punch cards | **Operation cards + Variable cards** | Instruction + data memory | | Conditional branching | **Conditional lever** | IF statement / branch instruction | | Looping | **Backing up the cards** | Loops /

iteration | | Output printer | **The Printing Apparatus** | Output device |

This is not a rough sketch. Babbage produced **thousands of engineering drawings** for the Analytical Engine. The conditional branching mechanism — the ability to take different actions based on the result of a computation — is the feature that makes a calculator into a computer.

**Why it was never built:** Babbage couldn't raise funding, couldn't get government support after the Difference Engine debacle, and spent his remaining years revising the design. He died in 1871 with the Analytical Engine unbuilt.

---

### Ada Lovelace — The First Programmer

In 1842, an Italian mathematician named Luigi Menabrea published a paper describing the Analytical Engine in French, based on lectures Babbage gave in Turin. A young English mathematician — **Augusta Ada King, Countess of Lovelace** — was asked to translate it.

Ada didn't just translate. She added her own **Notes**, labeled A through G, that were **three times longer than the original paper**.

**Note G** is the most famous document in the history of computing. It contains:

**The first published algorithm** intended to be executed by a machine — a procedure for computing Bernoulli numbers using the Analytical Engine

A complete understanding of *what an algorithm is*: a finite sequence of unambiguous steps that can be mechanically followed

A visionary observation: *"The Engine might act upon other things besides number... the Engine might compose elaborate and scientific pieces of music of any degree of complexity or extent."*

Ada understood something Babbage never fully articulated: the Engine's operations were not fundamentally about *numbers* — they were about *symbols* that could represent anything. This insight took the field another century to catch up to.

**Ada's algorithm for Bernoulli numbers** used:

• Variables stored in the "Store" (memory locations)

• Arithmetic operations in the "Mill" (the ALU)

• A looping structure (repeating a set of operations)

• A sequence of operation cards (the program)

It is, by every meaningful definition, a program.

---

### Why the Analytical Engine Matters

Babbage and Ada's machine was never built. So why does it matter?

**1. It proved the concept was coherent.** The design was complete and internally consistent. When the Science Museum built the Difference Engine from Babbage's drawings, it worked. The Analytical Engine would have worked too — just extremely slowly and at enormous cost.

**2. It established the architecture.** The Store + Mill + input/output + conditional branch is the **Von Neumann architecture** (1945), a century before Von Neumann wrote it down. Every computer you have ever used is a descendant of this architecture.

**3. Ada named the abstraction.** By writing the first algorithm and articul

## Illustrations

**Figure 15 • FIG 0.4-A**

### The Analytical Engine — Babbage's Vision of a General-Purpose Computer

*Illustration: FIG 0.4-A | The Analytical Engine — Babbage's Vision of a General-Purpose Computer [SVG diagram — see module-0-4-babbage-ada.html]*

Left: The Difference Engine — a finite-difference calculator with six columns of toothed brass wheels. The highlighted column ($\Delta^2$) is the second-order difference register. The printing mechanism at the base stamps results directly onto paper, eliminating human transcription errors. Right: The Analytical Engine's four-component architecture. Unlike the Difference Engine, it included conditional branching — the ability to execute different instruction sequences based on computed results. This is the first complete description of a general-purpose programmable computer, 108 years before the first

**Figure 16 • FIG 0.4-B**

### Method of Finite Differences — Computing f(x) = x² Without Multiplication

*Illustration: FIG 0.4-B | Method of Finite Differences — Computing f(x) = x² Without Multiplicat [SVG diagram — see module-0-4-babbage-ada.html]*

The method of finite differences applied to $f(x) = x^2$. The second-order difference ($\Delta^2$) is always 2 for any quadratic — a constant. The engine exploits this: at each step it adds $\Delta^2$ to $\Delta^1$ to produce a new first difference, then adds that to the current f(x) to produce the next f(x). No multiplication is ever performed. Babbage designed columns of digit wheels, one per order of difference, connected by an addition mechanism. The same principle applies to any polynomial up to any degree.

**Figure 17 • FIG 0.4-C**

### Ada Lovelace's Note G — The First Algorithm, 1843

*Illustration: FIG 0.4-C | Ada Lovelace's Note G — The First Algorithm, 1843 [SVG diagram — see module-0-4-babbage-ada.html]*

Ada Lovelace's Note G (1843) — the first published algorithm intended for machine execution. Left: manuscript style notes with the recursive Bernoulli formula and a fragment of her operation table. Centre: flowchart reconstruction showing the loop structure — Ada explicitly described the need for the engine to repeat a sequence of operations until a condition changes, which is the first documented algorithmic loop. Right: the output — Bernoulli numbers with operation counts showing the computational cost grows with n. Ada identified looping, variables, and subroutines as the three essential pr

**Figure 18 • FIG 0.4-D**

### Babbage's Analytical Engine vs Modern Von Neumann CPU — 186 Years Apart

*Illustration: FIG 0.4-D | Babbage's Analytical Engine vs Modern Von Neumann CPU — 186 Years Apar [SVG diagram — see module-0-4-babbage-ada.html]*

Direct architectural comparison between Babbage's 1837 design and a 2024 CPU. Every major component maps exactly: The Mill (gear-based arithmetic) → ALU/FPU (transistor-based arithmetic). The Store (brass registers) → RAM/Cache (DRAM/SRAM). The Reader (punch card input) → I/O Controller (USB/PCIe). The Printer (stereotype press) → GPU/Display. The logical structure Babbage described in the 1830s is the same structure inside every computer today. What changed is purely physical — metal gears became silicon transistors.

**Figure 19 • FIG 0.4-E**

### Difference Engine Simulation — Stepping Through f(x) = x²

*Illustration: FIG 0.4-E | Difference Engine Simulation — Stepping Through f(x) = x² [SVG diagram — see module-0-4-babbage-ada.html]*

Static simulation view: the Difference Engine stepping through f(x) = x² in three steps. Each register panel shows the three column values: f(x), $\Delta^1$, and $\Delta^2$ (constant 2). At each step the engine performs exactly two additions — $\Delta^2$ added to $\Delta^1$, then updated $\Delta^1$ added to f(x). The result matches x² exactly at every step, without ever performing a multiplication. Babbage's insight was that any polynomial function can be reduced to repeated addition through this method.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 0.4 Babbage & Ada Simulator: Difference Engine Polynomial Tables    Analytical Engine General Programming [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 0.4 Program the Analytical Engine **Type:** Simulation use + algorithm tracing + written analysis **Estimated time:** 3545 minutes **Tools needed:** `simulation.html` open in your browser + pen and paper --- Part A Difference Engine: Tabulate a Polynomial Using the **Difference Engine** tab in the simulation, tabulate the polynomial: **`f(n) = n + 3n + 2`** for n = 0 through 9 Step 1 Calculate by hand first Fill in this table manually before using the simulator: | n | f(n) = n+3n+2 | 1st diff | 2nd diff | |---|----------------|----------|----------| | 0 | ? | | | | 1 |

# Module 0.5 Hollerith and the Punch Card Era

## The Problem

*It is 1887. The US Census Bureau is staring at the results of the 1880 census — still being compiled, eight years later. The 1890 census is three years away and will count even more people. At the current rate, the results will still be coming in when the 1900 census begins.*

>

*A young statistician named Herman Hollerith has an idea. He watched a train conductor punch passenger tickets to record seat, destination, and fare. He remembered Jacquard's loom. He had worked briefly at the Census Bureau himself.*

>

*What if you could encode a person's census data as holes in a card — and have a machine read, sort, and tally those cards automatically?*

Hollerith's answer to this question didn't just solve the census problem. It invented the data processing industry, gave birth to IBM, and established the punch card as the dominant data storage medium for the next century.

## The Story

### Hollerith's Key Insight — The Conductor's Ticket

The story Hollerith told about inspiration involves a train conductor punching passenger descriptions into tickets — hair color, eye color, height — to prevent ticket fraud (someone buying a ticket and passing it to someone else). Each punch described a person.

Hollerith's leap: if you can describe a passenger in punched holes, you can describe a census respondent in punched holes. And if the card can represent a person, a machine that reads cards can process people — counting, sorting, tabulating — without any human doing the arithmetic.

### The Hollerith Machine — 1890

Hollerith's tabulating system had three components:

**1. The Pantograph Punch** An operator sat with the completed paper census form and used a hand punch to create holes in the card at positions corresponding to each answer. Age 34 → punch position 34. Occupation "Farmer" → punch position in the occupations zone. Foreign-born → punch a specific position.

Each card represented one person. The card had a grid of possible positions. The presence or absence of a hole at each position encoded that person's attributes as binary data.

**2. The Tabulator** The card was placed in a press. When the operator pressed down, a grid of spring-loaded pins descended onto the card. Where there was a hole, the pin passed through into a small cup of mercury, completing an electrical circuit. Where there was no hole, the pin was blocked by the card.

Each completed circuit incremented a mechanical counter by 1 — a small gear wheel, like the Pascaline, but driven by an electromagnet rather than a human hand.

This is the critical link: **a hole in a card** → **electrical signal** → **counter increments**. The physical hole becomes an electrical event becomes a counted datum.

**3. The Sorter** A separate device could sort cards into bins based on which circuit fired. This let operators group cards by any attribute — all farmers, all immigrants, all people over 65 — without manual searching.

---

### The 1890 Census — By the Numbers

| Metric | 1880 (manual) | 1890 (Hollerith) | |--------|---------------|-----------------| | Population counted | 50,189,209 | 62,979,766 | | Time to basic count | ~2.5 years | **6 weeks** | | Time to full tabulation | 8 years | ~2.5 years | | Error rate | ~5% | <1% | | Cost | ~$11.5M | ~$5M |

The Census Bureau saved over $5 million and delivered usable data in time to actually influence congressional apportionment. It was a decisive demonstration.

---

### The Birth of IBM

Hollerith founded the **Tabulating Machine Company** in 1896 to lease his equipment to businesses. The census had proven the concept; now he applied it to payroll processing, freight statistics, and insurance accounting.

In 1911, the company merged with two others to form the **Computing-Tabulating-Recording Company (CTR)**. In 1924, CTR was renamed by its new president, Thomas J. Watson Sr.:

**International Business Machines — IBM.**

The 80-column IBM punch card that became the standard by 1928 was a direct evolution of Hollerith's original design. It remained the dominant data input medium for computers until the early 1980s — nine decades of continuous use.

---

### How Hollerith Encoding Works

The Hollerith code assigned positions on a 12-row, 80-column card:

• **Rows 0–9**: digits 0 through 9

• **Rows 11–12** (above row 0): "zone punches" — combined with digit rows to encode letters

To encode the letter **A**: punch row 12 (zone) AND row 1 (digit 1) in the same column. To encode the letter **B**: punch row 12 AND row 2. ... To encode the letter **Z**: punch row 11 AND row 9.

A single column = one character. 80 columns = 80 characters per card. A "deck" of cards = a file of records.

This encoding scheme — a binary position encoding a character — is the ancestor of **ASCII** (1963), the character encoding system still used in every computer today.

---

### The Legacy: From Holes to Bits

Trace the lineage from Jacquard to Hollerith to modern computing:

```
Jacquard loom (1804)
    hole in card → thread lifts
    → stored instructions for weaving

Hollerith tabulator (1890)
    hole in card → electrical circuit closes → counter increments
    → stored data for processing

IBM punch cards (1928–1980s)
    80 columns × 12 rows = 960 bit positions per card
    → stored programs AND data for early mainframes

Magnetic storage (1950s onward)
    magnetic domain = bit
    → hard drives, tapes

Semiconductor memory (1970s onward)
    transistor state = bit
    → RAM, flash storage

The "bit" — the binary digit — is the hole in Jacquard's card,
abstracted through a century of technology changes.
```

The hole is gone. The bit remains.

---

## The Concept: Data Encoding

Every data encoding scheme answers the same question: **how do we represent information as a physical state that a machine can read?**

Hollerith's answers:

• Information unit: one census attribute (age, occupation, birthplace)

• Physical carrier: a stiff card

• Encoding method: hole position = attribut

## Illustrations

**Figure 20 • FIG 0.5-A**

### The Hollerith Tabulating Machine — 1890 US Census

*Illustration: FIG 0.5-A | The Hollerith Tabulating Machine — 1890 US Census [SVG diagram — see module-0-5-hollerith.html]*

The Hollerith system in operation. Left: the electric tabulating machine — a cabinet of circular counting dials (each incrementing as holes pass under the pin press) with a card press reader at the bottom and sorting compartments below. Right: an annotated 1890 census punch card with field zones and digit rows labelled. Each card encodes one person's demographic record across 80 columns. Bottom: time comparison — 8 years manually versus 6 weeks with the Hollerith system, a 78× improvement that determined the viability of the decennial census.

**Figure 21 • FIG 0.5-B**

### Hollerith Card Anatomy — How 80 Columns Encode Alphanumeric Data

*Illustration: FIG 0.5-B | Hollerith Card Anatomy — How 80 Columns Encode Alphanumeric Data [SVG diagram — see module-0-5-hollerith.html]*

The Hollerith card encoding scheme. Three zone rows (12, 11, 0) at the top and nine digit rows (1–9) below. A letter is encoded by punching a zone row plus a digit row in the same column. A digit is encoded by punching its digit row alone. Column A (zone 12 + digit 1) and column B (zone 12 + digit 2) are shown labelled. This two-punch scheme for letters vs one-punch for digits is a compressed encoding that maximises the information density of 80 physical columns.

**Figure 22 • FIG 0.5-C**

### Census Tabulation Workflow — Card to Sorted Count

*Illustration: FIG 0.5-C | Census Tabulation Workflow — Card to Sorted Count [SVG diagram — see module-0-5-hollerith.html]*

The complete 1890 census tabulation pipeline. Step 1: field enumerators collect household data on paper forms. Step 2: clerks key-punch the data onto cards. Step 3: cards pass through the Hollerith press — metal pins drop through holes to contact a mercury cup, closing an electrical circuit. Step 4: each closed circuit pulses an electromagnetic counter dial. Step 5: dial readings are transcribed into tabulated totals. The 62 million person census was fully processed in 6 weeks — complete results published 5 years ahead of the 1900 deadline.

**Figure 23 • FIG 0.5-D**

### Census Efficiency: 1880 Manual vs 1890 Hollerith

*Illustration: FIG 0.5-D | Census Efficiency: 1880 Manual vs 1890 Hollerith [SVG diagram — see module-0-5-hollerith.html]*

The census efficiency comparison quantifies the Hollerith impact precisely. The 1880 census took 8 years with 450 clerks handling 50 million records — an estimated 1.5 million tallying errors accumulated. The 1890 census processed 62 million records (24% more people) in 6 weeks using 56 machines at half the cost. Mechanical counting eliminated the arithmetic error pathway entirely; errors could only enter at the key-punch stage. Hollerith's company later became IBM.

**Figure 24 • FIG 0.5-E**

### Hollerith Card Encoder/Decoder — Text to Card to Decode

*Illustration: FIG 0.5-E | Hollerith Card Encoder/Decoder — Text to Card to Decode [SVG diagram — see module-0-5-hollerith.html]*

Static view of the Module 0.5 encoder/decoder simulation. Input "HELLO" — each letter resolves to a Hollerith zone+digit pair: H=12+8, E=12+5, L=12+3, L=12+3, O=12+6. The generated card shows two holes per column — one in the zone-12 row and one in the digit row. The decoder reads the same card and reconstructs the original five characters. The challenge asks students to encode their own name and decode a mystery card.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 0.5 Hollerith Tabulating System: Punch cards Mercury cups Counters The birth of data processing [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 0.5 The Hollerith Tabulator **Type:** Simulation use + encoding + data analysis + Level 0 capstone questions **Estimated time:** 4050 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Encode and Punch Using the **Card Puncher** in the simulation, encode the following 5 census respondents as punch cards. For each person, record which column positions are punched. | | Name | Age | Occupation | Birthplace | Family Size | |---|------|-----|------------|------------|-------------| | 1 | John Hayes | 34 | Farmer | Ireland | 5 | | 2 | Clara Schmidt | 28 | Tea

LEVEL 01

# Physical Computing

From Electricity to Logic — The Birth of Electronic Computation

*The gear gave way to the electron. Everything else followed.*

## What This Level Is About

Level 0 showed that humans needed machines to compute and that information could be stored in physical media — holes in cards, gear positions, ink on paper. But gears are slow, heavy, and wear out. The real revolution came when we replaced mechanical motion with **electrical switching**.

This level traces that journey — from the basic physics of electricity, through telegraph relays and vacuum tubes, to the transistor and the integrated circuit. Every step reduces the physical size of a "switch" while increasing its speed by orders of magnitude.

By the end of Level 1, you will understand:

• What electricity is and how it flows through circuits

• Why a switch is the fundamental atom of all computation

• How relay circuits implement logical decisions (AND, OR, NOT)

• Why vacuum tubes replaced relays, and why transistors replaced vacuum tubes

• What Moore's Law is, why it held for 50 years, and where it's going

## Modules

| Module | Title | Core Idea | |--------|-------|-----------| | [1.1](./module-1-1-electricity/lesson.md) | What is Electricity? | Voltage, current, resistanc

# Module 1.1 What is Electricity? (Only What We Need)

## The Problem

*Gears work. The Pascaline proved it. The Difference Engine proved it. But gears have a hard limit: to go faster, you need to move mass faster. Moving mass requires force. Force creates friction, heat, and wear.*

>

*By 1900, the fastest mechanical calculators could do a few operations per second. The 1890 census used Hollerith's electromechanical machines — the mechanical counters still moved physical gear wheels, just driven by electricity rather than hand-cranking.*

>

*To reach a million operations per second — let alone a billion — you need something that has almost no mass to move. You need **electrons**.*

Before we can understand how transistors and circuits work, we need to understand what electricity actually is — not as a mysterious force, but as the movement of charged particles through a material, governed by three simple relationships.

## The Story

### What Is Electricity?

Everything around you is made of atoms. Atoms have a nucleus (protons and neutrons) surrounded by electrons in orbits. Electrons carry a **negative charge**; protons carry a **positive charge**.

In most materials, electrons are tightly bound to their atoms. But in **conductors** — metals like copper, gold, and silver — the outermost electrons are loosely bound. They can drift from atom to atom through the material.

**Electric current** is the flow of these loosely-bound electrons through a conductor. Think of a copper wire as a tube packed with electrons. Push on one end (apply a voltage), and electrons drift through the tube from one end to the other.

One important convention: by historical accident, electric current is defined as flowing **from positive to negative** — the opposite direction from electron flow. We stick with this convention because it's universal, but always remember electrons actually move the other way.

### The Three Quantities: A Water Analogy

The best way to build intuition about electricity is through water flowing in pipes.

**Voltage (V) — Water Pressure**

Voltage is the "pressure" that pushes electrons through a circuit. A higher voltage means a stronger push. Measured in **Volts (V)**.

• A AA battery: 1.5V

• A phone charger: 5V

• A household outlet: 120V (US) / 230V (EU)

• A lightning bolt: ~300,000,000V

No voltage → no flow. Voltage is always measured *between two points* — it's a difference in electrical potential, like measuring the height difference between two water tanks.

**Current (I) — Water Flow Rate**

Current is how many electrons actually flow past a point per second. Measured in **Amperes (A)** — 1 Amp = 6.24 × 10¹■ electrons per second passing a point.

• An LED: ~0.02A (20 milliamps)

• A phone charging: ~1–2A

• A household circuit: 15–20A

• A lightning bolt: ~30,000A (briefly)

**Resistance (R) — Pipe Narrowness**

Resistance is how much a material opposes the flow of electrons. A narrow pipe resists water flow; a thin wire resists electron flow. Measured in **Ohms (Ω)**.

• A human body (dry): ~100,000 Ω

• A piece of copper wire (1m): ~0.017 Ω

• A 60W light bulb filament: ~240 Ω

• An open circuit (air gap): essentially infinite Ω

---

## Ohm's Law: The Fundamental Relationship

These three quantities are connected by one of the most important equations in physics:

```
V = I × R
```

**Voltage = Current × Resistance**

Rearranged:

• `I = V / R` — more voltage or less resistance → more current

• `R = V / I` — measure voltage and current to find resistance

This single equation predicts the behavior of most simple circuits. Every resistor, every LED, every transistor bias calculation starts here.

**Example:** A 5V battery connected to a 250Ω resistor:

```
I = V / R = 5 / 250 = 0.02A = 20mA
```

That's exactly the right current for a standard LED. This is why resistors are always paired with LEDs.

### Open vs Closed Circuits

A circuit needs to be a **complete loop** for current to flow.

• **Closed circuit**: the path from + terminal, through components, back to – terminal is unbroken → current flows

• **Open circuit**: the path is broken somewhere (a switch is open, a wire is cut) → no current flows

• **Short circuit**: the + and – terminals are connected directly with no resistance → enormous current flows, generating heat (dangerous)

A **switch** is simply a device that opens or closes a circuit on demand. When the switch is closed, the circuit is complete and current flows. When it's open, current stops.

This is the key connection to computing: **a switch is a binary device**. Closed = 1. Open = 0. Everything in digital electronics is built from switches.

### Series vs Parallel

**Series circuit**: components connected end-to-end in a single path.

• The same current flows through all components

• Voltages add up: `V_total = V1 + V2 + V3`

• Resistances add up: `R_total = R1 + R2 + R3`

• If one component fails (opens), the entire circuit stops (old Christmas lights)

**Parallel circuit**: components connected side-by-side, sharing the same two endpoints.

• The same voltage appears across all components

• Currents add up: `I_total = I1 + I2 + I3`

• Total resistance is *less* than any individual resistor: `1/R_total = 1/R1 + 1/R2 + 1/R3`

• If one component fails, others keep working (modern Christmas lights, house wiring)

### Power: The Heat Problem

Power is the rate at which a circuit consumes energy:

```
P = V × I     (also P = I²×R = V²/R)
```

Measured in **Watts (W)**. Every electron pushed through a resistance generates heat. This is why:

• Your phone gets warm while charging

• Early computers (vacuum tube era) required massive cooling systems

• Modern CPUs have heat sinks and fans

• Power consumption is one of the central engineering constraints of chip design

The transistor's dominance over vacuum tubes came partly from power: a vacuum tube might dissipate 1–5 watts; a modern transistor dissipates picowatts (10■¹² W). A billion transistors at a picowatt each = 1mW total — manageable.

#

## Illustrations

**Figure 25 • FIG 1.1-A**

### Electricity as Controlled Flow — The Foundation of All Computing

*Illustration: FIG 1.1-A | Electricity as Controlled Flow — The Foundation of All Computing [SVG diagram — see module-1-1-electricity.html]*

A complete series circuit: a 9V battery, connecting wires, and an incandescent bulb. Red dots show electron flow direction (negative to positive terminal). Current flows continuously as long as the circuit is unbroken. The three core quantities — voltage (driving force), current (flow rate), and resistance (opposition) — are related by Ohm's Law: V = I × R. For computing, the key insight is that voltage levels can represent binary states: high voltage = 1, low voltage = 0.

**Figure 26 • FIG 1.1-B**

### The Water Analogy — Voltage, Current, and Resistance Made Intuitive

*Illustration: FIG 1.1-B | The Water Analogy — Voltage, Current, and Resistance Made Intuitive [SVG diagram — see module-1-1-electricity.html]*

The water analogy for electricity. Left: a plumbing system — reservoir (high pressure), valve (switch), narrow pipe (resistance), low basin, and pump (battery) returning water. Right: the electrical equivalent. Centre: the analogy map. The critical computing insight is the valve/switch row: an open valve allows flow (circuit ON), a closed valve blocks it (circuit OFF). A transistor is a voltage-controlled valve — the fundamental building block of all digital logic.

**Figure 27 • FIG 1.1-C**

### Open Circuit vs Closed Circuit — The Binary Reality of Switches

*Illustration: FIG 1.1-C | Open Circuit vs Closed Circuit — The Binary Reality of Switches [SVG diagram — see module-1-1-electricity.html]*

The two states of a switch map directly to the two binary values. Open circuit: the switch creates a gap, breaking the conducting path. No current flows. The bulb is dark. This represents logic 0 / LOW. Closed circuit: the switch connects the gap, completing the path. Current flows through the filament, producing light. This represents logic 1 / HIGH. Every bit in every computer is physically this — a switch (transistor) that is either open or closed.

**Figure 28 • FIG 1.1-D**

**Series vs Parallel Circuits — Two Bulbs, Two Behaviours**

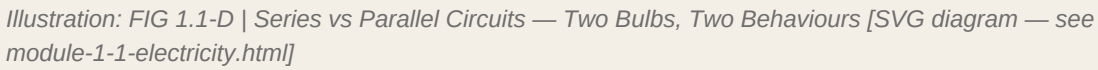


*Illustration: FIG 1.1-D | Series vs Parallel Circuits — Two Bulbs, Two Behaviours [SVG diagram — see module-1-1-electricity.html]*

Series vs parallel circuits — the simulation's second exercise. Series (left): both bulbs share the same current path. Each gets half the voltage, so both glow dimly. Remove one bulb and both go out — a cascade failure. Parallel (right): each bulb has its own independent path to the battery. Each receives the full 9V and glows at full brightness. Remove one bulb and the other stays on. CPU power delivery uses massive parallel distribution for exactly this reason: independent, redundant paths.

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 1.1 Electricity Simulator: Voltage Current Resistance the switch that connects them to computing [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 1.1 Ohm's Law and Circuit Basics **Type:** Calculation + simulation observation + written explanation **Estimated time:** 2535 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Ohm's Law Calculations Solve each problem. Show your working. Use `V = I R`. 1. A 9V battery is connected to a 180 resistor. What current flows? 2. A circuit carries 50mA of current through a 220 resistor. What is the voltage across the resistor? 3. A 12V power supply produces 0.5A of current. What is the resistance of the load? 4. An LED needs exactly 20mA to operate safely

# Module 1.2 Switches: The Atom of Computing

## The Problem

*We now know electricity flows through circuits and obeys Ohm's Law. We know a switch opens or closes a circuit. Simple enough.*

>

*But here is the question that changes everything:*

>

***What if the switch itself is controlled by electricity — not by a human hand?***

>

*If a tiny electrical signal can open and close a much larger circuit, you have a device that can make decisions automatically. You have a device that can respond to one electrical condition and change another. You have, in principle, the ability to build any logical operation from pure electricity.*

>

*That single idea — the electrically-controlled switch — is the atom from which all computing is built.*

---

## The Story

### The Telegraph and the Relay — 1837

Samuel Morse's telegraph (1837) sent electrical pulses down a wire to ring a bell at the other end. But there was a problem: over long distances, the signal weakened. After a few hundred kilometres, it was too faint to ring the bell.

The solution was a **relay station**: a box placed along the wire that used the weak incoming signal to switch a fresh, strong local battery into the line. The weak signal didn't drive the bell directly — it controlled a switch that let a strong local current drive the next segment.

This device was called a **relay** because it "relayed" the signal forward. And it was the first electrically-controlled switch.

**How a relay works:**

A weak current flows through a coil of wire wrapped around an iron core

The coil becomes an **electromagnet**

The electromagnet attracts a metal arm (the **armature**)

The armature moves, closing (or opening) a separate set of electrical contacts

Those contacts switch a completely separate circuit — which can carry much more current

The key insight: **the control circuit and the switched circuit are electrically isolated**. A tiny signal (5mA) can switch a circuit carrying amperes. The relay is an electrical amplifier of control.

---

### Binary Is the Natural Language of Switches

A switch has exactly two stable states:

• **Closed** (contacts touching) — current flows — call this **1**

• **Open** (contacts separated) — no current flows — call this **0**

There is no "half-closed" switch. There is no "37% closed." A switch is binary by nature.

This is why digital computing uses binary — not because binary is mathematically elegant (though it is), but because **the physical devices we use to build computers have exactly two stable states**. We could have built computers around three-state devices (ternary logic), and some researchers have tried. But binary won because every switch ever made — mechanical, relay, vacuum tube, transistor — naturally has two states.

The connection from Module 0.3 completes here: Jacquard's card had hole (1) or no hole (0). Hollerith's card had hole (electrical contact) or no hole (blocked). Relay contacts are closed (1) or open (0). Transistors are on (1) or off (0). The binary digit — the bit — is the universal language of switching.

### The Telegraph's Hidden Contribution

The telegraph didn't just send messages. It established the entire vocabulary of digital communication:

| Telegraph concept | Modern computing equivalent | |---|---| | Dot / dash pulse | Bit (0 / 1) | | Start / stop bits | Framing in serial communication | | Relay amplification | Signal regeneration / buffering | | Code (Morse) | Encoding (ASCII, Unicode) | | Transmission line | Network cable / optical fibre | | Repeater station | Network router / amplifier |

Morse code is, literally, a binary encoding: each letter is a sequence of short pulses (dots = 0) and long pulses (dashes = 1). Telegraph operators were the first data encoders.

### Relay Characteristics

Understanding what a relay can and cannot do sets up the entire story of why vacuum tubes and transistors were needed.

**What relays do well:**

• Electrically isolate control from output (safety, noise immunity)

• Switch high currents with tiny control signals

• Physically robust — relay contacts can carry 10A+

• Visible and audible (click!) — easy to debug

**What relays do poorly:**

• **Speed**: A relay's armature is a mechanical part. It takes 5–50 milliseconds to move. That limits switching to ~100–1000 times per second.

• **Size**: A relay is centimetres long — you can't pack many of them together

• **Power**: The electromagnet consumes power continuously while held closed

• **Wear**: Mechanical contacts eventually erode — relays have a finite lifespan

The first electromechanical computers — Zuse's Z3 (1941), Harvard Mark I (1944) — used relays. The Z3 had 2,600 relays and could do about 3 additions per second. ENIAC (1945), with vacuum tubes, could do 5,000 additions per second. Same logic, 1,000× faster.

### The Relay as a Binary Memory

Here's something surprising: a relay can remember a state. Consider this circuit:

Press button A → relay closes → contacts latch (hold) closed

Release button A → relay stays closed (the latch current flows through its own contacts)

Only pressing a separate "reset" button opens the circuit

This is called **self-latching** — a relay that holds its own state. It's the first mechanical implementation of a **flip-flop**: a device with two stable states that can be set and reset. This is the fundamental memory cell. Every RAM bit in every computer is a more sophisticated version of this circuit.

### From Relay to Every Switch That Followed

The relay established the template that every subsequent switching technology follows:

```
Control input  →  [switching device]  →  Output circuit

Relay:         Control coil current   →  Armature movement    →  Contact closure
Vacuum tube:   Grid voltage           →  Electron stream      →  Plate current
Transistor:    Base/gate current      →  Semiconductor state  →  Collector/drain current
```

Same structure. Shrinking size. Increasing speed. Decreasin

### Illustrations

**Figure 29 • FIG 1.2-A**

**The Relay — Electricity Controlling Electricity**

*Illustration: FIG 1.2-A | The Relay — Electricity Controlling Electricity [SVG diagram — see module-1-2-switches.html]*

A telegraph relay in cross-section. Left: the 5V control circuit contains the electromagnetic coil. When current flows through it, the coil becomes a magnet. Centre: the iron armature is pulled down against the spring tension, closing the gap between the two contact points. Right: this closure completes the 48V output circuit, powering the load. The relay is electrically isolated — the control and output circuits never touch. A 5mA control signal switches a 2A output: this is both amplification and binary switching.

**Figure 30 • FIG 1.2-B**

**Relay States — Deenergised (0) vs Energised (1)**

*Illustration: FIG 1.2-B | Relay States — Deenergised (0) vs Energised (1) [SVG diagram — see module-1-2-switches.html]*

The two states of a normally-open relay. Deenergised (left): no current in the coil — no magnetic field — spring holds the armature away from the fixed contact — gap remains open — output circuit broken — logic 0. Energised (right): current in the coil creates a magnetic field — field pulls the iron armature down against spring tension — moving contact touches fixed contact — output circuit completes — logic 1. This is the fundamental binary switching action that relay computers of the 1940s scaled to thousands of units.

**Figure 31 • FIG 1.2-C**

### Telegraph Relay Station — Signal Regeneration, 1840s

*Illustration: FIG 1.2-C | Telegraph Relay Station — Signal Regeneration, 1840s [SVG diagram — see module-1-2-switches.html]*

Telegraph relay stations (1840s) were the first practical application of the relay as a computing device. A signal sent from New York arrives in Philadelphia as a weak, attenuated pulse. The relay station uses this weak signal to trigger a relay, which uses local battery power to regenerate a full-strength signal for the next leg to Washington. Claude Shannon's 1937 MIT thesis showed that this regeneration logic — relay networks making decisions — is mathematically equivalent to Boolean algebra. The telegraph offices had been running digital circuits for 90 years without knowing it.

**Figure 32 • FIG 1.2-D**

### The Switch as a Binary Device — Every Switch in History

*Illustration: FIG 1.2-D | The Switch as a Binary Device — Every Switch in History [SVG diagram — see module-1-2-switches.html]*

The switch abstraction across six generations. Manual toggle (1800s), electromagnetic relay (1835), vacuum tube triode (1906), bipolar transistor (1947), CMOS logic gate (1963), 5nm FinFET (2011). Each is controlled by a different physical mechanism — mechanical force, electrical current, electron emission, semiconductor doping, voltage — but every one presents identical Boolean behaviour: exactly two states. This substrate-independence is what makes digital logic timeless.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 1.2 Relay & Switch Simulator: The electrically-controlled switch atom of all computation [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 1.2 Relay Logic and Binary Switching **Type:** Simulation + truth tables + circuit design **Estimated time:** 3040 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Relay Anatomy Using the **Relay Anatomy** tool: 1. At what control current percentage does the relay close? Record the threshold. 2. Once closed, lower the current slowly. At what percentage does it open again? Is this the same threshold? (This difference is called **hysteresis** explain in one sentence why it exists mechanically.) 3. Draw a simple diagram of the relay showing: coil, core,

# Module 1.3 Relay Logic: Building Decisions from Metal

## The Problem

*We can light a bulb with a relay. We can even make a relay remember a state. But can we make a relay compute?*

>

*In 1937, a 21-year-old MIT graduate student named Claude Shannon submitted a master's thesis that answered this question definitively — and in doing so, connected 83 years of Boole's abstract algebra to physical circuits for the first time.*

>

*The thesis was titled: "A Symbolic Analysis of Relay and Switching Circuits."*

>

*Its central claim:* ***any Boolean logical expression can be implemented directly as a relay circuit.***

>

*Every logic gate in every computer ever built traces its intellectual ancestry to that thesis.*

## The Story

### Claude Shannon's Thesis — 1937

George Boole had published *An Investigation of the Laws of Thought* in 1854. It described a complete algebra of TRUE/FALSE values with operations AND, OR, NOT. Mathematically elegant, but abstract — nobody had found a physical application for it.

Shannon was repairing a telephone switching network when he noticed that relay circuits — already in widespread use for telephone exchanges — behaved exactly like Boolean expressions:

• A closed contact = TRUE (1)

• An open contact = FALSE (0)

• Contacts in series = AND

• Contacts in parallel = OR

• A normally-closed contact = NOT

He proved that you could take any Boolean expression, however complex, and translate it directly into a relay circuit. And conversely, you could take any relay circuit and describe it as a Boolean expression.

This equivalence meant: **if you can express a computation as logic, you can build it from relays.**

His thesis examiner called it "possibly the most important master's thesis of the 20th century." He was 21.

## Boolean Algebra: The Rules

Boolean algebra operates on values that are either 0 (FALSE) or 1 (TRUE). The rules:

**AND (multiplication):**

```
0 AND 0 = 0
0 AND 1 = 0
1 AND 0 = 0
1 AND 1 = 1
```

AND is true only when ALL inputs are true.

**OR (addition):**

```
0 OR 0 = 0
0 OR 1 = 1
1 OR 0 = 1
1 OR 1 = 1
```

OR is true when ANY input is true.

**NOT (complement):**

```
NOT 0 = 1
NOT 1 = 0
```

NOT inverts the value.

**Key identities (you'll use these constantly):**

```
A AND 0 = 0          (anything ANDed with false is false)
A AND 1 = A          (anything ANDed with true is itself)
A OR 0 = A           (anything ORed with false is itself)
A OR 1 = 1           (anything ORed with true is true)
A AND A = A          (idempotent)
A AND (NOT A) = 0    (contradiction)
A OR (NOT A) = 1     (tautology)
NOT (NOT A) = A      (double negation)
```

**De Morgan's Laws** (extremely important):

```
NOT (A AND B) = (NOT A) OR (NOT B)
NOT (A OR B)  = (NOT A) AND (NOT B)
```

De Morgan's Laws let you convert between AND-circuits and OR-circuits with inversions. This is how NAND-only and NOR-only chip designs work.

## Building a Half Adder

The half adder is the simplest arithmetic circuit. It adds two 1-bit numbers and produces a 2-bit result: a **Sum** bit and a **Carry** bit.

Truth table:

```
A  B  | Sum  Carry
0  0  |  0     0
```

```
  0  1  |  1     0
  1  0  |  1     0
  1  1  |  0     1
```

Look carefully:

• **Carry = A AND B** — you only get a carry when both inputs are 1

• **Sum = A XOR B** — the sum is 1 when exactly one input is 1 (not both, not neither)

XOR (exclusive OR) is the key operation. How do you build XOR from AND, OR, NOT?

```
A XOR B = (A OR B) AND NOT(A AND B)
        = (A OR B) AND (NOT A OR NOT B)  ← De Morgan applied
```

In relay terms:

One relay per input (A, B)

OR path: contacts in parallel → fire if either A or B is high

NOT-AND path: normally-closed contacts in series → open only when BOTH are high (NAND)

AND the OR-result with the NAND-result → XOR

This is four relays. In 1940s relay computers, every XOR took four physical relay devices. A 16-bit adder needed hundreds of relays just for the sum logic. ENIAC's equivalent in vacuum tubes was already faster by orders of magnitude.

---

### Shannon's Minimization

Shannon also showed how to **minimize** logic circuits — reduce the number of gates needed to implement a function. This is the origin of **Karnaugh maps** and **Boolean minimization** algorithms used in every chip design tool today.

The principle: two relay networks that always produce the same output for every input are equivalent, even if they look different. The simpler one is preferable. Finding the simplest equivalent circuit is an optimization problem Shannon formalized.

Example: `(A AND B) OR (A AND C)` can be simplified to `A AND (B OR C)` — one fewer AND operation, saving one relay.

This kind of algebraic simplification is what synthesis tools (like those that compile Verilog/VHDL to chip layouts) do billions of times per chip design.

---

### The Zuse Z3 — First Programmable Computer (1941)

Konrad Zuse, working in his parents' living room in Berlin, built the **Z3** in 1941 — the world's first fully operational programmable computer. It used **2,600 telephone relays** and could execute programs stored on perforated film.

Key specs:

• Clock speed: ~5.3 Hz (5.3 relay switching cycles per second)

• Word length: 22-bit floating point

• Memory: 64 words

• Operations: Add, subtract, multiply, divide, square root

• Program input: punched 35mm film strip

The Z3 could multiply two 22-bit floating-point numbers in about 3 seconds. Impressive for relays — but the machine had fundamental speed limits. Each instruction required multiple relay switching cycles, each cycle took ~190ms.

The Z3 was destroyed in a 1943 Allied bombing raid. A working replica exists in the Deutsches Museum, Munich.

### The Mark I — Electromechanical Computing at Scale

Harvard's IBM Automatic Sequence Controlled Calculator (**Mark I**), completed in 1944, was the first large-scale electromechanical computer in the US. It had:

• 765,000 components

• ~3,000 electromechanical relays

• 800 km of wire

• 15 metres long, 2.4 metres tall

• Weight: 4,500 kg

It could do

### Illustrations

**Figure 33 • FIG 1.3-A**

**AND, OR, NOT — Three Relay Circuits, Three Logical Operations**

*Illustration: FIG 1.3-A | AND, OR, NOT — Three Relay Circuits, Three Logical Operations [SVG diagram — see module-1-3-relay-logic.html]*

The three fundamental relay logic circuits. AND gate (left): two relays in series — both A and B must be energised for current to reach the output. Truth table shows output is 1 only when A=1 AND B=1. OR gate (centre): two relays in parallel — either path can complete the circuit. Output is 1 when A=1 OR B=1 (or both). NOT gate (right): a normally-closed relay — the contact is closed by default and opens when the coil is energised, inverting the input. These three operations are the complete basis of all digital logic.

**Figure 34 • FIG 1.3-B**

### Relay Half Adder — Adding Two Bits with Metal and Wire

*Illustration: FIG 1.3-B | Relay Half Adder — Adding Two Bits with Metal and Wire [SVG diagram — see module-1-3-relay-logic.html]*

A relay half adder built from AND, OR, and NOT relay circuits. SUM = A XOR B (computed as (A OR B) AND NOT(A AND B)). CARRY = A AND B. The critical insight from the truth table: 1+1 in binary produces SUM=0 and CARRY=1 — the carry represents the overflow into the next column. Chaining two half adders with an OR gate for carries produces a full adder, which is the foundation of every arithmetic unit ever built.

**Figure 35 • FIG 1.3-C**

### Shannon 1937 — Circuits Are Boolean Algebra

*Illustration: FIG 1.3-C | Shannon 1937 — Circuits Are Boolean Algebra [SVG diagram — see module-1-3-relay-logic.html]*

Claude Shannon's 1937 master's thesis established that relay switching circuits implement Boolean algebra. Series relay contacts correspond to the AND operation. Parallel paths correspond to OR. Normally-closed contacts correspond to NOT. This means any Boolean expression can be directly translated into a relay circuit, and any relay circuit can be analysed and minimised using Boolean algebra. This thesis — submitted when Shannon was 21 — is arguably the most important master's thesis ever written, because it gave engineers a mathematical language to design digital systems.

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 1.3 Relay Logic Builder: Shannon's insight: Boolean algebra = relay circuit topology [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 1.3 Relay Logic and Boolean Algebra **Type:** Simulation circuit building + Boolean algebra + truth tables **Estimated time:** 3545 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Boolean Algebra Manipulation Simplify each expression using Boolean algebra identities. Show every step. 1. `A AND (A OR B)` *(Hint: Absorption law A AND (A OR B) = ?)* 2. `(A AND B) OR (A AND NOT B)` *(Factor out A)* 3. `NOT (NOT A AND NOT B)` *(Apply De Morgan's Law)* 4. `(A OR B) AND (A OR NOT B)` *(Distribute, then simplify)* 5. Verify De Morgan

# Module 1.4 Vacuum Tubes: Faster Switching

## The Problem

*Relay computers work. The logic is sound. The Z3 runs. The Mark I calculates.*

>

*But a relay takes 5–50 milliseconds to switch — 100 to 1,000 operations per second at best. The US military needed firing tables for hundreds of artillery configurations, each requiring thousands of trajectory calculations. At relay speeds, one complete firing table took days.*

>

*The question was brutally practical:* ***can we switch faster without moving any metal?***

>

*The answer already existed in every radio receiver in the world — a glass tube with no moving parts that could switch a million times per second. The vacuum tube.*

## The Story

### The Edison Effect — 1883

Thomas Edison, experimenting with his lightbulb in 1883, noticed something odd: a small current flowed between the hot filament and a metal plate sealed inside the bulb even though they weren't touching. He had no explanation, patented it as a curiosity, and moved on. It became known as the "Edison effect."

The explanation: **electrons were boiling off the hot filament** and drifting through the vacuum to the metal plate. This is **thermionic emission** — heat gives surface electrons enough energy to escape the metal.

### Fleming's Diode — 1904

British physicist John Ambrose Fleming connected the dots in 1904. If the plate is positive, emitted electrons (negative charge) are attracted to it — current flows. If the plate is negative, electrons are repelled — no current flows.

This one-way current flow made Fleming's **diode** perfect for detecting radio waves (rectifying the alternating signal into a detectable direct current). But a diode is not an amplifier and not a switch — it has no control input.

### De Forest's Triode — 1906

Lee de Forest added a third element in 1906: a wire mesh between cathode and plate called the **grid**. This created the **triode** — the device that changed everything.

**How a triode works:**

```
CATHODE (heated) → emits electrons → [GRID controls flow] → PLATE collects current
```

• Apply **negative voltage to grid**: repels electrons → almost no plate current → tube is **OFF**

• Apply **zero or positive voltage to grid**: electrons flow freely → large plate current → tube is **ON**

• The grid transition between these states takes **microseconds**, not milliseconds

A small voltage change on the grid produces a large current change at the plate — this is **amplification**. The ratio of plate voltage swing to grid voltage swing is the **amplification factor (μ)**, typically 20–100.

As a switch: the tube is either saturated (full current, ON=1) or cut off (no current, OFF=0). No moving parts. No mechanical wear. Switching time: ~1 microsecond.

### ENIAC — 18,000 Tubes, 1945

The Electronic Numerical Integrator and Computer (ENIAC) was the first general-purpose electronic computer, completed in 1945 at the University of Pennsylvania.

| Specification | Value | |---|---| | Vacuum tubes | 17,468 | | Resistors | 70,000 | | Weight | 27,000 kg | | Power | 150 kW | | Floor space | 167 m² | | Speed | ~5,000 additions/second | | Cost (1945) | ~$400,000 |

ENIAC was **1,000× faster than the Mark I**. Calculations that took the relay-based Mark I days took ENIAC minutes.

**The reliability problem**: With 17,468 tubes, one failed every few hours on average. A team of operators patrolled ENIAC continuously, replacing burned-out tubes. Running a long computation without failure required luck. This fragility was the tube's fundamental limitation — and the motivation for the transistor.

---

## The Concept: Thermionic Emission and Electron Control

**Why do electrons leave the cathode?**

In a metal, outermost electrons are only loosely bound. At room temperature, they don't have enough energy to escape. But heating the metal to ~1000°C gives electrons enough thermal energy to overcome the surface barrier — they literally boil off. This is **thermionic emission**.

**Why does the grid control the current?**

The grid is a mesh of fine wire sitting in the electron stream between cathode and plate. It creates an electric field:

• Negative grid → field points away from plate → electrons decelerate → most turn back → low current

• Positive grid → field points toward plate → electrons accelerate → all reach plate → high current

The grid needs only millivolts to milliamps to exert this control — while the plate circuit can handle volts and amps. **Small control energy, large output energy: this is amplification.**

**Why vacuum?**

Electrons scatter off air molecules. At atmospheric pressure, an electron travels only nanometres before colliding. A vacuum of 10■■ torr or better lets electrons travel centimetres without collision — essential for cathode-to-plate transit.

---

## Tube vs Relay: The Scorecard

| Property | Relay | Vacuum Tube | Winner | |----------|-------|-------------|--------| | Switching speed | 5–50 ms | 0.1–1 µs | Tube (×10,000) | | Moving parts | Yes | None | Tube | | Power per switch | ~0.1 W | 1–5 W | Relay | | Size | ~5 cm | ~3–8 cm | Roughly equal | | Lifespan | Millions of cycles | 1,000–10,000 hrs | Relay | | Fragility | Robust | Glass — fragile | Relay | | Heat generated | Low | Very high | Relay | | Control isolation | Complete | Partial | Relay |

The tube wins on speed decisively. It loses on power, heat, and reliability. This is precisely why it was replaced by the transistor — which kept the speed while fixing the other problems.

## The Williams Tube — First Electronic Memory, 1946

A key unsolved problem in early computers was memory. Paper tape and punch cards were slow. The Williams tube (1946, Manchester) used a cathode ray tube — normally a display device — as memory:

• Write a 1: fire the electron beam at a spot → leaves a positive charge on the phosphor

• Write a 0: don't fire (or fire differently) → no charge

• Read: scan the spot with a low-energy beam → the charge either absorbs or reflects the scan → detect the difference

### Illustrations

**Figure 36 • FIG 1.4-A**

**The Triode Vacuum Tube — Electrons Controlled by a Grid**

*Illustration: FIG 1.4-A | The Triode Vacuum Tube — Electrons Controlled by a Grid [SVG diagram — see module-1-4-vacuum-tubes.html]*

Triode vacuum tube anatomy. The heated cathode emits a cloud of electrons by thermionic emission. The control grid, positioned between cathode and anode, acts as an electrostatic gate: a negative grid voltage repels electrons, blocking current (tube OFF = logic 0). A zero or positive grid voltage allows electrons to stream through to the anode (tube ON = logic 1). The switching action is purely electronic — no moving parts — making the triode approximately 100× faster than an electromagnetic relay. ENIAC (1945) used 18,000 of these to reach 5,000 operations per second.

**Figure 37 • FIG 1.4-B**

### ENIAC — 18,000 Vacuum Tubes, 1945

*Illustration: FIG 1.4-B | ENIAC — 18,000 Vacuum Tubes, 1945 [SVG diagram — see module-1-4-vacuum-tubes.html]*

ENIAC (Electronic Numerical Integrator and Computer, 1945) — the first general-purpose electronic computer. Ten panels of vacuum tube circuitry filling a 30-metre room. 17,468 tubes, 70,000 resistors, 10,000 capacitors, consuming 150kW of power. One tube failed on average every two days, requiring a dedicated maintenance team. Despite this, ENIAC computed 5,000 additions per second — thousands of times faster than a human computer and 100× faster than a relay machine. It was programmed by physically rewiring patch cables between panels.

**Figure 38 • FIG 1.4-C**

### Relay vs Vacuum Tube — Five Dimensions of Comparison

*Illustration: FIG 1.4-C | Relay vs Vacuum Tube — Five Dimensions of Comparison [SVG diagram — see module-1-4-vacuum-tubes.html]*

Five-dimension comparison of relays vs vacuum tubes. The tube wins on switching speed (1000×) and electromagnetic noise immunity (no arcing contacts). The relay wins on reliability (filaments burn out) and power consumption (heaters draw significant current). Physical size is comparable — both are centimetre-scale. The transistor (Module 1.5) wins every dimension, which is why it replaced both within a decade of its invention.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 1.4 Vacuum Tube Simulator: From Edison's curiosity to ENIAC's 17,468 tubes [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 1.4 Vacuum Tubes and Switching Speed **Type:** Calculation + simulation + written analysis **Estimated time:** 3040 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A ENIAC Reliability Math ENIAC had 17,468 vacuum tubes. Each tube had a mean lifetime of approximately 2,500 hours under normal operating conditions. 1. If all tubes were installed at the same time and had exactly equal lifetimes, how many hours until the first tube fails? (This is the ideal case in reality, lifetimes vary.) 2. In reality, lifetimes follow a statistical distribution. If yo

# Module 1.5 The Transistor: Revolution in a Grain of Sand

## The Problem

*ENIAC works. It's fast. But 17,468 tubes, 150kW of power, and a tube failure every few hours is not a scalable path.*

>

*Bell Labs in 1945 had a very practical problem: their telephone network used millions of vacuum tube amplifiers in repeater stations along long-distance lines. Each station needed maintenance, cooling, and regular tube replacement. The cost and unreliability were becoming unmanageable.*

>

*Bell Labs set up a solid-state physics group with one mission: find a replacement for the vacuum tube. Something smaller, cooler, more reliable. Something that didn't need a glowing hot filament to work.*

>

*On December 16, 1947, John Bardeen and Walter Brattain, supervised by William Shockley, made the first working transistor. It was the size of a fingernail and made of germanium. It amplified a signal with no moving parts, no vacuum, and no heater.*

>

*It changed everything.*

## The Story

### What is a Semiconductor?

Recall from Module 1.1: conductors conduct freely, insulators don't conduct. Semiconductors sit in between — and their conductivity can be controlled.

**Silicon** (the element, Si) is the key material. Pure silicon at room temperature is a poor conductor: its four valence electrons form stable covalent bonds with neighbouring atoms, leaving almost no free electrons to carry current.

**Doping** changes this. Adding tiny amounts of impurity atoms alters silicon's conductivity dramatically:

**N-type silicon** (doped with phosphorus or arsenic):

• Phosphorus has 5 valence electrons; silicon needs only 4 to bond

• The extra electron is loosely bound and free to move

• Result: extra free electrons → current carried by negative charges → **N**-type

**P-type silicon** (doped with boron or aluminium):

• Boron has only 3 valence electrons; there's a "hole" where an electron is missing

• Neighbouring electrons can jump into the hole, making it appear to move

• Result: current carried by moving holes (positive charges) → **P**-type

### The P-N Junction: The Diode

Put N-type and P-type silicon in contact. At the junction:

• Free electrons from the N-side diffuse across and fill holes on the P-side

• This creates a **depletion region** — a zone with no free carriers, acting as an insulator

• An electric field builds across the junction opposing further diffusion

**Forward bias** (apply + to P-side, – to N-side): the electric field is reduced → carriers flow → current passes. **Reverse bias** (apply + to N-side): the field increases → no current flows.

This one-way current is a **diode** — identical function to Fleming's 1904 vacuum diode, but solid-state, small, and durable.

### The Bipolar Junction Transistor (BJT)

Add a second junction and you get a transistor. The **NPN transistor** has three layers: N-P-N, with three terminals:

• **Emitter (E)**: the N-region that emits carriers

• **Base (B)**: the thin middle P-region — the control terminal

• **Collector (C)**: the N-region that collects carriers

**How it works:**

Apply a small forward-bias voltage to the base-emitter junction ($V_{BE} \approx 0.6V$ for silicon)

This injects electrons from the emitter into the thin base region

Most electrons don't recombine in the thin base — they diffuse across into the strong reverse-biased collector field

A small base current (μA range) controls a large collector current (mA range)

**Current gain ($\beta$ or hFE)**: $\beta = I_C / I_B$, typically 50–500.

As a switch:

• Base voltage = 0 → transistor OFF (collector current ≈ 0)

• Base voltage = 0.7V → transistor ON (collector current = $V_{CC} / R_C$, limited by load)

• No vacuum needed. No heater. No moving parts.

• Switching time: ~1 nanosecond for early transistors, picoseconds for modern ones.

### The MOSFET: The Transistor That Runs the World

The **Metal-Oxide-Semiconductor Field-Effect Transistor (MOSFET)** is the dominant transistor in digital chips today — every CPU, GPU, and memory chip is built from billions of MOSFETs.

Three terminals: **Gate (G)**, **Source (S)**, **Drain (D)**.

An N-channel enhancement-mode MOSFET:

A thin layer of silicon dioxide (SiO■ — glass!) insulates the gate from the silicon

Applying voltage to the gate creates an electric field through the oxide

This field attracts electrons to the surface of the P-type body, creating an **inversion channel** — a thin conducting path of N-type carriers between source and drain

Current flows from drain to source through this channel

Remove gate voltage → channel disappears → no current

**Key advantage over BJT for digital logic:**

• Gate is insulated → draws essentially **zero current** (only capacitive charging)

• This means millions of MOSFETs can switch simultaneously without excessive current draw

• Enables the extreme integration density of modern chips

---

### The First Transistor Computer — 1953

The University of Manchester built the first transistor computer in 1953, using 92 point-contact transistors. By 1955, Bell Labs' TRADIC was the first transistor computer to fly in an aircraft.

IBM's first fully transistorized mainframe, the 7090 (1959), replaced the vacuum-tube IBM 709:

• 50× faster

• Smaller

• Used far less power

• Far more reliable

By 1960, vacuum tube computers were obsolete for general computing.

---

### Why Transistors Beat Tubes

| Property | Vacuum Tube | BJT Transistor | MOSFET | |----------|-------------|----------------|--------| | Size | ~3 cm | ~1 cm (discrete) | nm scale | | Power per switch | 1–5 W | 0.01–0.1 W | Picowatts | | Switching speed | ~1 µs | ~1 ns | <0.1 ns | | Needs heater | Yes (5–12V) | No | No | | Fragility | Glass — fragile | Robust | Robust | | Lifespan | 1,000–10,000 hrs | 10■ cycles+ | $10^{12}$+ cycles | | Control current | Small (mA) | Small (µA) | Near zero | | Works at room temp | No (needs heat) | Yes | Yes |

The transistor didn't just replace the tube — it enabled a density of switching that tubes could never approach. You simply cannot pack 10 billio

### Illustrations

**Figure 39 • FIG 1.5-A**

### The NPN Transistor as a Switch — Base Controls Collector-Emitter

*Illustration: FIG 1.5-A | The NPN Transistor as a Switch — Base Controls Collector-Emitter [SVG diagram — see module-1-5-transistor.html]*

NPN bipolar transistor used as a digital switch. The base terminal (B) receives a small control current through a resistor. When base voltage is low (0V), no base current flows, the transistor is in cutoff, and the collector-emitter path is an open circuit — load off, output 0. When base voltage is high (5V), a small base current (microamps) flows, forward-biasing the base-emitter junction, which collapses the depletion region and allows a large collector current (milliamps) to flow through the load — output 1. The transistor amplifies current while acting as a perfect binary switch, with no v

**Figure 40 • FIG 1.5-B**

### Relay → Vacuum Tube → Transistor — The Switching Technology Progression

*Illustration: FIG 1.5-B | Relay → Vacuum Tube → Transistor — The Switching Technology Progressio [SVG diagram — see module-1-5-transistor.html]*

Three generations of electronic switching technology. The relay (1835) uses electromagnetism to move a mechanical contact — slow, large, reliable. The vacuum tube (1906) eliminates mechanical movement using electron emission in vacuum — 100× faster but fragile, power-hungry, bulky. The transistor (1947) uses semiconductor junctions — no vacuum, no heater, no moving parts, inherently scalable. Each generation transitions from a physical phenomenon (mechanical, thermionic, quantum-mechanical tunnelling) to a purely solid-state operation. The transistor's scalability is the critical property that

**Figure 41 • FIG 1.5-C**

### MOSFET Cross-Section — Gate Voltage Controls the Channel

*Illustration: FIG 1.5-C | MOSFET Cross-Section — Gate Voltage Controls the Channel [SVG diagram — see module-1-5-transistor.html]*

MOSFET cross-section — the transistor architecture that powers all modern chips. When gate voltage exceeds the threshold (Vth), an inversion layer of electrons forms in the P-type silicon beneath the ultra-thin gate oxide, creating a conducting channel between source and drain. Below Vth, no channel exists: the transistor is off. The gate oxide thickness (tox) is the critical dimension scaled with each process node, now approaching just a few atomic layers — at which point quantum tunnelling causes leakage current even when the gate is off.

**Figure 42 • FIG 1.5-D**

**From Discrete Transistors to Integrated Circuits — Bell Labs 1947 to Apple M3 2023**

*Illustration: FIG 1.5-D | From Discrete Transistors to Integrated Circuits — Bell Labs 1947 to A [SVG diagram — see module-1-5-transistor.html]*

The integration revolution from Bell Labs to modern VLSI. Jack Kilby's 1958 integrated circuit (Nobel Prize 2000) was the pivotal insight: instead of wiring together discrete components, etch all of them onto a single piece of semiconductor. This made manufacturing complexity essentially free — a chip with one transistor and one with a billion cost similar amounts to fabricate. The Apple M3 (2023) contains 25 billion transistors on a die smaller than a postage stamp, all operating within a 22W thermal envelope.

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 1.5 Transistor Simulator: P-N Junction BJT Switch MOSFET from atoms to computers [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 1.5 The Transistor **Type:** Calculation + simulation + written analysis **Estimated time:** 3545 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Power Calculations 1. A vacuum tube consumes 2.5W when switched ON. A BJT transistor consumes 50mW when ON. How many times less power does the transistor use? 2. ENIAC had 17,468 tubes at ~2.5W each. A transistor equivalent with the same number of switches at 50mW each would consume how much power? Compare to ENIAC's 150kW. 3. A modern MOSFET in a CPU switches at a power of 1 picowatt (10 W) per switching

# Module 1.6 From Transistors to Chips: Moore's Law

## The Problem

*The transistor was invented in 1947. It was a breakthrough. But each transistor was still a separate component — you bought them individually, wired them together by hand, soldered connections one at a time.*

>

*A computer with 1,000 transistors meant 1,000 separate parts, thousands of hand-soldered joints, and a reliability nightmare (every solder joint could fail). Building a computer with a million transistors by hand was practically impossible.*

>

*The question:* ***can we build the entire circuit — transistors, resistors, connections — all at once on a single piece of silicon?***

The answer came from two people independently in 1958–1959, and it launched the modern world.

## The Story

### The Integrated Circuit — 1958

**Jack Kilby** at Texas Instruments built the first working integrated circuit in 1958. Using germanium, he placed a transistor, resistors, and capacitors on a single chip and connected them with fine gold wires. It was messy, but it worked.

**Robert Noyce** at Fairchild Semiconductor independently invented a cleaner approach in 1959: use deposited aluminium traces on top of silicon dioxide insulation to connect components without physical wires. This **planar process** is the foundation of every chip made since.

Both Kilby and Noyce are credited as co-inventors of the IC. Kilby won the Nobel Prize in Physics in 2000 (Noyce had died in 1990).

**Why the IC was revolutionary:**

• All components and connections fabricated simultaneously in one process

• Reliability improved: no hand-soldered joints to fail

• Size: the entire circuit is photolithographically patterned — smaller than any hand assembly

• Cost: manufacturing cost scales with chip area, not with number of components

• The same photomask that makes 1 transistor can make 1 million in the same process step

### Moore's Law — 1965

Gordon Moore, co-founder of Fairchild and later Intel, observed in 1965 that the number of transistors on an integrated circuit had been doubling roughly every year since the IC's invention. He predicted this would continue for at least 10 more years.

In 1975 he revised the prediction: doubling every **two years**.

This became **Moore's Law**: *the number of transistors on a chip doubles approximately every two years.*

| Year | Chip | Transistors | |------|------|-------------| | 1971 | Intel 4004 | 2,300 | | 1974 | Intel 8080 | 4,500 | | 1978 | Intel 8086 | 29,000 | | 1982 | Intel 286 | 134,000 | | 1985 | Intel 386 | 275,000 | | 1989 | Intel 486 | 1,200,000 | | 1993 | Pentium | 3,100,000 | | 2000 | Pentium 4 | 42,000,000 | | 2006 | Core 2 Duo | 291,000,000 | | 2012 | Ivy Bridge | 1,400,000,000 | | 2020 | Apple M1 | 16,000,000,000 | | 2023 | Apple M3 | 25,000,000,000 | | 2024 | Apple M4 | 28,000,000,000 |

From 2,300 to 28 billion in 53 years. That's a factor of **12 million**.

### How Chips Are Made: Photolithography

The magic of ICs is **photolithography** — printing circuit patterns onto silicon using light.

**Start with a silicon wafer** — a disc of highly pure silicon, 300mm in diameter

**Grow oxide**: heat the wafer to grow a thin layer of SiO■ on the surface

**Apply photoresist**: coat the wafer with a light-sensitive chemical

**Expose through a mask**: shine UV light through a photomask (like a film negative) — where light hits, photoresist becomes soluble

**Develop**: wash away the exposed photoresist — the pattern remains

**Etch or dope**: etch away the exposed oxide, or implant dopant atoms into the exposed silicon

**Repeat**: each chip layer requires multiple lithography-etch cycles (modern chips have 100+ layers)

**Deposit metal**: aluminium or copper traces connect transistors

The minimum feature size — the width of the smallest line that can be printed — is the **process node** (or technology node). This is the number you see: 90nm, 45nm, 14nm, 7nm, 5nm, 3nm.

**Shrinking the node** by 2× means:

• Each transistor occupies 4× less area (2D shrink)

• 4× more transistors fit on the same chip area

• Transistors are closer together → signals travel shorter distances → faster switching

• Smaller transistors have less capacitance → less energy per switch → lower power

This is the physical mechanism behind Moore's Law: **photolithography precision doubling every ~18 months**.

### Why Moore's Law Is Slowing

By the early 2010s, shrinking transistors was running into fundamental physical limits:

**The quantum tunnelling problem**: When a transistor gate is only a few atoms thick, electrons can **quantum tunnel** through the gate oxide even when the transistor is supposed to be OFF. This causes leakage current — the transistor can't fully shut off, wasting power and causing errors. At 2nm, the gate is roughly 10 atoms wide.

**Heat density**: Transistors have always generated heat. Smaller transistors generate less heat each, but you can pack so many more of them that total heat density has hit a ceiling. Most CPUs can't exceed ~100W/cm² without exotic cooling.

**Economic limits**: Each new fab (semiconductor fabrication facility) costs $10–20 billion to build. The number of companies that can afford leading-edge fabs has shrunk to: TSMC, Samsung, Intel. Each process node requires completely new equipment.

**The industry response:**

• **3D stacking**: stack multiple die layers (memory on top of logic) — going vertical instead of lateral

• **Heterogeneous integration**: combine specialized chips (CPU + GPU + memory) in one package

• **New transistor designs**: FinFET (2011), Gate-All-Around (2022) — new 3D transistor geometries instead of flat planar devices

• **Specialized hardware**: GPUs for graphics, TPUs for AI — don't need a general CPU for everything

### What Moore's Law Actually Enabled

Moore's Law isn't just an interesting observation. It's the engine behind every major computing development since 1965:

| What changed | Because of Moore's Law | |----------|----------| |

### Illustrations

**Figure 43 • FIG 1.6-A**

**Moore's Law — Transistor Count Doubles Every Two Years, 1971–2024**

*Illustration: FIG 1.6-A | Moore's Law — Transistor Count Doubles Every Two Years, 1971–2024 [SVG diagram — see module-1-6-moores-law.html]*

Moore's Law chart on a logarithmic scale, 1971–2023. The dashed line shows the ideal doubling-every-2-years trajectory. Real chip counts follow it with remarkable fidelity across 7 orders of magnitude — from 2,300 transistors (Intel 4004, 1971) to 80 billion (NVIDIA H100, 2023). Three eras are marked: micron-scale microprocessors (1970s–1990s), the submicron transition (2000s), and the sub-10nm FinFET/GAA era (2010s–present). The logarithmic scale makes linear growth on this chart correspond to exponential growth in reality.

**Figure 44 • FIG 1.6-B**

**Physical Limits — When Does Transistor Scaling End?**

*Illustration: FIG 1.6-B | Physical Limits — When Does Transistor Scaling End? [SVG diagram — see module-1-6-moores-law.html]*

The physical limits of transistor scaling. The top bar shows relative size from 5nm transistors to a human hair at 70,000nm — the scale compression is extraordinary. Three fundamental limits are approaching: quantum tunnelling (electrons pass through the gate even when off, causing leakage and errors), heat density (more transistors per mm² generates more heat, currently around 100W/cm² for high-performance chips), and the silicon atomic floor (a silicon atom is 0.28nm wide — you cannot build a transistor smaller than the atoms it's made of). The industry response is architectural: 3D chip sta

**Figure 45 • FIG 1.6-C**

### Consequences of Moore's Law — Why 53 Years of Doubling Changed Everything

*Illustration: FIG 1.6-C | Consequences of Moore's Law — Why 53 Years of Doubling Changed Everyth [SVG diagram — see module-1-6-moores-law.html]*

The consequence of 53 years of exponential transistor growth. Compared against linear improvements in other technologies (cars, aircraft), Moore's Law doubling stands apart. Cost per transistor fell from roughly $300 in 1971 to fractions of a nanodollar today — a 100-billion-fold reduction. This is why a modern smartphone carries more computational power than the entire 1969 Apollo mission, and why AI systems that required rooms full of hardware in 2015 fit on a consumer GPU in 2024. Exponential compounding, sustained long enough, produces qualitatively different civilisations.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 1.6 Moore's Law Simulator: From 2,300 transistors to 28 billion in 53 years [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 1.6 Moore's Law and the Level 1 Capstone **Type:** Calculation + simulation + comprehensive written capstone **Estimated time:** 4560 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Moore's Law Mathematics 1. The Intel 4004 (1971) had 2,300 transistors. Moore's Law predicts doubling every 2 years. How many transistors does the model predict for 2023 (52 years later)? Show: `2,300 2^(52/2)` 2. The Apple M3 (2023) actually has ~25 billion transistors. How does this compare to your prediction? Express the real count as a percentage of the prediction. W

LEVEL 02

# Digital Foundations

Binary, Boolean Algebra, and the Language of Machines

*A transistor switches in nanoseconds. But how does switching fast become thinking?*

## What This Level Is About

Level 1 gave you the atom of computing: a switch with two states. Level 2 shows how those switches, combined by the millions, build up into a machine that can perform any calculation ever conceived.

The path: **two physical states** → **binary digits** → **Boolean algebra** → **logic gates** → **circuits that add** → **circuits that remember** → **a simple processor core.**

Every CPU ever made — from the 4004 to the M4 — is built from exactly the ideas in this level, just repeated billions of times with extraordinary precision.

By the end of Level 2, you will be able to:

• Explain why binary is the only practical number system for electronics

• Convert fluently between binary, decimal, octal, and hexadecimal

• Design any logic circuit from truth table to gate diagram

• Build a working 4-bit adder from first principles

• Explain how a flip-flop stores one bit, and how flip-flops chain into counters

• Describe the structure and operation of a simple ALU

## Modules

| Module | Title | Core Concept | |--------|-------|-------------| | [2.1](./module-2-1-why-binary/lesson.md) | Why

# Module 2.1 Why Binary? The Physics of Two States

## The Problem

*You have a circuit carrying voltage. You want to represent the number 7. The obvious approach: use 7 volts. Store a number as its voltage.*

>

*The problem: real-world circuits are noisy. A "7V" signal might arrive as 6.8V. Is that 6 or 7? A "3V" signal in a noisy environment might read as 2.6V, 3.1V, or 3.4V. You've built a calculator that can't be trusted to read its own numbers.*

>

*The 1940s engineers trying to build reliable computers faced this exact crisis. The solution was counterintuitive:* ***use less information, not more.***

Instead of representing a number as a precise voltage level, represent it as a *region*: anything above 2.5V is a 1. Anything below 0.5V is a 0. A 0.5V–2.5V "no-man's-land" is forbidden — the circuit is designed to push signals to one extreme or the other.

This is binary. Its superiority over analog representation isn't mathematical elegance — it's **noise immunity**.

## The Story

### The Analog Disaster

Early computing attempts in the 1930s and 1940s tried analog computers — machines where voltage levels directly encoded values. They could solve differential equations faster than any human. But they had a fundamental problem: every component added noise.

A resistor adds thermal noise (electrons jiggling from heat). A wire acts as an antenna and picks up electromagnetic interference. Capacitors leak. Amplifiers amplify noise alongside signal. Chain together a hundred components and you've amplified the noise a hundred times.

Analog computers were accurate to perhaps 3–4 decimal places under ideal conditions. That's fine for controlling a gun turret. It's catastrophically wrong for financial calculations or scientific tables where you need 10+ digits of precision.

**Shannon's information theory (1948)** put this on rigorous mathematical footing: you can transmit information reliably over a noisy channel if and only if you have error correction — and the simplest error-correcting code is binary, because there are only two valid values to distinguish between.

### Why Two States Win

Compare three approaches to encoding the digit 5 in electronics:

**Analog (10 voltage levels, 0–9V):**

• Each level is 1V apart

• Noise margin: ±0.5V before misread

• Chain 20 amplifiers: noise accumulates, levels blur together

• Precision: ~3–4 decimal digits maximum

**Decimal (10 discrete levels):**

• Tried! The ENIAC used decimal ring counters — 10 flip-flops per digit

• A decimal digit required 10 transistors

• A binary digit requires 1 transistor

• 10× less efficient in hardware

**Binary (2 levels: 0 and 1):**

• 0V = 0, 5V = 1 (for a 5V logic family)

• Noise margin: ±2V before misread — 4× better than analog

• Every transistor is either saturated (fully on) or cut off (fully off)

• After every gate, the signal is *regenerated* to full swing — noise resets

• Precision: unlimited (just use more bits)

The binary system transforms *analog noise* into a *digital decision*. At each gate boundary, a comparator decides: is this above the threshold? If yes, drive the output to +5V. If no, drive it to 0V. The noise is erased. Precision is restored.

---

## Bits, Bytes, and Words

**Bit**: one binary digit — 0 or 1. The smallest unit of information. One bit encodes one of 2 possibilities.

**Byte**: 8 bits. Encodes one of 2■ = 256 possibilities. Chosen historically to represent one character (ASCII uses 7 bits; the 8th allows error checking or extended characters).

**Word**: the natural integer width of a processor. Was 8-bit (8086 era), then 16, 32, and now 64 bits. A 64-bit processor can represent integers from 0 to 2■■–1 = 18,446,744,073,709,551,615.

**Nibble** (yes, really): 4 bits = one hexadecimal digit. Used constantly in practice.

---

## The Threshold and Noise Margin

In a 5V CMOS logic family (common from the 1970s–1990s):

• Input LOW: 0V to 1.5V → read as 0

• Input HIGH: 3.5V to 5V → read as 1

• Forbidden zone: 1.5V to 3.5V (output should never be in this range)

• Output LOW: guaranteed ≤ 0.5V

• Output HIGH: guaranteed ≥ 4.5V

**Noise margin** = (guaranteed output) – (minimum acceptable input) = 4.5 – 3.5 = 1.0V This means up to 1V of noise on any wire before a logical error occurs.

Modern 1.8V and 1.2V logic (used in current chips) has tighter margins, which is one reason modern chips require extremely clean power supplies and careful layout.

---

### Binary and Power of 2

With n bits you can represent 2■ distinct values.

| Bits | Values | Range (unsigned) | Common name | |------|--------|-----------------|-------------| | 1 | 2 | 0–1 | bit | | 4 | 16 | 0–15 | nibble | | 8 | 256 | 0–255 | byte | | 16 | 65,536 | 0–65,535 | word | | 32 | 4,294,967,296 | 0–4.3 billion | double word | | 64 | 1.8×10¹■ | 0–18.4 quintillion | quad word |

The reason computers count in powers of 2 isn't arbitrary — it falls directly out of the physics of switches.

---

## The Concept: Regeneration

The single most important property of digital systems is **signal regeneration**.

In an analog system: noise accumulates indefinitely as signals pass through components.

In a digital system: each logic gate acts as a threshold detector. It receives a potentially noisy input, makes a binary decision, and drives a fresh, noise-free output. The noise resets to zero at every gate.

This is why digital audio doesn't degrade when copied. Why a file sent across the internet is bit-for-bit identical at the destination. Why your computer gives the same answer to 7+8 a billion times in a row without ever computing 15.0001.

**Analog computes with voltages. Digital computes with decisions.**

---

## Hands-On Exercise

**Open the simulation:** [`simulation.html`](./simulation.html)

Three tools:

**Tool 1 — Signal Noise Demonstrator**

• Drag an "analog" signal (sine wave representing a value 0–9)

• Add noise using the noise slider

• Watch analog values blur and misread at realistic noise levels

• Switch to binary: same noise, zero misreads

**Tool 2 — Th

### Illustrations

**Figure 46 • FIG 2.1-A**

### Analog vs Digital — Why Noise Destroys Analog but Not Digital

*Illustration: FIG 2.1-A | Analog vs Digital — Why Noise Destroys Analog but Not Digital [SVG diagram — see module-2-1-binary.html]*

Analog vs digital noise immunity. Left: an analog sine wave passes through a transmission medium with added noise. The received signal is permanently corrupted — every intermediate value is a valid voltage, so noise cannot be distinguished from data. Right: a digital square wave with the same noise added. At each receiver, a threshold comparator decides: above 2.4V → 1, below 0.4V → 0. The noise (small fluctuations) never crosses the threshold, so the signal is regenerated perfectly. This noise immunity property is why all modern computing uses binary — not because two is a convenient number,

**Figure 47 • FIG 2.1-B**

### Voltage Levels — TTL and CMOS Logic Thresholds

*Illustration: FIG 2.1-B | Voltage Levels — TTL and CMOS Logic Thresholds [SVG diagram — see module-2-1-binary.html]*

Logic voltage thresholds for TTL (5V, left) and CMOS (3.3V, right). Each supply voltage is divided into three zones: guaranteed HIGH output, undefined region (to be avoided), and guaranteed LOW output. The noise margins (green/grey bands) represent how much noise the signal can absorb before being misread. TTL provides ±0.4V margins. CMOS provides ±0.7V margins on a smaller supply. Key insight: any additional state (ternary, quaternary) would divide the voltage range further, shrinking each noise margin proportionally — making circuits less reliable, not more efficient.

**Figure 48 • FIG 2.1-C**

### Bits, Bytes, Words — Information Units in Binary

*Illustration: FIG 2.1-C | Bits, Bytes, Words — Information Units in Binary [SVG diagram — see module-2-1-binary.html]*

The binary information hierarchy. A single bit holds two states (0 or 1). Four bits (a nibble) hold 16 states — enough for one hexadecimal digit. Eight bits (a byte) hold 256 states — the ASCII character set fits in 7 bits with one spare. A 32-bit word holds over 4 billion states, sufficient for IPv4 addresses or 32-bit integers. A 64-bit word holds 1.8×10¹■ states — enough for modern 64-bit memory addresses. Each additional bit doubles capacity: states = 2■, so bits = log■(states).

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 2.1 Why Binary?: Noise immunity Signal regeneration Two physical states [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 2.1 Why Binary? **Type:** Calculation + simulation + written analysis **Estimated time:** 2535 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Noise Margin Calculations A **5V CMOS** logic family has these specifications: - Output HIGH guaranteed: 4.5V - Output LOW guaranteed: 0.5V - Input HIGH minimum: 3.5V - Input LOW maximum: 1.5V A **3.3V LVTTL** logic family has: - Output HIGH guaranteed: 2.4V - Output LOW guaranteed: 0.4V - Input HIGH minimum: 2.0V - Input LOW maximum: 0.8V 1. Calculate the **HIGH noise margin** for both families: `NM

# Module 2.2 Number Systems: Binary, Octal, Hex, Two's Complement

## The Problem

*You've accepted binary. Everything is 0s and 1s. Now you need to check what's stored at memory address 0b0001110101001100.*

>

*That's: memory address 7,500. Not hard in decimal. But in binary? Reading 16 bits of binary at a glance takes a trained eye — and you'll need to read thousands of them while debugging.*

>

*There is also the problem of negative numbers. Binary naturally represents only non-negative integers. How do you represent –1 in bits?*

Both problems have elegant solutions that every computer scientist uses daily: hexadecimal as shorthand for binary, and two's complement for signed integers. Understanding these isn't optional — they appear in every debugger, every disassembler, every network packet, every colour code on a webpage.

## The Story

### Positional Notation

Every number system we use is **positional** — the value of a digit depends on its position.

In decimal (base 10):

```
  3 4 7
  ■ ■ ■■■ 7 × 10■ = 7 × 1   =    7
  ■ ■■■■■ 4 × 10¹ = 4 × 10  =   40
  ■■■■■■■ 3 × 10² = 3 × 100 =  300
                              ■■■■■
                                347
```

The *base* tells you how many symbols exist (10: 0–9) and what each position is worth (powers of 10).

The same principle works for any base:

**Binary (base 2):** symbols are 0,1. Positions are powers of 2.

```
1 0 1 1 0 1
■ ■ ■ ■ ■ ■■■ 1 × 2■ = 1
■ ■ ■ ■ ■■■■■ 0 × 2¹ = 0
■ ■ ■ ■■■■■■■ 1 × 2² = 4
■ ■ ■■■■■■■■■ 1 × 2³ = 8
■ ■■■■■■■■■■■ 0 × 2■ = 0
■■■■■■■■■■■■■ 1 × 2■ = 32
                    ■■
                    45
```

So `0b101101` = 45 in decimal.

---

### Binary ↔ Decimal Conversions

**Decimal to binary** — repeated division by 2, read remainders bottom to top:

```
45 ÷ 2 = 22 remainder 1   ← LSB (least significant bit)
22 ÷ 2 = 11 remainder 0
11 ÷ 2 =  5 remainder 1
 5 ÷ 2 =  2 remainder 1
 2 ÷ 2 =  1 remainder 0
 1 ÷ 2 =  0 remainder 1   ← MSB (most significant bit)

Reading bottom to top: 101101
```

**Binary to decimal** — sum the positional values of each 1 bit (as shown above).

**Shortcut for powers of 2:**

```
2■=1, 2¹=2, 2²=4, 2³=8, 2■=16, 2■=32, 2■=64, 2■=128
2■=256, 2■=512, 2¹■=1024, 2¹■=65536, 2³²≈4 billion
```

Memorise up to 2¹■. The rest follow by doubling.

---

### Hexadecimal — Binary's Shorthand

**Hexadecimal (base 16):** symbols are 0–9 then A–F (where A=10, B=11, C=12, D=13, E=14, F=15).

The key insight: 16 = 2■. So **one hex digit encodes exactly 4 binary bits**. This makes conversion trivial:

```
Binary:  1010  1111  0011  1100
Hex:       A     F     3     C
= 0xAF3C
```

To convert any binary number to hex: group bits in fours from the right, convert each group. To convert hex to binary: expand each digit to 4 bits.

**Why hex is everywhere:**

• Memory addresses: `0x7FFF0000` is much more readable than `0111 1111 1111 1111 0000 0000 0000 0000`

• Colour codes: #FF5733 = RGB(255, 87, 51) — three bytes in six hex digits

• Machine code: `0x48 0x89 0xE5` is the first instruction of almost every x86-64 function

• Ethernet addresses: `A4:C3:F0:85:AC:2D`

• IPv6: `2001:0db8:85a3:0000:0000:8a2e:0370:7334`

---

### Octal — The Forgotten Sibling

Octal (base 8) groups binary in threes. It was used in old minicomputers and Unix file permissions.

```
Binary: 111 101 100
Octal:    7   5   4
= 0754 (a Unix file permission: rwxr-xr--)
```

Octal has largely been replaced by hex in modern systems, but you'll still encounter it in Unix `chmod` commands.

---

### Binary Arithmetic: Addition

Binary addition follows the same rules as decimal, with carries:

```
Bit addition table:
0 + 0 = 0          (no carry)
0 + 1 = 1          (no carry)
1 + 0 = 1          (no carry)
1 + 1 = 0 carry 1  (like decimal: 9+1=0 carry 1)
1 + 1 + 1 = 1 carry 1

Example: 0101 + 0011
  0101   (5)
+ 0011   (3)
■■■■■■
  1000   (8)   ✓
```

The carry propagation rule is exactly what the Pascaline implemented mechanically in 1642 — and what every adder circuit in every CPU implements today.

---

### Two's Complement: Negative Numbers

How do you represent –1 in binary? Several approaches exist; only one is used in practice.

**Sign-magnitude** (naive approach): use the leftmost bit as a sign (0=positive, 1=negative).

• Problem: two representations of zero (0000 and 1000)

• Problem: addition doesn't work normally — need separate add/subtract circuits

**Two's complement** (the universal standard):

**Rule**: to negate a number, invert all bits and add 1.

```
+5 in 8-bit binary:  00000101
Invert all bits:      11111010
Add 1:                11111011
= –5 in two's complement
```

Verify: 5 + (–5) = 0?

```
   00000101   (+5)
 + 11111011   (–5)
 ■■■■■■■■■■■
  100000000   = 9 bits, but we only keep 8 → 00000000 = 0 ✓
```

**Two's complement range** for n bits: –2^(n-1) to +2^(n-1)–1

• 8-bit: –128 to +127

• 16-bit: –32,768 to +32,767

• 32-bit: –2,147,483,648 to +2,147,483,647

**Why two's complement wins:**

• Only one representation of zero

• Addition and subtraction work with the **same circuit** — no special cases

• The most significant bit still acts as a sign indicator (1 = negative)

• Hardware is simpler: one adder handles both add and subtract

---

### Overflow

When a result exceeds the representable range, **overflow** occurs.

```
8-bit signed: max positive = 127 = 0111 1111
127 + 1:
  0111 1111
+ 0000 0001
■■■■■■■■■■■
  1000 0000  = 128 in unsigned, but = –128 in signed!
```

The CPU sets an **overflow flag** when this happens. If you're writing a program and ignore overflow, you get silent corruption — the classic source of security vulnerabilities. The `signed integer overflow` bug has caused spacecraft failures, financial miscalculations, and legendary security exploits.

---

## The Concept: Every Computer Uses These

Every value in your computer right now — every pixel colour, every instruction address, every floating-point number, every timestamp — is encoded in binary and often displayed in hex. Two's complement is how every signed integer works on every mainstream CPU (x86, ARM, MIPS, RI

### Illustrations

**Figure 49 • FIG 2.2-A**

### Positional Notation — The Same Number in Four Bases

*Illustration: FIG 2.2-A | Positional Notation — The Same Number in Four Bases [SVG diagram — see module-2-2-numbers.html]*

The number 42 expressed in four positional bases. Decimal (base 10): 4×10 + 2×1 = 42. Binary (base 2): 1×32 + 0×16 + 1×8 + 0×4 + 1×2 + 0×1 = 42, written 0b101010. Octal (base 8): 5×8 + 2×1 = 42, written 0o52 — each octal digit corresponds to exactly 3 binary bits. Hexadecimal (base 16): 2×16 + 10×1 = 42, written 0x2A — each hex digit corresponds to exactly 4 binary bits, making hex a compact notation for binary data.

**Figure 50 • FIG 2.2-B**

### Two's Complement — Representing Negative Numbers in Binary

*Illustration: FIG 2.2-B | Two's Complement — Representing Negative Numbers in Binary [SVG diagram — see module-2-2-numbers.html]*

Two's complement representation. Left: the 4-bit integer number line arranged in a circle. Positive values (0–7) have sign bit 0. Negative values (–1 to –8) have sign bit 1, with –8 as the only asymmetric value. Right: negation procedure for 8-bit +42. Step 1: invert all bits (bitwise NOT). Step 2: add 1. Verification: adding +42 and –42 produces 0 with a carry bit that is discarded. The key insight is that subtraction becomes addition — A – B = A + (–B) — so CPUs only need one arithmetic unit for both operations.

**Figure 51 • FIG 2.2-C**

### Binary Addition with Carry — Column-by-Column

*Illustration: FIG 2.2-C | Binary Addition with Carry — Column-by-Column [SVG diagram — see module-2-2-numbers.html]*

Binary addition of 183 (0b10110111) and 105 (0b01101001). Carries are shown in red above each column. The result 288 exceeds 8-bit capacity (max 255), so the final carry-out is set and the stored result wraps to 32 (0b00100000 = 288 – 256). The processor's Carry Flag (CF) is set to 1, alerting software that overflow has occurred. Eight addition rules cover all cases; a CPU's adder circuit implements exactly this truth table in hardware.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 2.2 Number Systems: Binary Octal Hexadecimal Two's Complement [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 2.2 Number Systems **Type:** Conversion + arithmetic + analysis **Estimated time:** 3545 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Base Conversions Show all working. Do not use a calculator these must be done by hand, then verified in the simulator. Decimal Binary 1. Convert 173 to binary 2. Convert 255 to binary 3. Convert 1024 to binary (shortcut exists!) Binary Decimal 4. Convert `0b10110101` to decimal 5. Convert `0b11111111` to decimal 6. Convert `0b1000000000000000` to decimal (count the zeros carefully) Binary Hex (group in fours)

# Module 2.3 Boolean Algebra and Logic Gates

## The Problem

*You have transistors. Each one is a switch: on or off, 1 or 0.*

>

*You have Boolean algebra: a complete mathematical system for manipulating true/false values.*

>

*Claude Shannon's 1937 thesis proved they're the same thing.*

>

*Now: how do you turn that equivalence into a practical engineering toolkit?*

The answer is **logic gates** — standardised circuit symbols that hide the transistor implementation and expose only the logical behaviour. A designer drawing a CPU schematic doesn't draw transistors; they draw gates. The gates abstract away the physics and let you reason in pure logic.

This module is where abstraction happens for the first time. You'll stop thinking about voltages and start thinking about truth.

## The Story

### George Boole's Algebra of Thought (1854)

George Boole was a self-taught English mathematician who published *An Investigation of the Laws of Thought* in 1854. He proposed that logical reasoning could be reduced to algebra where variables take only two values (TRUE/FALSE) and three operations (AND, OR, NOT) are sufficient to express any logical statement.

He had no application in mind. It was pure mathematics.

83 years later, Claude Shannon — reading Boole as a graduate student — realised that relay circuits behaved identically to Boolean algebra. The paper he wrote became the theoretical foundation of all digital electronics.

## The Seven Gates

Every digital circuit is built from combinations of these seven elementary gates:

### NOT (Inverter)

• **Symbol:** triangle with a bubble

• **Truth table:** output is the opposite of input

• **Boolean:** Y = ■ (A-bar)

• **Transistors needed:** 1 (CMOS uses 2: one NMOS + one PMOS)

| A | Y | |---|---| | 0 | 1 | | 1 | 0 |

### AND

• **Symbol:** D-shape, flat on input side

• **Truth table:** output 1 only when ALL inputs are 1

• **Boolean:** Y = A · B (dot means AND)

• **Transistors:** 4 (CMOS)

| A | B | Y | |---|---|---| | 0 | 0 | 0 | | 0 | 1 | 0 | | 1 | 0 | 0 | | 1 | 1 | 1 |

### OR

• **Symbol:** curved on both sides, pointed output

• **Truth table:** output 1 when ANY input is 1

• **Boolean:** Y = A + B (plus means OR)

• **Transistors:** 4

| A | B | Y | |---|---|---| | 0 | 0 | 0 | | 0 | 1 | 1 | | 1 | 0 | 1 | | 1 | 1 | 1 |

### NAND (NOT-AND)

• **Symbol:** AND gate + output bubble

• **Truth table:** opposite of AND — output 0 only when ALL inputs are 1

• **Boolean:** Y = A■·■B■ = NOT(A AND B)

• **Transistors:** 4 — actually **simpler** than AND in CMOS!

| A | B | Y | |---|---|---| | 0 | 0 | 1 | | 0 | 1 | 1 | | 1 | 0 | 1 | | 1 | 1 | 0 |

**NAND is the universal gate.** Any Boolean function can be implemented using only NAND gates. Every logic function — NOT, AND, OR, XOR, everything — can be constructed from NANDs alone. This is why real chips use NAND-based logic internally: one gate type is all you need to manufacture.

### NOR (NOT-OR)

• **Symbol:** OR gate + output bubble

• **Boolean:** `Y = NOT(A OR B)`

• Also universal — NOR alone can implement everything

### XOR (Exclusive OR)

• **Symbol:** OR gate with extra curved line on input

• **Truth table:** output 1 when inputs are DIFFERENT

• **Boolean:** `Y = A ⊕ B`

• Key application: **binary addition** (the sum bit of a half adder)

| A | B | Y | |---|---|---| | 0 | 0 | 0 | | 0 | 1 | 1 | | 1 | 0 | 1 | | 1 | 1 | 0 |

### XNOR (Exclusive NOR)

• Output 1 when inputs are the SAME

• Boolean: `Y = NOT(A ⊕ B)` = `A ■ B`

• Application: **equality comparator** (is A = B?)

---

## Boolean Algebra Laws

These identities let you simplify circuits — fewer gates, less cost, less power.

### Basic identities

```
A · 0 = 0          A + 0 = A
A · 1 = A          A + 1 = 1
A · A = A          A + A = A
A · ■ = 0          A + ■ = 1
NOT(NOT A) = A
```

### Commutative

```
A · B = B · A
A + B = B + A
```

### Associative

```
A · (B · C) = (A · B) · C
A + (B + C) = (A + B) + C
```

### Distributive

```
A · (B + C) = (A · B) + (A · C)
A + (B · C) = (A + B) · (A + C)
```

### De Morgan's Laws ← memorise these

```
NOT(A AND B) = (NOT A) OR  (NOT B)
NOT(A OR  B) = (NOT A) AND (NOT B)
```

In words: to negate a product, negate each factor and change AND↔OR.

**Why De Morgan's matters:** it lets you convert between AND-based and OR-based implementations. NAND universality comes directly from De Morgan — `NOT(NOT(A AND B)) = A AND B`, so you can build AND from two NANDs.

### Absorption and Simplification

```
A · (A + B) = A          (absorption)
A + (A · B) = A          (absorption)
(A + B)(A + C) = A + BC  (factoring)
```

---

## From Truth Table to Gate Circuit

Given any truth table, you can mechanically derive a circuit using **Sum of Products (SoP)** or **Product of Sums (PoS)**.

**Sum of Products:**

Write one AND term for every row where output = 1

In each AND term, include each input: as-is if input = 1, inverted if input = 0

OR all the AND terms together

**Example:** build a circuit that is 1 when A=1, B=0, C=1 OR when A=1, B=1, C=0:

```
Term 1: A=1, B=0, C=1  →  A · B■ · C
Term 2: A=1, B=1, C=0  →  A · B · C■
Output: Y = (A · B■ · C) + (A · B · C■)
       = A · (B■ · C + B · C■)       ← factor out A
       = A · (B ⊕ C)                  ← that's XOR!
```

Three gates (one AND + one XOR) instead of two three-input ANDs and one OR. Simplification saves hardware.

---

## NAND-Only Implementations

Since NAND is universal:

**NOT from NAND:** `NOT A = NAND(A, A)` **AND from NAND:** `A AND B = NOT(NAND(A,B)) = NAND(NAND(A,B), NAND(A,B))` **OR from NAND:** `A OR B = NAND(NOT A, NOT B) = NAND(NAND(A,A), NAND(B,B))`

This is how real chip logic libraries work — everything compiled down to NAND trees.

---

## Gate Delay and Propagation

Gates are not instantaneous. In a 5nm process (2022), a typical gate switches in about 10–20 picoseconds. The time from input change to stable output is the **propagation delay** (`t_pd`).

When you chain gates, delays add:

```
Chain of 10 gates: 10 × 20ps = 200ps
Maximum clock frequency: 1 / 20
```

## Illustrations

**Figure 52 • FIG 2.3-A**

### Logic Gate Symbol Atlas — All 7 Standard Gates

*Illustration: FIG 2.3-A | Logic Gate Symbol Atlas — All 7 Standard Gates [SVG diagram — see module-2-3-gates.html]*

Complete logic gate reference. Seven standard gates are shown with IEEE symbols, Boolean expressions, and truth tables. AND requires both inputs HIGH; OR requires at least one. NOT inverts. NAND (highlighted red border) is the universal gate: NOT, AND, and OR can all be synthesised from NAND gates alone. This universality means any digital circuit can be fabricated on a chip using only one type of transistor arrangement, which dramatically simplifies semiconductor manufacturing. NOR is also universal; both NAND and NOR flash memory chips derive their names from this property.

**Figure 53 • FIG 2.3-B**

### NAND Universality — Building XOR from 4 NAND Gates

*Illustration: FIG 2.3-B | NAND Universality — Building XOR from 4 NAND Gates [SVG diagram — see module-2-3-gates.html]*

XOR built from 4 NAND gates — the canonical proof of NAND universality. G1 = NAND(A,B) acts as a shared intermediate. G2 = NAND(A, G1) implements NOT(A AND NOT(AB)). G3 = NAND(B, G1) implements NOT(B AND NOT(AB)). G4 = NAND(G2, G3) combines them to produce A XOR B. The truth table verifies all four input combinations. This construction is used in CMOS logic libraries where all cells are ultimately implemented as NAND chains.

**Figure 54 • FIG 2.3-C**

### All Seven Standard Gates — Symbol · Boolean Expression · Truth Table

*Illustration: FIG 2.3-C | All Seven Standard Gates — Symbol · Boolean Expression · Truth Table [SVG diagram — see module-2-3-gates.html]*

Complete reference for all seven standard logic gates, showing IEEE schematic symbol, Boolean expression, and full truth table. NAND and NOR are marked as universal gates — a fact of deep importance: any digital circuit, no matter how complex, can be constructed from NAND gates alone. This is why CMOS logic libraries are largely implemented as NAND chains. XOR outputs 1 when inputs differ, making it the equality tester and the sum-bit generator in binary addition.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 2.3 Logic Gates: AND OR NOT NAND NOR XOR XNOR [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 2.3 Boolean Algebra and Logic Gates **Type:** Truth tables + algebra + circuit design **Estimated time:** 4050 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Truth Tables Build the complete truth table for each expression. 1. `Y = A AND (NOT B)` | A | B | NOT B | Y | |---|---|-------|---| | 0 | 0 | | | | 0 | 1 | | | | 1 | 0 | | | | 1 | 1 | | | 2. `Y = (A OR B) AND (NOT A OR C)` 3 variables, 8 rows 3. `Y = NOT (A XOR B)` what common gate is this? --- Part B Boolean Algebra Simplification Simplify each expression. Show every step and name the

# Module 2.4 Combinational Circuits

## The Problem

*You can compute AND, OR, NOT. You can build any logic function from gates.*

>

*But the fundamental operation every computer performs millions of times per second is **addition**. How do you add two binary numbers using only logic gates?*

>

*And once you can add, how do you chain that addition across multiple bits — from one-bit arithmetic to 4-bit, 8-bit, 32-bit?*

This module builds the **adder** — the most important combinational circuit — from first principles, one gate at a time. By the end, you'll have a working 4-bit ripple-carry adder that performs real binary addition, and understand every gate inside it.

**Combinational circuit**: output depends only on current inputs. No memory. No state. Input → output, instantly (within propagation delay).

## The Story

### The Half Adder

Adding two 1-bit numbers produces a 2-bit result: a sum bit and a carry bit.

```
0 + 0 = 00  (sum=0, carry=0)
0 + 1 = 01  (sum=1, carry=0)
1 + 0 = 01  (sum=1, carry=0)
1 + 1 = 10  (sum=0, carry=1)
```

Truth table:

| A | B | Sum | Carry | |---|---|-----|-------| | 0 | 0 | 0 | 0 | | 0 | 1 | 1 | 0 | | 1 | 0 | 1 | 0 | | 1 | 1 | 0 | 1 |

Looking at the patterns:

• **Sum = A XOR B** — output is 1 when exactly one input is 1

• **Carry = A AND B** — output is 1 only when both inputs are 1

Implementation: **one XOR gate + one AND gate**.

```
A ■■■[XOR]■ Sum
  ■
B ■■
  ■
  ■■[AND]■ Carry
```

The half adder is "half" because it can't handle an incoming carry from a previous stage. For multi-bit addition, we need the full adder.

---

### The Full Adder

A **full adder** adds three 1-bit inputs: A, B, and Cin (carry-in from the previous stage).

| A | B | Cin | Sum | Cout | |---|---|-----|-----|------| | 0 | 0 | 0 | 0 | 0 | | 0 | 0 | 1 | 1 | 0 | | 0 | 1 | 0 | 1 | 0 | | 0 | 1 | 1 | 0 | 1 | | 1 | 0 | 0 | 1 | 0 | | 1 | 0 | 1 | 0 | 1 | | 1 | 1 | 0 | 0 | 1 | | 1 | 1 | 1 | 1 | 1 |

Derivation:

• **Sum = A XOR B XOR Cin**

• **Cout = (A AND B) OR (Cin AND (A XOR B))**

Implementation: **two half adders + one OR gate** (5 gates total, or 9 gates in NAND-only).

```
A ■[HA]■ S1 ■[HA]■ Sum
B ■■  ■Cout1       ↑ Cin also goes in
                   ■
     Cout1 ■[OR]■ Cout
     Cout2 ■■
```

---

### The 4-Bit Ripple Carry Adder

To add two 4-bit numbers A[3:0] and B[3:0], chain four full adders:

```
A0 B0     A1 B1     A2 B2     A3 B3
 ■  ■      ■  ■      ■  ■      ■  ■
[FA0]■■C1■[FA1]■■C2■[FA2]■■C3■[FA3]■■Cout
Cin=0      ■         ■         ■         ■
           S0        S1        S2        S3
```

• FA0 gets Cin=0 (no incoming carry for the least significant bit)

• FA0's Cout feeds FA1's Cin

• FA1's Cout feeds FA2's Cin

• FA2's Cout feeds FA3's Cin

• FA3's Cout is the overflow/carry-out of the entire addition

**Example:** 5 + 6 = ?

```
A = 0101  (5)
B = 0110  (6)

Bit 0: 1+0+0=1, Cout=0  → S0=1
Bit 1: 0+1+0=1, Cout=0  → S1=1
Bit 2: 1+1+0=0, Cout=1  → S2=0
Bit 3: 0+0+1=1, Cout=0  → S3=1

Result: 1011 = 11 ✓
```

This is called "ripple carry" because the carry signal **ripples** from bit 0 to bit 3. FA3 cannot compute its result until FA2's carry is ready — which waits for FA1 — which waits for FA0. The critical path is 4 full-adder delays long.

For a 64-bit ripple-carry adder: 64 × (full-adder delay). Too slow for high-speed CPUs!

---

### Carry Lookahead — Fixing the Speed Problem

**Carry lookahead** (invented in the 1950s, still used today) eliminates the ripple wait by computing all carries in parallel.

Define:

• **Generate:** `Gi = Ai AND Bi` — this bit *generates* a carry regardless of Cin

• **Propagate:** `Pi = Ai XOR Bi` — this bit *propagates* an incoming carry

Then:

```
C1 = G0 OR (P0 AND C0)
C2 = G1 OR (P1 AND C1)
   = G1 OR (P1 AND G0) OR (P1 AND P0 AND C0)
C3 = G2 OR (P2 AND G1) OR (P2 AND P1 AND G0) OR (P2 AND P1 AND P0 AND C0)
C4 = G3 OR (P3 AND G2) OR ...
```

Each carry is now a direct function of the original inputs A, B, C0 — no chaining. All carries computed in 2 gate delays instead of n × (full adder delay). This is why modern 64-bit adders run at GHz speeds.

---

### Other Important Combinational Circuits

**Multiplexer (MUX):** selects one of N inputs based on a select signal.

• 2-to-1 MUX: `Y = (S AND B) OR (NOT S AND A)` — if S=0, pass A; if S=1, pass B

• Used everywhere: routing data, implementing registers, building ALU operations

**Demultiplexer (DEMUX):** routes one input to one of N outputs based on select.

**Decoder:** converts n-bit binary input to one of 2■ one-hot outputs.

• Example: 3-bit input → 8 output lines, exactly one high

• Application: memory address decoding, opcode decoding in CPUs

**Encoder:** opposite of decoder — 2■ inputs to n-bit output.

**Comparator:** compares two n-bit numbers, outputs A>B, A=B, A<B.

• A=B: XOR each bit pair, NOR all results

• A>B: more complex — requires looking at most significant differing bit

---

## The Concept: No State, Just Wires and Gates

A combinational circuit has no memory. Change the inputs, the outputs change (after propagation delay). Remove the inputs, the outputs become undefined. There is no "stored" value.

Everything we've built so far — adders, multiplexers, decoders — is combinational. The next module (2.5) introduces **sequential circuits**: circuits that remember their state. The union of combinational and sequential circuits is everything inside a CPU.

---

## Hands-On Exercise

**Open the simulation:** [simulation.html](./simulation.html)

**Tool 1 — Half Adder**

• Toggle A and B, watch Sum and Carry update

• Gate-level view shows which gates are active

**Tool 2 — Full Adder**

• Toggle A, B, Cin

• Step through the carry propagation

• See both internal half adders working

**Tool 3 — 4-Bit Ripple Carry Adder**

• Enter two 4-bit numbers

• Watch carry ripple from bit 0 to bit 3, one step at a time

• Overflow flag lights up when result excee

### Illustrations

**Figure 55 • FIG 2.4-A**

### Half Adder → Full Adder — Adding Bits with Gates

*Illustration: FIG 2.4-A | Half Adder → Full Adder — Adding Bits with Gates [SVG diagram — see module-2-4-combinational.html]*

Half adder (left) and full adder (right). The half adder adds two 1-bit inputs using XOR (sum) and AND (carry out) — no carry input. The full adder extends this by chaining two half adders with an OR gate. It accepts three inputs: A, B, and a carry-in (Cin) from the previous column. The carry-out is generated by ORing the carry outputs of both half adders. Full adders can be chained to add multi-bit numbers — four full adders build a 4-bit ripple carry adder.

**Figure 56 • FIG 2.4-B**

### 4-Bit Ripple Carry Adder — Carry Propagation Chain

*Illustration: FIG 2.4-B | 4-Bit Ripple Carry Adder — Carry Propagation Chain [SVG diagram — see module-2-4-combinational.html]*

4-bit ripple carry adder computing 0111 + 0001 = 1000 (7 + 1 = 8). Each full adder processes one bit column with a carry-in from the previous. The carry propagates through all four stages — bit 0 generates carry 1, which propagates through bits 1 and 2 (producing sum 0 each time), until bit 3 produces the final sum bit 1. The "ripple" refers to this sequential carry propagation: the adder cannot produce the final result until all carries have settled. This O(n) delay motivated the invention of carry lookahead adders, which compute all carries in parallel in O(log n) time.

**Figure 57 • FIG 2.4-C**

### Multiplexer and Decoder — Selecting and Routing Binary Data

*Illustration: FIG 2.4-C | Multiplexer and Decoder — Selecting and Routing Binary Data [SVG diagram — see module-2-4-combinational.html]*

Two fundamental combinational building blocks: the multiplexer (MUX) and the decoder. A 4-to-1 MUX routes one of four data inputs to a single output, selected by two control lines — used in CPU datapaths to choose between ALU operands. A 2-to-4 decoder converts a 2-bit binary address into four one-hot outputs (exactly one high at a time) — used in memory address decoding to select which memory bank or register to activate. Both circuits appear repeatedly at every level of computer architecture.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 2.4 Combinational Circuits: Half Adder Full Adder 4-Bit Ripple Carry Adder [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 2.4 Combinational Circuits **Type:** Circuit tracing + arithmetic + design **Estimated time:** 4050 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Full Adder Truth Table Complete the full truth table for a full adder (A + B + Cin). | A | B | Cin | Sum | Cout | |---|---|-----|-----|------| | 0 | 0 | 0 | | | | 0 | 0 | 1 | | | | 0 | 1 | 0 | | | | 0 | 1 | 1 | | | | 1 | 0 | 0 | | | | 1 | 0 | 1 | | | | 1 | 1 | 0 | | | | 1 | 1 | 1 | | | | Verify that Sum = A XOR B XOR Ci

# Module 2.5 Sequential Circuits: Memory from Gates

## The Problem

*The 4-bit adder from Module 2.4 computes the sum of two numbers instantly — as fast as the signals propagate through the gates. Perfect.*

>

*Now: what happens after the addition? Where does the result go?*

>

*If you wire the output directly to the next operation, the values are gone the moment the inputs change. There is no memory. A computer that can't remember intermediate results can't execute even the simplest program.*

>

*You need circuits that **hold a value** — circuits whose output persists even when the input is removed. You need **state**.*

State changes everything. A circuit with state is no longer purely combinational — its output depends not only on current inputs but on **history**. This is the distinction between a calculator (combinational) and a computer (sequential).

## The Story

### Feedback — The Key Insight

All memory in digital electronics comes from one idea: **feedback**. Route the output of a gate back to its own input. The circuit can now hold a state indefinitely.

Consider two NAND gates cross-coupled:



If Q=1 and Q■=0:

• Top NAND: one input is S, other is Q■=0. Output Q = NAND(S, 0) = 1 — stable.

• Bottom NAND: one input is R, other is Q=1. Output Q■ = NAND(R, 1) = NOT R.

Normally S=R=1 (inactive). Then Q■=0 and Q=1 — stable. Set S=0: Q becomes 1 (set). Release S: circuit stays Q=1. Set R=0: Q■ becomes 1, which feeds back to force Q=0 (reset). Release R: stays at 0.

This is the **SR NAND latch** — the simplest possible 1-bit memory cell. It holds its value indefinitely as long as power is supplied.

---

### The SR Latch

**SR Latch (NOR-based):**

| S | R | Q (next) | Q■ (next) | Action | |---|---|----------|-----------|--------| | 0 | 0 | Q (hold) | Q■ (hold) | Hold state | | 0 | 1 | 0 | 1 | Reset | | 1 | 0 | 1 | 0 | Set | | 1 | 1 | invalid | invalid | Forbidden! |

The forbidden state (S=R=1) forces both outputs to 0, violating the invariant Q≠Q■. If both inputs then return to 0 simultaneously, the final state is unpredictable — a **race condition**. Real systems must avoid this.

**The SR latch stores 1 bit.** It's the ancestor of every RAM cell in every computer ever built.

---

### The D Latch (Transparent Latch)

The SR latch has the forbidden state problem. The **D latch** solves it with one extra input:

• **D (Data):** the value to store (0 or 1)

• **E (Enable):** when high, output follows D; when low, output is held

Implementation: `S = D AND E`, `R = (NOT D) AND E`

This prevents S=R=1: when E=1, if D=1 then S=1,R=0; if D=0 then S=0,R=1. Never both 1.

**Problem with transparent latches:** when E is high, the output *follows* D continuously — the latch is "transparent." If D is changing (because the output of an adder is rippling), the latch sees every intermediate value. This causes glitches in cascaded circuits.

---

### The D Flip-Flop (Edge-Triggered)

The **D flip-flop** (DFF) solves the transparency problem by only capturing D at the **edge** of a clock signal (either rising edge 0→1 or falling edge 1→0).

Implementation: two D latches in series (master-slave configuration):

• When CLK=0: master latch is transparent, slave is held

• When CLK=1: master is held, slave captures master's output

The net effect: Q updates to D's value at the instant CLK transitions 0→1. After that, Q is held regardless of D changes until the next clock edge.

```
D ■■[master latch]■■[slave latch]■■ Q
         CLK              NOT CLK
```

**Why the edge-triggered flip-flop is the foundation of CPUs:**

• All registers in a CPU are flip-flops

• The clock signal synchronises all flip-flops — they all update at the same moment

• This makes the entire processor operate in lockstep

• Clock frequency = how fast the flip-flops toggle = operations per second

---

### Registers

A **register** is simply N flip-flops connected in parallel, sharing the same clock and enable signals.

• 4 flip-flops → 4-bit register (stores one nibble)

• 8 flip-flops → 8-bit register (byte)

• 64 flip-flops → 64-bit register (modern CPU register width)

A CPU's general-purpose registers (RAX, RBX, etc. in x86-64) are 64-bit D flip-flop arrays clocked at the CPU's operating frequency (3–5 GHz).

---

### Counters — Adding State to Addition

Chain flip-flops together and you get a **counter**: a circuit that increments a binary number on each clock edge.

**Ripple (asynchronous) counter:**

• Each flip-flop's output feeds the clock of the next flip-flop

• Q0 toggles every clock; Q1 toggles when Q0 goes 1→0; Q2 when Q1 goes 1→0…

• Simple but slow — the toggle ripples through stages (like ripple carry)

**Synchronous counter:**

• All flip-flops share the same clock

• Combinational logic computes next state: each bit toggles if all lower bits are 1

• Faster — all bits update simultaneously

**4-bit synchronous counter sequence:**

```
0000 → 0001 → 0010 → 0011 → 0100 → ... → 1111 → 0000 (wraps)
  0  →   1  →   2  →   3  →   4  → ... →  15  →   0
```

Counters appear everywhere: program counters (track which instruction to execute), timer circuits, frequency dividers, and pseudorandom number generators.

---

## The Concept: The Clock

The clock signal is the heartbeat of a synchronous digital system. It's a square wave oscillating between 0 and 1 at a fixed frequency.

```
CLK: ■■   ■■■   ■■■   ■■■   ■■
     ■■■■   ■■■■   ■■■■   ■■■
```

At each rising edge, all flip-flops sample their D inputs and update their Q outputs simultaneously. Between edges, the combinational logic settles to its new values, ready for the next clock edge.

**Setup time** (t_su): D must be stable this long *before* the clock edge **Hold time** (t_h): D must be stable this long *after* the clock edge **Propagation delay** (t_pd): how long after the clock edge Q is stable

Maximum clock frequency:

```
f_max = 1 / (t_pd_flop + t_pd_logic + t_su)
```

This is the clock constrai

## Illustrations

**Figure 58 • FIG 2.5-A**

### From SR Latch to D Flip-Flop — How Memory Emerges from Feedback

*Illustration: FIG 2.5-A | From SR Latch to D Flip-Flop — How Memory Emerges from Feedback [SVG diagram — see module-2-5-sequential.html]*

Left: SR (Set-Reset) latch built from two cross-coupled NOR gates. The feedback loops create bistable behaviour — when S=R=0, the latch holds its current Q value unchanged. This is the emergence of memory from pure combinational gates. The invalid state (S=R=1) forces both outputs to 0, violating the Q■ = NOT Q invariant. Right: the D flip-flop solves the invalid state problem by eliminating the R input and using a clock signal. D data is captured only on the rising edge of CLK, making the storage synchronous and predictable. The timing diagram shows Q updating at each rising clock edge to mat

**Figure 59 • FIG 2.5-B**

### 4-Bit Register — Four D Flip-Flops Sharing a Clock

*Illustration: FIG 2.5-B | 4-Bit Register — Four D Flip-Flops Sharing a Clock [SVG diagram — see module-2-5-sequential.html]*

A 4-bit register: four D flip-flops connected to a shared clock line. On every rising edge of CLK, all four D inputs are simultaneously sampled and stored in their respective flip-flops. The Q outputs maintain their values until the next rising edge. This synchronous, parallel capture is the foundation of CPU registers. A modern 64-bit processor register is simply 64 D flip-flops sharing a clock — conceptually identical to this diagram, scaled by 16×.

**Figure 60 • FIG 2.5-C**

**4-Bit Register — Four D Flip-Flops Sharing a Clock**

*Illustration: FIG 2.5-C | 4-Bit Register — Four D Flip-Flops Sharing a Clock [SVG diagram — see module-2-5-sequential.html]*

A 4-bit register formed by four D flip-flops sharing a single clock signal. On every rising clock edge, all four D inputs are simultaneously captured and held at the Q outputs until the next rising edge. This is the fundamental storage mechanism in every processor: the CPU's general-purpose registers (e.g., x86's RAX, RBX) are 64-bit versions of exactly this structure — 64 D flip-flops, one clock, 64 bits of state held between clock cycles.

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 2.5 Sequential Circuits: SR Latch D Flip-Flop 4-Bit Counter [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 2.5 Sequential Circuits **Type:** State tracing + design + written analysis **Estimated time:** 4050 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A SR Latch State Trace Trace the SR latch through this sequence of inputs. Start with Q=0. | Step | S | R | Q (before) | Q (after) | Action | |------|---|---|------------|-----------|--------| | 1 | 0 | 0 | ? | | | 2 | 1 | 0 | ? | ? | | | 3 | 0 | 0 | ? | ? | | | 4 | 0 | 1 | ? | ? | | | 5 | 0 | 0 | ? | ? | | | 6 | 1 | 0 | ? | ? | | | 7 | 1 | 1 | ? | ? | FORBIDDEN | | 8 | 0 | 0 |

# Module 2.6 From Gates to a Simple ALU

## The Problem

*You have:*

*- Logic gates that compute AND, OR, NOT, XOR*

*- Adders that add binary numbers*

*- Multiplexers that select between signals*

*>*

*Now: how do you combine these into a single unit that can perform any arithmetic or logical operation on demand?*

*>*

*The answer is the **Arithmetic Logic Unit (ALU)** — the computational core of every processor ever built.*

The ALU is the machine that executes the instructions of every program ever written. When Python evaluates `x + 1`, the compiler translates this to an ADD instruction, the CPU fetches it, decodes it, and routes the operands through the ALU. The ALU computes the result in one clock cycle.

Understanding the ALU is the bridge between Level 2 (circuits) and Level 3 (machine architecture). Once you have an ALU, you have a machine that can compute. Add memory, a program counter, and control logic — and you have a CPU.

---

## The Story

### What an ALU Must Do

A minimal ALU for a real processor needs to support at least:

**Arithmetic operations:**

• ADD: A + B

• SUB: A – B (= A + (–B) = A + two's complement of B)

• INC: A + 1

• DEC: A – 1

**Logical operations:**

• AND: A AND B

• OR: A OR B

• XOR: A XOR B

• NOT: NOT A (bitwise invert)

**Comparison (produces flags, not data):**

• Zero flag: result = 0?

• Carry flag: unsigned overflow?

• Negative flag: most significant bit = 1?

• Overflow flag: signed overflow?

**Shifts (often included):**

• LSR: logical shift right (divide by 2)

• LSL: logical shift left (multiply by 2)

All of these from gates, adders, and multiplexers.

---

### Building the 4-Bit ALU

**Step 1: Arithmetic unit**

ADD and SUB use the same adder. For SUB, we negate B using two's complement (invert B and set Cin=1).

Control signal: `sub` (0=add, 1=subtract)

```
B_effective = B XOR sub  (when sub=1, inverts B)
Cin = sub                (when sub=1, adds 1 to complete two's complement)
Result = A + B_effective + Cin
```

One 4-bit adder handles both ADD and SUB. This is elegant: subtraction is free once you have addition and XOR.

**Step 2: Logic unit**

For each bit position i, compute:

```
AND_result[i]  = A[i] AND B[i]
OR_result[i]   = A[i] OR  B[i]
XOR_result[i]  = A[i] XOR B[i]
NOT_result[i]  = NOT A[i]
```

All four computed simultaneously — no selection yet.

**Step 3: Multiplexer selects the operation**

A 2-bit **operation select** (op[1:0]) controls a 4-to-1 multiplexer:

```
op = 00 → select AND result
op = 01 → select OR result
op = 10 → select XOR result
op = 11 → select NOT result
```

**Step 4: Arithmetic/Logic mode select**

A 1-bit `mode` signal selects between arithmetic unit output and logic unit output:

```
mode = 0 → arithmetic (ADD/SUB)
mode = 1 → logic (AND/OR/XOR/NOT)
```

**Combined control word: 3 bits (sub, mode, op[1:0])**

| mode | sub | op | Operation | |------|-----|----|-----------| | 0 | 0 | — | ADD | | 0 | 1 | — | SUB | | 1 | 0 | 00 | AND | | 1 | 0 | 01 | OR | | 1 | 0 | 10 | XOR | | 1 | 0 | 11 | NOT A |

---

## Flags

The ALU doesn't just produce a result — it also produces **status flags** that the control unit reads to make decisions (for branch instructions like `if`, `while`, comparisons).

**Zero flag (Z):**

```
Z = NOR(result[3], result[2], result[1], result[0])
Z = 1 when all result bits are 0
```

Application: `JZ` (jump if zero) — the basis of all equality tests.

**Carry flag (C):**

```
C = carry-out of the most significant adder stage
C = 1 when unsigned result overflowed (exceeded 15 for 4-bit)
```

Application: multi-precision arithmetic — add 64-bit numbers as two 32-bit adds using the carry from the first.

**Negative flag (N):**

```
N = result[3]  (most significant bit)
N = 1 when result is negative in two's complement
```

Application: `JN` (jump if negative) — signed comparisons.

**Overflow flag (V):** Signed overflow — result's sign is wrong.

```
V = Cout XOR Cin_of_MSB
V = 1 when adding two positives gives a negative (or two negatives gives a positive)
```

Application: detecting signed arithmetic errors.

---

### Real-World ALU: Intel x86-64 EFLAGS

The x86-64 processor's EFLAGS register contains all four flags plus more:

• **CF** — Carry Flag

• **ZF** — Zero Flag

• **SF** — Sign Flag (= Negative flag)

• **OF** — Overflow Flag

• **PF** — Parity Flag (count of 1 bits in result)

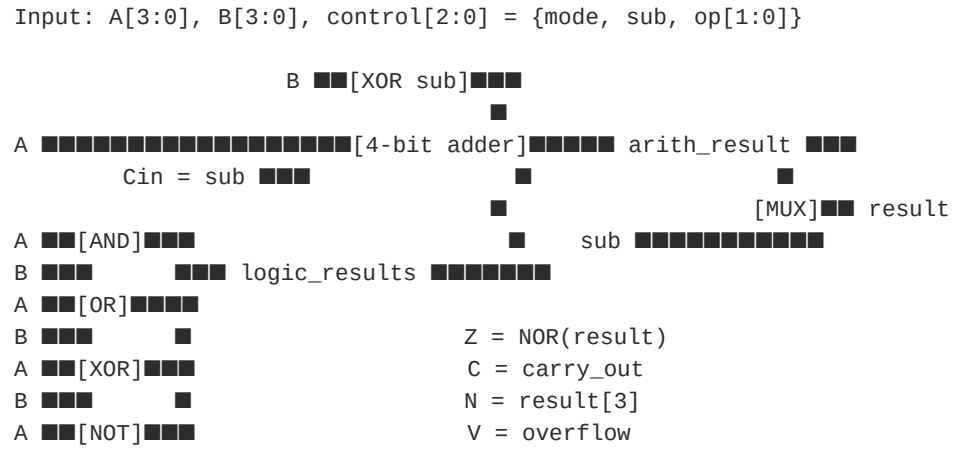
• **AF** — Auxiliary Carry (carry from bit 3 to bit 4, used in BCD arithmetic)

Every arithmetic instruction sets these flags. Conditional jump instructions (JE, JNE, JL, JG, etc.) test these flags and branch accordingly. Your `if (x == 0)` in C compiles to a CMP + JE — the ALU does the comparison, sets ZF, and the JE checks it.

---

### Putting It All Together

Here is the complete datapath for our 4-bit ALU:




This 4-bit ALU uses approximately 100 gates. A modern CPU's 64-bit ALU uses thousands. The structure is identical — wider data paths, more operations, more flags.

---

## The Concept: One Hardware, Many Operations

The ALU's elegance is that it contains **multiple computation paths running simultaneously**. The ADD path is always computing A+B. The AND path is always computing A AND B. The XOR path is always computing A XOR B.

The **multiplexer** at the output selects which result to forward based on the instruction opcode. The "unused" computation results are simply discarded.

This is a form of **spatial parallelism** —

### Illustrations

**Figure 61 • FIG 2.6-A**

**4-Bit ALU Architecture — Inputs, Operation Select, Outputs, and Flags**

*Illustration: FIG 2.6-A | 4-Bit ALU Architecture — Inputs, Operation Select, Outputs, and Flags [SVG diagram — see module-2-6-alu.html]*

4-bit ALU architecture. Two 4-bit operands A (11 = 1011) and B (5 = 0101) enter from the left. The operation select input (3 bits, selecting from 8 possible operations) currently selects ADD (000). The ALU performs the operation and outputs the 4-bit result plus a flags register. Here, 11+5=16 exceeds 4-bit capacity (max 15), so the result truncates to 0000 and the Carry Flag (CF=1) signals overflow. The Zero Flag (ZF=1) is also set because the stored result is 0. These flags are examined by conditional branch instructions to implement if/else and loop logic in software.

**Figure 62 • FIG 2.6-B**

**ALU Flags — What Each Flag Means and When It Is Set**

*Illustration: FIG 2.6-B | ALU Flags — What Each Flag Means and When It Is Set [SVG diagram — see module-2-6-alu.html]*

The five essential ALU flags. Zero Flag (ZF): result is zero — used by equality comparisons (JE = "jump if equal" checks ZF=1). Carry Flag (CF): unsigned arithmetic overflow or borrow — used in multi-word arithmetic. Overflow Flag (OF): signed arithmetic overflow — distinct from CF because signed and unsigned overflow happen at different points. Sign Flag (SF): the result's most significant bit — set when the result is negative in two's complement. Parity Flag (PF): even number of 1-bits in the low byte — historically used for serial communication error detection. Every conditional instruction

**Figure 63 • FIG 2.6-C**

### ALU Operation Table — Control Lines, Operations, and Status Flags

*Illustration: FIG 2.6-C | ALU Operation Table — Control Lines, Operations, and Status Flags [SVG diagram — see module-2-6-alu.html]*

ALU operation table and status flags for a minimal 4-bit ALU. Eight operations are encoded in 3 control bits, covering the essential arithmetic (ADD, SUB), logical (AND, OR, XOR, NOT), and shift (SHL, SHR) operations needed by a working processor. Four status flags capture properties of every result: Zero (result is zero — used for equality tests), Carry (result exceeds N bits — used in multi-word arithmetic), Overflow (signed arithmetic produced an incorrect result), and Negative (sign bit is set). These four bits directly drive conditional branch instructions in the CPU.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 2.6 Simple ALU: Arithmetic + Logic Unit Flags ALU Program [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 2.6 The Simple ALU (Level 2 Capstone) **Type:** ALU execution + flag analysis + design + capstone **Estimated time:** 5060 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A ALU Operations by Hand Execute each operation on the 4-bit ALU. Inputs are 4-bit unsigned. Predict results AND flags before simulating. The ALU operations: ADD, SUB (using two's complement), AND, OR, XOR, NOT A. | A | B | Operation | Result (binary) | Decimal | Z | C | N | V | |------|------|-----------|-----------------|---------|---|---|---|---| | 0101 | 0011 | ADD |

LEVEL 03

# Machine Architecture

How CPUs Fetch, Decode, and Execute Instructions

# Module 3 1 Memory

■# Module 3.1 — Memory: How Machines Remember

## The Problem

*You have an ALU that can add, subtract, and perform logic operations.*

*You have registers that hold values for a single clock cycle.*

>

*But here is the problem: where do the values come from before they reach the ALU?*

*And where do results go after the ALU produces them?*

>

*The answer is **memory** — the system that lets a machine remember anything at all.*

Without memory, a processor can only react to its current inputs. It cannot store a result, recall a previous value, or execute a program. Memory is what transforms a calculator into a computer.

## The Story

### The Human Analogy

A human performing arithmetic uses two types of memory simultaneously:

• **Working memory** — the few numbers held in mind right now (small, fast, volatile)

• **Long-term memory** — everything learned and stored (large, slow, persistent)

Computer memory works the same way, at every scale from a single chip to an entire data centre. The hierarchy exists because of a fundamental physical tradeoff: **fast memory is expensive and small; slow memory is cheap and large.**

Every computer architecture in history has navigated this tradeoff.

### From Flip-Flops to Memory Arrays

In Module 2.5 you built a D flip-flop — a single bit of memory. One flip-flop stores one bit.

Eight flip-flops wired together store one byte. That is a **register**.

Now imagine 256 rows of 8 flip-flops each. That is 256 bytes of **SRAM** — Static Random Access Memory. Add an address decoder to select which row to read or write, and you have a functioning memory array.

This is literally how CPU cache memory works today, scaled to millions of cells.

### SRAM vs DRAM

**SRAM (Static RAM)** — each bit stored in a flip-flop (6 transistors per bit)

• Fast: access time ~1 nanosecond

• Expensive: large die area

• Used for: CPU registers, L1/L2/L3 cache

**DRAM (Dynamic RAM)** — each bit stored as charge on a capacitor (1 transistor + 1 capacitor per bit)

• Slower: access time ~50–100 nanoseconds

• Cheap: very small cell, high density

• Problem: capacitors leak — must be **refreshed** every ~64 milliseconds

• Used for: main system RAM (DDR4, DDR5)

The "dynamic" in DRAM refers to this need for constant refreshing. If the refresh circuit fails, all data is lost within milliseconds — even with power connected.

---

### The Memory Hierarchy

Modern systems layer memory into a hierarchy based on speed and capacity:

| Level | Type | Size | Speed | Location | |---|---|---|---|---| | L0 | CPU Registers | 16–32 × 64-bit | 1 cycle (~0.3 ns) | On-chip | | L1 | SRAM Cache | 32–64 KB | 4 cycles (~1 ns) | On-chip | | L2 | SRAM Cache | 256 KB–1 MB | 12 cycles (~4 ns) | On-chip | | L3 | SRAM Cache | 4–32 MB | 40 cycles (~12 ns) | On-chip | | RAM | DRAM | 4–64 GB | ~200 cycles (~60 ns) | Off-chip | | SSD | Flash | 256 GB–4 TB | ~100,000 ns | Off-chip | | HDD | Magnetic | 1–20 TB | ~10,000,000 ns | Off-chip |

The CPU operates on data in registers. When it needs data not in registers, it looks in L1 cache. On a miss, it checks L2, then L3, then RAM, then disk. Each level is roughly 10–100× slower than the one above.

This is why **cache locality** matters so much in software performance: a program that accesses memory in sequential, predictable patterns keeps data in fast cache. A program that jumps around randomly keeps forcing expensive cache misses to main RAM.

---

### Address Space and Addressability

Every byte in memory has a unique **address** — a number that identifies its location.

For a 32-bit address space: $2^{32}$ = 4,294,967,296 addresses = 4 GB maximum. For a 64-bit address space: 2■■ = 18,446,744,073,709,551,616 addresses = 16 exabytes.

**Addressability** refers to what unit each address points to. Most modern architectures are **byte-addressable**: each address identifies one byte. Some older or embedded systems are word-addressable (each address identifies 16 or 32 bits).

**Memory map**: not all addresses point to RAM. A typical x86-64 system uses its address space for:

• RAM (the majority)

• Memory-mapped I/O (peripherals that respond to memory read/write)

• BIOS/UEFI ROM

• Graphics framebuffer

• Reserved regions

### How a Read Operation Works

When the CPU issues a **read**:

CPU places the **address** on the address bus

CPU asserts the **read control signal**

Memory decodes the address to select the correct row and column

Memory places the **data** on the data bus

CPU latches the data from the data bus into a register

For DRAM, step 3–4 involves:

• **Row address strobe (RAS)**: selects the row, copies it into a row buffer

• **Column address strobe (CAS)**: selects the column from the row buffer

• CAS latency (CL) is the key DRAM timing spec (e.g., CL16 = 16 clock cycles)

### How a Write Operation Works

When the CPU issues a **write**:

CPU places the **address** on the address bus

CPU places the **data** on the data bus

CPU asserts the **write control signal**

Memory decodes the address, stores data at that location

## The Concept: Memory as an Addressed Array

Think of memory as a post office with numbered mailboxes. Each mailbox (address) holds a fixed-size value (1 byte for byte-addressable memory). The postmaster (address decoder) routes requests to the correct box. Reading opens the box and copies its contents. Writing replaces its contents.

The key abstractions:

• **Address**: which location

• **Data**: what value

• **Control**: read or write

These three signals — address, data, control — form the **memory bus interface**. Everything from a 1980s microprocessor to a modern server uses this same abstraction.

## Hands-On Exercise

**Open the simulation:** [`simulation.html`](./simulation.html)

**Tool 1 — Memory Array Explorer**

• A 16-byte memory array with addresses 0x00–0x0F

• Click any cell to select it; enter a value (0–255) to write

• Use Read/Write buttons to simul

## Illustrations

**Figure 64 • FIG 3.1-A**

### Memory Hierarchy — Speed vs Capacity vs Cost

*Illustration: FIG 3.1-A | Memory Hierarchy — Speed vs Capacity vs Cost [SVG diagram — see module-3-1-memory.html]*

The memory hierarchy pyramid. Each level trades speed for capacity. CPU registers (apex): sub-nanosecond access, ~1KB total, implemented as flip-flops on the CPU die. L1/L2/L3 caches: SRAM, 1–20ns, 32KB–16MB — automatically managed by hardware. Main RAM: DRAM, ~60ns, 8–64GB — software-addressable. SSD/NVMe: ~100µs, terabytes — persistent. HDD: ~10ms, unlimited — archival. The processor tries to keep frequently used data in higher levels; a cache miss cascades down the hierarchy, stalling the CPU until data arrives from the slower level.

**Figure 65 • FIG 3.1-B**

### SRAM vs DRAM — Two Technologies, Two Trade-offs

*Illustration: FIG 3.1-B | SRAM vs DRAM — Two Technologies, Two Trade-offs [SVG diagram — see module-3-1-memory.html]*

SRAM vs DRAM at the cell level. SRAM (left): six transistors form two cross-coupled inverters that latch a bit stably — no refresh needed, sub-nanosecond access, but 6× the silicon area. Used exclusively for CPU caches. DRAM (right): one transistor gates a capacitor that stores charge representing the bit — but capacitors leak, so every cell must be read and rewritten (refreshed) every 64ms. Far denser and cheaper than SRAM — a single DRAM chip holds billions of cells. The 50–100ns access time is why cache misses are so expensive: the CPU must wait 50–100 cycles for data from RAM.

**Figure 66 • FIG 3.1-C**

**Memory Address Space — 8-byte Array in RAM**

*Illustration: FIG 3.1-C | Memory Address Space — 8-byte Array in RAM [SVG diagram — see module-3-1-memory.html]*

Memory as an addressable array. Each cell holds exactly one byte (8 bits). Addresses are sequential integers starting at 0. The string "HELLO" is stored as five ASCII bytes (72, 69, 76, 76, 79) followed by a null terminator (0) at addresses 0x0000–0x0005. Reading any byte requires specifying its address — the memory controller puts the address on the address bus and the value appears on the data bus. A 16-bit address bus can access 65,536 unique bytes. Modern 64-bit CPUs theoretically address 16 exabytes, though only 48 bits are typically implemented (256TB).

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 3.1 Memory: How Machines Remember: Memory Array Cache Simulator Hierarchy Explorer [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 3.1 Memory: How Machines Remember **Type:** Calculation + trace + hands-on **Estimated time:** 4050 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Memory Hierarchy Calculations Use the latency table from the lesson. 1. A program runs a loop that accesses the **same 4 values** repeatedly. All 4 fit in L1 cache. If the loop runs 1,000,000 iterations and each iteration does 4 memory reads: - How many total memory reads occur? - Assuming all reads hit L1 cache (1 ns each), what is the total memory access time? 2. The same loop is rewritten to acc

# Module 3 2 Fetch Decode Execute

■# Module 3.2 — The Fetch-Decode-Execute Cycle

## The Problem

*You now have:*

*- An ALU that performs arithmetic and logic*

*- Memory that stores bytes at addresses*

*- Registers that hold values temporarily*

>

*The missing piece: what makes the CPU actually do something?*

>

*What starts the process? What decides which operation to perform next?*

*What keeps it going, instruction after instruction, without stopping?*

The answer is the **Fetch-Decode-Execute cycle** — the three-step loop that has driven every program ever run on every computer ever built.

---

## The Story

### The Problem of Sequential Control

Imagine you have a list of instructions written on paper:

Load the number at address 5 into Register A

Load the number at address 6 into Register B

Add Register A and Register B

Store the result at address 7

A human can read this list and execute it. But how does a machine do it?

The machine needs:

• A way to **remember where it is** in the instruction list (the Program Counter)

• A way to **read the next instruction** from memory (the Fetch step)

• A way to **understand what the instruction means** (the Decode step)

• A way to **carry out the instruction** (the Execute step)

• A way to **automatically move to the next instruction** (PC increment)

These five elements, working in a continuous loop, are the CPU.

---

### The Three Registers You Cannot Live Without

**Program Counter (PC)** — holds the address of the *next instruction* to fetch. After each fetch, it automatically increments. A jump instruction sets it to a new value, breaking the sequential flow.

**Instruction Register (IR)** — holds the *current instruction* being decoded and executed. The fetched instruction is copied here and stays until the next fetch.

**Memory Address Register (MAR)** — holds the address being read from or written to memory. When the CPU needs to access memory, it first loads the address here.

**Memory Data Register (MDR)** also called the Memory Buffer Register (MBR) — holds the data traveling to or from memory. Reads store their result here; writes take their data from here.

---

### The Cycle — Step by Step

**FETCH:**

Copy PC → MAR (tell memory which address to read)

Read memory[MAR] → MDR (get the instruction bytes)

Copy MDR → IR (load instruction into the instruction register)

Increment PC (point to the next instruction)

**DECODE:**

The control unit examines IR

It identifies the **opcode** (what operation to perform)

It identifies the **operands** (which registers, addresses, or immediate values to use)

It activates the correct control signals for the next phase

**EXECUTE:**

The identified operation is performed:

- ALU operation: route operands to ALU, store result - Memory read: MAR ← address, MDR ← memory[MAR], register ← MDR - Memory write: MAR ← address, MDR ← register, memory[MAR] ← MDR - Jump: PC ← new address

Return to FETCH.

The cycle repeats forever — or until the machine is powered off or halts.

---

### Why This Is Profound

Every program ever written — every web browser, every game, every operating system — runs on this three-step loop. The loop does not know about Python or JavaScript or databases. It knows only: fetch an instruction, decode it, execute it, repeat.

The sophistication of modern software exists entirely in how instructions are sequenced and what they instruct the hardware to do. The hardware itself is doing nothing more than this ancient, simple cycle.

---

### A Concrete Example — Tracing ADD

Suppose memory contains:

```
Address 0x00: 0x01  ← LOAD_A instruction (load next byte into Register A)
Address 0x01: 0x05  ← operand: value 5
Address 0x02: 0x02  ← LOAD_B instruction
Address 0x03: 0x03  ← operand: value 3
Address 0x04: 0x10  ← ADD instruction (A + B → A)
Address 0x05: 0x30  ← STORE instruction (store A at address 0x08)
Address 0x06: 0x08  ← operand: store address
```

PC starts at 0x00.

**Cycle 1 — LOAD_A 5:**

• Fetch: IR ← 0x01, PC becomes 0x01

• Decode: opcode 0x01 = LOAD_A; operand follows

• Execute: fetch operand (0x05=5), load 5 into Register A; PC becomes 0x02

**Cycle 2 — LOAD_B 3:**

• Fetch: IR ← 0x02, PC becomes 0x03

• Decode: opcode 0x02 = LOAD_B; operand follows

• Execute: fetch operand (0x03=3), load 3 into Register B; PC becomes 0x04

**Cycle 3 — ADD:**

• Fetch: IR ← 0x10, PC becomes 0x05

• Decode: opcode 0x10 = ADD; uses Reg A and Reg B

• Execute: ALU computes A + B = 8; result stored in Register A

**Cycle 4 — STORE:**

• Fetch: IR ← 0x30, PC becomes 0x06

• Decode: opcode 0x30 = STORE; address follows

• Execute: fetch address (0x08); write Register A (8) to memory[0x08]

After 4 cycles, memory[0x08] = 8. The computation 5 + 3 = 8 is complete.

---

### The Role of the Control Unit

The **Control Unit (CU)** is the conductor of this orchestra. It:

• Generates all timing and control signals

• Tells the ALU which operation to perform

• Opens and closes the gates that route data between registers and memory

• Manages the PC increment and jump logic

• Handles exceptions (invalid instruction, division by zero, memory fault)

The CU does not compute. It directs the components that do.

Two implementations: **hardwired control** (logic gates implementing the state machine — fast, inflexible) and **microprogrammed control** (a small internal ROM of microinstructions — slower, but easy to modify and extend).

---

### Clock Speed and IPC

The FDE cycle is driven by a clock. Modern CPUs run at 3–6 GHz — 3 to 6 billion clock cycles per second. But a single instruction often takes multiple cycles (memory accesses are slow). The key performance metric is not just clock speed but **IPC — Instructions Per Clock**.

Modern superscalar CPUs execute multiple instructions per clock using **pipelining**: while one instruction is in the Execute stage, the next is in Decode, and the one after is being Fetched — simultaneously. A 4-stage pipeline theoretically achieves 4× throughput.

## Illustrations

**Figure 67 • FIG 3.2-A**

### Fetch–Decode–Execute — The Three-Step Cycle That Runs Every Program

*Illustration: FIG 3.2-A | Fetch–Decode–Execute — The Three-Step Cycle That Runs Every Program [SVG diagram — see module-3-2-fde.html]*

The Fetch-Decode-Execute cycle — the fundamental operation of every processor ever built. The Program Counter (PC, red) holds the address of the next instruction. Fetch: the CPU reads the byte(s) at address PC into the Instruction Register (IR, gold) and increments PC. Decode: the control unit examines the IR opcode bits to determine what operation to perform and which registers are involved. Execute: the ALU performs the computation (here ADD R1, R2 adds R1's value to R2's value) and writes the result to the destination register. The cycle repeats immediately — billions of times per second in

**Figure 68 • FIG 3.2-B**

### PC and IR State Trace — Five Instructions Step by Step

*Illustration: FIG 3.2-B | PC and IR State Trace — Five Instructions Step by Step [SVG diagram — see module-3-2-fde.html]*

A five-instruction state trace. Cycles 1–4 execute sequentially: PC increments by 1 after each fetch. Cycle 5 (JMP 0x0000) overwrites PC with 0x0000 instead of incrementing it — this is how conditional branches, function calls, and loops are implemented at the machine level. Every high-level control flow construct (if, while, for, return) compiles down to a variation of the JMP instruction altering the PC.

**Figure 69 • FIG 3.2-C**

### FDE Trace — Executing ADD R1, R2 Step by Step

*Illustration: FIG 3.2-C | FDE Trace — Executing ADD R1, R2 Step by Step [SVG diagram — see module-3-2-fde.html]*

Full trace of the Fetch-Decode-Execute cycle for a single ADD R1, R2 instruction. Fetch: the Program Counter (0x100) addresses memory, loads the encoded instruction into the Instruction Register, and advances to 0x101. Decode: the control unit splits the IR into opcode (ADD), destination register (R1), and source register (R2). Execute: both register values feed the ALU, which adds them and writes the result (0x08) back to R1. Every program ever executed on a stored-program computer is simply this three-step loop, repeated billions of times per second.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 3.2 The Fetch-Decode-Execute Cycle: Single-Step CPU Program Editor Pipeline Visualiser [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 3.2 The Fetch-Decode-Execute Cycle **Type:** Trace + reasoning + design **Estimated time:** 4555 minutes **Tools needed:** `simulation.html` + pen and paper --- Setup The Toy CPU This challenge uses a toy CPU with: - 4 general-purpose registers: **A, B, C, D** (each 8 bits, initially 0) - A **Program Counter (PC)** (initially 0x00) - 16 bytes of memory (addresses 0x000x0F) - A simple instruction set: | Opcode | Mnemonic | Description | |--------|----------|-------------| | 0x01 | LOAD_A imm | Load immediate value into A | | 0x02 | LOAD_B imm | Load immediate value into B |

# Module 3 3 Isa

■# Module 3.3 — Instruction Set Architecture (ISA)

## The Problem

*The ALU can add, subtract, AND, OR, XOR.*

*The FDE cycle can fetch and execute instructions.*

*Memory can store and retrieve bytes.*

>

*But how does the CPU know what to do with a byte like 0x83?*

*How does it distinguish "add these two registers" from "load this value" from "jump to that address"?*

>

*The answer is the **Instruction Set Architecture** — the contract between hardware and software that defines every operation a CPU can perform.*

The ISA is the most important design decision in computer architecture. It is the interface that software speaks and hardware implements. Every program ever compiled targets a specific ISA. Change the ISA and every program must be recompiled or rewritten.

## The Story

### The First ISAs

Early computers had no standard instruction set. Each machine had its own unique set of operations, wired directly into its control unit. Writing a program for one machine meant it ran on nothing else.

Moving programs between machines meant rewriting them from scratch.

The **IBM System/360** (1964) was the first family of computers to share a common ISA across machines of different speeds and prices. A program written for the cheapest System/360 would run unchanged on the most powerful one. This was revolutionary: **software became an asset that outlasted hardware**.

The System/360 ISA introduced concepts still present in x86-64 today.

### What an ISA Defines

An ISA specifies:

**Registers** — how many, how wide, which are special

**Data types** — integers (8/16/32/64-bit), floats, vectors

**Instructions** — every operation the CPU supports

**Addressing modes** — how operands are located (immediate, register, memory, indirect)

**Instruction encoding** — how instructions are represented as bits

**Memory model** — byte order (endianness), alignment rules

**Privilege levels** — which instructions only the OS can use

**Exception model** — what happens on errors

### RISC vs CISC

Two competing philosophies dominate ISA design:

**CISC — Complex Instruction Set Computer**

• Many instructions, some very complex (string copy, matrix operations)

• Instructions of variable length (1–15 bytes on x86)

• Single instruction can do a memory read, an operation, and a memory write

• Fewer instructions needed in programs → smaller code size

• Complex decoder hardware needed

• Examples: Intel x86-64, AMD64

**RISC — Reduced Instruction Set Computer**

• Few, simple instructions — each does exactly one thing

• Fixed instruction length (typically 32 bits)

• Memory access only via explicit load/store instructions

• More instructions needed for complex operations → larger programs

• Simple decoder → easier to pipeline, easier to optimise

• Examples: RISC-V, ARM (modern), MIPS, SPARC

**The modern reality**: ARM processors (RISC) run almost all smartphones and tablets. x86-64 (CISC) runs almost all desktop/laptop/server computing. Apple's M-series chips (ARM RISC) now run high-performance Macs, challenging x86-64 dominance. RISC-V is an open-source ISA gaining traction in embedded and research systems.

*Interesting nuance*: modern x86-64 CPUs internally translate CISC instructions into simpler micro-operations (μops) that look a lot like RISC. The CISC/RISC distinction has blurred at the implementation level, even if the ISA surface remains different.

---

### Addressing Modes

How does an instruction specify where its data comes from? **Addressing modes** are the answer.

**Immediate**: the value is encoded directly in the instruction. `LOAD A, 42` — loads the literal value 42 into register A.

**Register**: the value is in a register. `ADD A, B` — adds the value in register B to register A.

**Direct (absolute)**: the value is at a specific memory address. `LOAD A, [0x1000]` — loads the byte at memory address 0x1000 into A.

**Indirect**: the address of the value is in a register. `LOAD A, [B]` — loads the byte from the address stored in register B.

**Base + offset**: address = register + constant. `LOAD A, [B + 4]` — loads from the address (B's value + 4). Used for array indexing and struct field access.

**PC-relative**: address = PC + offset. Used for branch instructions. `JMP +8` — jump forward 8 bytes from the current PC. Produces position-independent code.

---

### Instruction Encoding

Every instruction must be represented as a pattern of bits. Two main approaches:

**Fixed-length** (RISC): every instruction is exactly 32 bits.

```
RISC-V R-type (32-bit):
[31:25] funct7  [24:20] rs2  [19:15] rs1  [14:12] funct3  [11:7] rd  [6:0] opcode
```

Simple decoder: always knows exactly where each field is.

**Variable-length** (CISC): instructions range from 1 to 15 bytes.

```
x86-64 examples:
NOP:           0x90           (1 byte)
INC  eax:      0xFF 0xC0      (2 bytes)
MOV  rax, imm: 0x48 0xB8 + 8-byte immediate  (10 bytes)
```

Complex decoder: must figure out instruction length before it can decode it. One reason x86 decoders are expensive in silicon area.

---

### Our Toy ISA — 8 Instructions

For this module, we design and use a minimal ISA:

| Opcode (hex) | Mnemonic | Format | Description | |---|---|---|---| | 0x01 | LOAD imm | 0x01 rr vv | Load immediate value vv into register rr | | 0x02 | LOAD_MEM | 0x02 rr aa | Load from memory[aa] into register rr | | 0x03 | STORE | 0x03 rr aa | Store register rr to memory[aa] | | 0x10 | ADD | 0x10 r1 r2 | r1 ← r1 + r2 | | 0x11 | SUB | 0x11 r1 r2 | r1 ← r1 – r2 | | 0x20 | JMP | 0x20 aa | PC ← aa (unconditional) | | 0x21 | JZ | 0x21 rr aa | If rr=0: PC ← aa | | 0xFF | HALT | 0xFF | Stop |

Register encoding: A=0x00, B=0x01, C=0x02, D=0x03

**Example — sum of 1+2+3+4+5:**

```
Address: Bytes:   Meaning:
0x00:    01 00 01  LOAD A, 1        ; accumulator = 1
0x03:    01 01 06  LOAD B, 6        ; loop limit (stop before adding 6)
0x06:    01 02 02  LOAD C, 2        ; next value to add
0x09:    11 01 02  SUB B, C         ; B =
```

## Illustrations

**Figure 70 • FIG 3.3-A**

### Instruction Encoding — How an Instruction Becomes Bits

*Illustration: FIG 3.3-A | Instruction Encoding — How an Instruction Becomes Bits [SVG diagram — see module-3-3-isa.html]*

MIPS-style 32-bit instruction encoding for ADD R1, R2, R3. The instruction word is divided into fields: opcode (6 bits, identifies instruction type), RS and RT (source registers, 5 bits each — can address 32 registers), RD (destination register), SHAMT (shift amount for shift instructions, zero here), and FUNCT (refinement of opcode for R-type instructions). The entire instruction fits in one 32-bit word = 0x00422020 = four bytes in memory. The three instruction formats (R, I, J) cover all MIPS instructions — the simplicity is intentional and is the defining characteristic of RISC design.

**Figure 71 • FIG 3.3-B**

### RISC vs CISC — Two Design Philosophies

*Illustration: FIG 3.3-B | RISC vs CISC — Two Design Philosophies [SVG diagram — see module-3-3-isa.html]*

RISC vs CISC design philosophies. CISC (x86-64, left): a single MOVSX instruction computes an address with a scaled index and offset, reads a 16-bit value, and sign-extends it — four logical operations in one binary instruction. RISC (ARM, right): the same operation requires three instructions, but each is exactly 4 bytes and does exactly one thing. RISC designs are easier to decode in parallel and pipeline. Modern x86 CPUs resolve the debate pragmatically — the frontend decodes complex x86 instructions into simple micro-operations (μops) before issuing to an internal RISC-like execution engin

**Figure 72 • FIG 3.3-C**

**RISC vs CISC — Two Philosophies for Designing an ISA**

*Illustration: FIG 3.3-C | RISC vs CISC — Two Philosophies for Designing an ISA [SVG diagram — see module-3-3-isa.html]*

The RISC-CISC design spectrum. RISC (ARM, RISC-V) uses uniform-width, single-cycle instructions — simple to pipeline and decode but requires more instructions per task. CISC (x86) provides powerful multi-cycle instructions — compact programs but complex decoders. The apparent conflict resolved itself in practice: modern x86 processors internally translate variable-length CISC instructions into fixed-length micro-operations and execute them on a RISC-style pipeline. The ISA is CISC; the microarchitecture is RISC.

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 3.3 Instruction Set Architecture: Toy ISA CPU Assembler Sum 15 Challenge [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 3.3 Instruction Set Architecture **Type:** Programming + encoding + design + analysis **Estimated time:** 5060 minutes **Tools needed:** `simulation.html` + pen and paper --- Setup The Toy ISA (same as Module 3.2) | Opcode | Mnemonic | Format | Description | |--------|----------|--------|-------------| | 0x01 | LOAD | 01 rr vv | Load immediate vv into register rr | | 0x02 | LOAD_MEM | 02 rr aa | Load memory[aa] into register rr | | 0x03 | STORE | 03 rr aa | Store register rr to memory[aa] | | 0x10 | ADD | 10 r1 r2 | r1 r1 + r2 | | 0x11 | SUB | 11 r1 r2 | r1 r1 r2 | | 0x2

# Module 3 4 Io

■# Module 3.4 — Input/Output: Connecting to the World

## The Problem

*You have a CPU that fetches, decodes, and executes instructions.*

*You have memory that stores programs and data.*

>

*But a computer that can only read from and write to its own memory is useless.*

*How does it accept input from a keyboard?*

*How does it send output to a screen?*

*How does it read a file from disk?*

>

*The answer is **Input/Output (I/O)** — the mechanisms by which a CPU communicates with the world outside its own chip.*

Without I/O, a computer is a sealed box performing calculations no one can see and responding to nothing. I/O is what connects the silicon to the world.

---

## The Story

### The First I/O: Punch Cards and Lights

The earliest computers had no keyboard, no screen. Programs were loaded via **punched cards** fed into a card reader — a mechanical device that detected holes and converted them to electrical signals. Output was printed on paper or displayed as patterns of lights on a front panel.

The operator's interaction with the computer was entirely through these physical media. A program could take minutes to load and hours to run. Output was not real-time — it was retrieved after the job completed.

The concept of **interactive computing** — where a human types and the computer responds immediately — did not exist until the 1960s. The development of time-sharing operating systems and terminals made it possible. This was considered revolutionary.

---

### Three I/O Methods

#### Method 1 — Programmed I/O (Polling)

The CPU directly controls the I/O device by reading and writing specific addresses. To check if a key has been pressed, the CPU reads a "status register" address and tests a bit. If not ready, it reads it again. And again. And again.

```
loop:
  READ status_register → A
  AND A, 0x01          ; test the "data ready" bit
  JZ loop              ; if not ready, keep polling
  READ data_register → B  ; now read the actual data
```

This is called **polling** or **busy-waiting**. It works, but the CPU wastes 100% of its time checking — it can do nothing else while waiting for I/O.

Analogy: standing at the front door staring at the letterbox, checking every second if a letter has arrived.

#### Method 2 — Interrupts

The CPU continues executing its program normally. When an I/O device has data ready, it sends an **interrupt signal** to the CPU.

The CPU:

Finishes the current instruction

Saves the current PC and registers to the stack

Jumps to an **Interrupt Service Routine (ISR)** — a special function that handles the event

The ISR reads the data from the I/O device

The ISR restores registers and returns to the interrupted program

The CPU can do useful work between I/O events. The keyboard only interrupts when a key is actually pressed — perhaps 10 times per second, consuming a tiny fraction of CPU time.

Analogy: put a doorbell on the front door. You do your work. The doorbell rings when a letter arrives.

**Method 3 — Direct Memory Access (DMA)**

For high-bandwidth I/O (disk, network, video), even interrupt-driven I/O is too slow — the CPU would spend all its time copying bytes from the I/O device into memory.

DMA gives the I/O device direct access to memory **without involving the CPU**:

CPU programs the DMA controller: "copy 4096 bytes from disk to address 0x8000"

CPU continues executing other code

DMA controller handles all the data transfer autonomously

When done, DMA controller interrupts the CPU

CPU reads the data at 0x8000 — already there

Modern SSDs, GPUs, and network cards all use DMA. Without it, a CPU reading a 1 GB file would spend nearly all its time doing memory copies instead of useful computation.

---

### Memory-Mapped I/O

How does the CPU "talk to" an I/O device? There are two approaches:

**Port-mapped I/O**: separate IN/OUT instructions with a separate I/O address space (x86 uses this historically).

```
IN  al, 0x60   ; read from keyboard port 0x60
OUT 0x61, al   ; write to speaker port 0x61
```

**Memory-mapped I/O (MMIO)**: I/O devices appear at specific memory addresses. Reading from or writing to those addresses communicates with the device — no special instructions needed.

```
Address 0xFFFF0000: keyboard status register
Address 0xFFFF0001: keyboard data register
Address 0xFFFF0010: screen framebuffer start
```

When the CPU writes to 0xFFFF0010, the value doesn't go to RAM — it goes to the video controller, which displays it on screen.

**Modern practice**: almost all modern systems use MMIO. ARM processors use MMIO exclusively. Memory-mapped I/O makes device drivers simpler — they just use regular memory read/write operations.

---

### The Journey of a Keystroke

Let's trace what happens when you press the 'A' key:

**Physical**: the key closes a circuit, sending a voltage signal

**Keyboard controller**: detects the closed circuit, generates a scancode (0x1C for 'A' on a standard layout)

**Keyboard buffer**: stores the scancode in a small internal buffer

**Hardware interrupt**: keyboard controller asserts the IRQ1 interrupt line

**CPU**: finishes current instruction, saves state, jumps to keyboard ISR

**ISR**: reads the scancode from the keyboard data port, translates to ASCII ('A' = 0x41), stores in OS keyboard buffer

**ISR return**: CPU restores state, continues previous program

**Application**: calls `read()` system call, OS returns 'A' from the keyboard buffer

**Terminal/shell**: receives 'A', echoes it to screen, adds to input line

This entire chain — from key press to character appearing on screen — takes approximately 1–10 milliseconds. Most of that time is in the OS and display pipeline. The hardware interrupt itself is handled in microseconds.

### From Punch Cards to USB

**Punch cards (1890s–1970s)**: data encoded as holes; mechanical reader converts holes to signals; output via line printer **Serial terminals (1960s–1980s)**: RS-232 serial connection; te

### Illustrations

**Figure 73 • FIG 3.4-A**

**Memory-Mapped I/O — Peripherals as Memory Addresses**

*Illustration: FIG 3.4-A | Memory-Mapped I/O — Peripherals as Memory Addresses [SVG diagram — see module-3-4-io.html]*

Memory-mapped I/O maps peripheral hardware registers into the CPU's address space alongside RAM. Addresses 0x0000–0x7FFF are normal RAM. Addresses 0x8000–0x8005 map to hardware registers: reading 0x8000 returns the last keypress; writing to 0x8002 sets a display pixel. The CPU uses the exact same MOV instructions for both — no special I/O instructions needed. The memory controller routes the access to the appropriate hardware based on the address range. This is how virtually all modern embedded and desktop systems handle peripherals.

**Figure 74 • FIG 3.4-B**

### Polling vs Interrupts — Two Ways to Handle I/O Events

*Illustration: FIG 3.4-B | Polling vs Interrupts — Two Ways to Handle I/O Events [SVG diagram — see module-3-4-io.html]*

Polling vs interrupt-driven I/O. Polling (left): the CPU checks a status register on every loop iteration, wasting cycles even when no input is available. Suitable only for very simple embedded systems where the CPU has nothing better to do. Interrupts (right): the peripheral asserts an IRQ (Interrupt Request) line when it has data. The CPU hardware pauses the current instruction stream, saves state on the stack, and jumps to the Interrupt Service Routine (ISR). After the ISR completes, execution resumes exactly where it was interrupted. Modern OSs handle keyboard, network, disk, and timer eve

**Figure 75 • FIG 3.4-C**

### Polling vs Interrupts — CPU Utilisation Under I/O Wait

*Illustration: FIG 3.4-C | Polling vs Interrupts — CPU Utilisation Under I/O Wait [SVG diagram — see module-3-4-io.html]*

CPU timeline comparison between polling and interrupt-driven I/O. With polling, the CPU repeatedly checks a status register — burning cycles even when the device has nothing to report. With interrupts, the CPU runs uninterrupted until the I/O controller raises a hardware signal (IRQ), triggering the Interrupt Service Routine (ISR). After the ISR completes, the CPU resumes exactly where it left off. Modern operating systems use interrupts for virtually all I/O: keyboards, network cards, disk controllers, and timers all signal the CPU rather than requiring it to ask.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 3.4 Input/Output: Connecting to the World: Polling vs Interrupts DMA Transfer MMIO Memory Map [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 3.4 Input/Output: Connecting to the World **Type:** Calculation + trace + code + reasoning **Estimated time:** 4050 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Polling vs Interrupts: Cost Analysis A CPU runs at 1 GHz (1 cycle = 1 ns). A keyboard generates keypresses at an average rate of **10 per second** (once every 100 ms). The polling loop checks the keyboard status register every **1,000 cycles** (1 s). 1. How many times does the CPU check the status register per second? 2. How many of those checks actually find a keypress waiting? (Assume a

# Module 3 5 Bus

■# Module 3.5 — The Bus: Moving Data Around

## The Problem

*The CPU computes. Memory stores. I/O devices communicate.*

>

*But how do these components actually talk to each other?*

*The CPU is a chip on a motherboard. Memory is another chip. The GPU is a third.*

>

*They cannot communicate by thought. They need a shared pathway — a physical medium, a protocol, and a set of rules for who talks when.*

>

*That pathway is the **bus**.*

A bus is a shared communication channel — a set of wires connecting multiple components, plus the protocol that governs who uses those wires and when.

## The Story

### The Early Bus: Everything Shared One Wire

The earliest computers wired components point-to-point: the CPU had a dedicated wire to memory, another to the printer, another to the card reader. Every new device required new wires and new control logic. Adding a device to a computer was a hardware engineering project.

The invention of the **shared bus** — a common set of wires that all components connect to — was transformative. Now adding a device meant plugging it into the existing bus. The protocol managed who could talk and when.

The **Intel 8086 (1978)** used a 16-bit data bus and 20-bit address bus. The **IBM PC (1981)** standardised the ISA bus (not to be confused with Instruction Set Architecture — the same acronym used for different purposes). **PCI (1992)** replaced ISA. **PCIe (2003)** replaced PCI with a point-to-point switched fabric that looks like a bus to software but is physically very different.

### The Three Sub-Buses

A classical system bus consists of three logically separate groups of signals:

**Address Bus**

• Carries the address of the memory location or I/O device being accessed

• Unidirectional: CPU → memory/devices

• Width determines addressable space: 32-bit → 4 GB, 64-bit → 16 EB

• Example: CPU places 0x0000_1000 on address bus → selects address 4096

**Data Bus**

• Carries the actual data being read or written

• Bidirectional: CPU ↔ memory/devices

• Width determines how much data moves per transfer: 8, 16, 32, or 64 bits

• A 64-bit data bus moves 8 bytes per bus cycle

**Control Bus**

• Carries control signals that coordinate the operation

• Signals include:

- `READ` / `WRITE` — direction of transfer - `MEMORY_SELECT` / `IO_SELECT` — is this a memory or I/O access? - `CLOCK` — timing signal - `INTERRUPT_REQUEST` — device requests CPU attention - `BUS_GRANT` / `BUS_REQUEST` — bus arbitration - `RESET` — system reset

All three buses operate simultaneously, coordinated by the clock.

---

## Bus Arbitration

A bus is a **shared resource**. Only one device can use it at a time. When multiple components want to use the bus simultaneously, someone must decide who goes first. This is **bus arbitration**.

**Daisy-chain arbitration**: devices are chained in priority order. A bus grant signal passes down the chain — the first device in the chain that wants the bus takes it. Simple but unfair to low-priority devices.

**Centralised arbitration**: a dedicated **bus arbiter** receives requests from all devices and grants access based on a scheduling policy (round-robin, priority, etc.). Fairer but requires extra hardware.

**Distributed arbitration**: devices use a shared wire and a collision-detection protocol (similar to early Ethernet). Each device can "bid" for the bus simultaneously; the highest-priority wins based on wire-OR logic.

Modern systems like PCIe avoid shared-bus arbitration entirely by using **point-to-point switched connections** — each device has its own dedicated link to a switch (like Ethernet vs a hub). But the software model still looks like a bus.

---

## Von Neumann vs Harvard Architecture

**Von Neumann architecture**: instructions and data share the same memory and the same bus. The CPU fetches an instruction, then fetches data — both using the same path. This creates the **Von Neumann bottleneck**: the bus bandwidth limits performance because the same resource is contended for both code and data.

Almost all general-purpose computers (x86, ARM application cores) use Von Neumann architecture.

**Harvard architecture**: separate memory and separate buses for instructions and data. The CPU can fetch an instruction and read data simultaneously — no contention. Simpler to implement but more expensive (two separate memory systems).

Used in: microcontrollers (PIC, AVR), DSPs, many embedded systems.

**Modified Harvard**: use separate L1 instruction cache and L1 data cache (effectively separate buses at the cache level), but share memory at L2 and beyond. This gives most of Harvard's performance benefit while keeping one unified address space. Used by: modern x86-64, ARM Cortex-A. This is why L1 is listed as "L1i" (instruction) and "L1d" (data) in CPU specs.

---

### Bus Bandwidth and Latency

**Bandwidth**: how much data can flow per second.

```
Bandwidth = (bus width in bytes) × (bus clock frequency)
Example: 64-bit bus at 100 MHz = 8 bytes × 100 MHz = 800 MB/s
DDR4-3200: 64-bit × 1,600 MHz (double-pumped) × 2 channels = 51.2 GB/s
```

**Latency**: how long until the first data byte arrives after a request.

```
Latency = (bus request overhead) + (memory access time)
Typical DRAM latency: 60–80 ns regardless of bus bandwidth
```

Bandwidth and latency are independent. A high-bandwidth bus can still have high latency. Latency dominates for small random accesses; bandwidth dominates for large sequential transfers.

**Bus width vs bus speed**: doubling bus width doubles bandwidth but requires more pins/wires (expensive). Doubling clock speed doubles bandwidth but increases power and electromagnetic interference. Real systems balance both.

---

### PCIe: The Modern "Bus"

PCIe (Peripheral Component Interconnect Express) is the dominant high-speed interconnect in modern PCs. Technically it is not a shared bus — it is a **switched point-to-point serial fabric**.

• Each device has a **dedicated serial link** (lane) to the P

### Illustrations

**Figure 76 • FIG 3.5-A**

### The System Bus — Address, Data, and Control Lines

*Illustration: FIG 3.5-A | The System Bus — Address, Data, and Control Lines [SVG diagram — see module-3-5-bus.html]*

The system bus connects CPU, RAM, I/O controllers, and ROM through three parallel wire groups. The address bus (red, top) carries the target memory address from the CPU to all components — only the component whose address range matches responds. The data bus (indigo, middle) carries the actual data value in either direction. The control bus (gold, bottom) carries read/write signals, clock, interrupt lines, and bus arbitration signals. A single memory read requires four steps: put address on address bus, assert READ on control bus, the target component puts data on the data bus, CPU reads the d

**Figure 77 • FIG 3.5-B**

### Von Neumann vs Harvard Architecture

*Illustration: FIG 3.5-B | Von Neumann vs Harvard Architecture [SVG diagram — see module-3-5-bus.html]*

Von Neumann (left): instructions and data share a single memory and a single bus. Simple and flexible — the program can modify itself or use memory for either instructions or data. The bottleneck: fetching the next instruction and loading a data operand require two sequential bus cycles. Harvard (right): separate instruction and data memories with separate buses. The CPU can fetch the next instruction and access a data operand simultaneously — removing the Von Neumann bottleneck. Used in microcontrollers (AVR, PIC), DSPs, and GPU shader processors where deterministic throughput matters. Modern

**Figure 78 • FIG 3.5-C**

### Von Neumann vs Harvard Architecture — One Bus vs Two

*Illustration: FIG 3.5-C | Von Neumann vs Harvard Architecture — One Bus vs Two [SVG diagram — see module-3-5-bus.html]*

Von Neumann architecture (left) places all code and data in a single unified memory, accessed over one shared bus — creating the Von Neumann bottleneck where instruction fetching and data access compete. Harvard architecture (right) uses physically separate buses for instructions and data, allowing the CPU to fetch the next instruction while simultaneously reading or writing data. Modern CPUs use a hybrid: a unified DRAM but separate on-chip instruction and data caches — giving Harvard-like performance with Von Neumann-like programming simplicity.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 3.5 The Bus: Moving Data Around: Bus Transaction Animator Arbitration Von Neumann vs Harvard [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 3.5 The Bus: Moving Data Around **Type:** Calculation + trace + design **Estimated time:** 4050 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Bus Bandwidth Calculations 1. A system has a **32-bit data bus** running at **66 MHz**. What is the theoretical peak bandwidth in MB/s? 2. A **64-bit data bus** at **133 MHz** (double data rate transfers on both clock edges): what is the peak bandwidth in MB/s? In GB/s? 3. DDR4-3200 RAM has a 64-bit bus running at 1,600 MHz (double data rate = 3,200 MT/s effective). Calculate the bandwidth in GB/s for a sin

LEVEL 04

# Software Foundations

From Machine Code to Operating Systems

# Module 4 1 Machine Code

■# Module 4.1 — Machine Code and Assembly Language

## The Problem

*You can now build a CPU and execute instructions from memory.*

*The CPU understands binary — 0x01 0x05 means "load the value 5 into register A."*

>

*But nobody writes programs in hex bytes.*

>

*How did programmers get from "a CPU that reads binary" to "a Python script you can write in plain English"?*

>

*The first step up from raw binary is **assembly language** — and the tool that bridges them is the **assembler**.*

Every high-level program you will ever write sits atop a chain of translations. Understanding the first link in that chain — from human-readable mnemonics to machine binary — is essential for understanding everything above it.

---

## The Story

### The First Programmers Had No Language

ENIAC (1945) was programmed by physically rewiring the machine. There were no instructions in memory — the circuit topology *was* the program. Changing the program meant changing the hardware.

The stored-program computer (1948–1949) was the breakthrough: programs could be loaded into memory as data. But the first stored programs were still written as raw binary numbers. Programmers kept notebooks mapping opcodes to numbers. A single typo in a binary string produced a program that crashed silently or produced wrong results with no indication of where.

**Kathleen Booth** (1947) and **David Wheeler** (1949) at Cambridge are among the earliest people credited with writing what we would today recognise as assembly language — symbolic mnemonics like `ADD` and `JMP` that a secondary program (the assembler) would translate into binary.

The insight was simple but profound: **let the computer do the translation**. Write in symbols; let a program convert them to numbers.

---

### The Compilation Pipeline

Modern programs go through multiple translation stages before the CPU can execute them:

```
  Source code (.c, .py, .rs)
```

```
       ↓  Preprocessor
Preprocessed source
       ↓  Compiler
Assembly code (.s / .asm)
       ↓  Assembler
Object code (.o / .obj)  ← machine code + relocation metadata
       ↓  Linker
Executable (.exe / ELF binary)  ← ready to load
       ↓  Loader (OS)
Process in memory  ← CPU executes this
```

**Preprocessor** — expands macros, includes header files, strips comments **Compiler** — translates high-level code to assembly (register allocation, optimisation) **Assembler** — translates assembly mnemonics to binary machine code **Linker** — combines multiple object files, resolves symbol references, produces one executable **Loader** — the OS reads the executable, allocates memory, copies segments into place, sets PC to entry point

---

### What Machine Code Looks Like

x86-64 machine code is variable-length. Here is the same simple computation in three representations:

```
// C source
int x = 5;
int y = 3;
int z = x + y;
```

```
; x86-64 Assembly (AT&T syntax)
movl  $5, -4(%rbp)      ; x = 5  (store 5 at stack slot)
movl  $3, -8(%rbp)      ; y = 3
movl  -4(%rbp), %eax    ; load x into eax
addl  -8(%rbp), %eax    ; eax = eax + y
movl  %eax, -12(%rbp)   ; z = eax
```

```
; Machine code (hex bytes)
C7 45 FC 05 00 00 00    ; movl $5, -4(%rbp)
C7 45 F8 03 00 00 00    ; movl $3, -8(%rbp)
8B 45 FC                ; movl -4(%rbp), %eax
03 45 F8                ; addl -8(%rbp), %eax
89 45 F4                ; movl %eax, -12(%rbp)
```

The CPU executes the bottom representation. The middle is what a human can read. The top is what an application programmer writes.

---

### Anatomy of an Assembly Instruction

```
label:    mnemonic    operand1, operand2    ; comment
```

**Label** — optional name for this address. Used as jump targets and function names. **Mnemonic** — the instruction name: MOV, ADD, SUB, JMP, CALL, RET, etc. **Operands** — what the instruction operates on: - Immediate: $5 (literal value) - Register: `%rax`, `%rbx` (CPU register) - Memory: `-4(%rbp)` (address = rbp – 4)

**Two syntax flavours:**

• **Intel syntax** (Windows, NASM): `mov rax, 5` — destination first

• **AT&T syntax** (Linux, GAS): `movq $5, %rax` — source first, register prefixed with %

### Real x86-64 Registers

| Register | 64-bit | 32-bit | Purpose | |---|---|---|---| | Accumulator | `rax` | `eax` | Return values, arithmetic | | Base | `rbx` | `ebx` | General purpose | | Counter | `rcx` | `ecx` | Loop counter | | Data | `rdx` | `edx` | I/O, multiply/divide | | Stack Pointer | `rsp` | `esp` | Top of stack | | Base Pointer | `rbp` | `ebp` | Stack frame base | | Instruction Pointer | `rip` | `eip` | Current instruction address | | Source Index | `rsi` | `esi` | String source | | Dest Index | `rdi` | `edi` | String dest, arg 1 (Linux ABI) | | R8–R15 | — | — | General purpose (64-bit only) |

### Hello World in Assembly (x86-64 Linux)

```
; hello.asm — NASM syntax, x86-64 Linux
section .data
    msg db "Hello, World!", 10   ; 10 = newline character
    len equ $ - msg              ; length = current position minus msg label

section .text
    global _start

_start:
    ; write(1, msg, len)  — syscall number 1 = write
    mov rax, 1          ; syscall: write
    mov rdi, 1          ; fd: stdout (1)
    mov rsi, msg        ; pointer to message
    mov rdx, len        ; length of message
    syscall             ; invoke kernel

    ; exit(0)  — syscall number 60 = exit
    mov rax, 60         ; syscall: exit
    mov rdi, 0          ; exit code: 0 (success)
    syscall
```

This is the bedrock. Every `print("Hello")` in Python eventually produces a sequence of operations equivalent to this — layers of abstraction translate the readable to the executable.

### The Assembler's Job

An assembler makes two passes through the source:

**Pass 1 — Symbol table construction:**

• Record the address of every label

• Compute the size of every instruction

• Build a table: `{label → address}`

**Pass 2 — Code generation:**

• For each instruction, look up the opcode encoding

• Substitute label references wi

## Illustrations

**Figure 79 • FIG 4.1-A**

### The Compilation Pipeline — Source Code to Running Program

*Illustration: FIG 4.1-A | The Compilation Pipeline — Source Code to Running Program [SVG diagram — see module-4-1-assembly.html]*

The complete compilation pipeline for a C program. The compiler (gcc -S) translates C source into x86-64 assembly mnemonics. The assembler (as) converts each mnemonic into its byte encoding — PUSH RBP becomes 0x55, MOV RBP,RSP becomes 0x48 0x89 0xE5. The linker (ld) combines object files with library stubs, resolves symbol references, and produces an ELF executable with sections for code (.text), initialised data (.data), read-only data (.rodata), and others. The OS exec() call loads the executable into virtual memory and creates the process. The stack and heap grow dynamically at runtime.

**Figure 80 • FIG 4.1-B**

### Assembly Instruction Anatomy — Opcode, Operands, Addressing Modes

*Illustration: FIG 4.1-B | Assembly Instruction Anatomy — Opcode, Operands, Addressing Modes [SVG diagram — see module-4-1-assembly.html]*

Assembly instruction structure. Every instruction has an opcode (operation code) and zero or more operands. MOV EAX, 42 loads the value 42 into register EAX using immediate addressing — the value is embedded in the instruction bytes (0xB8 0x2A 0x00 0x00 0x00). ADD EAX, EBX uses register-register addressing — fastest possible, no memory access needed. MOV EAX, [RBX + RCX*4 + 8] uses the most complex x86-64 addressing mode: base register + index register × scale factor + displacement, all in one instruction. Assembly is the lowest readable representation before machine code bytes.

**Figure 81 • FIG 4.1-C**

### Compilation Pipeline — From C Source to Running Machine Code

*Illustration: FIG 4.1-C | Compilation Pipeline — From C Source to Running Machine Code [SVG diagram — see module-4-1-assembly.html]*

The four-stage C compilation pipeline. The preprocessor (cpp) expands #include directives and macros, producing an intermediate .i file. The compiler (cc1) translates to human-readable assembly (.s). The assembler (as) encodes assembly mnemonics as binary machine code, producing a relocatable object file (.o) with unresolved symbol references (marked ?? for the printf call address). The linker (ld) resolves all symbol references by combining object files with library code, filling in the correct addresses and producing the final self-contained executable.

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 4.1 Machine Code and Assembly Language: Assembler Compilation Pipeline Disassembler [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 4.1 Machine Code and Assembly Language **Type:** Translation + encoding + trace + reasoning **Estimated time:** 4555 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A C to Assembly Translate each C snippet into assembly using the toy ISA from Level 3 (or simplified x86-like pseudocode). Show register assignments and every instruction required. 1. `int x = 10; int y = 4; int z = x - y;` Translate to assembly. Where is z stored? 2. `if (a > b) { result = a; } else { result = b; }` Translate to assembly. Use CMP and conditional jumps. (This is `max(a,

# Module 4 2 Stack Calling

■# Module 4.2 — The Stack and Calling Conventions

## The Problem

*You can write assembly instructions.*

*You know how to jump to a label.*

>

*But how do functions work at the machine level?*

*How does a function receive its arguments?*

*How does it return a value?*

*How does it return to exactly where it was called from — even when functions call other functions?*

>

*The answer is the **stack** — and the rules that govern its use are the **calling convention**.*

Without the stack, you cannot have functions. Without functions, you cannot have programs longer than a few dozen lines. The stack is the structural foundation on which all procedural programming rests.

## The Story

### The Problem of Return Addresses

Suppose function A calls function B, which calls function C. Each call must eventually return to exactly the right place. How?

The naive solution: store the return address in a fixed register. But if B calls C, B's return address gets overwritten. The solution to nested calls is a **stack** — a last-in, first-out structure that grows as calls nest and shrinks as they return.

The stack was invented independently by several people in the early 1950s. **Friedrich Bauer** and **Klaus Samelson** in Germany, **Charles Leonard Hamblin** in Australia, and others all recognised that a pushdown automaton could solve the nested-call problem elegantly. By the late 1950s, the call stack was universal.

---

### The Stack in Hardware

The x86-64 stack is a region of memory that grows **downward** (toward lower addresses). Two registers manage it:

**RSP (Stack Pointer)** — always points to the current top of the stack (the most recently pushed value). Lower RSP = more stack used.

**RBP (Base Pointer / Frame Pointer)** — points to the base of the **current stack frame**. Fixed while a function executes, allowing stable access to local variables and parameters regardless of what RSP does.

**PUSH instruction:**

```
push rax     ; RSP -= 8; memory[RSP] = rax
```

**POP instruction:**

```
pop rbx      ; rbx = memory[RSP]; RSP += 8
```

The stack is not a special hardware structure — it is ordinary memory, accessed through a convention. Any memory region could be a stack if you use it this way.

---

### The CALL and RET Instructions

**CALL target:**

```
; equivalent to:
push rip + instruction_size   ; save return address (next instruction)
jmp target                    ; jump to function
```

**RET:**

```
; equivalent to:
pop rip       ; restore return address
; (CPU resumes execution at the saved address)
```

This is how nested calls work. Each CALL pushes a return address. Each RET pops and jumps to it. They interleave correctly because of LIFO order:

```
main calls A    → stack: [main_ret]
A    calls B    → stack: [main_ret, A_ret]
B    calls C    → stack: [main_ret, A_ret, B_ret]
C    returns    → pops B_ret, resumes in B
B    returns    → pops A_ret, resumes in A
A    returns    → pops main_ret, resumes in main
```

---

### The Stack Frame

When a function is called, it creates a **stack frame** — a region of the stack containing:

The saved return address (pushed by CALL)

The saved RBP (pushed by the function prologue)

Local variables (allocated by subtracting from RSP)

Saved registers (if the function uses callee-saved registers)

**Standard function prologue (x86-64):**

```
push rbp          ; save caller's frame pointer
mov  rbp, rsp     ; set frame pointer to current stack top
sub  rsp, N       ; allocate N bytes for locals
```

**Standard function epilogue:**

```
mov  rsp, rbp     ; restore stack pointer (deallocate locals)
pop  rbp          ; restore caller's frame pointer
ret               ; pop return address and jump
```

After the prologue, local variables live at negative offsets from RBP:

```
[rbp - 4]  → local int a
[rbp - 8]  → local int b
[rbp - 12] → local int c
```

And the saved return address and caller's RBP live above RBP:

```
[rbp + 8]  → return address (pushed by CALL)
[rbp + 0]  → saved RBP (pushed by prologue)
[rbp - 4]  → first local variable
```

---

### Calling Conventions — The Rules Everyone Agrees On

A **calling convention** is the agreement between caller and callee about:

• Which registers hold function arguments

• Which register holds the return value

• Which registers the callee must preserve (callee-saved)

• Which registers the callee may destroy (caller-saved)

• How the stack must be aligned

**Linux x86-64 (System V AMD64 ABI):**

| Purpose | Registers | |---|---| | Integer arguments 1–6 | `rdi, rsi, rdx, rcx, r8, r9` | | Floating-point args 1–8 | `xmm0–xmm7` | | Return value | `rax` (integer), `xmm0` (float) | | Callee-saved | `rbx, rbp, r12–r15` (must restore) | | Caller-saved | `rax, rcx, rdx, rsi, rdi, r8–r11` (may be destroyed) | | Stack alignment | RSP must be 16-byte aligned at CALL site |

**Windows x64 ABI (different!):**

• First 4 integer args: `rcx, rdx, r8, r9`

• 32-byte "shadow space" on stack required by callee

• Different callee/caller-saved register split

This is why Windows DLLs and Linux shared libraries are binary-incompatible even on the same CPU.

### A Complete Function Call Example

```
// C
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(3, 4);
    return 0;
}
```

```
; Linux x86-64 — what the compiler generates

add:
    push rbp
    mov  rbp, rsp
    ; a is in edi (first arg), b is in esi (second arg)
    mov  eax, edi       ; eax = a
    add  eax, esi       ; eax = a + b
    pop  rbp
    ret                 ; return value is in eax

main:
    push rbp
    mov  rbp, rsp
    sub  rsp, 16        ; allocate space for local 'result'
    mov  edi, 3         ; first arg = 3
    mov  esi, 4         ; second arg = 4
    call add            ; call add(3, 4)
    mov  [rbp-4], eax   ; result = return value
    mov  eax, 0         ; return 0
    pop  rbp
    ret
```

### Stack Overflow and Buffer Overflows

**Stack overflow**: when rec

### Illustrations

**Figure 82 • FIG 4.2-A**

### Stack Frames — main() calling add(5, 3)

*Illustration: FIG 4.2-A | Stack Frames — main() calling add(5, 3) [SVG diagram — see module-4-2-stack.html]*

Stack frame layout for main() calling add(5, 3). The stack grows downward (high addresses at top). main's frame contains its locals (x=5, y=3, r). When main calls add, the CALL instruction pushes the return address onto the stack. add's prologue saves main's RBP and establishes a new stack frame pointer. Parameters a=5 and b=3 are passed in registers (x86-64 System V ABI) but the stack also shows their spilled copies. add's epilogue restores RBP and SP, then RET pops the return address back into RIP, resuming execution in main. This precise bookkeeping is how recursion, exception handling, and

**Figure 83 • FIG 4.2-B**

### Stack Frame Layout During Nested Function Calls

*Illustration: FIG 4.2-B | Stack Frame Layout During Nested Function Calls [SVG diagram — see module-4-2-stack.html]*

The call stack during nested function calls main() → foo(3) → bar(7). Each function call pushes a new stack frame containing the return address, saved registers, parameters, and local variables. RSP (Stack Pointer) always points to the top of the stack; RBP (Base Pointer) anchors each frame, allowing variables to be accessed via fixed offsets from RBP. When bar() returns, its frame is destroyed and RSP moves back up — restoring the exact execution state of foo() at the call site. The same mechanism underlies exceptions, coroutines, and stack traces in every debugger.

**Figure 84 • FIG 4.2-C**

### push / pop Mechanics — How the Stack Pointer Moves

*Illustration: FIG 4.2-C | push / pop Mechanics — How the Stack Pointer Moves [SVG diagram — see module-4-2-stack.html]*

push and pop mechanics. push rax: RSP decrements by 8 (one 64-bit word), then the value of RAX is written to the new top of stack. pop rcx: the value at the current RSP is read into RCX, then RSP increments by 8. The stack grows towards lower addresses on x86-64. This LIFO structure means function parameters and return addresses are naturally cleaned up in reverse order — the last function called is the first to return.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 4.2 The Stack and Calling Conventions: Stack Frame Visualiser Calling Convention Explorer Stack Overflow Demo [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 4.2 The Stack and Calling Conventions **Type:** Trace + layout + reasoning + security **Estimated time:** 5060 minutes **Tools needed:** `simulation.html` + pen and paper --- Setup Stack grows downward. Initial RSP = 0x7FFF0000. Each push decrements RSP by 8 (64-bit). Each pop increments RSP by 8. All values are 64-bit (8 bytes). --- Part A Stack Trace Trace the stack contents after each instruction sequence. Show RSP value and what is at each stack address. ```nasm ; Initial: RSP = 0x7FFF0000 push rax ; rax = 0x0000000A (10) push rbx ; rbx = 0x00000005 (5)

# Module 4 3 Os Concepts

■# Module 4.3 — Operating System Concepts

## The Problem

*You have a CPU that can execute programs.*

*You have memory that stores them.*

*You have I/O devices to communicate with the world.*

>

*Now run two programs at once.*

>

*How do you prevent program A from reading program B's memory?*

*Who decides which program gets the CPU, and for how long?*

*What happens when a program wants to write to disk — does it get direct access to the hardware?*

>

*These are the problems the **operating system** exists to solve.*

An OS is not magic. It is a program — but a privileged one, with exclusive access to hardware, that manages resources on behalf of all other programs and enforces the boundaries between them.

## The Story

### Early Computers Had No OS

On early computers, one program ran at a time. The programmer submitted a job (usually a deck of punch cards), the operator loaded it, it ran, and the output was collected. If the program crashed or ran forever, the operator killed it manually.

The **batch OS** (late 1950s) automated job sequencing: when one program finished, the OS automatically loaded the next. Still one program at a time, but no human required between jobs.

**Time-sharing** (1960s) was the revolution: multiple users sharing one computer simultaneously. The OS rapidly switched between programs, giving each a short burst of CPU time. Each user felt they had a dedicated machine. **CTSS** (MIT, 1961), **Multics** (1965–1969), and **Unix** (1969–1973) emerged from this era.

Unix's influence dominates to this day. Linux, macOS, Android, iOS, and BSD are all Unix-descended systems. Even Windows NT borrowed heavily from Unix/VMS concepts.

---

### Kernel vs User Space

The operating system enforces a hardware-enforced boundary between two privilege levels:

**Kernel space**: where the OS kernel runs. Direct access to hardware, memory, all CPU instructions. A bug here crashes the entire system.

**User space**: where all applications run. Cannot directly access hardware. Cannot access another process's memory. Cannot execute privileged CPU instructions. A crash here only kills that one process.

The CPU enforces this via **privilege rings** (or protection levels). x86 has 4 rings (0–3); modern OSes use only ring 0 (kernel) and ring 3 (user). ARM uses EL0 (user) through EL3 (firmware).

The boundary exists for security and stability: a buggy or malicious user program cannot crash the system or access another program's data. Without this boundary, every program crash would bring down the whole machine — which is exactly what happened on early unprotected systems.

---

### System Calls — The Bridge

User programs cannot access hardware directly. But they need to — to read files, write to screen, open network connections, allocate memory. The mechanism is the **system call** (syscall).

A system call is a controlled transition from user space to kernel space:

User program places syscall number in `rax` and arguments in `rdi, rsi, rdx, ...`

User program executes the `syscall` instruction

CPU switches to ring 0, jumps to the kernel's syscall handler

Kernel validates arguments, performs the requested operation

Kernel places result in `rax`, returns to user space (ring 3)

```
; Linux x86-64 — write "hello" to stdout
mov rax, 1          ; syscall #1 = write
mov rdi, 1          ; fd = 1 (stdout)
mov rsi, msg_ptr    ; pointer to message
mov rdx, 5          ; length = 5
syscall             ; enter kernel
; rax now holds number of bytes written (or negative error code)
```

**Common Linux syscalls:**

| Number | Name | Description | |---|---|---| | 0 | `read` | Read from file descriptor | | 1 | `write` | Write to file descriptor | | 2 | `open` | Open a file | | 3 | `close` | Close a file descriptor | | 9 | `mmap` | Map memory | | 11 | `munmap` | Unmap memory | | 12 | `brk` | Adjust heap break | | 60 | `exit` | Terminate process | | 57 | `fork` | Create child process | | 59 | `execve` | Execute new program |

The C standard library (`libc`) wraps syscalls in convenient functions: `printf()` calls `write()`, `malloc()` calls `brk()` or `mmap()`, `fopen()` calls `open()`. You use these every day without thinking about the syscall layer beneath.

---

### Processes and Context Switching

A **process** is an instance of a running program. It consists of:

• A **virtual address space** (more on this in Module 4.4)

• A set of open **file descriptors**

• A **process ID (PID)**

• A **program counter** and register state

• A **stack** and **heap**

The OS gives the illusion that multiple processes run simultaneously. In reality (on a single core), only one runs at a time. The OS uses a **timer interrupt** to periodically preempt the running process and switch to another.

**Context switch:**

Timer interrupt fires — CPU jumps to kernel

Kernel saves all registers of the running process to its **Process Control Block (PCB)**

Kernel scheduler selects the next process to run

Kernel restores the registers of the new process from its PCB

Kernel returns to user space — the new process resumes exactly where it left off

A context switch takes ~1–10 microseconds. A modern system performs thousands per second. This is why time-sharing feels instantaneous to humans, whose perception latency is ~10 milliseconds.

---

### The Scheduler

The **scheduler** decides which process runs next. Key policies:

**Round Robin**: each process gets a fixed time quantum (e.g., 10 ms). Fair but ignores priority.

**Priority scheduling**: higher-priority processes preempt lower-priority ones. Risk: **starvation** — a low-priority process may never run if high-priority ones keep arriving.

**Completely Fair Scheduler (CFS)** — used by Linux: tracks each process's "virtual runtime" (time actually executed, normalised by priority). Always runs the process with the smallest virtual runtime. Achieves fairness without explicit time quanta.

**Real-time scheduling**: hard deadlines. A proc

## Illustrations

**Figure 85 • FIG 4.3-A**

### Kernel vs Userspace — The Protected Boundary

*Illustration: FIG 4.3-A | Kernel vs Userspace — The Protected Boundary [SVG diagram — see module-4-3-os.html]*

Kernel vs userspace separation. User space (top) runs with restricted CPU privileges (Ring 3 on x86) — processes cannot directly access hardware, other processes' memory, or kernel data structures. Kernel space (bottom) runs at Ring 0 with full hardware access. The system call interface is the only sanctioned crossing point: user code executes SYSCALL, the CPU switches to Ring 0, the kernel handles the request, then returns to Ring 3. This enforced boundary is the fundamental security and stability mechanism of modern OSes — a buggy user program cannot corrupt the kernel or other processes.

**Figure 86 • FIG 4.3-B**

### Process Lifecycle and Context Switching

*Illustration: FIG 4.3-B | Process Lifecycle and Context Switching [SVG diagram — see module-4-3-os.html]*

Process lifecycle states and transitions. NEW: process created by fork()/exec(). READY: waiting in the scheduler queue for CPU time. RUNNING: actively executing on a CPU core. WAITING/BLOCKED: suspended waiting for I/O completion, a lock, or a timer. EXIT: process has returned or called exit(). Context switching (gold annotation) occurs at preemption boundaries — the scheduler saves the running process's complete CPU state (all registers, PC, flags) into its Process Control Block, then restores the next process's state from its PCB. This creates the illusion of simultaneous execution across mu

**Figure 87 • FIG 4.3-C**

### Process Lifecycle — Five-State Model

*Illustration: FIG 4.3-C | Process Lifecycle — Five-State Model [SVG diagram — see module-4-3-os.html]*

The five-state process lifecycle model used by most operating system schedulers. A process is created in the NEW state via fork(). It enters the READY queue waiting for CPU time. The scheduler dispatches it to RUNNING when a CPU core is available. If the time quantum expires or a higher-priority process arrives, it is preempted back to READY. If the running process requests I/O, it moves to WAITING until an interrupt signals the I/O is complete, whereupon it returns to READY. Finally, exit() moves it to TERMINATED and releases its Process Control Block. This model is implemented in Linux, Wind

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 4.3 Operating System Concepts: Process Scheduler Syscall Tracer Kernel/User Space [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 4.3 Operating System Concepts **Type:** Scheduling + syscall tracing + privilege reasoning **Estimated time:** 4555 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Process Scheduling Four processes arrive at time 0. A Round Robin scheduler uses a time quantum of 4 ms. Context switch overhead = 1 ms (deducted from the quantum of the switching-out process). | PID | Burst time (ms) | Arrival | Priority | |-----|----------------|---------|----------| | P1 | 12 | 0 | 2 | | P2 | 6 | 0 | 4 (highest) | | P3 | 8

# Module 4 4 Memory Management

■# Module 4.4 — Memory Management

## The Problem

*A 64-bit CPU can address 16 exabytes of memory.*

*A typical machine has 16 gigabytes of RAM.*

>

*Every program believes it has its own private address space starting at 0x0000.*

*Two different programs can both have a variable at address 0x4000 — and they never interfere.*

>

*How?*

>

*And how does a 4 GB program run on a machine with only 8 GB of RAM when other programs are also running?*

These two mysteries have the same answer: **virtual memory** — one of the most elegant engineering solutions in the history of computing.

## The Story

### The Problem of Running Multiple Programs

On early computers, programs ran at fixed physical addresses. Loading two programs simultaneously was impossible — they would overwrite each other. The programmer had to know exactly where in physical RAM the program would be placed and write addresses accordingly.

**Relocation** (1950s) was the first solution: a hardware register held the base address of the current program. Every memory access added this base. Programs could move anywhere in RAM without being rewritten. But they still shared the same physical address space — a bug in one program could corrupt another's memory.

**Segmentation** (1960s, Multics) added boundaries: each program had a base and a limit. Hardware enforced that no access exceeded the limit. But segments were variable-size, leading to fragmentation.

**Paging** (late 1960s) solved fragmentation: divide all memory into fixed-size **pages** (typically 4 KB). Give each process its own mapping from virtual pages to physical pages. This is the system used today in every general-purpose OS.

---

### Virtual vs Physical Addresses

Every user-space program uses **virtual addresses**. These are what the program sees — addresses starting at 0x0000_0000_0000_0000 and extending to 0xFFFF_FFFF_FFFF_FFFF (on 64-bit, though only 48 bits are used in practice).

**Physical addresses** are the actual locations in RAM chips. The CPU's **Memory Management Unit (MMU)** translates every virtual address into a physical address on the fly, for every memory access.

```
Virtual address  →  [MMU / Page Table Walk]  →  Physical address
0x00007FFF_1000          translation              0x0000_1A40_3000
```

Two processes can both use virtual address 0x401000 — they map to different physical addresses. Neither can see the other's memory. The OS controls the mapping.

---

### Pages, Frames, and Page Tables

**Page**: a fixed-size block of virtual memory. Typically 4 KB (4096 bytes). **Frame**: a fixed-size block of physical memory. Same size as a page. **Page table**: a data structure mapping virtual page numbers to physical frame numbers.

For a 32-bit address space with 4 KB pages:

• Virtual address = 20-bit page number + 12-bit page offset

• $2^{20}$ = 1,048,576 possible virtual pages

• Each page table entry (PTE) = 4 bytes → 4 MB page table per process!

That is too large. The solution: **multi-level page tables**.

A 2-level page table for 32-bit systems:

```
Virtual address: [10 bits dir] [10 bits table] [12 bits offset]
                       ↓               ↓              ↓
                 Page Directory → Page Table → Physical Frame + Offset
```

x86-64 uses a **4-level page table**:

```
Virtual address: [9 bits PML4] [9 bits PDP] [9 bits PD] [9 bits PT] [12 bits offset]
```

Only page directories that are actually in use need to exist. A sparse address space uses very little memory for its page table.

---

### Page Table Entries (PTEs)

Each PTE contains:

• **Physical frame number** — which RAM frame this page maps to

• **Present bit (P)** — 1 = this mapping exists; 0 = page not in memory (triggers page fault)

• **Writable bit (W)** — 0 = read-only; 1 = writable. Enforces code section immutability.

• **User bit (U)** — 0 = kernel only; 1 = user space accessible

• **Accessed bit (A)** — set by hardware when the page is read. Used by page replacement algorithms.

• **Dirty bit (D)** — set by hardware when the page is written. Used to know if a page needs to be written back to disk before reclaiming.

• **NX bit (No-Execute)** — prevents code execution from this page. Makes data pages non-executable — critical for stack/heap security.

---

### The TLB — Translation Lookaside Buffer

A page table walk requires up to 4 memory accesses (on x86-64). That would make every memory access 5× slower. The solution: the **TLB** — a small, fast cache of recent virtual→physical translations.

• Capacity: 64–4096 entries (tiny)

• Speed: 1 cycle (same as L1 cache)

• Hit rate: >99% for well-behaved programs (due to temporal and spatial locality)

**TLB miss**: the hardware page table walker reads the page table hierarchy, loads the translation into the TLB, and retries. On x86, this is done entirely by hardware (hardware page table walker). On ARM, software can handle TLB misses.

**TLB shootdown**: when the OS modifies a process's page table (e.g., unmaps a page), it must invalidate the corresponding TLB entries on *all* CPU cores. On a multiprocessor system, this requires an inter-processor interrupt (IPI) to each core. TLB shootdowns are expensive and a significant performance cost of shared-memory multiprocessors.

---

### Demand Paging and the Page Fault

A process does not need all its pages in physical RAM simultaneously. The OS uses **demand paging**: pages are loaded into RAM only when first accessed.

When a process accesses a virtual address whose PTE has Present=0:

The CPU raises a **page fault** exception

The kernel's page fault handler runs

It checks: is this a valid virtual address for this process?

- Yes: load the page from disk (swap space / file), update PTE, return - No: invalid access → send SIGSEGV to the process (segmentation fault)

The process resumes, unaware that it paused

**Copy-on-Write (CoW)**: when `fork()` creates a child process, the OS does not immediately copy the parent's memory. Instead, both parent an

## Illustrations

**Figure 88 • FIG 4.4-A**

### Virtual Memory — Every Process Gets Its Own 48-bit Address Space

*Illustration: FIG 4.4-A | Virtual Memory — Every Process Gets Its Own 48-bit Address Space [SVG diagram — see module-4-4-memory.html]*

Virtual memory gives each process a private, contiguous-looking address space regardless of where its pages actually live in physical RAM. Both Process A and Process B see the same virtual address layout (kernel at top, stack below, heap above code). The MMU (Memory Management Unit) hardware — guided by each process's page table — transparently translates every virtual address to a physical frame address. Physical frames are scattered and interleaved between processes with no visible correlation to virtual addresses. A process cannot access another process's frames because it has no valid page

**Figure 89 • FIG 4.4-B**

### Virtual Address Translation — 2-Level Page Table Walk

*Illustration: FIG 4.4-B | Virtual Address Translation — 2-Level Page Table Walk [SVG diagram — see module-4-4-memory.html]*

Two-level page table walk for virtual-to-physical address translation. The 32-bit virtual address is split into three fields: a 10-bit Page Directory index, a 10-bit Page Table index, and a 12-bit byte offset (supporting 4KB pages). The MMU uses CR3 to find the Page Directory, reads the entry at PD[1] to locate the Page Table at physical address 0x2000, then reads PT[3] to find the physical page frame at 0xD000. Adding the offset (0x010) gives the final physical address 0xD010. The Translation Lookaside Buffer (TLB) caches recent translations to avoid this expensive two-memory-access walk on e

**Figure 90 • FIG 4.4-C**

**Segmentation Fault — Accessing a Non-Mapped Virtual Address**

*Illustration: FIG 4.4-C | Segmentation Fault — Accessing a Non-Mapped Virtual Address [SVG diagram — see module-4-4-memory.html]*

The anatomy of a segmentation fault, explained from hardware upward. Every process has a virtual address space divided into mapped regions (stack, heap, code, data) and unmapped regions. When a program accesses an address in an unmapped region — the classic case being a null pointer dereference — the MMU finds no valid page table entry and raises a hardware Page Fault exception. The OS kernel handles this fault: if the address should be in swap space, it loads the page from disk (a valid soft fault). If the address is truly unmapped, the kernel sends SIGSEGV to the process, which terminates it

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 4.4 Memory Management: Address Translator Page Fault Handler Process Memory Map [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 4.4 Memory Management **Type:** Calculation + trace + reasoning **Estimated time:** 4555 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Page Address Arithmetic System: 32-bit virtual addresses, 4 KB pages (2 bytes = 4096 bytes). 1. A virtual address is 32 bits: [20-bit page number | 12-bit offset]. Break down virtual address `0x0040_1B8C`: - What is the page number (in decimal and hex)? - What is the page offset (in decimal and hex)? 2. Physical memory has 256 MB. Page frames are also 4 KB. How many physical frames exist? How many bits are ne

# Module 4 5 File Systems

■# Module 4.5 — File Systems

## The Problem

*You can read and write bytes at memory addresses.*

*But when the power goes off, RAM forgets everything.*

>

*How do you give data a name that persists across power cycles?*

*How do you organise billions of bytes on a disk into something human-navigable?*

*How does the OS know where one file ends and another begins?*

>

*The answer is the **file system** — the layer that imposes order on the chaos of raw storage.*

A file system solves three problems simultaneously: naming (how to find data by name), organisation (how to structure data into directories), and persistence (how to survive power loss without corruption).

---

## The Story

### Files Before File Systems

Early computers had no persistent storage. Programs and data were loaded from punch cards each run. The concept of a "file" — data with a name that persists — did not exist.

Magnetic tape (1950s) introduced persistence but no random access. Data was stored sequentially; to find record 1000, you had to scan past records 1–999. The concept of a "directory" had no meaning.

**Magnetic disk** (IBM RAMAC, 1956) introduced random access: the read/write head could seek to any location in milliseconds. Now you could have many files — but you needed a way to find them. The first file system concepts emerged: a table mapping names to disk locations.

Unix (1969) introduced the inode-based file system. This design — with refinements — is still used in ext4 (Linux) and HFS+ (macOS) today.

---

### Disks and Blocks

A magnetic hard disk is divided into fixed-size **blocks** (also called sectors or clusters). Typical block size: 4096 bytes (4 KB).

A file system works entirely in terms of blocks, not bytes:

• A 1-byte file occupies one full block (4 KB wasted internally — called **internal fragmentation**)

• A 5000-byte file occupies two blocks (8 KB on disk, 3 KB wasted)

The block is the smallest unit of allocation. The file system maintains a data structure tracking which blocks are free and which are in use.

---

### Inodes — The Heart of Unix File Systems

An **inode** (index node) is a data structure that stores all metadata about a file except its name:

```
inode {
  mode:      permissions (rwxrwxrwx) + file type
  uid, gid:  owner user and group IDs
  size:      file size in bytes
  atime:     last access time
  mtime:     last modification time
  ctime:     last metadata change time
  nlinks:    number of hard links to this inode
  blocks[]:  array of block addresses where data lives
             [direct blocks: 12 entries]
```

```
            [single indirect block pointer]
            [double indirect block pointer]
            [triple indirect block pointer]
}
```

**Direct blocks**: the first 12 entries point directly to data blocks. Supports files up to 12 × 4 KB = 48 KB with no indirection.

**Single indirect**: one block used as an array of block pointers. 4 KB / 4 bytes = 1024 pointers → 1024 × 4 KB = 4 MB additional data.

**Double indirect**: pointer → block of pointers → data. 1024 × 1024 × 4 KB = 4 GB.

**Triple indirect**: $1024^3 \times 4$ KB = 4 TB.

This structure allows small files to be accessed very efficiently (direct blocks) while still supporting large files.

---

### Directories — Files About Files

A directory is simply a file whose data is a table of `(name → inode_number)` pairs:

```
Directory /home/user:
.    → inode 2  (this directory)
..   → inode 1  (parent directory)
hello.c → inode 417
notes.txt → inode 821
src  → inode 1024  (subdirectory inode)
```

When you access `/home/user/hello.c`:

Start at root inode (inode 2 on most systems)

Read root's directory data → find "home" → get its inode number

Read "home"'s directory data → find "user" → get inode number

Read "user"'s directory data → find "hello.c" → get inode 417

Read inode 417 → get list of data blocks

Read data blocks → file contents

This is why deep directory nesting (100 levels) is slow: each level requires at least one disk read.

---

### Hard Links and Soft Links

**Hard link**: a second directory entry pointing to the *same inode*. The inode's `nlinks` counter increments. The file has two names. Deleting one name decrements `nlinks`; data is only freed when `nlinks` reaches 0.

```
ln file.txt link.txt    # hard link: same inode, two names
```

**Symbolic link (symlink)**: a file whose data is a path string. Following a symlink reads its data (the path) and redirects the lookup. Symlinks can point to directories, span file systems, and point to things that don't exist.

```
ln -s /path/to/target link.txt   # symlink
```

Hard links: same inode, same file, multiple names. Symlinks: separate inode, data = path string, like a shortcut.

### The Free Space Bitmap

The file system tracks which blocks are free using a **bitmap**: one bit per block. Bit = 0 means free; bit = 1 means in use. Allocating a block: find a 0 bit, flip it to 1. Freeing a block: find the bit, flip it to 0.

For a 1 TB disk with 4 KB blocks: 1 TB / 4 KB = $2^{28}$ blocks → bitmap = $2^{28}$ bits = 32 MB. A very compact structure that fits easily in memory.

### Journaling — Surviving Power Loss

Suppose the system loses power halfway through creating a file:

• The directory was updated (points to a new inode)

• But the inode was never written

• Or the data blocks were written but the inode size was not updated

Without protection, the file system is now **inconsistent** — internal data structures contradict each other. `fsck` (file system check) was the traditional solution: scan the entire disk on every boot after an unclean shutdown. For a 1 TB disk this takes minutes.

**Journaling** (ext3, ext4, NTFS, HFS+): before making changes, write a **journal entry** describing what changes are about to be made. After all changes are safely written, mark the journal entry as committed.

On recovery after a crash: replay the journal. Any committed entries not yet applied are re-applied. The file system is a

### Illustrations

**Figure 91 • FIG 4.5-A**

**Inodes and Directory Trees — How Files Are Named and Located**

*Illustration: FIG 4.5-A | Inodes and Directory Trees — How Files Are Named and Located [SVG diagram — see module-4-5-filesystem.html]*

Filesystem structure on Linux ext4. The directory tree (left) stores name-to-inode mappings only — the file name "hello.txt" exists only in the directory alice/. The inode (centre, inode 203) stores all file metadata: permissions, owner, timestamps, size, and an array of disk block pointers. The first direct block pointer (block[0] = 7821) points to the 4KB disk block that contains the actual file data "Hello, World!\n" (13 bytes). The critical insight: the file name and the file data are completely separate. This enables hard links (multiple names pointing to the same inode) and explains why

**Figure 92 • FIG 4.5-B**

### Inode Structure — Separating File Metadata from File Data

*Illustration: FIG 4.5-B | Inode Structure — Separating File Metadata from File Data [SVG diagram — see module-4-5-filesystem.html]*

The inode architecture used by ext4, XFS, and most Unix file systems. A directory entry maps a filename to an inode number. The inode stores all metadata (permissions, timestamps, size, owner) and a set of block pointers: 12 direct pointers for small files, and an indirect pointer (pointing to a block full of more pointers) for larger ones. Critically, the filename is not stored in the inode — it lives only in the directory. This enables hard links: multiple directory entries pointing to the same inode. The file persists on disk until its link count reaches zero, at which point the OS frees it

**Figure 93 • FIG 4.5-C**

### Path Resolution — Walking /home/user/hello.txt from the Root

*Illustration: FIG 4.5-C | Path Resolution — Walking /home/user/hello.txt from the Root [SVG diagram — see module-4-5-filesystem.html]*

Path resolution in a Unix file system. Opening /home/user/hello.txt requires the kernel to walk each path component: read the root inode (#2) to find "home"→#100, read inode #100 to find "user"→#450, read inode #450 to find "hello.txt"→#4821, read inode #4821 to get block pointers, then finally read the data block. This is a minimum of 4 inode accesses before any file content is transferred. The kernel's directory cache (dcache) keeps recently resolved path components in memory, reducing this cost to near zero for hot paths — but cold path performance explains why deep directory hierarchies ca

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 4.5 File Systems: Inode Explorer Directory Lookup Journal Visualiser [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 4.5 File Systems **Type:** Calculation + trace + reasoning + design **Estimated time:** 4555 minutes **Tools needed:** `simulation.html` + pen and paper --- Setup A Unix-like file system with: - Block size: 4 KB (4096 bytes) - Inode structure: 12 direct block pointers + 1 single-indirect + 1 double-indirect - Each block pointer: 4 bytes - Inode table starts at block 1; data blocks start at block 100 --- Part A File Size Limits 1. A file uses only **direct blocks** (12 entries). What is the maximum file size? 2. The **single-indirect block** holds block pointers. How man

LEVEL 05

# Programming

C, Python, Algorithms, and Computational Thinking

# Module 5 1 C Language

■# Module 5.1 — C: Close to the Metal

## The Problem

*Assembly language gives you full control over the hardware.*

*But assembly is architecture-specific: x86-64 assembly won't run on ARM.*

*Writing an operating system, a compiler, or a database in pure assembly means rewriting it for every CPU.*

>

*In 1972, Dennis Ritchie at Bell Labs asked:*

*Can we have a language that is portable across hardware but still compiles down to efficient machine code with no hidden overhead?*

>

*The answer was **C** — and it changed computing permanently.*

C sits at the exact boundary between human-readable and machine-executable. Every concept in C maps directly to a small number of machine instructions. There is no garbage collector, no runtime, no virtual machine. If you write x++, the CPU executes one instruction. This is why C is called "portable assembly."

## The Story

### Why C Was Invented

Before C, operating systems were written in assembly. The PDP-7 version of Unix (1969) was written entirely in assembly. Porting it to a new machine meant rewriting every line.

Ritchie's predecessor language, **B** (1969), was portable but too slow for OS work — it had no types, so every operation treated data as machine words, requiring extra work to handle bytes or floats.

C added:

• **Types**: `int`, `char`, `float`, `struct` — the compiler uses type information to generate correct machine code without programmer overhead

• **Structs**: group related data into a single compound type

• **Pointers**: explicit memory addresses — full hardware access

• **Arrays**: contiguous memory blocks with typed access

In 1973, the Unix kernel was rewritten in C. This was the first major operating system written in a high-level language. It could now be ported to new hardware in weeks instead of years. This single decision — rewriting Unix in C — is why C became universal.

### The Compilation Pipeline (Review)

```
Source (.c) → Preprocessor → Compiler → Assembler → Linker → Executable
```

**Preprocessor** (`cpp`): expands `#include`, `#define`, `#ifdef`. Produces pure C. **Compiler** (`cc1`): translates C to assembly. Performs optimisation. **Assembler** (`as`): translates assembly to object code (`.o`). **Linker** (`ld`): combines object files, resolves symbols, produces final binary.

Understanding this pipeline explains why a missing semicolon in `foo.c` can produce an error message referencing `foo.h` — the preprocessor merged them first.

---

## The Concept

### Pointers — The Core of C

A **pointer** is a variable that holds a memory address.

```
int  x = 42;      // x is an int stored at some address
int *p = &x;      // p holds the address of x (& = "address of")
int  y = *p;      // y = the value at address p (* = "dereference")
*p = 100;         // write 100 to the address p points to → x is now 100
```

This is not abstraction — this is exactly what the CPU does. `p` contains a 64-bit integer that happens to be a memory address. The dereference `*p` compiles to a single `MOV` instruction using an indirect addressing mode.

**Why pointers matter:**

• Pass large data to functions without copying (pass by reference)

• Build dynamic data structures (linked lists, trees) that grow at runtime

• Interface with hardware registers (embedded systems)

• Implement your own memory allocator

**Common pointer mistakes:**

• **Null pointer dereference**: `int <i>p = NULL; </i>p = 5;` → segfault

• **Dangling pointer**: pointer to memory that has been freed

• **Buffer overrun**: writing past the end of an array via pointer arithmetic

• **Memory leak**: allocating with `malloc()` without ever calling `free()`

### Memory Allocation

C has two kinds of memory:

**Stack** — automatic, managed by the compiler:

```
void fn() {
    int x = 5;      // allocated on stack when fn() called
    char buf[64];   // 64 bytes on stack
}                   // all freed when fn() returns
```

**Heap** — manual, managed by the programmer:

```
int *arr = malloc(100 * sizeof(int));  // allocate 400 bytes
if (arr == NULL) { /* handle error */ }
arr[0] = 42;        // use it
free(arr);          // you must free it — no garbage collector
arr = NULL;         // good practice: clear the pointer after free
```

This manual memory management is what makes C fast and dangerous simultaneously. No GC pause, but also no safety net.

### Structs — User-Defined Types

```
struct Point {
    int x;
    int y;
};

struct Point p1;
p1.x = 10;
p1.y = 20;

struct Point *pp = &p1;
pp->x = 30;          // arrow operator: (*pp).x = 30
```

Structs are laid out contiguously in memory. `struct Point` occupies 8 bytes: 4 for x, 4 for y. The compiler may add **padding bytes** to ensure alignment rules are met, which is why `sizeof(struct)` can surprise you.

### Functions and the Call Stack

C functions map directly to the calling convention from Module 4.2:

```
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(3, 4);  // a→rdi, b→rsi; return→rax
    return 0;
}
```

Recursion works naturally because each call gets its own stack frame:

```
int factorial(int n) {
    if (n <= 1) return 1;         // base case
    return n * factorial(n - 1);  // recursive case
}
```

### Building a Memory Allocator

To truly understand C, you build a simplified `malloc()`. The core idea:

```
// A minimal slab allocator — conceptual sketch
static char heap[65536];  // 64 KB of pretend heap
static int  top = 0;      // current allocation pointer

void *my_malloc(int size) {
    if (top + size > sizeof(heap)) return NULL;  // out of memory
```

```
    void *ptr = &heap[top];
    top += size;
    return ptr;
}
// (No free() in this sketch — a real allocator needs a free list)
```

A real `malloc()` implementation (like `ptmalloc` in glibc) manages a free list of blocks, coalesces adjacent free blocks, and calls `brk()`/`mmap()` to request more memory from the OS when needed.

## Hands-On Exercise

**Open the simulation:**

### Illustrations

**Figure 94 • FIG 5.1-A**

**Stack vs Heap — Two Memory Regions, Two Lifetimes**

*Illustration: FIG 5.1-A | Stack vs Heap — Two Memory Regions, Two Lifetimes [SVG diagram — see module-5-1-c.html]*

C memory model: stack vs heap. Local variables like int x = 42 are allocated automatically on the stack at function entry and freed automatically on return — the compiler manages this. Heap allocations via malloc() give the programmer explicit control: the memory persists until free() is called. The pointer p (the variable holding the address) lives on the stack; the pointed-to data lives on the heap. Forgetting to call free() is a memory leak. Calling free() twice, or dereferencing p after free(), is undefined behaviour. These are the most common class of C bugs and the primary motivation for

**Figure 95 • FIG 5.1-B**

**Pointers in Memory — Address, Dereference, Pointer Arithmetic**

*Illustration: FIG 5.1-B | Pointers in Memory — Address, Dereference, Pointer Arithmetic [SVG diagram — see module-5-1-c.html]*

Pointer mechanics in C. An integer array arr[4] occupies four contiguous 4-byte slots starting at address 0x1000. The pointer p stores the address 0x1000 — p itself is a variable on the stack holding a number. The dereference operator *p reads the value at that address (10). Pointer arithmetic p+1 does not add 1 to the address — it adds sizeof(int) = 4 bytes, advancing to the next array element at 0x1004. This is why p[i] is exactly equivalent to *(p + i) in C — array subscripting is defined in terms of pointer arithmetic. This direct correspondence between arrays, pointers, and memory address

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 5.1 C: Close to the Metal: Pointer Visualiser Stack vs Heap Linked List Builder [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 5.1 C: Close to the Metal **Type:** Pointer trace + memory layout + implementation **Estimated time:** 4555 minutes **Tools needed:** `simulation.html` + pen and paper --- Part A Pointer Arithmetic Trace the following C code. For each line, write the value of every variable and the state of memory. ```c int arr[] = {10, 20, 30, 40, 50}; int *p = arr; // (1) printf("%d\n", *p); // (2) p++; // (3) printf("%d\n", *p); // (4) p += 2; // (5) printf("%d\n", *p); // (6) int *q = arr + 4; // (7) printf("%d\n", q - p); // (8) pointer sub

# Immutable: creates a NEW object

■# Module 5.2 — Python: Abstraction for Humans

## The Problem

*C gives you total control and total responsibility.*

*Writing a web scraper in C means managing memory, parsing strings byte by byte,*

*handling network sockets manually.*

>

*Most programmers most of the time are not writing operating systems.*

*They are solving problems: process this data, build this web service, train this model.*

>

**What if the language handled memory, types, and low-level details automatically*

*so the programmer could focus entirely on the problem?**

This question produced Python (1991), and then a generation of languages with similar philosophy. Python's design principle: **readability counts**. Code is read far more than it is written. A program that takes 10 lines in C and 2 lines in Python is almost always faster to write, debug, and maintain — even if it runs 50× slower at execution time.

## The Story

### Guido van Rossum and the Christmas Holiday

Guido van Rossum was a Dutch programmer working at CWI (Netherlands) in December 1989. During the holiday break, he decided to write a new scripting language as a personal project — something easier to use than C for quick programming tasks, better-designed than the shells and AWK scripts he

was using.

He named it Python after Monty Python's Flying Circus, not the snake. His design goals:

• Elegant syntax that reads almost like pseudocode

• A comprehensive standard library ("batteries included")

• Automatic memory management — no `malloc`/`free`

• Dynamic typing — no type declarations needed

• A single, clear way to do each thing

Python 1.0 was released in 1994. Python 2.0 (2000) added garbage collection and list comprehensions. Python 3.0 (2008) was a backward-incompatible redesign that fixed accumulated design mistakes. By 2020, Python was the most-used programming language in the world by survey.

## The Concept

### Interpreted vs Compiled

C source code is compiled to machine code once; the CPU runs the machine code directly.

Python source code is interpreted at runtime by the Python interpreter (CPython, written in C):

```
Python source (.py)
      ↓ Compilation (fast, at runtime)
Bytecode (.pyc)
      ↓ Interpretation (CPython virtual machine)
Result
```

The CPython VM executes bytecode instructions in a loop. Each Python operation (like `x + y`) may compile to hundreds of machine instructions — type-checking, object dispatch, reference counting. This is why Python is 50–100× slower than C for CPU-intensive code.

**When Python speed doesn't matter:**

• I/O-bound tasks (network requests, file reads) — most time is waiting, not computing

• NumPy/SciPy/PyTorch operations — these call into C/Fortran/CUDA code, so the speed is C speed

• Prototyping — development time matters more than runtime for one-off scripts

### Dynamic Typing

In C: `int x = 5;` — the type `int` is fixed at compile time.

In Python: `x = 5` — the type is attached to the *value*, not the variable. `x` can point to any type:

```
x = 5         # x refers to an int object
x = "hello"   # now x refers to a str object
x = [1, 2, 3] # now x refers to a list object
```

Under the hood, every Python value is a C struct (`PyObject`) containing:

• A reference count (for garbage collection)

• A pointer to the type object

• The actual value

This indirection — going through a type pointer for every operation — is most of Python's overhead compared to C.

### The Python Data Model

Everything in Python is an **object**. Integers, strings, functions, classes, modules — all objects. Every object has:

• An **identity** (`id(x)` — the memory address)

• A **type** (`type(x)`)

• A **value**

**Immutable types**: `int`, `float`, `str`, `tuple`, `frozenset` — cannot be changed after creation. **Mutable types**: `list`, `dict`, `set`, custom objects — can be modified in-place.

```
# Immutable: creates a NEW object
x = "hello"
y = x.upper()    # y is a new string "HELLO", x unchanged

# Mutable: modifies IN PLACE
lst = [1, 2, 3]
lst.append(4)    # lst is now [1, 2, 3, 4] — same object
```

This distinction matters for correctness and performance. Passing a list to a function and modifying it changes the original (pass by reference to the object). Passing an int and "modifying" it just rebinds the local variable (pass by value of the reference — effectively pass by value for immutables).

### Garbage Collection

Python uses **reference counting** as its primary GC mechanism:

```
x = [1, 2, 3]    # ref count of the list = 1
y = x             # ref count = 2
del x             # ref count = 1
del y             # ref count = 0 → object freed
```

When an object's reference count reaches 0, it is immediately freed. This is fast and deterministic — no GC pause.

Problem: **circular references**:

```
a = {}
b = {}
a['other'] = b    # b's ref count = 2 (b itself + a['other'])
b['other'] = a    # a's ref count = 2
del a             # a's ref count = 1 (still referenced by b)
del b             # b's ref count = 1 (still referenced by a)
# Both are now unreachable but neither is freed!
```

Python's **cycle garbage collector** (runs periodically) detects and collects such cycles. This is why Python can have occasional GC pauses for long-lived programs.

### Key Built-in Types and Their Internals

**List** — a dynamic array of object pointers:

```
lst = [1, 2, 3]
lst.append(4)     # O(1) amortised — pre-allocates extra space
lst.insert(0, 0)  # O(n) — shifts all elements right
lst[2]            # O(1) — direct index access
```

**Dict** — a hash table:

```
d = {'name': 'Alice', 'age': 30}
d['name']         # O(1) average — hash lookup
d['city'] = 'NYC' # O(1) average — insert
'name' in d       # O(1) average — membership test
```

Python dicts since 3.7 maintain insertion order. The implementation uses open addressing with a compact layout since Python 3.6.

**Set** — a hash table without values:

```
s = {1, 2, 3, 4}
s.add(5)           # O(1) a
```

## Illustrations

**Figure 96 • FIG 5.2-A**

### Interpreted vs Compiled — Two Paths from Source to Execution

*Illustration: FIG 5.2-A | Interpreted vs Compiled — Two Paths from Source to Execution [SVG diagram — see module-5-2-python.html]*

Compiled vs interpreted execution models. C (top): source compiles directly to native CPU instructions — the CPU executes them directly at hardware speed. Python (bottom): source compiles to platform-independent bytecode, which a Python Virtual Machine (CPython) interprets one opcode at a time. Each Python bytecode LOAD_CONST or BINARY_ADD triggers a C function call inside the VM, adding overhead. Python's 50–100× slowdown versus C comes from this interpretation loop plus the dynamic type-checking overhead on every operation. JIT compilers (PyPy) watch which bytecode loops run frequently and c

**Figure 97 • FIG 5.2-B**

**Python Object Model — Everything Is an Object**

*Illustration: FIG 5.2-B | Python Object Model — Everything Is an Object [SVG diagram — see module-5-2-python.html]*

Python's uniform object model. Every value in Python — integers, strings, lists, functions, even types — is a PyObject structure on the heap containing a type pointer, a reference count, and the value. When you write b = a, Python does not copy the integer 42; it increments the reference count on the existing integer object and makes b point to it. When a variable goes out of scope or is reassigned, the reference count decrements. When it hits zero, the memory is freed immediately (reference counting garbage collection). Cyclic collector handles the edge case of objects that reference each oth

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 5.2 Python: Abstraction for Humans: Reference Counter Data Structure Internals Decorator Builder [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 5.2 Python: Abstraction for Humans **Type:** Implementation + trace + reasoning **Estimated time:** 4555 minutes **Tools needed:** Python interpreter (or `simulation.html`) --- Part A Linked List in Python Implement a singly linked list class with the following interface: ```python class Node: def __init__(self, data, next=None): ... class LinkedList: def __init__(self): ... def append(self, data): ... add to tail O(n) acceptable def prepend(self, data): ... add to head O(1) def delete(self, data): ... remove first occurrence def

# Module 5 3 Algorithms Intro

■# Module 5.3 — Algorithms: Thinking in Steps

## The Problem

*You have a program that works.*

*It processes 100 records in 0.01 seconds.*

*You run it on 1,000,000 records.*

*It takes 27 hours.*

>

*What went wrong?*

Nothing *crashed*. The logic is correct. But the **algorithm** — the sequence of steps used to solve the problem — scales so poorly that the program becomes useless at the scale it was designed for.

Understanding algorithms means understanding not just *how* to solve a problem, but *how efficiently* — and being able to predict what happens when the input grows by 10×, 100×, or 1,000,000×.

---

## The Story

### The Algorithm Predates the Computer

The word **algorithm** comes from the name of the 9th-century Persian mathematician Muhammad ibn Musa al-Khwarizmi, whose book *Kitab al-mukhtasar fi hisab al-jabr wal-muqabala* (825 AD) introduced systematic procedures for solving equations. "Algorithm" is a Latinisation of al-Khwarizmi. "Algebra" comes from "al-jabr" in the title.

Long before computers, people described algorithms: Euclid's algorithm for the GCD of two numbers (300 BC) is still used in every modern computer's arithmetic library. Long division is an algorithm. A recipe is an algorithm. The question is always the same: *how many steps does it take, and how does that grow as the input grows?*

Alan Turing formalised the concept of a computable algorithm in 1936 with the Turing machine — a theoretical model that defines exactly what it means for a problem to be solvable by a mechanical process.

---

## The Concept

### Big-O Notation

Big-O notation describes how an algorithm's runtime (or memory usage) scales as the input size *n* grows. It describes the **worst-case upper bound**, ignoring constant factors.

| Notation | Name | Example | n=100 | n=1,000 | n=1,000,000 | |---|---|---|---|---|---| | O(1) | Constant | Array index access | 1 op | 1 op | 1 op | | O(log n) | Logarithmic | Binary search | 7 ops | 10 ops | 20 ops | | O(n) | Linear | Linear scan | 100 | 1,000 | 1,000,000 | | O(n log n) | Linearithmic | Merge sort | 664 | 9,966 | 19,931,568 | | O(n²) | Quadratic | Bubble sort | 10,000 | 1,000,000 | 10¹² | | O(2■) | Exponential | Naive TSP | 10³■ | — | — |

The difference between O(n log n) and O(n²) is the difference between a sort that handles a million records in 1 second and one that takes 12 days.

**Formal definition**: f(n) = O(g(n)) if there exist constants c > 0 and n■ such that f(n) ≤ c·g(n) for all n ≥ n■. In practice: keep only the fastest-growing term, drop coefficients.

### Analysing an Algorithm

```
  def find_duplicates(lst):          # n = len(lst)
      for i in range(len(lst)):      # n iterations
          for j in range(len(lst)):  # n iterations (inner)
```

```
            if i != j and lst[i] == lst[j]:
                return True
    return False
```

Inner loop runs n times for each of n outer iterations: **O(n²)**.

Better approach:

```
def find_duplicates_fast(lst):
    seen = set()                    # O(1) lookup
    for x in lst:                   # n iterations
        if x in seen:               # O(1) average
            return True
        seen.add(x)                 # O(1) average
    return False
```

Single loop, O(1) operations per iteration: **O(n)**. For n=1,000,000: the first version does $10^{12}$ comparisons; the second does 1,000,000.

### Space Complexity

Runtime is not the only resource that scales. Memory usage also has a Big-O:

```
def sum_list(lst):
    total = 0              # O(1) space — one variable
    for x in lst:
        total += x
    return total           # Total: O(1) space

def reverse_list(lst):
    result = []            # O(n) space — new list same size as input
    for i in range(len(lst) - 1, -1, -1):
        result.append(lst[i])
    return result          # Total: O(n) space
```

There is often a space-time tradeoff: use more memory to get faster runtime (hash tables, memoisation) or use less memory at the cost of more computation (in-place sorts).

### Recursion and the Recurrence Relation

Many algorithms are naturally recursive. Their complexity is expressed as a **recurrence relation**:

```
def binary_search(arr, target, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search(arr, target, mid + 1, high)
    else:
        return binary_search(arr, target, low, mid - 1)
```

At each step, the problem size halves: T(n) = T(n/2) + O(1). Solving: T(n) = **O(log n)**.

**The Master Theorem** gives a formula for recurrences of the form T(n) = aT(n/b) + O(n^d):

• If d > log_b(a): T(n) = O(n^d)

• If d = log_b(a): T(n) = O(n^d log n)

• If d < log_b(a): T(n) = O(n^(log_b a))

Merge sort: T(n) = 2T(n/2) + O(n), so a=2, b=2, d=1. log_2(2)=1=d → T(n) = O(n log n).

### Greedy, Divide-and-Conquer, Dynamic Programming

These three paradigms cover most algorithm design:

**Greedy**: at each step, make the locally optimal choice and never reconsider.

• Coin change (specific denominations), activity selection, Dijkstra's shortest path

• Fast, simple — but often does not produce the global optimum

**Divide and Conquer**: split the problem into independent sub-problems, solve each, combine.

• Merge sort, binary search, quicksort, fast Fourier transform

• Enables O(n log n) where naive approaches are $O(n^2)$

**Dynamic Programming**: break into overlapping sub-problems; store results to avoid recomputation.

• Fibonacci, longest common subsequence, shortest path (Bellman-Ford), knapsack

• Turns exponential recursion into polynomial iteration

---

## Hands-On Exercise

**Open the simulation:** [`simulation.html`](./simulation.html)

**Tool 1 — Complexity Plotter**

• Enter two algorithm complexities (e.g., $O(n^2)$ vs O(n log n))

• See the actual operation counts plotted as n grows

• Observe the crossover point where the faster algorithm wins

**Tool 2 — Linear vs

### Illustrations

**Figure 98 • FIG 5.3-A**

### Big-O Growth Curves — How Different Complexities Scale



*Illustration: FIG 5.3-A | Big-O Growth Curves — How Different Complexities Scale [SVG diagram — see module-5-3-algorithms-intro.html]*

Big-O growth curves showing seven complexity classes. O(1) is constant — runtime does not depend on input size at all (array index lookup). O(log n) barely grows (binary search). O(n) grows linearly (linear scan). O(n log n) is the best possible for comparison-based sorting (merge sort, heap sort). O(n²) grows quickly (bubble sort, nested loops). O(2■) and O(n!) are computationally infeasible beyond tiny inputs — an O(2■) algorithm processing n=100 items would require more operations than atoms in the observable universe.

**Figure 99 • FIG 5.3-B**

### Linear Search vs Binary Search — The Algorithmic Advantage



*Illustration: FIG 5.3-B | Linear Search vs Binary Search — The Algorithmic Advantage [SVG diagram — see module-5-3-algorithms-intro.html]*

Linear search versus binary search finding value 73 in a sorted 16-element array. Linear search checks every element left to right — finding 73 at index 11 requires 12 comparisons, and the worst case is 16. Binary search exploits the sorted order: compare the middle element (61 at index 7), determine which half contains the target, repeat. Found in 2 comparisons. For a 1-million-element array, linear search may require 1,000,000 checks; binary search needs at most 20 (log■ 1,000,000 ≈ 20). The right algorithm dominates hardware speed.

**Figure 100 • FIG 5.3-C**

### Algorithm Complexity Reference — Real Operation Counts at Scale



*Illustration: FIG 5.3-C | Algorithm Complexity Reference — Real Operation Counts at Scale [SVG diagram — see module-5-3-algorithms-intro.html]*

Concrete operation counts for seven complexity classes across four input sizes. The table makes the abstract growth rates visceral: an O(n log n) algorithm handles a million inputs in ~20 million operations — fine for a modern processor. An O(n²) algorithm requires a trillion — still possible but slow. An O(2■) algorithm on 100 inputs requires 10³■ operations — physically impossible on any computer that could ever be built. Algorithm selection is not an optimisation — it is the difference between a program that finishes and one that never will.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 5.3 Algorithms: Thinking in Steps: Complexity Plotter Linear vs Binary Search Recurrence Solver [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 5.3 Algorithms: Thinking in Steps **Type:** Analysis + implementation + complexity proof **Estimated time:** 4555 minutes **Tools needed:** pen and paper + Python --- Part A Complexity Analysis For each code snippet, determine the time complexity as a function of n. Show your reasoning. **Snippet 1:** ```python def f1(n): total = 0 for i in range(n): for j in range(n): total += i * j return total ``` **Snippet 2:** ```python def f2(arr): n = len(arr) for i in range(n): for j in range(i + 1, n): inner loop starts at i+1

# Module 5 4 Recursion

■# Module 5.4 — Recursion: Functions That Call Themselves

## The Problem

*Some problems are defined in terms of themselves.*

*The factorial of n is n times the factorial of n-1.*

*The height of a tree is one plus the maximum height of its subtrees.*

*The files in a folder include all the files in all its subfolders.*

>

*Writing loops to solve these problems forces you to manage explicit stacks,*

*state variables, and awkward index arithmetic.*

*The code obscures the problem's natural structure.*

>

*Recursion expresses these problems directly — the code mirrors the definition.*

## The Story

### Recursive Thinking Before Computers

Long before programming languages, mathematicians defined functions recursively. **Giuseppe Peano** (1889) defined the natural numbers recursively: 0 is a number; if n is a number, then the successor of n is a number. **Ackermann's function** (1928) was one of the first examples of a function that is computable but not *primitive recursive* — it grows faster than any fixed power of n.

In programming, recursive functions became possible once CPUs had the call stack (Module 4.2). The ability to call a function from within itself, with each call getting its own stack frame, is precisely what makes recursion work. Early languages like FORTRAN (1957) did not support recursion — function calls used a fixed return address, so a function calling itself would overwrite its own return address.

ALGOL 60 introduced proper recursion with a hardware-supported call stack.

## The Concept

### The Structure of a Recursive Function

Every correct recursive function has two parts:

**Base case**: the simplest input that can be answered directly, without further recursion. Without a base case, the function recurses forever → stack overflow.

**Recursive case**: the function calls itself with a *smaller* version of the problem, moving toward the base case.

```
def factorial(n):
    # Base case
    if n <= 1:
        return 1
    # Recursive case: n! = n × (n-1)!
    return n * factorial(n - 1)
```

Trace for `factorial(4)`:

```
factorial(4)
  → 4 × factorial(3)
       → 3 × factorial(2)
            → 2 × factorial(1)
                 → 1          (base case returns)
            ← 2 × 1 = 2
       ← 3 × 2 = 6
  ← 4 × 6 = 24
```

Each call waits on the stack until its sub-call returns. The call stack holds 4 frames simultaneously for `factorial(4)`.

### Stack Frames and Depth

Every recursive call creates a new stack frame (as shown in Module 4.2). The maximum recursion depth is limited by the stack size. Python's default limit is 1,000 frames:

```
import sys
print(sys.getrecursionlimit())   # 1000

def infinite(n):
    return infinite(n + 1)

infinite(0)  # RecursionError: maximum recursion depth exceeded
```

For problems that require deep recursion (trees with depth 10,000, files nested 500 levels), you need either:

• An iterative solution using an explicit stack data structure

• Tail call optimisation (TCO) — the compiler reuses the stack frame when the recursive call is the very last thing done (Python does not do TCO; Scheme requires it)

• Increasing the stack size (risky)

### Fibonacci — The Classic Trap

```
def fib_naive(n):
    if n <= 1: return n
    return fib_naive(n-1) + fib_naive(n-2)
```

This is **O(2■)** — exponential time. `fib(50)` takes about 10¹■ operations. The problem: the same sub-problems are computed repeatedly:

```
fib(5)
■■■ fib(4)
■   ■■■ fib(3)        ← computed again below
■   ■   ■■■ fib(2)
■   ■   ■■■ fib(1)
■   ■■■ fib(2)
■■■ fib(3)            ← duplicate
    ■■■ fib(2)        ← duplicate
    ■■■ fib(1)
```

**Memoisation** fixes this in one line:

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

Now O(n) — each sub-problem computed once, result cached. This is the core idea of **dynamic programming**: cache sub-problem results.

### Recursion on Data Structures

Recursion shines on naturally recursive structures — particularly **trees** (covered in Level 6):

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val   = val
        self.left  = left
        self.right = right

def tree_height(node):
    if node is None:           # base case: empty tree has height 0
        return 0
    left_h  = tree_height(node.left)   # height of left subtree
    right_h = tree_height(node.right)  # height of right subtree
    return 1 + max(left_h, right_h)    # this node adds 1

def tree_sum(node):
    if node is None: return 0
    return node.val + tree_sum(node.left) + tree_sum(node.right)
```

An iterative tree traversal requires an explicit stack. The recursive version is identical in structure to the mathematical definition.

### Mutual Recursion

Two or more functions can call each other:

```
def is_even(n):
    if n == 0: return True
    return is_odd(n - 1)

def is_odd(n):
    if n == 0: return False
    return is_even(n - 1)
```

This is elegant but not efficient — use `n % 2 == 0` in practice. The concept matters: parsers for context-free grammars are often built as mutually recursive functions, where each function handles one grammar rule.

### Tower of Hanoi

The canonical recursive problem: move n disks from peg A to peg C, using peg B as auxiliary, without placing a larger disk on a smaller one.

```
def hanoi(n, source, target, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    hanoi(n-1, source, auxiliary, target)   # move n-1 disks out of the way
    print(f"Move disk {n} from {source} to {target}")
    hanoi(n-1, auxiliary, target, source)   # move n-1 disks to target

hanoi(3, 'A', 'C', 'B')
```

The solution requires exactly **2■ – 1** moves. There is no simpler way — the recursion captures the entire problem structure. The Tower of Hanoi is also a model for the recursive nature of the call stack itself: each

### Illustrations

**Figure 101 • FIG 5.4-A**

**Recursive Call Tree — factorial(5) Unrolled**

*Illustration: FIG 5.4-A | Recursive Call Tree — factorial(5) Unrolled [SVG diagram — see module-5-4-recursion.html]*

Recursive call tree for factorial(5). The function descends through 6 levels of calls, each generating a new stack frame. The base case (factorial(0) = 1) terminates the descent. The return phase propagates back up: 1 × 1 = 1 → 2 × 1 = 2 → 3 × 2 = 6 → 4 × 6 = 24 → 5 × 24 = 120. Every recursive algorithm has this two-phase structure: a descent phase that reduces the problem, and an ascent phase that combines sub-results. The depth of recursion determines stack space consumption — factorial(5) uses 6 frames, factorial(10000) would use 10001.

**Figure 102 • FIG 5.4-B**

### Fibonacci: Naive Recursion vs Memoisation — Eliminating Redundant Work

*Illustration: FIG 5.4-B | Fibonacci: Naive Recursion vs Memoisation — Eliminating Redundant Work [SVG diagram — see module-5-4-recursion.html]*

Naive recursive Fibonacci (left) recomputes overlapping subproblems: fib(3) is called twice, fib(2) three times, for a total of 15 calls to compute fib(5). For fib(50) the naive approach makes approximately 2■■ ≈ 10¹■ calls — infeasible. Memoisation (right) stores each result in a dictionary on first computation and returns the cached value on subsequent lookups. fib(5) with memoisation makes exactly 5 unique calls — O(n). This insight — that recursive problems with overlapping subproblems can be solved in polynomial time by storing sub-results — is the core idea of dynamic programming.

**Figure 103 • FIG 5.4-C**

### Tail Recursion — When the Compiler Can Eliminate Stack Frames

*Illustration: FIG 5.4-C | Tail Recursion — When the Compiler Can Eliminate Stack Frames [SVG diagram — see module-5-4-recursion.html]*

Tail recursion places the recursive call as the absolute last operation, with no work remaining for the caller after the recursive call returns. A compiler can recognise this and reuse the same stack frame instead of pushing a new one — Tail Call Optimisation (TCO). Regular factorial must keep its frame alive to perform the final multiplication; tail-recursive factorial with an accumulator parameter has no such obligation. Languages with mandatory TCO (Scheme, Haskell) can recurse indefinitely without stack overflow. Python deliberately does not implement TCO to preserve readable tracebacks.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 5.4 Recursion: Functions That Call Themselves: Call Stack Visualiser Memoisation Demo Tower of Hanoi [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 5.4 Recursion: Functions That Call Themselves **Type:** Trace + implementation + induction proof **Estimated time:** 4555 minutes **Tools needed:** pen and paper + Python --- Part A Stack Frame Trace Trace the execution of `factorial(5)` completely. For each function call, show: - The call being made - The current stack depth - The return value when it unwinds ``` factorial(5) 5 factorial(4) 4 factorial(3) ... ``` 1. Complete the trace all the way to the base case and back. 2. How many stack frames are on the call stack simultaneously at the deep

# Module 5 5 Sorting

■# Module 5.5 — Sorting: Putting Things in Order

## The Problem

*A database table has 50 million rows.*

*A search engine needs to rank 10 billion web pages.*

*A hospital system needs to sort 100,000 patient records by urgency in real time.*

>

*Sorting is one of the most fundamental operations in computing.*

*It is estimated that 25% of all CPU time in the world is spent sorting.*

*And the difference between a bad sort algorithm and a good one*

*is the difference between a program that works and one that never finishes.*

## The Story

### The Sorting Problem Drove Algorithm Theory

In the 1950s and 60s, IBM's core business was data processing — payroll, census tabulation, inventory management. Sorting punched cards and magnetic tape records was the bottleneck. IBM researchers spent enormous effort on sort algorithms because a 2× speedup in sort performance would save millions of dollars in computing time.

**John von Neumann** wrote Mergesort in 1945 — the first O(n log n) sort algorithm. **Tony Hoare** invented Quicksort in 1959 while working on machine translation at Moscow State University. The sorting problem was so important that it drove major theoretical advances: the proof that no comparison-based sort can be faster than O(n log n) in the worst case (the information-theoretic lower bound), and the discovery of Radix sort which breaks this barrier by not using comparisons.

The standard sort algorithm in production today (Timsort, used in Python, Java, and Android) was invented in 2002 by Tim Peters specifically for Python. It exploits real-world patterns in data — partially sorted runs — to achieve near-linear performance on common inputs.

## The Concept

### The Sorting Problem

**Input**: an array of n elements with a total order (we can compare any two elements). **Output**: the same elements in non-decreasing order. **Constraint**: we count comparisons and swaps; we want to minimise them.

### Bubble Sort — O(n²) — The Slow Baseline

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

Each pass "bubbles" the largest unsorted element to its final position. n passes × n comparisons each = O(n²).

In practice: never use bubble sort. It is taught because it is simple to understand and implement, but its performance is catastrophically bad for n > 1,000.

### Merge Sort — O(n log n) — Guaranteed

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left  = merge_sort(arr[:mid])    # sort left half
    right = merge_sort(arr[mid:])    # sort right half
    return merge(left, right)        # merge two sorted halves

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

**Why O(n log n)?** Divide: log■(n) levels of recursion. Each level merges total n elements. Total: n × log n comparisons.

**Stable**: elements with equal keys maintain their original relative order. **Not in-place**: requires O(n) extra space for the merge step. **Predictable**: always O(n log n), never worse. Preferred when worst-case matters (e.g., real-time systems).

### Quick Sort — O(n log n) average, O(n²) worst

```
def quick_sort(arr, low=0, high=None):
    if high is None: high = len(arr) - 1
    if low < high:
        p = partition(arr, low, high)
        quick_sort(arr, low, p - 1)
        quick_sort(arr, p + 1, high)

def partition(arr, low, high):
    pivot = arr[high]    # choose last element as pivot
    i = low - 1
    for j in range(low, high):
```

```
            if arr[j] <= pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]
        arr[i+1], arr[high] = arr[high], arr[i+1]
        return i + 1
```

**Pivot selection** is everything. With a good pivot (median-of-three, random), the array splits roughly in half each time: O(n log n). With the worst pivot (always the smallest or largest element, e.g., sorted input with naive pivot choice): one partition gets n-1 elements → $O(n^2)$.

**In-place**: O(log n) stack space for recursion, O(1) extra space otherwise. **Cache-friendly**: accesses memory sequentially — excellent real-world performance. **Not stable**: equal elements may be reordered.

In practice, Quicksort with randomised or median-of-three pivot is faster than Mergesort on random data by a constant factor, despite the same O(n log n) asymptotics.

### Heap Sort — O(n log n), in-place

Builds a max-heap from the array (O(n)), then repeatedly extracts the maximum (O(log n) each × n times).

• Always O(n log n), in-place, not stable

• Cache-unfriendly (heap accesses jump around memory)

• Used when guaranteed in-place O(n log n) is required

### Radix Sort — O(nk) — Beats the Lower Bound

The O(n log n) lower bound applies only to **comparison-based** sorts. Radix sort does not compare elements — it distributes them into buckets by individual digits.

```
def radix_sort(arr, base=10):
    max_val = max(arr)
    exp = 1
    while max_val // exp > 0:
        counting_sort_by_digit(arr, base, exp)
        exp *= base
```

For 32-bit integers with base 256: 4 passes, each O(n) → O(4n) = **O(n)**. In practice, the constant factor is large, and Radix sort is only faster than comparison sorts for very large n with bounded key size (e.g., sorting IP addresses, DNA sequences).

### Timsort — The Real-World Champion

Python's built-in `sorted()` and `list.sort()` use **Timsort** (Tim Peters, 2002):

Find natural **runs** of already-sorted data in the input (forward or backward)

Ensure each run is at least `minrun` elements (typically 32–64) using insertion sort

**Merge** runs pairwise using a merge sort

### Illustrations

**Figure 104 • FIG 5.5-A**

### Bubble Sort — Step-by-Step Trace on [5, 3, 8, 1, 9, 2]

*Illustration: FIG 5.5-A | Bubble Sort — Step-by-Step Trace on [5, 3, 8, 1, 9, 2] [SVG diagram — see module-5-5-sorting.html]*

Bubble sort pass 1 on [5, 3, 8, 1, 9, 2]. The algorithm compares adjacent elements and swaps them if out of order. After pass 1, the largest element (9) has "bubbled" to its final position. After pass 2, the second-largest reaches its place, and so on. $O(n^2)$ total comparisons: n(n-1)/2. For n=1000, that is about 500,000 comparisons. Bubble sort is studied for its algorithmic clarity, not its practical performance — it motivates the search for O(n log n) algorithms that can sort a million elements in ~20 million operations instead of ~500 billion.

**Figure 105 • FIG 5.5-B**

### Merge Sort — Divide, Conquer, Combine

*Illustration: FIG 5.5-B | Merge Sort — Divide, Conquer, Combine [SVG diagram — see module-5-5-sorting.html]*

Merge sort on [5, 3, 8, 1]. The divide phase recursively splits the array in half until single-element arrays are reached (trivially sorted). The conquer phase merges adjacent sorted pairs into larger sorted arrays. The merge step compares the front elements of two sorted arrays and takes the smaller — O(n) work per level. With log n recursive levels, total work is O(n log n) — guaranteed regardless of input order. Merge sort is stable (preserves equal elements' order) and optimal for comparison-based sorting by the lower-bound proof.

**Figure 106 • FIG 5.5-C**

### Sorting Algorithm Comparison — Complexity, Stability, Space

*Illustration: FIG 5.5-C | Sorting Algorithm Comparison — Complexity, Stability, Space [SVG diagram — see module-5-5-sorting.html]*

Sorting algorithm comparison across complexity, space, and stability properties. The O(n log n) lower bound applies to comparison-based sorting — no algorithm that only compares elements can sort faster in the general case. Practical choice: Quicksort for general purpose (fastest in cache, O(n log n) average), Merge Sort when stability is required, Heap Sort when guaranteed worst-case O(n log n) with O(1) space is needed, Insertion Sort for small or nearly-sorted arrays, Radix Sort when keys are integers and the digit count k is small. Python's Timsort is a hybrid merge-insertion sort optimise

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 5.5 Sorting: Putting Things in Order: Sort Race Merge Sort Tree Quicksort Partition [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 5.5 Sorting: Putting Things in Order **Type:** Trace + implementation + analysis **Estimated time:** 4555 minutes **Tools needed:** pen and paper + Python --- Part A Merge Sort Trace Sort the array `[5, 3, 8, 1, 9, 2, 7, 4]` using Merge Sort. Show every step. 1. Draw the **divide phase**: show the array split recursively into sub-arrays until you reach single elements. 2. Draw the **merge phase**: show every merge operation, listing the two input sub-arrays and the merged output. 3. Count the total number of comparisons made during all merge operations. Compare to the th

# Module 5 6 Searching

■# Module 5.6 — Searching: Finding What You Need

## The Problem

*A phone book has 5 million entries.*

*Finding a name by scanning every entry takes minutes.*

*Finding it by binary search takes microseconds.*

>

*Google's index has 100 billion web pages.*

*Finding pages matching a query in under 100 milliseconds*

*requires not just a fast search algorithm but an entirely*

*different data structure designed for search from the ground up.*

Searching is the inverse of sorting, and understanding how to find things efficiently — from a sorted array, an unsorted collection, a graph, or a text string — is foundational to almost every software system ever built.

## The Story

### From Card Catalogues to Search Engines

Before computers, librarians invented efficient search structures: alphabetical catalogues, Dewey Decimal classification, card indexes. These were manual implementations of data structures designed to reduce lookup time from O(n) (scan every card) to O(log n) (binary search through a sorted index).

**Vannevar Bush** (1945) described the Memex in his essay "As We May Think" — a hypothetical device that could store and rapidly retrieve vast amounts of information using associative links. This vision

directly inspired hypertext and the web.

The first web search engine, **Archie** (1990), simply indexed FTP filenames. **AltaVista** (1995) indexed full page text. **Google** (1998) introduced PageRank — ranking by link structure rather than just keyword frequency — and transformed search from a text-matching problem into a graph analysis problem. Today, a Google search traverses an index of 100 billion pages in under 100ms on a cluster of thousands of machines.

---

## The Concept

### Linear Search — O(n)

The simplest possible search: check every element until you find the target or exhaust the collection.

```
def linear_search(arr, target):
    for i, x in enumerate(arr):
        if x == target:
            return i      # found at index i
    return -1             # not found
```

**When to use**: unsorted data, small collections (n < ~50), or when you need to find all occurrences. **When not to use**: large sorted collections — binary search is orders of magnitude faster.

### Binary Search — O(log n)

Requires the array to be sorted. At each step, compare the target to the middle element and eliminate half the remaining search space.

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1    # target is in the right half
        else:
            high = mid - 1   # target is in the left half
    return -1                 # not found
```

**Why O(log n)?** Each comparison halves the remaining search space. Starting with n elements, after k comparisons we have n/2^k elements left. When n/2^k = 1, k = log■(n). For n = 1,000,000: at most 20 comparisons.

**Invariant**: at every iteration, if the target exists, it is in `arr[low..high]`. Maintaining this invariant is the key to getting the boundary conditions right.

**Common mistakes**:

• Off-by-one in `mid + 1` / `mid - 1` — leads to infinite loops or missed elements

• Integer overflow in `(low + high) // 2` for very large arrays (use `low + (high - low) // 2` in C)

### Binary Search Variants

Binary search has many useful generalisations:

**Find first occurrence of target** (duplicates in array):

```
def first_occurrence(arr, target):
    low, high, result = 0, len(arr) - 1, -1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            result = mid
            high = mid - 1   # keep searching left for earlier occurrence
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return result
```

**Find insertion point** (like Python's `bisect.bisect_left`): Returns the index where target should be inserted to keep the array sorted.

**Search on answer** — a powerful technique: binary search on the *answer space* rather than the data. For example: "what is the minimum speed at which I can complete a task within T hours?" Binary search over possible speeds, using a feasibility function to check each candidate.

### Hash Tables — O(1) Average

Binary search requires sorted data and gives O(log n). For O(1) average-case search, we use a **hash table** (covered in depth in Level 6):

```
d = {'alice': 30, 'bob': 25, 'carol': 28}
d['alice']        # O(1) average — hash lookup
'bob' in d        # O(1) average
```

The hash function maps the key to an array index. Lookup: hash the key, go directly to that index. No comparisons needed. The price: hash tables use more memory, and worst-case is O(n) if many keys hash to the same slot.

### Graph Search — BFS and DFS

When data has relationships (not just linear order), we need graph search. Covered fully in Level 6 (Module 6.6), but the foundations are:

**Breadth-First Search (BFS)**: explore level by level, using a queue.

• Finds the shortest path (fewest edges) in an unweighted graph

• O(V + E) where V = vertices, E = edges

**Depth-First Search (DFS)**: go as deep as possible before backtracking, using a stack (or recursion).

• Detects cycles, topological sort, connected components

• O(V + E)

### String Search — KMP Algorithm

Finding a pattern of length m within a text of length n. Naive: try every position → O(nm). The **Knuth-Morris-Pratt (KMP)** algorithm preprocesses the pattern to avoid redundant comparisons → O(n + m).

```
Text:     AAABAAABAAAC
Pattern:  AAABAAAC

Naive: would restart from scratch on each mismatch
KMP: uses a failure function to skip characters already matched
```

The core idea: when a mismatch occurs, the pattern's internal structure tells us how far to shift without losing already-matched characters. This makes substring search in a 1GB log file fast enough

## Illustrations

**Figure 107 • FIG 5.6-A**

### Binary Search — Interval Halving on a Sorted Array of 32 Elements

*Illustration: FIG 5.6-A | Binary Search — Interval Halving on a Sorted Array of 32 Elements [SVG diagram — see module-5-6-searching.html]*

Binary search on a 32-element sorted array. Each comparison eliminates exactly half the remaining candidates. The search interval [lo, hi] shrinks from [0,31] to [0,14] to [8,14] to [12,14] to [12,12] — found in 5 steps. The maximum steps needed equals log■(n): for n=32 that is 5, for n=1,000,000 that is 20. Binary search requires the array to be sorted — if the data arrives in arbitrary order, the cost of maintaining sorted order (via an index or sorted insert) must be weighed against the search speedup.

**Figure 108 • FIG 5.6-B**

### Hash Table — O(1) Average Lookup via Key → Hash → Bucket

*Illustration: FIG 5.6-B | Hash Table — O(1) Average Lookup via Key → Hash → Bucket [SVG diagram — see module-5-6-searching.html]*

Hash table architecture. The hash function converts any key (string, integer, object) to an integer index modulo the bucket count. Keys "alice" and "eve" both hash to bucket 3 — a collision. Chaining resolves this by storing a linked list in each bucket; the worst case (all keys collide) degrades to O(n), but with a good hash function and load factor below 0.75, average performance is O(1) for insert, lookup, and delete. This is why Python dict and set, Java HashMap, and C++ unordered_map all achieve O(1) average operations — and why hash table selection and implementation is one of the most i

**Figure 109 • FIG 5.6-C**

**Search Algorithm Selection Guide — Structure, Complexity, Trade-offs**

*Illustration: FIG 5.6-C | Search Algorithm Selection Guide — Structure, Complexity, Trade-offs [SVG diagram — see module-5-6-searching.html]*

Search algorithm selection guide. Linear search requires no preconditions but scales poorly. Binary search gives O(log n) but requires sorted data. Hash tables achieve O(1) average for exact key lookup — the right choice for dictionaries, caches, and sets. Binary search trees (BSTs) support range queries and ordered iteration that hash tables cannot. Tries excel at string prefix matching (autocomplete). Bloom filters achieve space-efficient probabilistic membership testing at the cost of false positives. The choice is always driven by the query pattern, not just the complexity class.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 5.6 Searching: Finding What You Need: Linear vs Binary Search Binary Search Variants Trie Builder [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 5.6 Searching: Finding What You Need **Type:** Trace + implementation + analysis **Estimated time:** 4050 minutes **Tools needed:** pen and paper + Python --- Part A Binary Search Trace Sort the array `[2, 5, 8, 12, 16, 23, 38, 44, 56, 72, 91]` (11 elements, indices 010). 1. Trace binary search for target = **38**: show `low`, `high`, and `mid` at each step, and which half is eliminated. 2. Trace binary search for target = **50** (not present): show every step until `low > high`. 3. Binary search on a 1-billion-element sorted array requires at most how many comparisons?

LEVEL 06

# Data Structures & Algorithms

Organising Information for Efficient Computation

# ... append until capacity full

■# Module 6.1 — Arrays and Linked Lists

## The Problem

*You need to store a collection of items.*

*You will add items, remove items, and access them by position or by searching.*

>

*The obvious choice: just put them all in a row.*

*But "in a row" has hidden costs.*

*Inserting in the middle means moving everything else.*

*Not knowing the size in advance means either wasting space or copying everything.*

>

*The first decision in every data structure course is the most fundamental:*

***contiguous or linked?***

## The Story

### Arrays — The First Data Structure

The array is as old as stored-program computing. A memory address is a number; an array is just a block of memory at a base address, with each element at `base + index × element_size`. The CPU can compute any element's address in one multiplication and one addition — O(1) access.

FORTRAN (1957), the first widely-used programming language, was built around arrays. Matrix arithmetic in FORTRAN was the entire point: scientific computing required fast access to elements by index. The array's O(1) access is why cache memory (Level 3) was designed the way it was — sequential memory access patterns fill cache lines efficiently.

Linked lists were invented slightly later, when programmers needed to handle collections whose size wasn't known at compile time. John McCarthy's **Lisp** (1958) was the first language built around linked lists — in Lisp, everything is a list. The language's name stands for "LISt Processing."

## The Concept

### Arrays

An array stores elements in **contiguous memory**. For an array of `int` (4 bytes each) starting at address A:

```
Element:  arr[0]   arr[1]   arr[2]   arr[3]
Address:  A+0      A+4      A+8      A+12
```

**Random access**: `arr[i]` → address `A + i * sizeof(element)` → one instruction. **O(1)**.

**Insertion at end**: if capacity remains, write at `arr[length]` and increment length. **O(1) amortised** (Python lists, C++ `vector` double capacity when full).

**Insertion at middle**: must shift all elements after the insertion point right. **O(n)**.

**Deletion at middle**: shift all elements after the deletion point left. **O(n)**.

**Search (unsorted)**: scan from index 0 until found. **O(n)**.

**Search (sorted)**: binary search. **O(log n)**.

**Memory**: no overhead per element — just the data itself.

### Dynamic Arrays

Static C arrays have fixed size declared at compile time. **Dynamic arrays** (Python `list`, C++ `vector`, Java `ArrayList`) grow automatically:

```
lst = []           # empty, capacity 0
lst.append(1)      # allocates initial capacity (e.g., 4 slots)
lst.append(2)      # fits in existing capacity
# ... append until capacity full
lst.append(x)      # RESIZE: allocate new array (2× capacity),
                   # copy all elements, free old array
```

The resize is O(n) work, but it happens rarely. Amortised analysis: n appends on a fresh list do at most O(2n) total work (geometric series: n + n/2 + n/4 + ...) → **O(1) amortised per append**.

### Linked Lists

A linked list stores elements in **nodes** scattered anywhere in memory. Each node contains the data and a pointer to the next node:

```
[data:1 | next:→] → [data:2 | next:→] → [data:3 | next:None]
  head                                    tail
```

In Python:

```
class Node:
    def __init__(self, data, next_node=None):
        self.data = data
        self.next = next_node

class LinkedList:
    def __init__(self):
        self.head = None

    def prepend(self, data):
        self.head = Node(data, self.head)   # O(1)

    def append(self, data):                  # O(n) — must traverse to tail
        if not self.head:
            self.head = Node(data); return
        curr = self.head
        while curr.next:
            curr = curr.next
```

```
            curr.next = Node(data)

    def delete(self, data):                  # O(n) — must find node
        if not self.head: return
        if self.head.data == data:
            self.head = self.head.next; return
        curr = self.head
        while curr.next:
            if curr.next.data == data:
                curr.next = curr.next.next; return
            curr = curr.next
```

**Insert at head**: update head pointer only. **O(1)**. **Insert at tail** (without tail pointer): traverse to last node. **O(n)**. **Insert at tail** (with tail pointer maintained): **O(1)**. **Delete given node pointer**: update previous node's `next`. **O(1)** if you have the previous node, **O(n)** if you must find it. **Random access** (`list[5]`): must traverse from head. **O(n)** — no index formula exists. **Memory**: each node stores a pointer (8 bytes on 64-bit) plus the data. More overhead than arrays.

### Doubly Linked Lists

Each node has both a `next` and `prev` pointer:

```
None ← [1] ↔ [2] ↔ [3] → None
```

Enables O(1) deletion when you have a pointer to the node (update both neighbours without traversing). Python's `collections.deque` is implemented as a doubly linked list — O(1) append and pop from both ends.

### When to Use Each

| Operation | Array (dynamic) | Linked List | |---|---|---| | Access by index | O(1) | O(n) | | Insert at end | O(1) amortised | O(1) with tail ptr | | Insert at start | O(n) | O(1) | | Insert at middle | O(n) | O(1) with node ptr | | Delete at middle | O(n) | O(1) with node ptr | | Memory per element | Low | High (pointer overhead) | | Cache friendliness | Excellent | Poor (nodes scattered) |

**In practice**: dynamic arrays (Python lists, C++ vectors) are used for almost everything. Their cache friendliness makes them faster than linked lists even for operations where linked lists have better asymptotic complexity. Linked lists are used when: insertions/deletions in the middle are extremely frequent and you already have node pointers, or when elements must not be moved in memory (e.g., intrusive linked lists in the Linux kernel, where struct members point to each other directly).

---

## Hands-On Exercis

### Illustrations

**Figure 110 • FIG 6.1-A**

### Array vs Linked List — Memory Layout Comparison



*Illustration: FIG 6.1-A | Array vs Linked List — Memory Layout Comparison [SVG diagram — see module-6-1-arrays-linked-lists.html]*

Array vs linked list memory layout. Arrays store elements at consecutive addresses — index i is found by one multiply-and-add: base + i×size, giving O(1) random access. Linked list nodes are scattered anywhere in the heap, connected by pointers. Accessing element i requires following i pointers from the head — O(n). However, inserting or deleting in a linked list only requires updating two pointers — O(1) once the position is known. The fundamental trade-off: arrays are fast to read, linked lists are fast to modify structurally.

**Figure 111 • FIG 6.1-B**

### Linked List Insert and Delete — Pointer Manipulation



*Illustration: FIG 6.1-B | Linked List Insert and Delete — Pointer Manipulation [SVG diagram — see module-6-1-arrays-linked-lists.html]*

Linked list pointer manipulation for insert and delete. Inserting a new node between A and B requires exactly two pointer updates: new_node.next = A.next, then A.next = new_node — no element shifting, no reallocation, O(1). Deleting node B requires one update: A.next = B.next — then free B's memory. The critical caveat: both operations require knowing the predecessor node. Finding that predecessor still costs O(n) traversal from the head. Doubly-linked lists add a prev pointer to each node, enabling O(1) delete given only the node itself.

**Figure 112 • FIG 6.1-C**

### Array vs Linked List — Operation Complexity Summary



*Illustration: FIG 6.1-C | Array vs Linked List — Operation Complexity Summary [SVG diagram — see module-6-1-arrays-linked-lists.html]*

Operation complexity table for arrays, singly linked lists, and doubly linked lists. The doubly linked list achieves O(1) delete when given a direct node reference because each node knows its predecessor through prev. Arrays achieve O(1) append amortized — dynamic arrays (Python list, C++ vector) occasionally resize by doubling capacity, but the average cost per append over N operations is O(1). The practical choice depends on the dominant operation: favour arrays for indexed reads, favour doubly linked lists for frequent arbitrary insertions and deletions.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 6.1 Arrays and Linked Lists: Dynamic Array Linked List Visualiser Head-to-Head Comparison
[Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 6.1 Arrays and Linked Lists **Type:** Implementation + trace + analysis **Estimated time:** 4555 minutes **Tools needed:** pen and paper + Python --- Part A Linked List Implementation Implement a singly linked list with these methods: ```python class Node: def __init__(self, data): ... class LinkedList: def prepend(self, data): ... O(1) def append(self, data): ... O(n) no tail pointer def delete(self, data): ... remove first occurrence def reverse(self): ... in-place, O(n) def to_list(self): ... convert to Py

# Module 6 2 Stacks Queues

■# Module 6.2 — Stacks and Queues

## The Problem

*Two fundamental questions about collections:*

>

*1. When you take something out, should it be the most recently added item (Last In, First Out)?*

*→ A stack of plates, the call stack, the browser back button.*

>

*2. Or should it be the least recently added item (First In, First Out)?*

*→ A print queue, a line at a ticket counter, CPU job scheduling.*

>

*Stacks and queues are not clever data structures.*

*They are principled restrictions on how a collection is accessed —*

*and those restrictions turn out to match the natural structure of dozens of important problems.*

## The Story

### The Call Stack and the History of Stacks

The concept of a stack in computing is inseparable from the call stack — the hardware mechanism for function calls described in Module 4.2. Every time you call a function, the return address is pushed; every time you return, it is popped. This is a stack in its purest form.

Alan Turing described stack-like mechanisms in 1946 for his ACE computer design. Friedrich Bauer and Klaus Samelson in Germany, and Charles Hamblin in Australia, independently formalised the push-down stack in the late 1950s.

The calculator application of stacks came from Hamblin: **Reverse Polish Notation (RPN)** — write `3 4 +` instead of `3 + 4`. RPN evaluates perfectly with a stack and requires no parentheses, no operator precedence rules, no parse tree. HP's engineering calculators used RPN through the 1970s–80s. Understanding why RPN works is the clearest introduction to stack-based computation.

Queues formalise the fair waiting structure that humans already understood intuitively — but computers needed an explicit data structure to implement scheduling, buffering, and breadth-first search.

## The Concept

### Stacks — LIFO

A stack supports two operations:

• **push(item)**: add item to the top

• **pop()**: remove and return the top item

• **peek()**: look at the top item without removing it

• **is_empty()**: return True if no items

```
class Stack:
    def __init__(self):
        self._data = []

    def push(self, item):
        self._data.append(item)       # O(1) amortised

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self._data.pop()       # O(1) amortised

    def peek(self):
        return self._data[-1]         # O(1)

    def is_empty(self):
        return len(self._data) == 0
```

All operations O(1) amortised using a dynamic array as backing store.

**Key applications:**

**Expression evaluation** (RPN):

```
def eval_rpn(tokens):
    stack = Stack()
    for t in tokens:
        if t in '+-*/':
            b, a = stack.pop(), stack.pop()
            result = {'+':a+b, '-':a-b, '*':a*b, '/':a/b}[t]
            stack.push(result)
        else:
            stack.push(float(t))
```

```
    return stack.pop()

eval_rpn(['3','4','+','2','*'])   # (3+4)*2 = 14
```

**Balanced parentheses check**:

```
def is_balanced(s):
    stack = Stack()
    pairs = {')':'(', ']':'[', '}':'{'}
    for c in s:
        if c in '([{':
            stack.push(c)
        elif c in ')]}':
            if stack.is_empty() or stack.pop() != pairs[c]:
                return False
    return stack.is_empty()

is_balanced("({[]})")   # True
is_balanced("({[}])")   # False
```

**Undo/redo**: push each action onto the undo stack. `Ctrl+Z` pops from the undo stack and pushes onto the redo stack.

**DFS without recursion**:

```
def dfs(graph, start):
    stack = Stack()
    visited = set()
    stack.push(start)
    while not stack.is_empty():
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            for neighbour in graph[node]:
                stack.push(neighbour)
```

### Queues — FIFO

A queue supports:

• **enqueue(item)**: add item to the back

• **dequeue()**: remove and return the front item

• **peek()**: look at the front item

• **is_empty()**

Implementing a queue with a Python list is inefficient: `list.pop(0)` is O(n) because it shifts every element. Use Python's `collections.deque` (doubly linked list):

```
from collections import deque

class Queue:
    def __init__(self):
        self._data = deque()

    def enqueue(self, item):
```

```
        self._data.append(item)         # O(1)

    def dequeue(self):
        if self.is_empty():
            raise IndexError("dequeue from empty queue")
        return self._data.popleft()     # O(1)

    def is_empty(self):
        return len(self._data) == 0
```

**Key applications:**

**BFS (Breadth-First Search)**:

```
def bfs(graph, start):
    queue = Queue()
    visited = set([start])
    queue.enqueue(start)
    while not queue.is_empty():
        node = queue.dequeue()
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.enqueue(neighbour)
```

**Process scheduling**: the OS maintains a run queue of processes waiting for CPU time. Round Robin scheduling enqueues a process when its turn ends and dequeues the next one.

**Printer spooler**: jobs arrive and are enqueued. The printer dequeues and prints one at a time.

### Circular Buffer (Ring Buffer)

A fixed-size queue implemented as an array with two pointers (`head` and `tail`). When they reach the end of the array, they wrap around. No dynamic resizing needed:

```
Array: [_, _, 3, 4, 5, _, _]   (size 7)
           ↑           ↑
          head        tail (next write position)
```

Used in: audio buffers (steady stream of samples), network packet buffers, producer-consumer queues in embedded systems. O(1) enqueue and dequeue with a fixed, pre-allocated memory footprint.

### Monotonic Stack

A stack where elements are always in sorted order (either monotonically increasing or decreasing). Used to solve problems like "next greater element" in O(n):

```
def next_greater_element(arr):
    result = [-1
```

### Illustrations

**Figure 113 • FIG 6.2-A**

### Stack (LIFO) vs Queue (FIFO) — Structural Comparison

*Illustration: FIG 6.2-A | Stack (LIFO) vs Queue (FIFO) — Structural Comparison [SVG diagram — see module-6-2-stacks-queues.html]*

Stack and Queue are the two fundamental access-ordered data structures. A Stack enforces Last-In-First-Out (LIFO): push adds to the top, pop removes from the top. A Queue enforces First-In-First-Out (FIFO): enqueue adds to the rear, dequeue removes from the front. Both operations are O(1) regardless of structure size when implemented with a linked list or circular buffer. The key distinction is the relationship between insertion order and retrieval order — stacks reverse it, queues preserve it.

**Figure 114 • FIG 6.2-B**

### Stack Applications — Call Stack, Undo/Redo, Expression Evaluation

*Illustration: FIG 6.2-B | Stack Applications — Call Stack, Undo/Redo, Expression Evaluation [SVG diagram — see module-6-2-stacks-queues.html]*

Three canonical stack applications. The function call stack tracks activation frames — each function call pushes a frame containing local variables and the return address; return pops the frame and resumes the caller. Undo/redo systems maintain two stacks: Ctrl+Z pops from the undo stack and pushes onto the redo stack; Ctrl+Y reverses this. Postfix (Reverse Polish Notation) expression evaluation uses a single stack — operands are pushed, operators pop their operands, compute, and push the result. All three rely on LIFO order as a correctness requirement, not an optimisation.

**Figure 115 • FIG 6.2-C**

### Queue Applications — BFS Frontier, Task Scheduling, Print Queue

*Illustration: FIG 6.2-C | Queue Applications — BFS Frontier, Task Scheduling, Print Queue [SVG diagram — see module-6-2-stacks-queues.html]*

Three canonical queue applications. BFS uses a queue as its frontier — visited nodes' unvisited neighbours are enqueued, ensuring nodes are explored in order of increasing distance from the source, which guarantees shortest-path correctness on unweighted graphs. Round-robin CPU scheduling rotates processes through the ready queue, giving each a time slice in arrival order for fair CPU sharing. Print spoolers queue jobs in submission order so no document is skipped. All three require FIFO ordering as a correctness invariant, not merely a convenience.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 6.2 Stacks and Queues: Stack (LIFO) Queue (FIFO) BFS with Queue [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 6.2 Stacks and Queues **Type:** Implementation + trace + reasoning **Estimated time:** 4555 minutes **Tools needed:** pen and paper + Python --- Part A Stack with Two Queues Implement a stack using only two queues (FIFO structures). The stack must support `push(x)`, `pop()`, and `peek()` using only queue operations: `enqueue(x)`, `dequeue()`, `is_empty()`. 1. Implement `StackFromQueues`. You may make push O(1) and pop O(n), or push O(n) and pop O(1) choose one approach and implement it. 2. Trace the following sequence on your implementation, showing the state of both que

# Extremely simplified no collision handling

■# Module 6.3 — Hash Tables

## The Problem

*You have 10 million user records.*

*For each web request, you need to look up a user by username.*

*A sorted array gives O(log n) — about 23 comparisons per lookup.*

*A linear scan gives O(n) — up to 10 million comparisons.*

>

*Can we do better than O(log n)?*

*Can we achieve O(1) — constant time regardless of how many users exist?*

Yes. The answer is a **hash table** — one of the most important data structures ever invented, used in every database, every caching layer, every programming language's built-in dictionary type.

## The Story

### The Birth of Hashing

The concept of hash functions appeared in the early 1950s as a way to accelerate data retrieval on magnetic tape systems, where random access was impossible. **Hans Peter Luhn** of IBM described a hashing technique in 1953 for chained overflow buckets — essentially the same as what we use today.

The first thorough theoretical treatment came from **Arnold Dumey** (1956), who coined the term "hash" and analysed collision resolution strategies. By the 1960s, every major database system used hash indexes for O(1) lookup.

The insight: instead of storing elements in order (requiring log n steps to find), store each element at the index determined by a function of its key. Finding it later requires only recomputing that function — one step, regardless of table size.

## The Concept

### The Core Idea

A hash table uses an array of fixed size `M` (the "buckets") and a **hash function** `h(key) → [0, M-1]` to map keys to array indices.

**Insert**: compute `h(key)`, store the key-value pair at `table[h(key)]`. **Lookup**: compute `h(key)`, return `table[h(key)]`. **Delete**: compute `h(key)`, remove from `table[h(key)]`.

If the hash function maps different keys to different indices, all operations are O(1).

```
# Extremely simplified — no collision handling
class TinyHashTable:
    def __init__(self, size=16):
        self.size = size
        self.table = [None] * size

    def _hash(self, key):
        return hash(key) % self.size   # Python's built-in hash function

    def set(self, key, value):
        self.table[self._hash(key)] = value

    def get(self, key):
        return self.table[self._hash(key)]
```

### The Collision Problem

A collision occurs when two different keys hash to the same index. This is inevitable: if you store 1 million keys in a table of 1 million slots, by the Birthday Paradox, collisions start happening after only ~1,000 insertions. Two strategies:

**1. Chaining (Separate Chaining)**

Each bucket holds a linked list of all key-value pairs that hash there:

```
Table:
index 0: → [(alice, 30)] → None
index 1: → None
index 2: → [(bob, 25)] → [(carol, 28)] → None   ← collision
index 3: → [(dave, 22)] → None
...
```

Lookup: hash key → find bucket → scan linked list for matching key.

• **Average case**: O(1) if the load factor (n/M) is bounded by a constant

• **Worst case**: O(n) if all keys hash to the same bucket

• **Space**: extra pointer per element

Python's `dict` uses a variant of open addressing (below), but conceptually, chaining is the cleaner mental model.

**2. Open Addressing (Linear Probing)**

No linked lists — all entries stored in the array itself. On collision, probe for the next empty slot:

```
def _insert(self, key, value):
    idx = self._hash(key)
    while self.table[idx] is not None:
        if self.table[idx][0] == key:    # update existing
            self.table[idx] = (key, value); return
        idx = (idx + 1) % self.size      # linear probe: try next slot
    self.table[idx] = (key, value)
```

• **Better cache performance** than chaining (no pointer chasing, everything in one array)

• **Clustering problem**: long runs of occupied slots form, making probing slow

• **Load factor must stay below ~0.7** — resize when exceeded

• **Deletion is tricky**: cannot just clear a slot (breaks the probe chain). Use a "tombstone" marker.

**Quadratic probing** (`idx = (idx + c■i + c■i²) % M`) and **double hashing** (`idx = (h1(key) + i × h2(key)) % M`) spread keys more evenly and reduce clustering.

### The Hash Function

A good hash function:

• Is deterministic: `h(k) = h(k)` always

• Is fast to compute: ideally O(1) for fixed-size keys

• Distributes keys uniformly across buckets

• Minimises collisions for real-world key distributions

**For integers**: simple approaches include modular hashing `h(k) = k mod M`. But if M is a power of 2, only the low bits matter — if keys cluster in low bits, so do their hashes. Better: use a prime M, or multiply by a large constant and take the high bits.

**For strings**: sum character values mod M is poor (anagrams collide). Better:

```
def hash_string(s, M):
    h = 0
    for c in s:
        h = (h * 31 + ord(c)) % M   # Horner's method, prime multiplier
    return h
```

Python's `hash()` for strings uses **SipHash-1-3**, a cryptographic-strength hash function, specifically to prevent **hash flooding attacks** — an adversary who knows the hash function can craft inputs that all map to the same bucket, turning O(1) lookup into O(n). SipHash uses a secret seed randomised per process startup.

### Load Factor and Resizing

The **load factor** $\alpha$ = n / M (number of stored items / table size).

With chaining: expected O(1 + $\alpha$) per lookup. Keep $\alpha < 1$. With open addressing: performance degrades rapidly above $\alpha \approx 0.7$. Keep $\alpha < 0.7$.

When α exceeds the threshold, **resize**: allocate a new table of size ~2M, rehash all existing entries into the new table. This is O(n) work but happens infrequently.

Python's `dict` resizes when 2/3 full. Java's `HashMap` resizes at load factor 0.75.

### Python's dict Implementation

Python `dict` uses **open addressing with pseudo-random probing**:

Initial size: 8 slots

Hash function: `hash(key) % size` for initial index

On collision: use a probe sequence based on the full hash value (not linear) to reduce clustering

Load fact

### Illustrations

**Figure 116 • FIG 6.3-A**

**Hash Function Step-by-Step — Turning a String Key into a Bucket Index**

*Illustration: FIG 6.3-A | Hash Function Step-by-Step — Turning a String Key into a Bucket Index [SVG diagram — see module-6-3-hash-tables.html]*

The djb2 hash function traced step by step on key "cat". Each character's ASCII value is mixed into a running hash via multiply-by-33-and-add — a simple but effective avalanche operation. The final 193,488,125 is reduced to a bucket index by taking modulo the table size (8), landing in bucket 5. A good hash function must be deterministic, fast, and distribute keys uniformly to minimise collisions. Real implementations like MurmurHash3 use bitwise XOR and shifts for better statistical properties and near-zero correlation between similar keys.

**Figure 117 • FIG 6.3-B**

**Collision Resolution — Chaining vs Linear Open Addressing**

*Illustration: FIG 6.3-B | Collision Resolution — Chaining vs Linear Open Addressing [SVG diagram — see module-6-3-hash-tables.html]*

Two collision resolution strategies. Separate chaining stores a linked list in each bucket — colliding keys are appended to the list. Simple and robust at high load factors but each list node requires extra memory and pointer dereferences hurt CPU cache performance. Linear probing stores all data in the array itself — on collision, it scans forward to the next empty slot. Cache-friendly because data is contiguous, but clustering (long runs of filled slots) degrades performance, and deletion requires tombstone markers to avoid breaking probe chains. Python's dict uses a variant of open addressi

**Figure 118 • FIG 6.3-C**

**Load Factor and Rehashing — When Performance Degrades**

*Illustration: FIG 6.3-C | Load Factor and Rehashing — When Performance Degrades [SVG diagram — see module-6-3-hash-tables.html]*

Load factor α = items/buckets governs hash table performance. Chaining average lookup cost grows as 1 + α/2 — manageable even at higher load factors. Linear probing cost grows as 1/(1–α), approaching infinity as α approaches 1 (table full). Most implementations trigger a rehash at α = 0.75: allocate a new array of double the size, re-hash every existing key into the new table, then discard the old one. This O(n) rehash event occurs logarithmically rarely, so the amortized cost per insert remains O(1). Python's dict uses a load factor threshold of 2/3; Java's HashMap defaults to 0.75.

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 6.3 Hash Tables: Open Addressing (Linear Probing) Separate Chaining Hash Quality [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 6.3 Hash Tables **Type:** Trace + implementation + analysis **Estimated time:** 4555 minutes **Tools needed:** pen and paper + Python --- Part A Linear Probing Trace Hash table of size **M = 7**. Hash function: `h(k) = k % 7`. Collision resolution: **linear probing** (probe at `(h(k) + i) % M` for i = 0, 1, 2, ...). Insert keys in order: **10, 22, 31, 4, 15, 28, 17, 88, 59** 1. For each key, calculate `h(k)` and trace the probe sequence until an empty slot is found. Draw the final table. 2. How many total probes were needed across all insertions? 3. What is the load fac

# In-order: left root right sorted output for BST

■# Module 6.4 — Trees: BST, AVL, Red-Black

## The Problem

*A sorted array gives O(log n) search via binary search.*

*But inserting into a sorted array is O(n) — you must shift elements.*

>

*A linked list gives O(1) insert (given the right position).*

*But search is O(n) — no binary search without random access.*

>

*Can we have both O(log n) search AND O(log n) insert?*

Yes. The answer is the **balanced binary search tree** — a data structure that maintains sorted order dynamically, giving O(log n) for search, insert, and delete simultaneously.

---

## The Story

### Binary Search Trees — The Concept

A binary search tree (BST) stores keys in a tree structure such that for any node with key k:

• All keys in the **left subtree** < k

• All keys in the **right subtree** > k

This property enables binary-search-style lookups: at each node, compare the target to k and go left (smaller) or right (larger). With a balanced tree of n nodes, height ≈ log■ n, so search takes O(log n) steps.

**George Boole's** logical algebra (1854) and **Leonhard Euler's** graph theory (1736) laid mathematical groundwork, but BSTs as a data structure were formalised in the early 1960s in connection with symbol tables for compilers. Compilers needed to look up variable names rapidly — a BST on variable names gave O(log n) lookup vs O(n) for a linear scan.

The problem came quickly: trees go wrong. Insert keys in sorted order (1, 2, 3, 4, 5...) and the BST degenerates into a linked list — O(n) for everything. The solution: **self-balancing trees**.

**G.M. Adelson-Velsky and E.M. Landis** (1962) invented the AVL tree — the first self-balancing BST, named after their initials. **Rudolf Bayer and Ed McCreight** (1972) invented the B-tree (used in databases, Module 9.2). **Leonidas Guibas and Robert Sedgewick** (1978) invented the red-black tree, now used in Linux's scheduler, C++ `std::map`, Java `TreeMap`, and virtually every language's sorted map.

---

## The Concept

### Binary Search Tree (BST)

```
class BSTNode:
    def __init__(self, key, val=None):
        self.key   = key
        self.val   = val
        self.left  = None
        self.right = None

class BST:
    def search(self, node, key):
        if node is None or node.key == key:
            return node
        if key < node.key:
            return self.search(node.left, key)
```

```
            return self.search(node.right, key)

        def insert(self, node, key, val=None):
            if node is None:
                return BSTNode(key, val)
            if key < node.key:
                node.left  = self.insert(node.left, key, val)
            elif key > node.key:
                node.right = self.insert(node.right, key, val)
            else:
                node.val = val   # update existing
            return node
```

**In-order traversal** (left → root → right) produces keys in sorted order — this is the fundamental property:

```
def inorder(node):
    if node:
        inorder(node.left)
        print(node.key)
        inorder(node.right)
```

**Delete**: three cases:

Node has no children → remove it

Node has one child → replace node with child

Node has two children → replace node's key with in-order successor (smallest key in right subtree), then delete the successor

**Complexity (balanced tree)**: O(log n) search, insert, delete. **Complexity (worst case — degenerate)**: O(n) for all operations.

### AVL Trees — Height-Balanced

An AVL tree maintains the **balance factor** for every node: `height(left) - height(right) ∈ {-1, 0, 1}`. After every insert or delete, the tree **rotates** nodes to restore balance.

**Rotations** (O(1) operations):

**Right rotation** at node y:

```
     y              x
    / \            / \
   x   C    →     A   y
  / \                / \
 A   B              B   C
```

**Left rotation** at node x:

```
   x                y
  / \              / \
 A   y     →      x   C
    / \          / \
   B   C        A   B
```

Four cases after insertion:

• **Left-Left**: single right rotation at unbalanced node

• **Right-Right**: single left rotation

• **Left-Right**: left rotation on left child, then right rotation on node

• **Right-Left**: right rotation on right child, then left rotation on node

AVL trees guarantee height ≤ 1.44 × log■(n+2). All operations strictly O(log n).

### Red-Black Trees — Practically Faster

A red-black tree uses node colours (red/black) and a simpler set of invariants to maintain approximate balance:

Every node is red or black

The root is black

Every leaf (null sentinel) is black

If a node is red, both children are black (no two consecutive reds)

All paths from any node to its descendant leaves contain the same number of black nodes

These rules guarantee height ≤ 2 × log■(n+1). Red-black trees perform fewer rotations than AVL trees on average (AVL is more strictly balanced), making them faster in practice for workloads with many insertions and deletions.

**Used in**: Linux kernel (CFS scheduler, memory management), C++ `std::map`/`std::set`, Java `TreeMap`/`TreeSet`, Rust `BTreeMap`, most language standard library sorted containers.

### Tree Traversals

```
# In-order: left → root → right → sorted output for BST
def inorder(node, result=[]):
    if node:
        inorder(node.left, result)
        result.append(node.key)
        inorder(node.right, result)

# Pre-order: root → left → right → tree serialisation
def preorder(node):
    if node:
        print(node.key)
        preorder(node.left)
        preorder(node.right)

# Post-order: left → right → root → safe deletion order
def postorder(node):
    if node:
        postorder(node.left)
        postorder(node.right)
        print(node.key)         # process after children

# Level-order: breadth-first (uses a queue)
from collections import deque
```

```
  def level_order(root):
      q = deque([root])
      while q:
          node = q.popleft()
          print(node.key)
          if node.left:  q.append(node.left)
          if node.right: q.append(node.right)
```

##

## Illustrations

**Figure 119 • FIG 6.4-A**

### Binary Search Tree — Inserting [5, 3, 7, 1, 4, 6, 8] in Sequence

*Illustration: FIG 6.4-A | Binary Search Tree — Inserting [5, 3, 7, 1, 4, 6, 8] in Sequence [SVG diagram — see module-6-4-trees.html]*

BST construction by inserting [5, 3, 7, 1, 4, 6, 8]. Each value navigates left if smaller than the current node, right if larger, until an empty slot is reached. This particular insertion order produces a balanced tree of height 3 — close to the theoretical minimum of ■log■ 7■ = 3. The BST property (left subtree < root < right subtree at every node) guarantees that search, insert, and delete each require at most O(height) comparisons. The critical vulnerability: if keys arrive in sorted order, a BST degenerates into a linked list of height n, making all operations O(n). AVL and Red-Black

**Figure 120 • FIG 6.4-B**

### AVL Tree Rotations — LL, RR, LR, RL Cases

*Illustration: FIG 6.4-B | AVL Tree Rotations — LL, RR, LR, RL Cases [SVG diagram — see module-6-4-trees.html]*

AVL tree rotations — the four cases that restore the balance invariant after an insert or delete. The LL case (left-left heavy) requires a single right rotation: the left child B becomes the new root, A becomes B's right child. The RR case mirrors this with a left rotation. The LR and RL cases each require two rotations: first fix the inner imbalance at the child, then apply the outer rotation at the original root. At most two rotations per insert guarantees O(log n) amortized cost. Every rotation is O(1) — just three pointer reassignments.

**Figure 121 • FIG 6.4-C**

**Sorted Input Catastrophe — BST Degenerates, AVL Stays Balanced**

*Illustration: FIG 6.4-C | Sorted Input Catastrophe — BST Degenerates, AVL Stays Balanced [SVG diagram — see module-6-4-trees.html]*

The worst case for a plain BST: sorted or nearly-sorted input produces a degenerate right-skewed chain of height n — every operation degrades to O(n), the same as linear search. The AVL tree inserts the same sequence but applies rotations after insertions 3, 5, and 7, maintaining a tree of height 3. For n=1,000,000 sorted items, a BST requires up to one million comparisons to find a value; an AVL tree requires at most about 29 (■1.44 × log■ 10■■). This motivates self-balancing trees: AVL for lookup-heavy workloads, Red-Black trees (used in Linux kernel and std::map) for insert-heavy workloads

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 6.4 Trees: BST, AVL, Red-Black: BST Builder Tree Traversals Worst-Case Demonstration [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 6.4 Trees: BST, AVL, Red-Black **Type:** Build + trace + implementation + reasoning **Estimated time:** 5060 minutes **Tools needed:** pen and paper + Python --- Part A BST Construction and Search Insert the keys `[8, 3, 10, 1, 6, 9, 14, 4, 7]` into an empty BST in that order. 1. Draw the resulting BST. Verify the BST property: for every node, all left-subtree keys < node key < all right-subtree keys. 2. Trace search for key **7**: list every node visited and the comparison at each step. 3. Trace search for key **11** (not present): list every node visited until you reac

# Build heap in O(n):

■# Module 6.5 — Heaps and Priority Queues

## The Problem

*You are managing a hospital emergency room.*

*Patients arrive continuously, each with a severity score.*

*At any moment, the next patient to be treated should be the one with the highest severity — not the one who arrived first, and not a random one.*

>

*This is the **priority queue** problem: a collection where we always want efficient access to the minimum (or maximum) element.*

>

*An unsorted array gives O(n) for "find the minimum" — too slow.*

*A sorted array gives O(1) for "find the minimum" but O(n) for insert — still too slow.*

*A heap gives O(log n) for both insert and extract-min, with O(1) find-min.*

---

## The Story

### Heapsort and Priority Queues

The heap data structure was invented by **J. W. J. Williams** in 1964, in the same paper that introduced Heapsort. Williams needed a data structure that could both maintain a partially-sorted order and allow efficient extraction of the maximum element — essentially a priority queue. The key insight was to store the structure as a binary tree implicitly in an array, using index arithmetic instead of pointers.

**Robert Floyd** (1964) improved the heap construction algorithm from O(n log n) to O(n) in the same year — a beautiful result showing that you can build a heap from an arbitrary array in linear time by "heapifying" from the bottom up rather than inserting elements one by one.

Heaps are used pervasively in systems software: Linux's Completely Fair Scheduler uses a red-black tree (a tree with heap-like properties) to manage processes. Dijkstra's algorithm, first described in 1959, uses a priority queue (typically a heap) to always expand the nearest unvisited node.

---

## The Concept

### The Heap Property

A **binary heap** is a complete binary tree (every level filled except possibly the last, filled left to right) satisfying the **heap property**:

**Max-heap**: every node's value ≥ its children's values. The maximum is always at the root. **Min-heap**: every node's value ≤ its children's values. The minimum is always at the root.

```
Max-heap example:
        90
       /  \
      80   70
     / \  / \
    50  60 40 30
   / \
  20  10
```

### Array Representation — No Pointers Needed

A complete binary tree maps to an array without any pointers:

```
Index:   0   1   2   3   4   5   6   7   8
Value:  [90, 80, 70, 50, 60, 40, 30, 20, 10]
```

```
For node at index i:
  Parent:       (i - 1) // 2
  Left child:   2*i + 1
  Right child:  2*i + 2
```

This is elegant: the heap is just an array, and all navigation is index arithmetic. Cache-friendly, zero pointer overhead.

### Core Operations

**peek_max()** / **peek_min()**: return `heap[0]`. **O(1)**.

**insert(x)** — "sift up" (also called "bubble up"):

Append x to end of array (at position n)

While x is greater than its parent and we're not at root:

- Swap x with parent - Move up (i = parent index)

Heap property restored. **O(log n)** — tree height.

```
def insert(heap, x):
    heap.append(x)
    i = len(heap) - 1
    while i > 0:
        parent = (i - 1) // 2
        if heap[i] > heap[parent]:      # max-heap condition
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break
```

**extract_max()** — "sift down" (also called "heapify down"):

Root (maximum) is the answer to return

Move last element to root (to fill the gap)

While root is less than either child:

- Swap with the larger child - Move down

Return the original root. **O(log n)**.

```
def extract_max(heap):
    if not heap: raise IndexError
    max_val = heap[0]
    heap[0] = heap.pop()    # move last to root
    # sift down
    i = 0
    n = len(heap)
    while True:
        left, right = 2*i+1, 2*i+2
        largest = i
        if left < n and heap[left] > heap[largest]:   largest = left
        if right < n and heap[right] > heap[largest]: largest = right
        if largest == i: break
        heap[i], heap[largest] = heap[largest], heap[i]
```

```
            i = largest
        return max_val
```

**build_heap(arr)** — Floyd's algorithm, **O(n)**:

Rather than inserting n elements one by one (O(n log n)), start from the last non-leaf and sift down every internal node:

```
def build_heap(arr):
    n = len(arr)
    for i in range((n-2)//2, -1, -1):    # start from last non-leaf
        sift_down(arr, i, n)
    return arr
```

The O(n) analysis: most nodes are near the bottom of the tree and need to sift down very few levels. The total work is bounded by the sum of heights, which is O(n) (geometric series).

### Python's heapq Module

Python provides a **min-heap** implementation:

```
import heapq

heap = []
heapq.heappush(heap, 5)
heapq.heappush(heap, 3)
heapq.heappush(heap, 8)
heapq.heappush(heap, 1)

print(heapq.heappop(heap))   # 1 (minimum)
print(heapq.heappop(heap))   # 3

# Build heap in O(n):
data = [5, 3, 8, 1, 9, 2]
heapq.heapify(data)          # in-place, O(n)

# nlargest and nsmallest:
heapq.nlargest(3, data)      # [9, 8, 5]
heapq.nsmallest(2, data)     # [1, 2]
```

For a max-heap in Python, negate values: push `-x` and negate on pop.

### Heapsort — O(n log n) In-Place

```
def heapsort(arr):
    # Step 1: build max-heap O(n)
    n = len(arr)
    for i in range((n-2)//2, -1, -1):
        sift_down(arr, i, n)
    # Step 2: extract max n times O(n log n)
    for i in range(n-1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]   # swap max to end
        sift_down(arr, 0, i)               # restore heap on remaining
```

Heapsort is in-place (O(1) extra space) and always O(n log n). In practice it is slower than Quicksort due to poor cache behaviour — elements are accessed in a tree pattern rather than sequentially.

### The Priority Queue as an Interface

The heap implements the **priority queue abstract data type** (ADT):

• `insert(item, priority)` — add item with given priority

• `extract_min()` / `extract_max()` — remove an

### Illustrations

**Figure 122 • FIG 6.5-A**

**Max-Heap — Tree View and Array Representation Side by Side**

*Illustration: FIG 6.5-A | Max-Heap — Tree View and Array Representation Side by Side [SVG diagram — see module-6-5-heaps.html]*

A max-heap of 7 elements stored as a 1-indexed array. The heap property (every node ≥ its children) guarantees the maximum is always at index 1, accessible in O(1). Parent-child relationships are computed arithmetically: parent(i) = ■i/2■, left(i) = 2i, right(i) = 2i+1 — no pointers required. This array-backed representation makes heaps extremely cache-efficient compared to pointer-based trees: all elements occupy a contiguous memory block, and accessing a child is a multiply-by-two rather than a pointer dereference. The complete binary tree shape (filled level by level) guarantees height = ■l

**Figure 123 • FIG 6.5-B**

**Heap Insert (Sift-Up) and Extract-Max (Sift-Down)**

*Illustration: FIG 6.5-B | Heap Insert (Sift-Up) and Extract-Max (Sift-Down) [SVG diagram — see module-6-5-heaps.html]*

Heap insert (sift-up) and extract-max (sift-down). Insert appends the new element at the next array position, then bubbles it up by repeatedly swapping with its parent while it is larger — at most ■log■ n■ swaps. Extract-max removes the root (the maximum), moves the last element to the root position to maintain the complete tree shape, then sifts it down by swapping with the larger of its two children — until the heap property is restored. Both operations cost O(log n) — the key insight that makes heaps useful for priority queues where push and pop-max must both be fast.

**Figure 124 • FIG 6.5-C**

**Priority Queue Applications — Dijkstra, Task Scheduling, Heapsort**

*Illustration: FIG 6.5-C | Priority Queue Applications — Dijkstra, Task Scheduling, Heapsort [SVG diagram — see module-6-5-heaps.html]*

Three canonical priority queue applications, all powered by the heap. Dijkstra's shortest path algorithm uses a min-heap keyed by tentative distance — always extracting the nearest unvisited node ensures correctness. OS task schedulers maintain a priority queue of runnable tasks — the CPU always executes the highest-priority task, with O(log n) insert when new tasks arrive and O(log n) extract when the current task yields. Heapsort builds a max-heap in O(n), then repeatedly extracts the maximum to the end of the array — producing a sorted result in O(n log n) worst-case with O(1) extra space,

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 6.5 Heaps and Priority Queues: Heap Visualiser Floyd's Build Priority Queue [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 6.5 Heaps and Priority Queues **Type:** Trace + implementation + application **Estimated time:** 4555 minutes **Tools needed:** pen and paper + Python --- Part A Heap Operations Start with this **max-heap** (array representation): `[90, 80, 70, 50, 60, 40, 30, 20, 10]` Indices: 0=90, 1=80, 2=70, 3=50, 4=60, 5=40, 6=30, 7=20, 8=10 1. **Insert 85**: trace the sift-up operation. Show the array after every swap. 2. **Extract-max**: remove the maximum (90). Show every swap in the sift-down operation. What is the final array? 3. What is the height of a max-heap containing n e

# nn matrix where matrix[i][j] = 1 if edge exists

■# Module 6.6 — Graphs: BFS, DFS, Dijkstra, A*

## The Problem

*A road network. A social network. The internet. A circuit board. A gene regulatory network.*

*All of these are the same mathematical object: a **graph**.*

>

*How do you find the shortest route from A to B in a city with 10,000 intersections?*

*How does Google find the shortest degree of separation between two people?*

*How does a GPS recalculate a route in real time?*

>

*These all reduce to graph search problems — and understanding them*

*means understanding the structure of almost every networked system ever built.*

---

## The Story

### Leonhard Euler and the Bridges of Königsberg

In 1736, Leonhard Euler solved the first graph theory problem: is it possible to walk through Königsberg (now Kaliningrad, Russia) crossing each of its seven bridges exactly once? He proved it was impossible by abstraction: replace land masses with nodes, bridges with edges. The problem reduces to whether an **Eulerian path** exists — which requires at most two nodes of odd degree. Königsberg had four nodes of odd degree: impossible.

This was the birth of graph theory, 200 years before computers.

**Edsger Dijkstra** invented his shortest-path algorithm in 1956, reportedly in 20 minutes while sitting in a café in Amsterdam, without pen and paper. He was designing software for the first Dutch computer (ARMAC) and needed a test program. The algorithm he invented became one of the most widely used in computer science.

**Peter Hart, Nils Nilsson, and Bertram Raphael** invented A *at Stanford Research Institute in 1968, as part of the Shakey robot project — a robot that needed to navigate a room. A* added a heuristic to Dijkstra's algorithm to guide the search toward the goal, dramatically reducing unnecessary exploration.

---

## The Concept

### Graph Representation

**Adjacency List** (preferred for sparse graphs):

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}
```

Space: O(V + E). Checking if edge (u,v) exists: O(degree(u)).

**Adjacency Matrix** (preferred for dense graphs):

```
# n×n matrix where matrix[i][j] = 1 if edge exists
matrix = [
    [0, 1, 1, 0, 0, 0],  # A
    [1, 0, 0, 1, 1, 0],  # B
    ...
```

```
]
```

Space: $O(V^2)$. Checking edge existence: O(1).

### Breadth-First Search (BFS)

Explores all neighbours of a node before moving to their neighbours. Uses a **queue**. Finds the **shortest path** (fewest edges) in an unweighted graph.

```
from collections import deque

def bfs(graph, start):
    visited = {start: None}    # node → parent (for path reconstruction)
    queue   = deque([start])
    while queue:
        node = queue.popleft()
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited[neighbour] = node
                queue.append(neighbour)
    return visited

def shortest_path(graph, start, end):
    visited = bfs(graph, start)
    if end not in visited: return None
    path = []
    node = end
    while node is not None:
        path.append(node)
        node = visited[node]
    return list(reversed(path))
```

**Time**: O(V + E). **Space**: O(V). Guarantees shortest path in unweighted graphs.

### Depth-First Search (DFS)

Goes as deep as possible before backtracking. Uses a **stack** (or recursion). Finds connected components, detects cycles, produces topological orderings.

```
def dfs_recursive(graph, node, visited=None):
    if visited is None: visited = set()
    visited.add(node)
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs_recursive(graph, neighbour, visited)
    return visited

def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            for neighbour in graph[node]:
                if neighbour not in visited:
```

```
                    stack.append(neighbour)
    return visited
```

**Applications of DFS**:

• **Cycle detection**: if DFS encounters a grey (in-progress) node, there's a cycle

• **Topological sort**: DFS-based, used in build systems (make, Gradle)

• **Connected components**: run DFS from each unvisited node

• **Maze solving**: DFS finds *a* path (not necessarily shortest)

### Dijkstra's Shortest Path

For **weighted graphs** (each edge has a cost/distance). Generalises BFS using a **priority queue** instead of a plain queue — always expand the unvisited node with the smallest known distance.

```
import heapq

def dijkstra(graph_weighted, start):
    # graph_weighted: {node: [(neighbour, weight), ...]}
    dist = {start: 0}
    pq   = [(0, start)]        # (distance, node)
    while pq:
        d, node = heapq.heappop(pq)
        if d > dist.get(node, float('inf')):
            continue           # stale entry
        for neighbour, weight in graph_weighted[node]:
            new_dist = dist[node] + weight
            if new_dist < dist.get(neighbour, float('inf')):
                dist[neighbour] = new_dist
                heapq.heappush(pq, (new_dist, neighbour))
    return dist
```

**Time**: O((V + E) log V) with a binary heap. **Correctness**: only works on graphs with **non-negative edge weights**. Negative weights require Bellman-Ford.

### A* Search

Dijkstra explores in all directions equally — expanding the cheapest frontier node regardless of direction. A* adds a **heuristic** h(n) estimating the cost from n to the goal:

```
f(n) = g(n) + h(n)
```

where g(n) = actual cost from start to n, h(n) = estimated cost from n to goal.

If h(n) is **admissible** (never overestimates the true cost), A* always finds the optimal path while exploring fewer nodes than Dijkstra.

```
def astar(graph, start, goal, heuristic):
    pq = [(heuristic(start, goal), 0, start)]
    g  = {start: 0}
    while pq:
        f, cost, node = heapq.heappop(pq)
        if node
```

## Illustrations

**Figure 125 • FIG 6.6-A**

### Graph Representations — Adjacency Matrix vs Adjacency List

*Illustration: FIG 6.6-A | Graph Representations — Adjacency Matrix vs Adjacency List [SVG diagram — see module-6-6-graphs.html]*

The same undirected 4-node graph stored two ways. The adjacency matrix is a V×V boolean grid — O(1) edge existence check but O(V²) space regardless of edge count. The adjacency list stores only actual edges per node — O(V+E) space, O(degree) to iterate neighbours. For a social network with 1 billion users and 100 billion friendships (average degree 100), the list uses ~800 GB while the matrix would require 1 billion² bits ≈ 10^18 bytes — physically impossible. Adjacency lists are the universal choice for graph algorithms; matrices appear in dense graphs like fully-connected neural network laye

**Figure 126 • FIG 6.6-B**

### BFS vs DFS — Traversal Order on the Same Graph

*Illustration: FIG 6.6-B | BFS vs DFS — Traversal Order on the Same Graph [SVG diagram — see module-6-6-graphs.html]*

BFS (queue-based) explores level by level, guaranteeing it finds the shortest path in unweighted graphs — critical for social network degree separation, web crawlers, and GPS routing. DFS (stack-based, or recursive) plunges as deeply as possible before backtracking, naturally discovering connected components, topological ordering of dependencies, and strongly connected components (Tarjan's algorithm). Both run O(V+E) with an adjacency list. The choice is driven by the problem: shortest path → BFS; topological sort, cycle detection, maze → DFS.

**Figure 127 • FIG 6.6-C**

### Dijkstra's Shortest Path — Step-by-Step on a Weighted Graph

*Illustration: FIG 6.6-C | Dijkstra's Shortest Path — Step-by-Step on a Weighted Graph [SVG diagram — see module-6-6-graphs.html]*

Dijkstra's shortest path algorithm traced on a 4-node weighted graph. The priority queue (min-heap) always extracts the node with the smallest tentative distance. When node C (dist=2) is extracted, it updates D's distance to 3 via the C→D edge of weight 1 — cheaper than the direct A→B→D path of weight 4+5=9. The algorithm's correctness relies on non-negative edge weights: once a node is extracted from the priority queue, its distance is provably minimal. For negative weights, Bellman-Ford is needed instead. Real-world use: Google Maps, network routing (OSPF), game pathfinding.

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 6.6 Graphs: BFS, DFS, Dijkstra, A*: BFS vs DFS Dijkstra Step-by-Step A* Pathfinding [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 6.6 Graphs: BFS, DFS, Dijkstra, A* **Type:** Trace + implementation + reasoning **Estimated time:** 5060 minutes **Tools needed:** pen and paper + Python --- Part A BFS and DFS by Hand Given this undirected graph: ``` 1 --- 2 --- 5 | | | 3 --- 4 --- 6 | 7 ``` Adjacency list (sorted): `{1:[2,3], 2:[1,4,5], 3:[1,4], 4:[2,3,6,7], 5:[2,6], 6:[4,5], 7:[4]}` 1. Run **BFS** from node 1. List nodes in discovery order and show the queue state after each dequeue. 2. Run **DFS** from node 1 (use sorted adjacency lists, so always visit the smal

# Module 6 7 Sorting Advanced

■# Module 6.7 — Sorting: Merge, Quick, Radix

## The Problem

*Module 5.5 introduced sorting and Big-O complexity.*

*Now we go deeper: not just what these algorithms do,*

*but why they work, what guarantees they provide,*

*how to implement them correctly at the code level,*

*and when to choose each one for a real problem.*

>

*Three algorithms dominate practical sorting:*

*Merge Sort for guaranteed O(n log n),*

*Quick Sort for real-world speed,*

*and Radix Sort for when you can beat the comparison lower bound.*

## The Story

### The Race for Practical Sorting

By the 1960s, sorting was known to have an O(n log n) lower bound for comparison-based methods. Merge Sort (von Neumann, 1945) achieved this but required O(n) extra memory. Shell Sort (1959) was faster in practice but its worst case was unclear. **Tony Hoare's** Quicksort (1962) was average-case O(n log n) and in-place — a stunning combination.

For decades, the battle between Merge Sort and Quicksort dominated practical computing. IBM's OS/360 used Merge Sort for stability and predictability. Unix's `qsort` used Quicksort for speed. Java's Arrays.sort for primitives uses Dual-Pivot Quicksort (Yaroslavskiy, 2009). Python's `list.sort()` uses Timsort (Peters, 2002) — a hybrid Merge Sort that exploits real-world patterns.

The key insight that drove modern sort implementations: theoretical worst cases matter less than average-case performance on real data. Real data is often partially sorted, has repeated elements, or exhibits patterns. Good sorts exploit these patterns.

## The Concept

### Merge Sort — Full Implementation

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left  = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j  = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:     # <= for stability
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

**Bottom-up Merge Sort** (iterative, avoids recursion overhead):

```
def merge_sort_iterative(arr):
    n = len(arr)
    width = 1
    while width < n:
        for i in range(0, n, 2 * width):
            left  = arr[i : i + width]
            right = arr[i + width : i + 2 * width]
            arr[i : i + len(left) + len(right)] = merge(left, right)
        width *= 2
    return arr
```

Bottom-up avoids the O(log n) recursion stack — useful when stack space is limited.

**Correctness proof by induction**: base case (len ≤ 1 is sorted), inductive step (merge of two sorted arrays is sorted), merge is correct (takes the smaller of two heads at each step). Merging two arrays of total size m takes exactly m-1 comparisons at most.

### Quicksort — Correct and Efficient Implementation

The key to correct Quicksort is the **partition function**. The most common bug: incorrect boundary conditions.

```
import random

def quicksort(arr, lo=0, hi=None):
    if hi is None: hi = len(arr) - 1
    if lo >= hi: return
    pivot_idx = partition(arr, lo, hi)
    quicksort(arr, lo, pivot_idx - 1)
    quicksort(arr, pivot_idx + 1, hi)

def partition(arr, lo, hi):
    # Randomise pivot to avoid worst case on sorted input
    rand_idx = random.randint(lo, hi)
    arr[rand_idx], arr[hi] = arr[hi], arr[rand_idx]
    pivot = arr[hi]
    i = lo - 1
    for j in range(lo, hi):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[hi] = arr[hi], arr[i+1]
    return i + 1
```

**Three-way partition** (Dutch National Flag, Dijkstra 1976) — handles duplicates efficiently:

```
def quicksort_3way(arr, lo, hi):
    if lo >= hi: return
    lt, gt = lo, hi
    pivot  = arr[lo]
    i = lo + 1
    while i <= gt:
        if   arr[i] < pivot: arr[lt], arr[i] = arr[i], arr[lt]; lt += 1; i += 1
        elif arr[i] > pivot: arr[gt], arr[i] = arr[i], arr[gt]; gt -= 1
        else:                i += 1
    quicksort_3way(arr, lo, lt - 1)
    quicksort_3way(arr, gt + 1, hi)
```

With three-way partitioning, an array of all identical elements sorts in O(n) — each element is compared exactly once and placed in the "equal" region.

**Introsort** (David Musser, 1997): Quicksort with a fallback to Heapsort when recursion depth exceeds 2·log■(n). Guarantees O(n log n) worst case while keeping Quicksort's average-case speed. Used in C++ `std::sort`.

### Counting Sort and Radix Sort

When keys are integers in a bounded range [0, k], we can sort in **O(n + k)** — faster than O(n log n).

**Counting Sort**:

```
def counting_sort(arr, k):
    count = [0] * (k + 1)
    for x in arr:
        count[x] += 1
```

```
    # Cumulative counts (for stable positioning)
    for i in range(1, k+1):
        count[i] += count[i-1]
    output = [0] * len(arr)
    for x in reversed(arr):    # reversed for stability
        count[x] -= 1
        output[count[x]] = x
    return output
```

O(n + k) time and space. Optimal when k = O(n) — e.g., sorting ages (k=150), test scores (k=100), ASCII characters (k=128).

**Radix Sort** — sort digit by digit, least significant to most significant:

```
def radix_sort(arr):
    if not arr: return arr
    max_val = max(arr)
    exp = 1
    while max_val // exp > 0:
        arr = counting_sort_by_digit(arr, exp)
        exp *= 10
    return arr

def counting_sort_by_digit(arr, exp):
    n = len(arr)
    count  = [0] * 10
    output = [0] * n
    for x in arr:
        digit = (x // exp) % 10
        count[digit] += 1
    for i in range(1, 10):
        count[i] += count[i-1]
    for x in reversed(arr):
        digit = (x // exp) % 10
        count[digit] -= 1
        output[count[digit]] = x
    return output
```

For 32-bit integers with base 256: 4 passes, each O(n) → **O(4n) = O(n)** total.

**Why does it work?** Sorting by LSD

### Illustrations

**Figure 128 • FIG 6.7-A**

### Quicksort Partition Step — Lomuto Scheme with Pivot = Last Element

*Illustration: FIG 6.7-A | Quicksort Partition Step — Lomuto Scheme with Pivot = Last Element [SVG diagram — see module-6-7-sorting-advanced.html]*

Quicksort's partition step using the Lomuto scheme with the last element as pivot. The algorithm scans with index j; whenever an element ≤ pivot is found, it swaps into the "≤ pivot" region (tracked by i). After the scan, the pivot is placed at i+1 — its final sorted position. The partition takes O(n) and splits the array into two subproblems. Key insight: the pivot's final position is determined in a single linear pass, and the two sub-arrays are then sorted recursively. With a good pivot choice (random or median-of-3), the expected split is near 50/50, giving O(n log n) average. A terrible p

**Figure 129 • FIG 6.7-B**

### Quicksort Recursion Tree — Balanced (Best) vs Skewed (Worst)

*Illustration: FIG 6.7-B | Quicksort Recursion Tree — Balanced (Best) vs Skewed (Worst) [SVG diagram — see module-6-7-sorting-advanced.html]*

Quicksort's two extremes. With a median pivot, each partition splits evenly into n/2 + n/2, producing log■ n levels — at each level the total work is n (all partitions at that depth sum to n elements). Total: O(n log n). With the minimum element always chosen as pivot, partitions split into 0 + (n–1) — a degenerate chain of n levels. Total work: n + (n–1) + ... + 1 = O(n²). This worst case hits exactly when a sorted array is partitioned using the last element as pivot — a fatal interaction. The fix: choose a random pivot, or the median of the first, last, and middle elements. Production Quicks

**Figure 130 • FIG 6.7-C**

### Radix Sort (LSD) — Sorting Integers Digit by Digit

*Illustration: FIG 6.7-C | Radix Sort (LSD) — Sorting Integers Digit by Digit [SVG diagram — see module-6-7-sorting-advanced.html]*

Radix sort (Least Significant Digit) sorts integers one digit position at a time, using a stable sub-sort (counting sort) at each pass. After pass 1 (units), numbers are grouped by their last digit. After pass 2 (tens), the relative order within each tens-group is already correct from pass 1 — stability is essential. After pass 3 (hundreds), the array is fully sorted. The algorithm never compares elements — it only examines individual digits — breaking through the O(n log n) lower bound that applies to comparison-based sorts. For n integers with at most k digits, time complexity is O(nk), whic

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 6.7 Sorting: Merge, Quick, Radix: Merge Sort Tree Quicksort Partition Radix Sort Pass-by-Pass
[Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 6.7 Sorting: Merge, Quick, Radix **Type:** Trace + implementation + analysis **Estimated time:** 5060 minutes **Tools needed:** pen and paper + Python --- Part A Merge Sort Full Trace Sort `[5, 3, 8, 1, 9, 2, 7, 4]` using merge sort. 1. Draw the complete **recursion tree**: show every split and every merge, with the sub-array contents at each node. 2. Count exact comparisons per merge operation. Sum to get the total. Compare to the theoretical bound n log n. 3. Implement **bottom-up merge sort** (iterative, no recursion): start with sub-arrays of width 1, merge to width

# Or manually:

■# Module 6.8 — Dynamic Programming

## The Problem

*Naive recursion for Fibonacci: each call to fib(n) spawns two more.*

*fib(50) requires 2■■ ≈ 10¹■ calls — impossible.*

>

*But fib(50) only has 50 distinct sub-problems: fib(0), fib(1), ..., fib(49).*

*If we compute each one once and store the result, we need exactly 50 operations.*

>

*This is dynamic programming in its essence:*

*identify overlapping sub-problems, solve each once, store the result,*

*and build the final answer from stored sub-results.*

Dynamic programming (DP) transforms exponential-time recursive algorithms into polynomial-time ones — one of the most powerful algorithmic techniques in computer science.

## The Story

### Richard Bellman and the Name

**Richard Bellman** coined the term "dynamic programming" in 1953 while working at RAND Corporation on optimal control problems. The name was deliberately chosen to sound impressive and avoid scrutiny — Bellman was working under a Secretary of Defense who disliked mathematics research, so he named it to be vague and unchallengeable.

The actual technique — breaking problems into sub-problems and storing results — had been used informally for centuries. But Bellman formalised it as a general principle and developed the **principle of optimality**: an optimal solution to the whole problem contains optimal solutions to its sub-problems.

DP is fundamental to: sequence alignment in bioinformatics (finding how similar two DNA sequences are), compiler optimisation (code generation), network routing (Bellman-Ford), speech recognition (Viterbi algorithm), machine translation, and game playing (minimax with memoisation).

## The Concept

### Two Approaches: Top-Down and Bottom-Up

**Top-Down (Memoisation)**: write the natural recursion, cache results in a dictionary.

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)

# Or manually:
memo = {}
def fib_memo(n):
    if n in memo: return memo[n]
    if n <= 1: return n
    memo[n] = fib_memo(n-1) + fib_memo(n-2)
    return memo[n]
```

**Bottom-Up (Tabulation)**: fill a table from smallest sub-problems upward.

```
def fib_dp(n):
    if n <= 1: return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

# Space optimised (only need last two):
def fib_optimal(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

**When to use top-down vs bottom-up**:

• Top-down is easier to write (follow the recursion), handles sparse sub-problem spaces

• Bottom-up is often faster (no recursion overhead, better cache behaviour), more space-efficient

### Identifying a DP Problem

Two properties make a problem solvable with DP:

**Optimal substructure**: an optimal solution contains optimal solutions to sub-problems

**Overlapping sub-problems**: the same sub-problems are solved repeatedly in naive recursion

If a problem has both, DP applies.

### Canonical DP Problems

**Longest Common Subsequence (LCS)**:

Given two strings, find the length of their longest common subsequence (characters in order, not necessarily contiguous).

```
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

lcs("ABCBDAB", "BDCAB")   # 4 (BCAB or BDAB)
```

Time: O(mn). Used in: `diff` (finding differences between files), DNA sequence alignment, version control systems.

**0/1 Knapsack**:

n items, each with weight w[i] and value v[i]. A bag with capacity W. Maximise total value without exceeding capacity. Each item used at most once.

```
def knapsack(weights, values, W):
    n = len(weights)
    dp = [[0] * (W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            dp[i][w] = dp[i-1][w]    # don't take item i
            if weights[i-1] <= w:
                take = dp[i-1][w - weights[i-1]] + values[i-1]
                dp[i][w] = max(dp[i][w], take)   # take item i
    return dp[n][W]
```

Time: O(nW) — pseudo-polynomial (polynomial in W, but W can be exponentially large).

**Coin Change**:

Given coin denominations and a target amount, find the minimum number of coins.

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for amt in range(1, amount + 1):
        for coin in coins:
            if coin <= amt:
                dp[amt] = min(dp[amt], dp[amt - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

coin_change([1, 5, 10, 25], 36)   # 3 coins: 25+10+1
```

**Longest Increasing Subsequence (LIS)**:

```
def lis(arr):
    if not arr: return 0
    n = len(arr)
    dp = [1] * n            # each element alone is an LIS of length 1
    for i in range(1, n):
        for j in range(i):
            if arr[j] < arr[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

lis([10, 9, 2, 5, 3, 7, 101, 18])   # 4 (2,3,7,101 or 2,5,7,101)
```

O(n²) DP. There is an O(n log n) solution using binary search and patience sorting.

### The DP Design Process

**Step 1 — Define the sub-problem**: what does `dp[i]` or `dp[i][j]` represent?

**Step 2 — Write the recurrence**: how does `dp[i]` depend on smaller sub-problems?

**Step 3 — Identify base cases**: what are the smallest sub-problems with known answers?

**Step 4 — Determine computation order**: which sub-problems must be solved before others?

**Step 5 — Extract the answer**: which cell(s) of the table contain the final answer?

**Step 6 — Optimise space**: often only the last row (or last two values) of the DP table is needed.

---

## Hands-On Exercise

**Open the simulation:** [`simulation.html`](./simulation.html)

### Illustrations

**Figure 131 • FIG 6.8-A**

**Longest Common Subsequence — DP Table for "ABCB" and "BCAB"**

*Illustration: FIG 6.8-A | Longest Common Subsequence — DP Table for "ABCB" and "BCAB" [SVG diagram — see module-6-8-dynamic-programming.html]*

LCS DP table for "ABCB" (rows) and "BCAB" (columns). When characters match, the cell inherits the diagonal cell + 1, recording that the common subsequence can be extended. When characters differ, the cell takes the maximum of the cell above and the cell to the left — propagating the best known LCS length so far. The final answer in dp[4][4] = 3, representing "BCB". Backtracking diagonals reconstructs the actual sequence. Without memoisation the naive recursion would require O(2^n) calls; the DP table reduces this to O(m×n) — the defining transformation of dynamic programming.

**Figure 132 • FIG 6.8-B**

### 0/1 Knapsack — Choosing Items to Maximise Value within Capacity W=5

*Illustration: FIG 6.8-B | 0/1 Knapsack — Choosing Items to Maximise Value within Capacity W=5 [SVG diagram — see module-6-8-dynamic-programming.html]*

0/1 Knapsack DP table with 3 items and capacity W=5. At each cell dp[i][w], we choose the better of: not taking item i (same as dp[i–1][w]) or taking item i if it fits (dp[i–1][w–wt_i] + val_i). The maximum value achievable with all 3 items and capacity 5 is 7, achieved by selecting items 1 and 2 (weights 2+3=5, values 3+4=7). Item 3 (weight 4) alone yields only 5. Backtracking from dp[3][5] recovers which items were chosen. The algorithm runs O(nW) — pseudo-polynomial time — versus an exponential 2^n exhaustive search over all item subsets.

**Figure 133 • FIG 6.8-C**

### When to Use DP — Overlapping Subproblems + Optimal Substructure

*Illustration: FIG 6.8-C | When to Use DP — Overlapping Subproblems + Optimal Substructure [SVG diagram — see module-6-8-dynamic-programming.html]*

Dynamic programming decision framework. Overlapping subproblems alone justify memoisation (cache results of a top-down recursion). Optimal substructure alone may justify a greedy algorithm. Both properties together — the same subproblems recur AND the global optimum decomposes into subproblem optima — make DP the right tool. Classic DP problems: Longest Common Subsequence, Edit Distance, Coin Change, Knapsack, Matrix Chain Multiplication. Non-DP: binary search (no subproblem overlap), merge sort (no reuse of subproblems across calls), graph BFS/DFS (state visited once). Recognising the two pro

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 6.8 Dynamic Programming: Fibonacci DP LCS Grid Knapsack Table [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 6.8 Dynamic Programming **Type:** Trace + implementation + design **Estimated time:** 5060 minutes **Tools needed:** pen and paper + Python --- Part A LCS Table Compute the **Longest Common Subsequence** of `"ABCD"` and `"ACBD"`. 1. Fill in the complete DP table (55 including the empty-string base case). Use the recurrence: - `dp[i][j] = dp[i-1][j-1] + 1` if `s1[i-1] == s2[j-1]` - `dp[i][j] = max(dp[i-1][j], dp[i][j-1])` otherwise 2. What is the length of the LCS? 3. **Reconstruct** the actual LCS string by tracing back through the table. Show every step of the tra

LEVEL 07

# Systems Programming

Concurrency, Sockets, Shells, and HTTP Servers

# For CPU-bound parallel work:

■# Module 7.1 — Multithreading and Concurrency

## The Problem

*A web server must handle 10,000 simultaneous connections.*

*A video game must update physics, render frames, and play audio simultaneously.*

*A data pipeline must download, process, and store data in parallel.*

>

*A single-threaded program does one thing at a time.*

*The world does not.*

>

***Concurrency** is about dealing with multiple things happening at the same time.*

***Parallelism** is about doing multiple things at the same time.*

*They are related but distinct — and confusing them leads to bugs that are nearly impossible to reproduce.*

## The Story

### From Sequential to Concurrent

The earliest computers were strictly sequential: one program ran, finished, then the next ran. Time-sharing (Module 4.3) created the *illusion* of concurrency by rapidly context-switching between processes.

**Threads** (1960s–70s) took this further: lightweight "processes" sharing the same address space. Switching between threads in the same process is cheaper than between processes (no address space switch). Threads share heap memory, file descriptors, and global variables — enabling communication at the cost of coordination complexity.

**Dijkstra** formalised many of the synchronisation problems in the 1960s: the mutual exclusion problem (1965), semaphores (1965), the dining philosophers problem (1965). These problems, posed 60 years ago, are still the foundational examples taught in every operating systems course.

Intel's **Hyper-Threading** (2002) put two logical CPU cores on one physical core, enabling true thread-level parallelism within a single chip. Modern CPUs have 8–32 physical cores, each with hardware multithreading. Writing correct parallel code became a survival skill for systems programmers.

## The Concept

### Processes vs Threads

| Property | Process | Thread | |---|---|---| | Address space | Own (isolated) | Shared with other threads | | Memory | Own heap, stack, data | Own stack; shared heap, data | | Communication | IPC (pipes, sockets, shared mem) | Direct memory access | | Context switch cost | High (TLB flush, page table switch) | Low (same address space) | | Failure isolation | Crash doesn't affect others | Crash kills all threads in process | | Creation cost | High (`fork` + `exec`) | Low (1–10 µs) |

In Python:

```
import threading

def worker(n):
    print(f"Thread {n} running")
    # compute something
    print(f"Thread {n} done")

threads = [threading.Thread(target=worker, args=(i,)) for i in range(4)]
for t in threads: t.start()
for t in threads: t.join()   # wait for all threads to finish
```

### The Race Condition

The fundamental problem of concurrent programs:

```
counter = 0   # shared variable

def increment():
    global counter
    for _ in range(100_000):
        counter += 1   # NOT atomic!

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)
t1.start(); t2.start()
t1.join(); t2.join()
print(counter)   # Expected: 200,000. Actual: some random smaller number!
```

`counter += 1` compiles to three machine instructions:

LOAD counter into register

ADD 1 to register

STORE register back to counter

If two threads interleave between any of these steps, one thread's increment is lost. This is a **race condition** — the outcome depends on timing, making it non-deterministic and nearly impossible to reproduce consistently.

### Critical Sections and Mutual Exclusion

A **critical section** is any code that accesses a shared resource and must not be executed by two threads simultaneously.

**Mutex (Mutual Exclusion Lock)**: only one thread can hold the lock at a time. Others block until the holder releases it.

```
import threading

counter = 0
lock = threading.Lock()

def safe_increment():
    global counter
    for _ in range(100_000):
        with lock:           # acquire on entry, release on exit
            counter += 1    # critical section — only one thread at a time

t1 = threading.Thread(target=safe_increment)
t2 = threading.Thread(target=safe_increment)
t1.start(); t2.start()
t1.join(); t2.join()
print(counter)   # Always 200,000
```

The `with lock:` context manager ensures the lock is always released, even if an exception occurs.

### Thread Safety Patterns

**Immutable objects**: if shared data never changes, no synchronisation is needed. Pure functions and immutable data structures are inherently thread-safe.

**Thread-local storage**: data stored per-thread, not shared. Each thread has its own copy.

```
import threading
local_data = threading.local()

def worker():
    local_data.x = threading.current_thread().name   # each thread writes its own
    print(local_data.x)   # each thread reads its own, no conflict
```

**Producer-Consumer pattern**: producers add work to a queue; consumers take from it.

```
from queue import Queue

q = Queue(maxsize=10)

def producer():
    for i in range(20):
        q.put(i)           # blocks if queue full
    q.put(None)            # sentinel: no more work

def consumer():
    while True:
        item = q.get()     # blocks if queue empty
        if item is None: break
        process(item)

threading.Thread(target=producer).start()
threading.Thread(target=consumer).start()
```

Python's `queue.Queue` is thread-safe — all operations are internally synchronised.

### The GIL and True Parallelism in Python

Python's **Global Interpreter Lock (GIL)** prevents multiple threads from executing Python bytecode simultaneously. This means:

• Python threads are concurrent (interleaved) but not truly parallel for CPU-bound code

• For CPU-bound parallelism, use `multiprocessing` (separate processes, no GIL)

• For I/O-bound tasks (network, disk), threads work fine — the GIL is released during I/O

```
# For CPU-bound parallel work:
from multiprocessing import Pool

def compute(x):
    return x * x

with Pool(processes=4) as pool:
    results = pool.map(compute, range(1_000_
```

## Illustrations

**Figure 134 • FIG 7.1-A**

### Threads vs Processes — Shared Memory vs Isolated Address Spaces

*Illustration: FIG 7.1-A | Threads vs Processes — Shared Memory vs Isolated Address Spaces [SVG diagram — see module-7-1-multithreading.html]*

The fundamental distinction between processes and threads. Each process receives its own isolated virtual address space — process A cannot access process B's memory; communication requires explicit IPC (pipes, sockets, shared memory segments). Threads within a single process share the heap, global data, open file descriptors, and code segment — any thread can read or write any shared variable instantly. This sharing is both the strength (cheap communication, low creation cost) and the hazard (race conditions when multiple threads write the same location without synchronisation). Thread creatio

**Figure 135 • FIG 7.1-B**

### Race Condition — Two Threads Incrementing a Counter

*Illustration: FIG 7.1-B | Race Condition — Two Threads Incrementing a Counter [SVG diagram — see module-7-1-multithreading.html]*

Race condition on a shared counter. The C expression counter++ compiles to three machine instructions: READ (load counter into register), ADD (increment register), WRITE (store register back). When Thread A and Thread B interleave — both reading the value 0 before either writes back — both compute 0+1=1 and write 1. The second write overwrites the first, losing one increment entirely. The expected final value is 2 but the actual result is 1. This is a lost update. The fix is a mutex: only one thread at a time may execute the read-add-write sequence (the critical section). Alternatively, atomic

**Figure 136 • FIG 7.1-C**

**CPU Thread Scheduling — Time-Slice Context Switching**

*Illustration: FIG 7.1-C | CPU Thread Scheduling — Time-Slice Context Switching [SVG diagram — see module-7-1-multithreading.html]*

Preemptive time-slice scheduling with three threads on a single CPU. The OS scheduler allocates 10ms quanta, cycling through runnable threads. When Thread 2 blocks on I/O at t=40ms, the CPU immediately switches to Thread 3 — no busy-waiting. Thread 2 is moved to the WAITING state and only returns to READY when the I/O interrupt fires (at ~t=65ms), resuming in the next available slot. Context switching saves and restores the entire thread state (registers, program counter, stack pointer) from the Thread Control Block. The illusion of simultaneous execution emerges from rapid switching — at 10ms

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 7.1 Multithreading and Concurrency: Race Condition Thread States Producer-Consumer [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 7.1 Multithreading and Concurrency **Type:** Code analysis + implementation + reasoning **Estimated time:** 4555 minutes **Tools needed:** Python + pen and paper --- Part A Race Condition Identification ```python import threading balance = 1000 def withdraw(amount): global balance current = balance (1) if current >= amount: (2) balance = current - amount (3) t1 = threading.Thread(target=withdraw, args=(700,)) t2 = threading.Thread(target=withdraw, args=(700,)) t1.start(); t2.start() t1.join(); t2.join() print(balance) ``` 1. Identify

# Module 7 2 Sync Primitives

■# Module 7.2 — Mutexes, Semaphores, and Deadlocks

## The Problem

*Module 7.1 introduced race conditions and the basic mutex.*

*But real concurrent systems have more complex coordination needs:*

>

*- Allow up to N threads to access a resource simultaneously (database connection pool)*

*- Signal that an event has occurred (producer notifies consumer)*

*- Prevent a circular wait where threads block each other forever*

>

*These needs require richer synchronisation primitives —*

*and the coordination bugs they prevent are replaced by a new class of bugs*

*that are often harder to detect: **deadlocks**, **livelocks**, and **starvation**.*

---

## The Story

### Dijkstra's Semaphore

Edsger Dijkstra introduced the **semaphore** in his 1965 paper "Solution of a Problem in Concurrent Programming Control." He was solving the mutual exclusion problem: how can N processes share a resource safely?

A semaphore is an integer with two atomic operations:

• **P (proberen/wait)**: decrement, block if result is negative

• **V (verhogen/signal)**: increment, wake a blocked thread if any

Dijkstra was also the first to formally describe the **dining philosophers problem** (1965): five philosophers sit at a round table, each alternating between thinking and eating. To eat, a philosopher needs both the left and right fork. If all pick up their left fork simultaneously and wait for the right, they deadlock — a simple model of circular waiting.

This single thought experiment has driven 60 years of research into deadlock avoidance, lock ordering, and concurrent algorithm design.

---

## The Concept

### Mutex (Revisited)

A mutex is a binary semaphore with ownership semantics: only the thread that locked it can unlock it. Used for mutual exclusion — one thread in the critical section at a time.

```
import threading
lock = threading.Lock()

with lock:              # acquire
    # critical section
    pass              # release on exit (even on exception)
```

**Reentrant mutex (RLock)**: a thread can acquire the same lock multiple times without deadlocking. Must release the same number of times.

```
rlock = threading.RLock()
with rlock:
    with rlock:    # safe with RLock, deadlock with Lock
        pass
```

### Semaphores — Counting Synchronisation

A semaphore with initial value N allows up to N threads to be in the critical section simultaneously:

```
sem = threading.Semaphore(3)   # allow 3 concurrent accesses

def access_database():
    with sem:          # acquire (blocks when count reaches 0)
        # use database connection (max 3 simultaneous)
        query_db()
    # release: another thread can now enter
```

**Binary semaphore** (initial=1): equivalent to a mutex, but any thread can release it — useful for signalling.

**Signalling with semaphores** — producer/consumer:

```
filled = threading.Semaphore(0)   # how many items available to consume
empty  = threading.Semaphore(10)  # how many spaces available to produce
mutex  = threading.Semaphore(1)   # protect buffer

def producer():
    while True:
        item = produce()
        empty.acquire()    # wait for empty slot
        mutex.acquire()
        buffer.append(item)
        mutex.release()
        filled.release()   # signal: item is available

def consumer():
    while True:
        filled.acquire()   # wait for item
        mutex.acquire()
        item = buffer.pop()
        mutex.release()
        empty.release()    # signal: slot freed
        consume(item)
```

### Condition Variables

A condition variable allows threads to wait for a specific condition while atomically releasing a mutex:

```
lock = threading.Lock()
condition = threading.Condition(lock)
buffer = []

def producer():
    while True:
        item = produce()
        with condition:
            buffer.append(item)
            condition.notify()   # wake one waiting consumer

def consumer():
    while True:
```

```
        with condition:
            while not buffer:    # spurious wakeup protection
                condition.wait() # release lock, block, re-acquire on wake
            item = buffer.pop(0)
        consume(item)
```

The key operations:

• `wait()`: atomically release lock + block thread + re-acquire lock on wake

• `notify()`: wake one waiting thread

• `notify_all()`: wake all waiting threads

### Deadlock — The Circular Wait

A deadlock occurs when:

Thread A holds lock L1 and waits for lock L2

Thread B holds lock L2 and waits for lock L1

Neither can proceed

```
lock1 = threading.Lock()
lock2 = threading.Lock()

def thread_a():
    with lock1:              # acquires lock1
        with lock2: pass     # waits for lock2 → DEADLOCK if B holds it

def thread_b():
    with lock2:              # acquires lock2
        with lock1: pass     # waits for lock1 → DEADLOCK if A holds it
```

**The four conditions for deadlock** (Coffman, 1971):

**Mutual exclusion**: resources cannot be shared

**Hold and wait**: threads hold resources while waiting for more

**No preemption**: resources cannot be forcibly taken

**Circular wait**: a cycle of threads, each waiting for the next

Breaking any one condition prevents deadlock:

**Lock ordering** (break circular wait): always acquire locks in the same global order. If all threads acquire lock1 before lock2, circular wait is impossible:

```
def thread_a():
    with lock1:
        with lock2: pass   # same order as thread_b

def thread_b():
    with lock1:            # both acquire lock1 first
        with lock2: pass
```

**Timeout with trylock** (break hold-and-wait):

```
def safe_acquire(lock1, lock2, timeout=1.0):
```

```
    if lock1.acquire(timeout=timeout):
        if lock2.acquire(timeout=timeout):
            return True
        else:
            lock1.release()   # back off — try again later
    return False
```

### Livelock and Starvation

**Livelock**: threads keep responding to each other, constantly changing state, but making no progress. Like two pe

### Illustrations

**Figure 137 • FIG 7.2-A**

**Mutex — Protecting a Critical Section from Concurrent Access**

*Illustration: FIG 7.2-A | Mutex — Protecting a Critical Section from Concurrent Access [SVG diagram — see module-7-2-sync-primitives.html]*

Mutex (mutual exclusion lock) protecting a critical section. Thread A calls mutex_lock(), acquires the lock (mutex transitions to LOCKED), and enters the critical section. When Thread B calls mutex_lock() while the mutex is held by A, the OS suspends B — it moves to WAITING state and stops consuming CPU. When A calls mutex_unlock(), the OS wakes B, which then acquires the lock and executes its own critical section. The two increments now happen sequentially — no interleaving — and the final counter is the correct value 2. The mutex guarantees mutual exclusion (only one thread in the critical s

**Figure 138 • FIG 7.2-B**

**Deadlock — Circular Wait: Two Threads, Two Mutexes**

*Illustration: FIG 7.2-B | Deadlock — Circular Wait: Two Threads, Two Mutexes [SVG diagram — see module-7-2-sync-primitives.html]*

Deadlock arising from circular wait: Thread A holds Mutex M1 and waits for M2; Thread B holds M2 and waits for M1. Neither can proceed — they wait forever. Deadlock requires four Coffman conditions simultaneously: mutual exclusion (only one thread can hold a lock), hold-and-wait (holding a lock while waiting for another), no preemption (locks cannot be forcibly taken), and circular wait. Breaking any one condition prevents deadlock. The simplest prevention: impose a global lock ordering — all threads must acquire M1 before M2. This eliminates circular wait by construction. Tools like ThreadSan

**Figure 139 • FIG 7.2-C**

**Semaphore — Producer-Consumer with Bounded Buffer**

*Illustration: FIG 7.2-C | Semaphore — Producer-Consumer with Bounded Buffer [SVG diagram — see module-7-2-sync-primitives.html]*

Bounded-buffer producer-consumer using two semaphores and one mutex. Semaphore empty (initialised to N) counts available slots — the producer decrements it before adding; if empty=0, the producer blocks. Semaphore full (initialised to 0) counts available items — the consumer decrements it before removing; if full=0, the consumer blocks. A mutex protects the actual buffer operations from concurrent access. This classic synchronisation pattern appears in operating system I/O buffers, message queues, thread pool task queues, and network socket receive buffers. Two semaphores handle the bounded ca

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 7.2 Mutexes, Semaphores, and Deadlocks: Mutex Demo Semaphore Pool Deadlock Simulator [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 7.2 Mutexes, Semaphores, and Deadlocks **Type:** Analysis + implementation + trace **Estimated time:** 4555 minutes **Tools needed:** Python + pen and paper --- Part A Deadlock Identification ```python import threading, time lock1 = threading.Lock() lock2 = threading.Lock() lock3 = threading.Lock() def thread_A(): with lock1: time.sleep(0.01) with lock2: waits for lock2 with lock3: pass def thread_B(): with lock2: time.sleep(0.01) with lock3: waits for lock3 with lock1: pass def thread_C

# 1. Create socket IPv4, TCP (stream)

■# Module 7.3 — Network Sockets from Scratch

## The Problem

*Two programs. Different machines. Possibly different continents.*

*How do they exchange data?*

>

*The answer — at every layer above bare hardware — is sockets.*

*A socket is the API that Unix and Windows expose for network communication.*

*Every web server, every game server, every chat application,*

*every database client ultimately calls socket(), bind(), listen(), accept(), send(), recv().*

>

*Understanding sockets means understanding how every networked application is built.*

---

## The Story

### The Birth of the Socket API

The Berkeley Software Distribution (BSD) Unix at UC Berkeley introduced the **socket API** in 1983 (4.2BSD). The goal was to create a uniform interface for network communication that abstracted over different network protocols (TCP/IP, Xerox NS, UNIX domain sockets).

The BSD socket API was so well-designed that it was adopted by every Unix-like system, then by Windows (as WinSock, 1993), and is now the standard network API on every major platform. It has been essentially unchanged for 40 years.

The core insight: a socket is a **file descriptor** — it behaves like a file. You `open()` it (socket()), you `read()` from it (recv()), you `write()` to it (send()), you `close()` it. The "everything is a file" Unix philosophy (Module 4.3) applied to networking.

---

## The Concept

### TCP Socket — Client-Server Model

**Server side:**

```
import socket

# 1. Create socket — IPv4, TCP (stream)
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2. Allow reuse of address immediately after process ends
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# 3. Bind to address and port
server.bind(('0.0.0.0', 8080))

# 4. Listen for incoming connections (backlog = 5)
server.listen(5)
print("Server listening on port 8080")

while True:
    # 5. Accept a new connection (blocks until client connects)
    conn, addr = server.accept()
    print(f"Connection from {addr}")

    # 6. Receive data (up to 1024 bytes)
    data = conn.recv(1024)
    print(f"Received: {data.decode()}")

    # 7. Send response
```

```
    conn.send(b"Hello from server!")

    # 8. Close this connection
    conn.close()
```

**Client side:**

```
import socket

# 1. Create socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2. Connect to server
client.connect(('127.0.0.1', 8080))

# 3. Send data
client.send(b"Hello from client!")

# 4. Receive response
data = client.recv(1024)
print(f"Received: {data.decode()}")

# 5. Close
client.close()
```

### The TCP Handshake (Behind the Scenes)

When `client.connect()` is called, the OS performs the TCP three-way handshake:

Client sends SYN (synchronise) to server

Server replies SYN-ACK (synchronise-acknowledge)

Client sends ACK (acknowledge)

Connection established — bidirectional byte stream ready

This handshake takes one round trip (~10 ms on a LAN, ~50–100 ms across the internet). HTTPS adds TLS handshake on top: 1–2 more round trips. Modern TLS 1.3 reduces this to 1 round trip.

### UDP Sockets — Fire and Forget

UDP is connectionless — no handshake, no guarantee of delivery or ordering.

```
# UDP server
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(('0.0.0.0', 9090))
data, addr = server.recvfrom(1024)   # returns data + sender address
server.sendto(b"ACK", addr)          # reply (optional)

# UDP client
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.sendto(b"Hello", ('127.0.0.1', 9090))   # no connect() needed
```

UDP is used when: low latency matters more than reliability (gaming, video streaming, DNS lookups), or when the application handles its own reliability (QUIC, DTLS).

### Handling Multiple Clients

A single-threaded server handles clients one at a time. For concurrent connections, three approaches:

**1. Threaded server** (one thread per connection):

```
import threading

def handle_client(conn, addr):
    data = conn.recv(1024)
    conn.send(b"Hello " + data)
    conn.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('', 8080)); server.listen()
while True:
    conn, addr = server.accept()
    threading.Thread(target=handle_client, args=(conn, addr)).start()
```

Scales to ~1,000 clients (memory/thread overhead). Does not scale to 10,000+ connections (C10K problem).

**2. select/poll/epoll** — I/O multiplexing. One thread monitors many sockets simultaneously:

```
import selectors
sel = selectors.DefaultSelector()
# Register multiple sockets
# sel.select() blocks until any socket is ready
# No threads — one event loop handles all connections
```

This is how nginx handles 50,000+ concurrent connections on one thread — the **event-driven** model.

**3. asyncio** (Python's event-driven networking):

```
import asyncio

async def handle_client(reader, writer):
    data = await reader.read(1024)
    writer.write(b"Echo: " + data)
    await writer.drain()
    writer.close()

async def main():
    server = await asyncio.start_server(handle_client, '127.0.0.1', 8080)
    async with server:
        await server.serve_forever()

asyncio.run(main())
```

### Building a Tiny HTTP Server

HTTP/1.0 is just text over a TCP socket. A minimal server:

```
import socket, os

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('', 8080)); server.listen(5)
while True:
```

```
    conn, _ = server.accept()
    request = conn.recv(4096).decode()
    first_line = request.split('\r\n')[0]  # e.g., "GET /index.html HTTP/1.0"
    path = first_line.split(' ')[1]        # e.g., "/index.html"

    filename = '.' + path
    if os.path.exists(filename):
        with open(filename, 'rb') as f:
            body = f.read()
        response = (b"HTTP/1.0 200 OK\r\n"
                    b"Content-Type: text/html\r\n\r\n" + body)
    else:
        response = b"HTTP/1.0 404 Not Found\r\n\r\nNot Found"

    conn.send(response)
    conn.close()
```

T

## Illustrations

**Figure 140 • FIG 7.3-A**

### TCP Socket Lifecycle — Server bind/listen/accept, Client connect

*Illustration: FIG 7.3-A | TCP Socket Lifecycle — Server bind/listen/accept, Client connect [SVG diagram — see module-7-3-sockets.html]*

TCP socket lifecycle showing the exact sequence of system calls on both sides. The server creates a socket, binds to a port, marks it as listening, then blocks on accept() waiting for a client. When the client calls connect(), the OS performs the TCP three-way handshake (SYN → SYN-ACK → ACK) and accept() returns a new connected socket file descriptor. Both sides then exchange data via send()/recv() — these are just file I/O operations on the socket fd. Both sides call close() when finished, which triggers a TCP FIN exchange (four-way teardown). Every socket is a file descriptor — it can be pas

**Figure 141 • FIG 7.3-B**

### TCP vs UDP — Reliability, Ordering, and Overhead Trade-offs

*Illustration: FIG 7.3-B | TCP vs UDP — Reliability, Ordering, and Overhead Trade-offs [SVG diagram — see module-7-3-sockets.html]*

TCP and UDP trade reliability for efficiency. TCP guarantees delivery, ordering, and flow control through sequence numbers, acknowledgements, and retransmission — at the cost of a 3-way handshake on every connection, per-packet ACKs, and a 20-byte minimum header. UDP sends datagrams independently with an 8-byte header and no handshake — packets may arrive out of order, be duplicated, or be dropped silently. DNS uses UDP because its queries fit in one small packet and the application can retry if needed. Video streaming uses UDP because a retransmitted old frame would arrive too late to display

**Figure 142 • FIG 7.3-C**

**Client-Server Data Flow — HTTP Request over TCP Socket**

*Illustration: FIG 7.3-C | Client-Server Data Flow — HTTP Request over TCP Socket [SVG diagram — see module-7-3-sockets.html]*

End-to-end HTTP GET request traced at the socket system call level. The 3-way TCP handshake (SYN/SYN-ACK/ACK) establishes the connection before any data is exchanged — this alone adds one full network round-trip latency. The HTTP request is sent via send() as raw bytes in the TCP stream. The server's recv() returns those bytes, parses the HTTP request, and uses send() to write the response. Connection teardown requires a FIN/FIN-ACK exchange. HTTP/1.1 persistent connections reuse the TCP connection for multiple requests, amortising the handshake cost. HTTP/2 multiplexes multiple requests over

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 7.3 Network Sockets from Scratch: TCP Handshake Send/Receive Multi-Client [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 7.3 Network Sockets from Scratch **Type:** Implementation + trace + reasoning **Estimated time:** 4555 minutes **Tools needed:** Python --- Part A TCP Echo Server and Client 1. Implement a TCP **echo server** that: - Listens on port 9000 - Accepts one connection at a time - Reads up to 1024 bytes from the client - Sends the same bytes back (echo) - Closes the connection and accepts the next one - Runs until interrupted 2. Implement a TCP **echo client** that: - Connects to `127.0.0.1:9000` - Sends the string "Hello, server!" - Reads the response

# Module 7 4 Shell

■# Module 7.4 — Building a Tiny Shell

## The Problem

*The shell is the user's interface to the operating system.*

*Every time you type* `ls -la | grep .py > output.txt`,

*something reads that line, parses it, and executes it.*

>

*Building a tiny shell from scratch teaches you:*

*- How* `fork()` + `exec()` *work together to create processes*

*- How pipes are implemented at the OS level*

*- How input/output redirection works*

*- What the shell actually does when you type a command*

---

## The Story

### The Shell as the First Application

The shell was the very first interactive program run by Unix — it was already present in the initial PDP-7 Unix system in 1969. Ken Thompson wrote the first Unix shell (the Thompson shell, `sh`) in a single weekend.

**Stephen Bourne** rewrote it in 1977 as the Bourne shell (`/bin/sh`) — the ancestor of every POSIX-compatible shell today. **Bill Joy** (founder of Sun Microsystems) wrote the C shell (`csh`) in 1978, adding job control and command history. **David Korn** wrote the Korn shell (`ksh`) in 1983, combining features of both. The **Bash** shell (Bourne Again SHell, Brian Fox, 1989) is the default on most Linux systems today.

All of these shells — despite their different features and syntaxes — implement the same fundamental mechanics: read a line, parse it, `fork()`, `exec()`, `wait()`.

---

## The Concept

### The Core Loop: REPL

A shell is a **Read-Eval-Print Loop**:

```
import os, sys, subprocess

def shell():
    while True:
        # 1. Print prompt
        try:
            line = input(f"{os.getcwd()} $ ")
        except (EOFError, KeyboardInterrupt):
            print(); break

        line = line.strip()
        if not line: continue

        # 2. Parse
        parts = line.split()

        # 3. Built-in commands
        if parts[0] == 'exit':
            break
        if parts[0] == 'cd':
```

```
            try: os.chdir(parts[1] if len(parts) > 1 else os.environ.get('HOME', '/'))
            except FileNotFoundError as e: print(f"cd: {e}")
            continue

        # 4. Execute external command
        try:
            proc = subprocess.run(parts)
        except FileNotFoundError:
            print(f"{parts[0]}: command not found")

shell()
```

### fork() + exec() — The Unix Process Model

Under the hood, `subprocess.run(['ls', '-la'])` does exactly this:

```
import os

pid = os.fork()
if pid == 0:
    # Child process
    os.execvp('ls', ['ls', '-la'])   # replace child with 'ls'
    # execvp never returns on success
    sys.exit(1)
else:
    # Parent process (the shell)
    os.waitpid(pid, 0)               # wait for child to finish
```

`fork()` creates an exact copy of the current process. `exec()` replaces the running program with a new one. The child never returns from `exec()` on success. The parent waits.

This two-step design gives the shell a window between `fork()` and `exec()` to manipulate file descriptors — which is how pipes and redirections are implemented.

### I/O Redirection

```
ls > output.txt       # stdout → file
cat < input.txt       # stdin ← file
ls 2> errors.txt      # stderr → file
```

Between `fork()` and `exec()`, the shell redirects file descriptors using `dup2()`:

```
if redirect_stdout:
    fd = os.open(output_file, os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o644)
    os.dup2(fd, 1)    # make stdout (fd 1) point to the file
    os.close(fd)      # close original (now duplicated)
```

`dup2(src, dst)`: makes `dst` a duplicate of `src`. After `dup2(fd, 1)`, writes to stdout go to the file. The `exec()`'d program never knows — it just uses fd 1 as normal.

### Pipes

```
ls | grep .py | wc -l
```

A pipe `|` connects stdout of one process to stdin of the next.

```
def run_pipeline(commands):
    # commands = [['ls'], ['grep', '.py'], ['wc', '-l']]
```

```
    n = len(commands)
    pipes = [os.pipe() for _ in range(n - 1)]  # n-1 pipes for n commands

    pids = []
    for i, cmd in enumerate(commands):
        pid = os.fork()
        if pid == 0:  # child
            # Connect stdin to previous pipe's read end
            if i > 0:
                os.dup2(pipes[i-1][0], 0)  # stdin ← pipe read
            # Connect stdout to next pipe's write end
            if i < n - 1:
                os.dup2(pipes[i][1], 1)    # stdout → pipe write
            # Close all pipe fds (child doesn't need them)
            for r, w in pipes:
                os.close(r); os.close(w)
            os.execvp(cmd[0], cmd)
            sys.exit(1)
        pids.append(pid)

    # Parent: close all pipe fds
    for r, w in pipes:
        os.close(r); os.close(w)

    # Wait for all children
    for pid in pids:
        os.waitpid(pid, 0)
```

The kernel's `os.pipe()` creates a pair of file descriptors: `(read_end, write_end)`. Data written to `write_end` is buffered by the kernel and can be read from `read_end`. This is why `ls | grep` works: `ls`'s stdout writes to the pipe buffer; `grep`'s stdin reads from it.

### Job Control

Real shells support background jobs (`cmd &`), job suspension (`Ctrl+Z`), and foreground/background switching (`fg`, `bg`). This requires Unix signal handling:

• `SIGCHLD`: sent to parent when a child changes state

• `SIGINT` (Ctrl+C): interrupt the foreground job

• `SIGTSTP` (Ctrl+Z): suspend the foreground job

• `SIGCONT`: resume a suspended job

A minimal implementation:

```
import signal

background_jobs = []

def run_background(cmd):
    pid = os.fork()
    if pid == 0:
        os.execvp(cmd[0], cmd); sys.exit(1)
    background_jobs.append(pid)
```

```
    print(f"[{len(background_jobs)}] {pid}")
```

## Hands-On Exercise

**Open the simulation:** [`simulation.html`](./simulation.html)

**Tool 1 — Command Parser**

• Type shell commands; watch the tokenisation and parsing

• See quoted strings handled correctly: `echo "hello world"` → 2 tokens

**Tool 2 — fork/exec Visualiser**

• Type a command; watch the process tree

• See the parent shell fork a child, the child exec,

### Illustrations

**Figure 143 • FIG 7.4-A**

**Shell Internals — The Read-Parse-Fork-Exec-Wait Cycle**

*Illustration: FIG 7.4-A | Shell Internals — The Read-Parse-Fork-Exec-Wait Cycle [SVG diagram — see module-7-4-shell.html]*

The shell's core loop: read a line from the terminal, tokenise and parse it into a command and arguments, fork a child process (an identical copy of the shell), call exec in the child (replacing the child's memory image with the requested program), then wait for the child to finish before printing the next prompt. The shell itself is never replaced — it always returns to the top of the loop. This fork-exec separation is elegant: between fork() and exec(), the child can redirect file descriptors, set environment variables, or set up pipes — before the new program ever starts. This is how shell

**Figure 144 • FIG 7.4-B**

**fork() / exec() / wait() — Process Tree After Each System Call**

*Illustration: FIG 7.4-B | fork() / exec() / wait() — Process Tree After Each System Call [SVG diagram — see module-7-4-shell.html]*

The fork/exec/wait triad is the foundation of Unix process creation. fork() duplicates the calling process — both parent and child continue after the fork call, distinguished by the return value (0 in child, child PID in parent). exec() replaces the calling process's memory image with a new program — the PID is preserved but everything else changes. The parent calls waitpid() which suspends it until the child exits, preventing zombie processes. The crucial architectural insight: the gap between fork() and exec() lets the child rearrange its file descriptors (enabling redirection with dup2()) a

**Figure 145 • FIG 7.4-C**

**Shell Pipeline — ls | grep txt | wc -l**

*Illustration: FIG 7.4-C | Shell Pipeline — ls | grep txt | wc -l [SVG diagram — see module-7-4-shell.html]*

Shell pipeline ls | grep txt | wc -l implemented with two kernel pipes and three child processes. The shell calls pipe() twice to create two kernel pipe file descriptors, then forks three children. Before each child calls exec(), it uses dup2() to redirect: ls's stdout to pipe1's write end, grep's stdin from pipe1's read end and stdout to pipe2's write end, wc's stdin from pipe2's read end. All three processes run concurrently — ls writes output as grep reads it, grep filters and writes as wc reads. The kernel pipe buffer (typically 64KB) decouples their speeds. The shell waits for all three c

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 7.4 Building a Tiny Shell: Command Parser fork/exec Visualiser Pipe Diagram [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 7.4 Building a Tiny Shell **Type:** Implementation + trace + security **Estimated time:** 5060 minutes **Tools needed:** Python (on Linux/macOS for os.fork) or simulation --- Part A Command Parser 1. Implement `parse_command(line)` that tokenises a shell command string, handling: - Simple commands: `"ls -la"` `["ls", "-la"]` - Quoted strings: `"echo 'hello world'"` `["echo", "hello world"]` - Output redirection: `"ls > out.txt"` `{"cmd": ["ls"], "stdout": "out.txt"}` - Input redirection: `"sort < data.txt"` `{"cmd": ["sort"], "stdin": "data.txt"}` - Pipes

# Apply to all routes:

■# Module 7.5 — Building a Tiny HTTP Server

## The Problem

*Every website you visit is served by an HTTP server.*

*nginx, Apache, Express, Django, Flask — they all do the same core job:*

*listen on a port, accept TCP connections, parse HTTP requests, return HTTP responses.*

>

*When you understand how a tiny HTTP server works from the socket layer up,*

*every web framework becomes transparent.*

*You understand what Flask's* `@app.route` *is doing underneath.*

*You understand what "middleware" means mechanically.*

*You understand why HTTP/1.1 keep-alive matters for performance.*

---

## The Story

### HTTP Was Designed on a Napkin

Tim Berners-Lee invented HTTP at CERN in 1989. The original HTTP/0.9 was extraordinarily simple: one command (`GET /path`) over a TCP connection, response is the file content, connection closes. No headers, no status codes, no response format.

HTTP/1.0 (1996) added headers, status codes (200, 404, etc.), and support for content types — enabling images, not just text. Each request required a new TCP connection.

HTTP/1.1 (1997, still dominant) added keep-alive (reuse connections for multiple requests), chunked transfer encoding, host headers (enabling virtual hosting — multiple websites on one IP), and caching directives. It is the protocol powering the majority of the web today.

HTTP/2 (2015) added multiplexing (multiple requests per connection, no head-of-line blocking), header compression, and server push. HTTP/3 (2022) runs over QUIC instead of TCP.

A working understanding of HTTP/1.1 is sufficient to understand everything built on top of it.

---

## The Concept

### HTTP Request Format

An HTTP/1.1 request is plain text over a TCP connection:

```
GET /index.html HTTP/1.1\r\n
Host: www.example.com\r\n
User-Agent: Mozilla/5.0\r\n
Accept: text/html\r\n
\r\n
```

Structure:

• **Request line**: `METHOD PATH HTTP/VERSION`

• **Headers**: `Name: Value` pairs, each ending `\r\n`

• **Blank line**: `\r\n` marks end of headers

• **Optional body**: for POST/PUT requests (e.g., form data, JSON)

### HTTP Response Format

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html; charset=utf-8\r\n
Content-Length: 138\r\n
```

```
Connection: keep-alive\r\n
\r\n
<html>...</html>
```

Structure:

- **Status line**: `HTTP/VERSION STATUS_CODE REASON_PHRASE`
- **Headers**: content type, length, caching, cookies, etc.
- **Blank line**
- **Body**: the actual content

### Parsing an HTTP Request

```
def parse_request(raw: bytes) -> dict:
    text  = raw.decode('utf-8', errors='replace')
    lines = text.split('\r\n')

    # Request line
    method, path, version = lines[0].split(' ', 2)

    # Headers
    headers = {}
    i = 1
    while i < len(lines) and lines[i] != '':
        key, _, value = lines[i].partition(': ')
        headers[key.lower()] = value
        i += 1

    # Body (everything after the blank line)
    body = '\r\n'.join(lines[i+1:]) if i+1 < len(lines) else ''

    return {
        'method':  method,
        'path':    path,
        'version': version,
        'headers': headers,
        'body':    body,
    }
```

### Building the Server — Step by Step

```
import socket, os, mimetypes
from pathlib import Path

STATIC_ROOT = Path('./static')

STATUS = {200: 'OK', 404: 'Not Found', 405: 'Method Not Allowed',
          500: 'Internal Server Error'}

def build_response(status, headers, body=b''):
    header_lines = f"HTTP/1.1 {status} {STATUS[status]}\r\n"
    for k, v in headers.items():
        header_lines += f"{k}: {v}\r\n"
```

```
    return header_lines.encode() + b"\r\n" + body

def handle_request(raw):
    try:
        req = parse_request(raw)
    except Exception:
        return build_response(400, {'Content-Length': 0})

    if req['method'] not in ('GET', 'HEAD'):
        return build_response(405, {'Allow': 'GET, HEAD', 'Content-Length': 0})

    # Resolve path (security: prevent directory traversal)
    rel_path = req['path'].lstrip('/')
    if not rel_path: rel_path = 'index.html'
    file_path = (STATIC_ROOT / rel_path).resolve()

    # Security: ensure file is inside STATIC_ROOT
    if not str(file_path).startswith(str(STATIC_ROOT.resolve())):
        return build_response(403, {'Content-Length': 0})

    if not file_path.exists():
        return build_response(404, {'Content-Type': 'text/plain',
                                    'Content-Length': str(len(b'Not Found'))}, b'Not Found')

    body = b'' if req['method'] == 'HEAD' else file_path.read_bytes()
    mime = mimetypes.guess_type(str(file_path))[0] or 'application/octet-stream'
    return build_response(200, {
        'Content-Type': mime,
        'Content-Length': str(len(body)),
        'Connection': 'close',
    }, body)

def run_server(host='', port=8080):
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((host, port))
    server.listen(5)
    print(f"Serving on http://localhost:{port}")
    while True:
        conn, addr = server.accept()
        raw = b''
        while b'\r\n\r\n' not in raw:
            chunk = conn.recv(4096)
            if not chunk: break
            raw += chunk
        response = handle_request(raw)
        conn.sendall(response)
        conn.close()
```

### Adding a Router

A router maps URL patterns to handler functions — this is exactly what Flask's `@app.route` does:

```
routes = {}
```

```
def route(path):
    def decorator(fn):
        routes[path] = fn
        return fn
    return decorator

@route('/hello')
def hello(req):
    body = b'<h1>Hello, World!</h1>'
    return build_response(200, {'Content-Type': 'text/html',
                                'Content-Length': str(len(body))}, body)

@route('/echo')
def echo(req):
    body = req['body'].encode()
    return build_response(200, {'Content-Type': 'text/plain',
                                'Content-Length': str(len(body))}, body)

def handle_request(raw):
    req = parse_request(raw)
    handler = routes.get
```

## Illustrations

**Figure 146 • FIG 7.5-A**

### HTTP/1.1 Request and Response — Anatomy of Every Web Transaction

*Illustration: FIG 7.5-A | HTTP/1.1 Request and Response — Anatomy of Every Web Transaction [SVG diagram — see module-7-5-http-server.html]*

HTTP/1.1 request and response anatomy. Both are plain text transmitted over a TCP connection. A request opens with a request line (method, path, HTTP version), followed by headers (key: value pairs), a mandatory blank line, then an optional body (present in POST/PUT). The response opens with a status line (version, numeric code, reason phrase), then headers, blank line, body. The blank line (CRLF CRLF) is the delimiter between headers and body — a server must read until it finds this sequence before parsing headers. HTTP is intentionally human-readable, which makes debugging with nc or telnet

**Figure 147 • FIG 7.5-B**

### HTTP Server Models — Single-Threaded, Multi-Threaded, Event-Driven

*Illustration: FIG 7.5-B | HTTP Server Models — Single-Threaded, Multi-Threaded, Event-Driven [SVG diagram — see module-7-5-http-server.html]*

Three HTTP server concurrency models. Single-threaded is simple but blocks on I/O — second request waits for first. Thread-per-request handles concurrent I/O but costs ~1MB of stack RAM per thread: at 10,000 concurrent connections that is 10GB RAM just for stacks. The event-driven model (Nginx, Node.js) uses a single thread with epoll/kqueue to monitor thousands of file descriptors simultaneously — the thread only runs when a descriptor is ready, eliminating idle time. This solved the "C10K problem" (10,000 simultaneous connections on a single server) in the early 2000s. Modern servers combine

**Figure 148 • FIG 7.5-C**

### HTTP Request Processing Pipeline — accept to response

*Illustration: FIG 7.5-C | HTTP Request Processing Pipeline — accept to response [SVG diagram — see module-7-5-http-server.html]*

HTTP server request processing pipeline. accept() blocks until a new TCP connection arrives, returning a connected socket file descriptor. read() collects bytes until the CRLF-CRLF header terminator. parse() extracts method, path, HTTP version, and all headers. route() looks up the path in a routing table to find the appropriate handler function. handler() performs the application logic — reading a file, querying a database, calling another service. write() serialises the response (status line, headers, blank line, body) and sends it over the socket. Any stage can short-circuit with an error r

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 7.5 Building a Tiny HTTP Server: Request Parser Router Full Request Trace [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 7.5 Building a Tiny HTTP Server **Type:** Implementation + trace + extension **Estimated time:** 5060 minutes **Tools needed:** Python --- Part A HTTP Request Parsing Given this raw HTTP request as bytes: ``` POST /api/users HTTP/1.1\r\n Host: localhost:8080\r\n Content-Type: application/json\r\n Content-Length: 27\r\n Authorization: Bearer abc123\r\n \r\n {"name":"Alice","age":30} ``` 1. Parse this request into a dict with keys: `method`, `path`, `version`, `headers` (dict), `body` (string). Show your `parse_request` function. 2. What is the significance of the blank li

LEVEL 08

# Networks & Protocols

From Bits on Wire to HTTPS — The Internet Layer by Layer

# Module 8 1 Physical Datalink

■# Module 8.1 — Physical and Data Link Layers

## The Problem

*Two computers. A wire between them.*

*How do bits get from one to the other?*

*And if the wire is shared by 100 computers, how does each*

*computer know which bits were meant for it?*

>

*These questions are answered at the two lowest layers of the network stack:*

*the **Physical Layer** (turning bits into signals)*

*and the **Data Link Layer** (framing, addressing, and access control).*

## The Story

### From Morse Code to Ethernet

The physical layer problem is as old as electrical communication. Samuel Morse's telegraph (1844) was a one-bit physical layer: key down = 1, key up = 0. The modulation scheme (dots and dashes) was a data link layer that enabled human-readable encoding.

**Ethernet** was invented at Xerox PARC in 1973 by Robert Metcalfe and David Boggs. The name comes from "ether" — the medium through which light was once thought to propagate. The core problem Metcalfe solved was how to share a single cable among multiple computers without a central controller — the **CSMA/CD** (Carrier Sense Multiple Access with Collision Detection) protocol.

The 10Base5 "thick Ethernet" cable of the 1970s literally connected computers through a shared coaxial cable — any computer could listen to any other's transmissions. Modern Ethernet uses switches, but the addressing scheme (MAC addresses) and framing format are essentially unchanged from 1980.

## The Concept

### Layer 1: The Physical Layer

The physical layer is responsible for transmitting a sequence of bits over a physical medium. It is not concerned with what the bits mean — only with reliably transferring each 0 and 1.

**Encoding schemes:**

• **NRZ (Non-Return to Zero)**: 1 = high voltage, 0 = low voltage. Problem: long strings of same bit lose clock synchronisation.

• **Manchester encoding**: each bit includes a transition: 1 = low-to-high, 0 = high-to-low. Used in early Ethernet. Reliable but uses 2× bandwidth.

• **4B/5B**: encode every 4 bits as a 5-bit code chosen to prevent long runs of zeros. Used in 100Base-TX (Fast Ethernet).

• **PAM-4** (4-level pulse amplitude modulation): four voltage levels encode 2 bits per symbol. Used in 25G, 100G, 400G Ethernet.

**Physical media:**

• **Twisted pair copper** (Cat5e, Cat6, Cat7): standard for LAN. Pairs twisted to reduce electromagnetic interference. Up to 10 Gbps for Cat6A at 55m.

• **Fibre optic**: light pulses in glass or plastic fibre. Single-mode: 100 km+ at 400 Gbps. Multi-mode: shorter distances, cheaper. Used for all long-distance and data centre interconnects.

• **802.11 (Wi-Fi)**: radio waves. Physical layer uses OFDM (Orthogonal Frequency Division Multiplexing) — simultaneously transmit on many sub-carriers. Wi-Fi 6 (802.11ax): up to 9.6 Gbps theoretical.

### Layer 2: The Data Link Layer

The data link layer takes raw bits and organises them into **frames** — logical units with headers, payload, and error checking.

**Ethernet frame format** (IEEE 802.3):

```
Preamble (7B) | SFD (1B) | Dst MAC (6B) | Src MAC (6B) | EtherType (2B) | Payload (46-1500B) | FCS (4B
```

• **Preamble + SFD**: synchronisation — `10101010...10101011` lets the receiver lock onto the clock

• **Destination MAC**: 6-byte hardware address of the intended recipient

• **Source MAC**: sender's hardware address

• **EtherType**: what protocol is in the payload (`0x0800` = IPv4, `0x0806` = ARP, `0x86DD` = IPv6)

• **Payload**: 46–1500 bytes (the MTU — maximum transmission unit — is 1500 bytes for standard Ethernet)

• **FCS (Frame Check Sequence)**: CRC-32 checksum to detect transmission errors

**MAC Addresses**: Every network interface has a globally unique 48-bit MAC address burned in at manufacture. Format: `AA:BB:CC:DD:EE:FF`. First 24 bits = OUI (Organisationally Unique Identifier, assigned to the manufacturer). Last 24 bits = device-specific.

Special addresses:

• `FF:FF:FF:FF:FF:FF` = broadcast (all devices on the local segment receive it)

• `01:00:5E:xx:xx:xx` = IPv4 multicast

**MAC addresses are local**: they only matter within a single network segment (broadcast domain). The IP address is what matters for routing between networks.

### CSMA/CD — Ethernet's Collision Protocol

Early Ethernet (bus topology — all computers on one shared cable) used CSMA/CD:

**Carrier Sense**: before transmitting, listen if someone else is already transmitting. If so, wait.

**Multiple Access**: multiple computers share the same medium.

**Collision Detection**: while transmitting, listen. If what you hear differs from what you're sending, a collision occurred (two computers started transmitting simultaneously). Both stop, wait a **random backoff** time (exponential backoff), and retry.

Modern switched Ethernet is **full-duplex point-to-point** — each cable connects one computer to one switch port. No shared medium, no collisions possible. CSMA/CD is not used but is still specified by the standard.

### Switches — The Data Link Layer Device

A **network switch** operates at Layer 2. It:

**Learns** MAC addresses: when a frame arrives on a port, it associates the source MAC with that port in its **MAC address table** (CAM table).

**Forwards** frames: looks up the destination MAC; if found, sends only to that port; if not, **floods** to all ports except the source.

Creates **collision domains** per port: each port is a separate collision domain — no more shared medium.

A 24-port switch with all links active gives each device its full bandwidth (unlike a hub, where all devices share the bandwidth).

**VLANs** (Virtual LANs): divide a physical switch into multiple logical segments. Frames carry a **VLAN tag** (IEEE 802.1Q, 4 bytes added to the frame) identifying which VLAN they belong to. Frames from different VLANs cannot communicate directly — they must be routed (Layer 3).

---

## Hands-On Exercise

**Open the simulation:** [`simulation.html`](./simulation.html)

**Tool 1 — Manchester Encoding**

• Enter a bit sequence

### Illustrations

**Figure 149 • FIG 8.1-A**

### OSI 7-Layer Model — Encapsulation as Data Travels Down the Stack

*Illustration: FIG 8.1-A | OSI 7-Layer Model — Encapsulation as Data Travels Down the Stack [SVG diagram — see module-8-1-physical-datalink.html]*

The OSI 7-layer model with encapsulation. As data travels down the sender's stack, each layer adds its own header (and sometimes trailer): Transport adds TCP/UDP ports and sequence numbers, Network adds IP source/destination addresses, Data Link adds MAC addresses and a Frame Check Sequence. The receiver's stack strips these headers in reverse — each layer reads only its own header and passes the payload up. This layering is the key architectural principle: higher layers are completely insulated from the details of lower layers. An HTTP server does not know whether it is running over copper Et

**Figure 150 • FIG 8.1-B**

### Ethernet II Frame Structure — Every Field Labelled with Byte Widths

*Illustration: FIG 8.1-B | Ethernet II Frame Structure — Every Field Labelled with Byte Widths [SVG diagram — see module-8-1-physical-datalink.html]*

Ethernet II frame structure — the Data Link Layer PDU on virtually all modern LANs. The 7-byte preamble and 1-byte Start Frame Delimiter are used by hardware for clock synchronisation and are not counted in the frame length. The 6-byte destination and source MAC addresses identify which device should receive the frame and who sent it. EtherType (0x0800 for IPv4, 0x0806 for ARP, 0x86DD for IPv6) tells the receiver how to parse the payload. The 4-byte Frame Check Sequence (CRC-32) detects transmission errors — a mismatch causes the NIC to silently discard the frame. Minimum payload is 46 bytes (

**Figure 151 • FIG 8.1-C**

### Encapsulation Walk-Through — HTTP Data Wrapped at Each Layer

*Illustration: FIG 8.1-C | Encapsulation Walk-Through — HTTP Data Wrapped at Each Layer [SVG diagram — see module-8-1-physical-datalink.html]*

Encapsulation walk-through for an HTTP GET request. At Layer 7 (Application), the HTTP request is plain text. Layer 4 (Transport) wraps it in a TCP segment header adding source port, destination port 80, and sequence number. Layer 3 (Network) wraps the TCP segment in an IP packet header adding source and destination IP addresses. Layer 2 (Data Link) wraps the IP packet in an Ethernet frame adding MAC addresses and a CRC trailer. Layer 1 (Physical) transmits the entire frame as electrical signals or light pulses. The receiver performs the reverse — each layer reads its header, verifies integrit

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 8.1 Physical and Data Link Layers: Manchester Encoding Ethernet Frame Builder Switch MAC Table [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 8.1 Physical and Data Link Layers **Type:** Calculation + trace + reasoning **Estimated time:** 4050 minutes **Tools needed:** pen and paper --- Part A Manchester Encoding 1. Encode the bit sequence `10110010` using Manchester encoding (1 = low-to-high transition, 0 = high-to-low transition). Draw the resulting waveform showing voltage levels over time. 2. Decode this Manchester-encoded waveform back to bits: starting HIGH, the transitions are: (each pair = one bit). What is the decoded sequence? 3. Manchester encoding uses 2 the bandwidth of NRZ for the same bit

# Module 8 2 Network Layer

■# Module 8.2 — Network Layer: IP and Routing

## The Problem

*Ethernet connects computers on the same local segment.*

*But your laptop is not on the same cable as a server in Tokyo.*

*Billions of machines are connected through an interconnected web of networks.*

>

*How does a packet travel from your machine to one in Tokyo*

*— through dozens of intermediate routers, different network technologies,*

*different countries — and arrive at exactly the right destination?*

>

*This is the network layer's problem: **global addressing and routing**.*

## The Story

### The Internet Is a Network of Networks

The ARPANET (1969) connected four universities. By 1972 it had 40 nodes. By 1975 it was a US government backbone. The challenge: connect different networks — ARPANET, satellite links, radio networks — using a common protocol.

**Vint Cerf and Bob Kahn** designed IP (Internet Protocol) in 1974. The key principle: IP makes no assumptions about the underlying network. It doesn't care if the link is Ethernet, Wi-Fi, dial-up, or carrier pigeon (RFC 1149, 1990 — an actual internet standard). IP provides a common addressing scheme and best-effort delivery. Reliability was left to the transport layer (TCP).

**IPv4** (1981) remains the dominant protocol with 4.3 billion addresses. Address exhaustion, foreseen since the 1980s, forced the development of **IPv6** (1998) with $2^{12}$■ addresses. The internet has been running a simultaneous IPv4/IPv6 transition since the 1990s — still ongoing.

## The Concept

### IPv4 Addresses

An IPv4 address is a 32-bit number, written as 4 decimal octets: `192.168.1.100`.

```
192.168.1.100
■   ■   ■ ■
■   ■   ■ ■■■ Host (100) within the subnet
■   ■   ■■■■■ Subnet (1)
■   ■■■■■■■■■ Network (168)
■■■■■■■■■■■■■ Network (192)
```

**Subnet masks** define the boundary between network and host portions:

• `/24` (255.255.255.0): first 24 bits = network, last 8 bits = host. 256 addresses, 254 usable.

• `/16` (255.255.0.0): first 16 bits = network. 65,536 addresses.

• `/32`: single host.

**CIDR notation** (Classless Inter-Domain Routing): `192.168.1.0/24` means the 256 addresses `192.168.1.0` through `192.168.1.255`.

**Special addresses:**

• `127.0.0.0/8`: loopback (`127.0.0.1` = "this machine")

• `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`: private (not routable on the internet)

• `0.0.0.0/0`: default route (matches everything)

• Broadcast: last address in subnet (e.g., `192.168.1.255` for `/24`)

### IPv4 Packet Header

```
 0               1               2               3
 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|Ver=4|IHL| DSCP    |ECN|        Total Length                   |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|      Identification           |Flags| Fragment Offset         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   TTL         |  Protocol     |       Header Checksum         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                     Source IP Address                         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                  Destination IP Address                       |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

**TTL (Time to Live)**: decremented by each router. Prevents packets looping forever. If TTL reaches 0, packet is dropped and ICMP Time Exceeded is sent back. Starting TTL is typically 64 or 128.

**Protocol**: identifies the transport layer protocol. 6=TCP, 17=UDP, 1=ICMP.

**Fragmentation**: if a packet is larger than a link's MTU, it is split into fragments, each with the same Identification. Reassembled at the destination. Modern practice: use **Path MTU Discovery** instead — discover the smallest MTU on the path and never send packets that need fragmenting.

### Routing

Each router maintains a **routing table** — a set of rules mapping destination prefixes to next-hop routers or output interfaces:

```
Network          Netmask          Gateway        Interface
0.0.0.0          0.0.0.0          10.0.0.1       eth0    ← default route
10.0.0.0         255.255.255.0    0.0.0.0        eth0    ← directly connected
192.168.1.0      255.255.255.0    0.0.0.0        eth1    ← directly connected
```

**Longest prefix match**: when multiple routes match a destination, the most specific (longest prefix) wins. `10.1.2.3` matches both `10.0.0.0/8` and `10.1.0.0/16` — the `/16` wins because it's more specific.

### Routing Protocols

Routers learn routes from each other using routing protocols:

**RIP (Routing Information Protocol)**: distance-vector, hop count as metric. Simple, slow convergence, max 15 hops. Used in small networks.

**OSPF (Open Shortest Path First)**: link-state routing. Each router floods its link state to all others; every router builds a complete topology map and computes shortest paths (Dijkstra's algorithm). Fast convergence, no hop limit. Used in enterprise and ISP networks.

**BGP (Border Gateway Protocol)**: the protocol of the internet. Connects **autonomous systems** (ASes) — networks operated by ISPs, companies, universities. BGP carries policy information (not just cost) — an ISP can prefer routes through certain ASes, avoid others, or limit what routes it advertises. The internet's routing table has ~900,000+ BGP routes.

### NAT — Network Address Translation

Private addresses (`192.168.x.x`) are not routable on the internet. **NAT** (typically on a home router) rewrites source IP and port:

```
Outbound: 192.168.1.5:54321 → 8.8.8.8:53
NAT rewrites to: 203.0.113.1:41234 → 8.8.8.8:53
(203.0.113.1 = public IP of the router)

Return: 8.8.8.8:53 → 203.0.113.1:41234
NAT rewrites back to: 8.8.8.8:53 → 192.168.1.5:54321
```

NAT effectively allows millions of devices to share a few public IP addresses, delaying IPv4 exhaustion for decades. The downside: breaks the end-to-end principle — d

### Illustrations

**Figure 152 • FIG 8.2-A**

### IPv4 Packet Header — All 13 Fields with Bit Widths

*Illustration: FIG 8.2-A | IPv4 Packet Header — All 13 Fields with Bit Widths [SVG diagram — see module-8-2-network-layer.html]*

IPv4 packet header — 20 bytes minimum in 5 rows of 32 bits each. The most critical fields: Source and Destination IP (32 bits each) are the addresses every router uses to forward the packet. TTL (Time to Live) prevents routing loops — each router decrements it by 1; at 0 the packet is dropped and an ICMP Time Exceeded message is sent back. Protocol identifies the payload type (6=TCP, 17=UDP) so the receiving host knows how to parse the payload. IHL (Internet Header Length) specifies the header length in 32-bit words — necessary because the Options field makes the header variable-length. The Fr

**Figure 153 • FIG 8.2-B**

### Routing Table — Longest Prefix Match for 192.168.5.100

*Illustration: FIG 8.2-B | Routing Table — Longest Prefix Match for 192.168.5.100 [SVG diagram — see module-8-2-network-layer.html]*

Routing table lookup using longest prefix match. For destination 192.168.5.100, the router tests every entry: the default route /0 always matches, /8 matches on the first octet, /16 matches on the first two octets, and /24 matches on the first three octets. The /24 entry (192.168.5.0/24) is the most specific match — more bits agree with the destination — so the packet is forwarded to next-hop 10.0.1.1 on interface eth2. This algorithm runs on every router for every packet. Modern routers implement it in hardware using TCAM (Ternary Content-Addressable Memory) at line rate — billions of lookups

**Figure 154 • FIG 8.2-C**

### Subnetting and CIDR — How IP Addresses Are Allocated in Blocks

*Illustration: FIG 8.2-C | Subnetting and CIDR — How IP Addresses Are Allocated in Blocks [SVG diagram — see module-8-2-network-layer.html]*

CIDR (Classless Inter-Domain Routing) notation specifies how many leading bits are the network address. 192.168.1.0/24 means the first 24 bits (the first three octets) are fixed — they identify the network — and the remaining 8 bits identify individual hosts within that network. The subnet mask (255.255.255.0) is a bitmask with 1s for network bits and 0s for host bits; bitwise AND of any address with the mask yields the network address. The first address (all host bits 0) is the network identifier and the last (all host bits 1) is the broadcast address — both are reserved. This gives 254 usabl

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 8.2 Network Layer: IP and Routing: Subnet Calculator Routing Table Packet Routing [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 8.2 Network Layer: IP and Routing **Type:** Calculation + trace + reasoning **Estimated time:** 4050 minutes **Tools needed:** pen and paper --- Part A Subnet Calculations For each of the following, compute: network address, broadcast address, first and last usable host address, number of usable hosts. 1. `192.168.10.50/24` 2. `10.0.5.130/25` 3. `172.16.0.200/20` 4. How many /27 subnets can you create from a single /24 network? How many usable hosts does each /27 provide? 5. You have the block `10.10.0.0/22`. Divide it into exactly 4 equal subnets. List the network addre

# Module 8 3 Transport Layer

■# Module 8.3 — Transport Layer: TCP and UDP

## The Problem

*IP (Module 8.2) delivers packets between machines.*

*But a machine runs hundreds of programs, each communicating simultaneously.*

*How does the OS know which packets belong to which program?*

*How do we guarantee that data arrives completely and in order?*

*How do we prevent a fast sender from overwhelming a slow receiver?*

>

*These are the transport layer's problems:*

***multiplexing**, **reliability**, and **flow control**.*

## The Story

### TCP Was Born from Military Necessity

The ARPANET (1969) used a packet-switched protocol without reliability guarantees. When the US military needed reliable data transfer — routing commands, sensor data, and communications across potentially damaged or intermittent connections — a new protocol was needed.

Vinton Cerf and Robert Kahn published the TCP specification in 1974. The original design had TCP and IP as one protocol. Jon Postel's famous contribution (1978) was separating them — TCP (Transport Control Protocol) handled reliability and ordering; IP handled routing. This separation of concerns enabled the internet to accommodate different transport needs: UDP for simple datagram delivery, TCP for reliable streams.

Cerf and Kahn received the Turing Award in 2004 for this work.

## The Concept

### Port Numbers — Multiplexing

A machine has one IP address but hundreds of applications needing network access. **Port numbers** (16-bit, 0–65535) distinguish between applications:

• **Well-known ports** (0–1023): reserved for standard services. HTTP=80, HTTPS=443, SSH=22, DNS=53, FTP=21, SMTP=25.

• **Registered ports** (1024–49151): by application convention. MySQL=3306, PostgreSQL=5432, Redis=6379, MongoDB=27017.

• **Dynamic/ephemeral ports** (49152–65535): assigned by the OS for outbound connections.

A connection is uniquely identified by a **5-tuple**: `(protocol, src_IP, src_port, dst_IP, dst_port)`. Two different browsers can connect to the same server simultaneously because they have different ephemeral source ports.

### UDP — Simple Datagram

UDP adds only port numbers to IP:

```
UDP Header (8 bytes):
[src_port 2B] [dst_port 2B] [length 2B] [checksum 2B]
[payload]
```

**Properties**: connectionless, unreliable (no delivery guarantee), no ordering, no flow control. Fire and forget.

**When to use UDP**:

• **DNS**: a query fits in one packet; if no reply arrives, just retry. TCP overhead is unnecessary.

• **Real-time media** (VoIP, video streaming): a dropped packet is better than a delayed one. Applications handle loss gracefully (video skips a frame, audio conceals the gap).

• **Online games**: low latency matters more than reliability. Games send position updates 60/s — a missed one is just outdated data.

• **QUIC**: uses UDP as a substrate and builds its own reliability mechanism on top.

### TCP — Reliable Byte Stream

TCP provides an ordered, reliable, flow-controlled byte stream between two endpoints.

**TCP Header** (20 bytes minimum):

```
[src_port 2B] [dst_port 2B]
[sequence_number 4B]
[acknowledgement_number 4B]
[flags: SYN ACK FIN RST PSH URG] [window_size 2B]
[checksum 2B] [urgent_pointer 2B]
[options variable] [data variable]
```

### The Three-Way Handshake

Before data transfer, TCP establishes a connection:

```
Client → Server: SYN, seq=x
Server → Client: SYN-ACK, seq=y, ack=x+1
Client → Server: ACK, seq=x+1, ack=y+1
(connection established)
```

SYN (synchronise) starts a connection and carries the initial sequence number. The sequence number is a 32-bit counter for tracking byte positions.

**Connection termination** (four-way):

```
Client → Server: FIN, seq=m
Server → Client: ACK, ack=m+1
Server → Client: FIN, seq=n
Client → Server: ACK, ack=n+1
```

Both sides must explicitly close — TCP connections are full-duplex and each direction closes independently.

### Reliability — Sequence Numbers and ACKs

Each byte in the stream has a sequence number. The receiver sends acknowledgements (ACK) confirming bytes received.

**Retransmission**: if the sender doesn't receive an ACK within a timeout (RTO — Retransmission Timeout), it resends the unacknowledged data.

**Sliding window**: the sender can have multiple unacknowledged segments in flight (up to the window size). This is essential for performance on high-latency links — waiting for one ACK per segment at 100ms RTT means only 10 round trips per second regardless of bandwidth.

### Flow Control — Receiver Window

The receiver advertises how much buffer space it has (`window_size` in the header). The sender never sends more data than the receiver can buffer.

If the receiver's buffer fills (window=0), the sender stops and periodically probes with a 1-byte segment to check if the window reopened.

### Congestion Control

The network can become congested — routers drop packets when buffers overflow. TCP detects congestion via:

• **Packet loss** (timeout or duplicate ACKs)

• **ECN** (Explicit Congestion Notification): routers mark packets instead of dropping them

TCP's congestion control algorithms:

**Slow Start**: begin with cwnd (congestion window) = 1 MSS (maximum segment size). Double cwnd every RTT. This is "slow" compared to sending immediately at full rate, not actually slow.

**Congestion Avoidance**: when cwnd exceeds ssthresh (slow start threshold), increase linearly by 1 MSS per RTT.

**On packet loss** (Reno): set ssthresh = cwnd/2, reset cwnd = 1 MSS, restart slow start.

**BBR** (Bottleneck Bandwidth and RTT, Google 2016): models the network path's bandwidth and RTT rather than responding to loss. Achieves much better throughput on long-distance links and in data centres. Used by ~50% of TCP traffic on YouTube.

### TCP vs UDP Comparison

| Feature | TCP | UDP | |---|---|---| | Connection | Yes (3-way handshake) | No | | Reliability | Guaranteed delivery | Best effort | | Ordering | In-order delivery | Out of order possible | | Flow control | Yes (window) | No | | Congestion co

### Illustrations

**Figure 155 • FIG 8.3-A**

**TCP Three-Way Handshake — SYN, SYN-ACK, ACK**

*Illustration: FIG 8.3-A | TCP Three-Way Handshake — SYN, SYN-ACK, ACK [SVG diagram — see module-8-3-transport-layer.html]*

TCP three-way handshake establishes the connection and synchronises Initial Sequence Numbers (ISNs) on both sides. Step 1: Client sends SYN with its ISN (100). Step 2: Server responds with SYN-ACK — its own ISN (300) and acknowledgement of the client's ISN+1 (101). Step 3: Client sends ACK acknowledging the server's ISN+1 (301). Now both sides know each other's sequence numbers, enabling reliable ordered delivery. The handshake costs one full round-trip before any data transfers — at 100ms RTT, this adds 100ms latency to every new TCP connection. HTTP/1.1 persistent connections amortise this c

**Figure 156 • FIG 8.3-B**

**TCP Sliding Window — Flow Control in Action**

*Illustration: FIG 8.3-B | TCP Sliding Window — Flow Control in Action [SVG diagram — see module-8-3-transport-layer.html]*

TCP sliding window flow control. Rather than waiting for an ACK after each segment (stop-and-wait), TCP allows the sender to have a window of unacknowledged segments in flight simultaneously. The receiver's advertised window size tells the sender its available buffer space. When the sender receives ACK 3, segment 3 is confirmed received; the window slides right, freeing a slot for segment 8. Maximum throughput is bounded by window_size / RTT — at 100ms RTT and 64KB window the theoretical maximum is about 5 Mbps. TCP congestion control (AIMD — Additive Increase Multiplicative Decrease) dynamica

**Figure 157 • FIG 8.3-C**

**TCP vs UDP — When to Use Which Transport Protocol**

*Illustration: FIG 8.3-C | TCP vs UDP — When to Use Which Transport Protocol [SVG diagram — see module-8-3-transport-layer.html]*

Transport protocol selection guide. TCP's reliability machinery — sequence numbers, acknowledgements, retransmission, flow control, and congestion control — makes it the right choice whenever every byte must arrive correctly: HTTP, file transfer, database connections. UDP's minimal header (8 bytes vs TCP's 20+) and lack of handshake overhead make it right for latency-sensitive applications where occasional packet loss is acceptable or managed by the application. DNS uses UDP because queries fit in one packet and the application simply retries. Modern video streaming uses UDP because a retransm

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 8.3 Transport Layer: TCP and UDP: TCP Handshake Sliding Window Congestion Control [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 8.3 Transport Layer: TCP and UDP **Type:** Calculation + trace + implementation **Estimated time:** 4555 minutes **Tools needed:** pen and paper + Python --- Part A TCP Sequence Numbers A TCP connection transfers a 2,000-byte file. The initial sequence number (ISN) is 100. Each TCP segment carries 500 bytes of data. 1. Fill in the sequence and acknowledgement numbers for all segments and ACKs: | | Direction | Seq | Ack | Bytes | Notes | |---|---|---|---|---|---|---| | 1 | CS | SYN, x=100 | | 0 | Handshake | | 2 | SC | SYN-ACK, y=500 | ? | 0 | | | 3 | CS | ACK | ? | 0 | | | 4

# Module 8 4 Application Layer

■# Module 8.4 — Application Layer: DNS and HTTP

## The Problem

*TCP gives you a reliable byte stream between two ports.*

*But no one navigates to* `172.217.14.238:443`*.*

*People navigate to* `google.com`*.*

>

*And once you have a connection, how do you ask for a specific resource?*

*How does a single IP address serve thousands of different websites?*

>

*These are application-layer problems:*

***DNS** translates names to addresses,*

***HTTP** defines how to request and serve resources.*

---

## The Story

### DNS: The Phone Book of the Internet

In the early ARPANET, there were fewer than 1,000 machines. A single file — `HOSTS.TXT` — maintained at Stanford Research Institute listed every machine and its address. Administrators emailed updates; the file was downloaded regularly. As the ARPANET grew, this system became untenable.

**Paul Mockapetris** designed the Domain Name System (DNS) in 1983 (RFC 882/883). The key innovation: a distributed, hierarchical, delegated database. No single server holds all names. Instead, authority for each zone is delegated to nameservers — Google is responsible for `google.com`, Amazon for `amazon.com`, and so on. The root servers (13 groups, lettered A–M) sit at the top and delegate to top-level domain (TLD) servers (`.com`, `.org`, `.uk`), which delegate to domain nameservers.

DNS became operational in 1985. Today it handles trillions of queries per day — every web request, every email, every API call begins with a DNS lookup.

### HTTP: The Protocol That Built the Web

Tim Berners-Lee designed HTTP at CERN in 1989 (Module 7.5). HTTP/1.1 (RFC 2616, 1997) is still the most widely deployed version. HTTP/2 (2015) improved performance with multiplexing. HTTP/3 (2022) runs over QUIC instead of TCP.

The web as we know it — every website, web app, REST API, and web service — is built on HTTP.

---

## The Concept

### DNS Hierarchy




**Types of DNS servers:**

• **Root nameservers**: 13 groups (a–m.root-servers.net), replicated globally via anycast. Know the TLD servers.

• **TLD nameservers**: operated by registries (Verisign for `.com`). Know authoritative nameservers for each domain.

• **Authoritative nameservers**: operated by domain owners (or their DNS provider). Hold actual DNS records.

• **Recursive resolvers**: operated by ISPs or public DNS (8.8.8.8, 1.1.1.1). Cache results and perform the full lookup on behalf of clients.

### DNS Resolution Process

When you type `www.google.com` in a browser:

**Local cache check**: browser checks its own DNS cache. Found? Done (sub-millisecond).

**OS resolver cache**: OS checks its cache and `/etc/hosts`. Found? Done.

**Recursive resolver** (e.g., `8.8.8.8`): OS sends query to configured DNS server.

**Root query**: resolver asks a root server for `.com` TLD server address.

**TLD query**: resolver asks `.com` TLD server for `google.com` nameserver address.

**Authoritative query**: resolver asks Google's nameserver for `www.google.com`.

**Answer**: authoritative server returns A record: `172.217.14.238`.

**Cache**: resolver caches the result for TTL (e.g., 300 seconds).

**Return**: resolver returns the IP to the client.

A cold lookup (nothing cached) typically takes 50–200ms. Subsequent lookups hit the cache: < 1ms.

### DNS Record Types

| Type | Purpose | Example | |---|---|---| | A | IPv4 address | `www.example.com` → `93.184.216.34` | | AAAA | IPv6 address | `www` → `2606:2800:220:1:248:1893:25c8:1946` | | `CNAME` | Alias (canonical name) | `blog.example.com` → `example.com` | | `MX` | Mail server, with priority | `example.com` → `mail.example.com (priority 10)` | | `NS` | Authoritative nameserver | `example.com` → `ns1.example.com` | | `TXT` | Arbitrary text | SPF, DKIM, domain verification | | SOA | Start of Authority | Zone metadata, serial number | | `PTR` | Reverse lookup (IP → name) | `34.216.184.93.in-addr.arpa` → `www.example.com` |

### HTTP/1.1 Request-Response

A complete HTTP/1.1 exchange (from Module 7.5, extended):

**Request:**

```
GET /search?q=dns HTTP/1.1
Host: www.google.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: SID=abc123; HSID=xyz789
```

**Response:**

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Encoding: gzip
Content-Length: 12453
Cache-Control: private, max-age=0
Set-Cookie: NID=204=...; expires=...; path=/; domain=.google.com; HttpOnly
Transfer-Encoding: chunked
```

### HTTP Status Codes

| Range | Meaning | Key codes | |---|---|---| | 1xx | Informational | 100 Continue, 101 Switching Protocols | | 2xx | Success | 200 OK, 201 Created, 204 No Content | | 3xx | Redirection | 301 Moved Permanently, 302 Found, 304 Not Modified | | 4xx | Client error | 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests | | 5xx | Server error | 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable |

### HTTP Methods

| Method | Purpose | Body? | Safe? | Idempotent? | |---|---|---|---|---| | GET | Retrieve resource | No | Yes | Yes | | POST | Create / submit | Yes | No | No | | PUT | Replace resource | Yes | No | Yes | | PATCH | Partial update | Yes | No | No | | DELETE | Delete resource | No | No | Yes | | HEAD | GET without body | No | Yes | Yes | | OPTIONS | List allowed methods | No | Yes | Yes |

**Safe**: does not modify state. **Idempotent**: calling multiple times has the same effect as calling once.

### HTTP Caching

```
Cache-Control: max-age=3600          # cache for 1 hour
Cache-Control: no-cache              # must revalidate before using cache
Cache-Control: no-store              # never cache (passwords, bank data)
ETag: "abc123"                       # content
```

### Illustrations

**Figure 158 • FIG 8.4-A**

**DNS Resolution — How "example.com" Becomes an IP Address**

*Illustration: FIG 8.4-A | DNS Resolution — How "example.com" Becomes an IP Address [SVG diagram — see module-8-4-application-layer.html]*

DNS resolution chain for "example.com". The stub resolver in your OS contacts the recursive resolver (typically your ISP's or a public resolver like 8.8.8.8). If the answer is not cached, the recursive resolver queries the root nameservers, which direct it to the .com TLD nameservers. The TLD nameservers provide the address of example.com's authoritative nameserver (ns1.example.com). The authoritative nameserver holds the A record — the actual IP address (93.184.216.34) with a TTL indicating how long it may be cached. The recursive resolver returns the IP to your device. The entire chain typic

**Figure 159 • FIG 8.4-B**

### HTTP/1.1 vs HTTP/2 vs HTTP/3 — Connection Model Evolution

*Illustration: FIG 8.4-B | HTTP/1.1 vs HTTP/2 vs HTTP/3 — Connection Model Evolution [SVG diagram — see module-8-4-application-layer.html]*

HTTP version evolution driven by latency and throughput problems. HTTP/1.1 serialises requests on a persistent connection — the browser hacks around this by opening up to 6 connections per host. HTTP/2 introduces multiplexing: multiple request/response streams share one TCP connection via framing, eliminating the need for multiple connections. However, a single lost TCP segment blocks all streams (TCP head-of-line blocking). HTTP/3 over QUIC runs over UDP with independent streams at the transport layer — a lost UDP packet only blocks the stream that owns it. QUIC also incorporates TLS 1.3 into

**Figure 160 • FIG 8.4-C**

### URL Anatomy — Every Component of a Web Address

*Illustration: FIG 8.4-C | URL Anatomy — Every Component of a Web Address [SVG diagram — see module-8-4-application-layer.html]*

URL (Uniform Resource Locator) anatomy. The scheme identifies the protocol. Userinfo (rarely used in HTTP — credentials belong in headers, not URLs). Host is resolved via DNS. Port defaults to 443 for HTTPS and 80 for HTTP if omitted. Path identifies the resource on the server. Query string passes parameters to the server (sent in the HTTP request line). The fragment identifier (#section) is processed entirely by the browser — it is never transmitted to the server, making it useful for client-side navigation in single-page applications. URLs must percent-encode characters outside the allowed A

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 8.4 Application Layer: DNS and HTTP: DNS Resolver Walk HTTP Request Builder Cache Behaviour [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 8.4 Application Layer: DNS and HTTP **Type:** Trace + analysis + implementation **Estimated time:** 4050 minutes **Tools needed:** pen and paper --- Part A DNS Resolution Walk Trace the complete DNS resolution for `api.github.com` from a client with no cached entries. The client's recursive resolver is `8.8.8.8`. 1. List every query and response in order: - Who queries whom? - What is the exact query (qname, qtype)? - What does the response contain? There should be at least 4 distinct queries before the final answer arrives. 2. The final answer is an **A r

# Module 8 5 Tls Ssl

■# Module 8.5 — TLS/SSL from First Principles

## The Problem

*HTTP sends all data in plaintext.*

*Any router between your laptop and the server can read your password,*

*your bank balance, your private messages.*

*>*

*Three security properties are needed:*

*1. **Confidentiality** — only sender and recipient can read the data*

*2. **Integrity** — data cannot be modified in transit without detection*

*3. **Authentication** — you are really talking to google.com, not an impostor*

*>*

*TLS (Transport Layer Security) provides all three.*

*It is the protocol behind the padlock icon in your browser.*

## The Story

### From SSL to TLS

Netscape Communications invented **SSL** (Secure Sockets Layer) in 1994 to secure e-commerce on the early web. Taher Elgamal was the primary designer. SSL 2.0 was the first public release; SSL 3.0 (1996) fixed critical vulnerabilities.

The IETF standardised **TLS 1.0** (1999) based on SSL 3.0. TLS 1.2 (2008) remains widely deployed. **TLS 1.3** (2018) was a major redesign — removing obsolete cryptography (RC4, MD5, SHA-1, RSA key exchange, DH < 1024 bits), reducing handshake latency from 2 RTTs to 1 RTT, and achieving **0-RTT** (zero round-trip) for session resumption.

As of 2024: TLS 1.3 is used by ~70% of HTTPS connections. TLS 1.2 handles most of the rest. TLS 1.0 and 1.1 are deprecated.

## The Concept

### The Cryptographic Building Blocks

**Symmetric encryption**: one key, used for both encryption and decryption. Fast. Problems: key distribution (how do you securely share the key?). Examples: **AES-256-GCM** (the dominant cipher in TLS 1.3).

**Asymmetric encryption (public-key cryptography)**: two mathematically related keys — a public key anyone can know, and a private key only you know. Data encrypted with the public key can only be decrypted with the private key. Slow (1000× slower than AES for bulk data). Examples: **RSA**, **ECDH** (Elliptic Curve Diffie-Hellman).

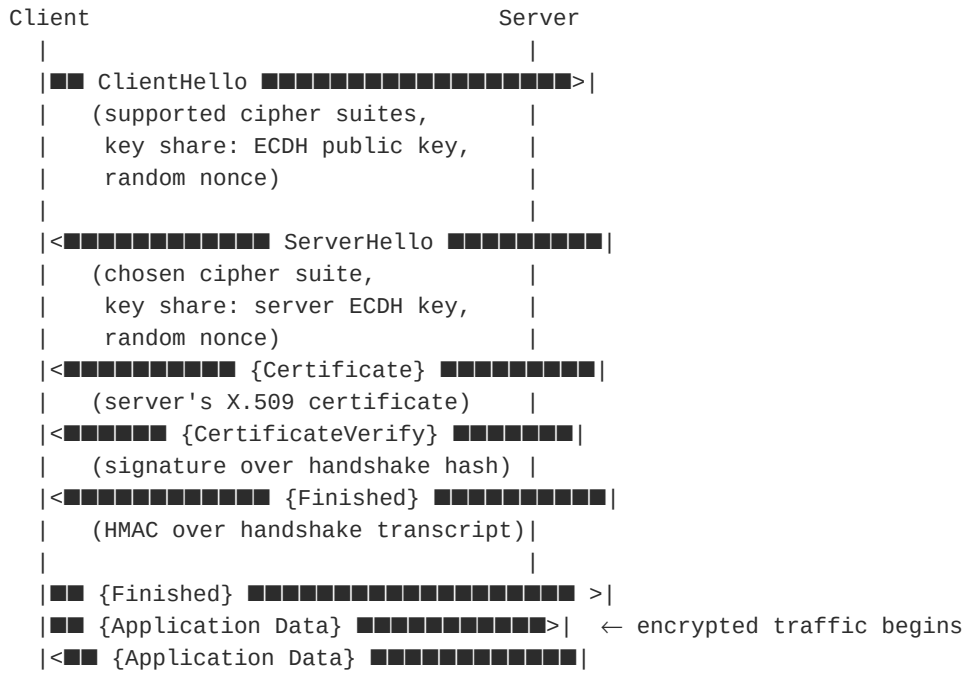
**Hash functions**: one-way transformation. Given input, produces a fixed-size digest. Computationally infeasible to reverse or find two inputs with the same digest. Examples: **SHA-256**, **SHA-384**.

**MACs (Message Authentication Codes)**: like a hash but keyed. Only someone with the key can compute or verify a MAC. Proves both integrity and authenticity. Example: **HMAC-SHA256**.

**Digital signatures**: a private key signs data; anyone with the public key can verify it was signed by the private key holder. Used in certificates to prove identity.

### The TLS 1.3 Handshake

TLS 1.3 completes the handshake in **one round trip** (1-RTT):




`{}` indicates the message is **encrypted** with handshake keys derived during the handshake.

### Key Exchange — ECDH

**Elliptic Curve Diffie-Hellman** allows two parties to agree on a shared secret over an insecure channel without ever transmitting the secret:

Both agree on a public curve and generator point G

Client chooses random private key a, sends public key `A = a×G`

Server chooses random private key b, sends public key `B = b×G`

Client computes shared secret: `S = a×B = a×b×G`

Server computes: `S = b×A = b×a×G`

Both have `S = a×b×G` — the shared secret — without it ever being transmitted

An eavesdropper sees `A` and `B` but cannot compute `S` without solving the **elliptic curve discrete logarithm problem** — computationally infeasible for 256-bit curves.

The shared secret is then used to derive symmetric keys for encrypting the actual data (AES-256-GCM in TLS 1.3).

### Certificates and the PKI

Authentication answers: "Is the server I'm talking to really `google.com`?" Without it, an attacker could intercept the connection and use their own keys (man-in-the-middle attack).

A **X.509 certificate** binds a public key to an identity:

```
Subject: CN=www.google.com, O=Google LLC, C=US
Issuer: CN=GTS CA 1C3, O=Google Trust Services LLC
Validity: Not Before: Aug 19 2026, Not After: Nov 11 2026
Public Key: [ECDSA P-256 public key]
Subject Alternative Names: www.google.com, *.google.com
Signature: [signed by GTS CA 1C3]
```

The certificate is signed by a **Certificate Authority (CA)**. Your browser trusts a pre-installed set of ~150 root CAs (Mozilla, Google, Apple, Microsoft maintain lists). The chain of trust:

```
Root CA (trusted by browser, self-signed)
   ↓ signs
Intermediate CA certificate
   ↓ signs
Server certificate (www.google.com)
```

The browser verifies the signature chain up to a trusted root. If valid, and the domain in the certificate matches the URL, authentication succeeds.

### After the Handshake — Record Protocol

All application data is encrypted using the symmetric key established during the handshake:

```
TLS Record:
[content type (1B)] [version (2B)] [length (2B)] [encrypted payload + auth tag]
```

**AES-256-GCM** provides both encryption and authentication in one operation:

• Encrypts data with AES-256 in Galois/Counter Mode

• Produces a 128-bit authentication tag

• Any modification to the ciphertext invalidates the tag — detected immediately

The **sequence number** is included in the authentication (as "additional

### Illustrations

**Figure 161 • FIG 8.5-A**

### TLS 1.3 Handshake — 1-RTT Connection with Forward Secrecy

*Illustration: FIG 8.5-A | TLS 1.3 Handshake — 1-RTT Connection with Forward Secrecy [SVG diagram — see module-8-5-tls-ssl.html]*

TLS 1.3 handshake — the mechanism behind HTTPS. After the TCP 3-way handshake, the client sends its Diffie-Hellman public key in ClientHello. The server responds with its own DH public key, its X.509 certificate, a signature proving it holds the private key, and a Finished HMAC. Both sides now derive an identical session key from the DH exchange without ever transmitting it — a man in the middle sees only public keys, from which deriving the session key is computationally infeasible. The client verifies the certificate chain and sends its own Finished HMAC. All subsequent HTTP data is encrypte

**Figure 162 • FIG 8.5-B**

### Certificate Chain — Root CA, Intermediate CA, Leaf Certificate

*Illustration: FIG 8.5-B | Certificate Chain — Root CA, Intermediate CA, Leaf Certificate [SVG diagram — see module-8-5-tls-ssl.html]*

X.509 certificate chain of trust. Root CAs are the anchors: ~150 of them are pre-installed in every operating system and browser trust store, and their private keys are kept offline in Hardware Security Modules. Root CAs sign Intermediate CAs, which are kept online and issue leaf certificates to websites. This two-level structure limits blast radius: if an intermediate CA is compromised, only certificates it issued are affected — the root remains secure. The leaf certificate contains the domain's public key, the allowed hostnames (SANs), validity period (90 days for Let's Encrypt), and the int

**Figure 163 • FIG 8.5-C**

### TLS Encryption Roles — Asymmetric for Key Exchange, Symmetric for Data

*Illustration: FIG 8.5-C | TLS Encryption Roles — Asymmetric for Key Exchange, Symmetric for Data [SVG diagram — see module-8-5-tls-ssl.html]*

TLS uses asymmetric and symmetric cryptography for complementary purposes. Asymmetric cryptography (RSA, ECDSA, ECDH) solves the key distribution problem — two parties can establish a shared secret over a public channel without ever transmitting the secret. But asymmetric operations are 1000× slower than symmetric ones. Symmetric cryptography (AES-256-GCM) is fast enough for bulk data — modern CPUs include AES-NI hardware instructions, enabling 10+ Gbps encrypted throughput. TLS therefore uses asymmetric crypto only in the handshake (to authenticate the server and derive the session key), then

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 8.5 TLS/SSL from First Principles: TLS 1.3 Handshake ECDH Key Exchange Certificate Chain [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 8.5 TLS/SSL from First Principles **Type:** Reasoning + calculation + trace **Estimated time:** 4050 minutes **Tools needed:** pen and paper --- Part A TLS 1.3 Handshake 1. Draw the TLS 1.3 handshake message sequence between Client and Server. Label each message and indicate which are encrypted (use `{}` notation). 2. The handshake takes **1 RTT** in TLS 1.3. In TLS 1.2, it took **2 RTTs**. For a user with 100ms RTT to a server, calculate the total connection setup time: - DNS: 1 RTT - TCP handshake: 1 RTT - TLS 1.2: 2 RTTs - TLS 1.3: 1 RTT How many mill

# Raw socket requires root/admin privileges

■# Module 8.6 — Building a Packet Analyzer

## The Problem

*You've studied the network stack from physical to application layer.*

*Theory is one thing. Seeing real bytes on the wire is another.*

>

*A packet analyzer — a program that captures and decodes network packets —*

*is the most powerful tool for understanding how networks actually work.*

*Wireshark, tcpdump, and tshark are the professionals' tools.*

*Building a minimal version teaches you every header and field you've studied,*

*as real hex bytes.*

## The Story

### Wireshark and Packet Capture

**tcpdump** was written by Van Jacobson and others at Lawrence Berkeley National Laboratory in 1988. It is still the primary command-line tool for packet capture on Unix systems, running on every Linux and macOS system.

**Ethereal** (later renamed **Wireshark**, 2006) was written by Gerald Combs starting in 1997. It added a GUI, protocol dissectors for hundreds of protocols, and became the de facto standard tool for network analysis. Used by security researchers, network engineers, protocol developers, and system administrators worldwide.

Both tools use **libpcap** (Linux: pcap, Windows: Npcap) to capture packets from network interfaces at the OS level. The OS kernel copies arriving packets into a ring buffer accessible to capture programs — without disrupting normal packet processing.

## The Concept

### Raw Sockets and libpcap

To capture all packets on a network interface (not just those addressed to this machine), you need a **raw socket** with promiscuous mode enabled:

```
import socket, struct

# Raw socket — requires root/admin privileges
sock = socket.socket(socket.AF_PACKET,   # Linux: packet-level access
                     socket.SOCK_RAW,
                     socket.htons(0x0003))  # ETH_P_ALL: capture all protocols

while True:
    raw_data, addr = sock.recvfrom(65535)
    parse_ethernet(raw_data)
```

### Parsing Ethernet Frames

```
def parse_ethernet(frame):
    # Ethernet header: 6B dst, 6B src, 2B ethertype
    dst_mac = format_mac(frame[0:6])
    src_mac = format_mac(frame[6:12])
    ethertype = struct.unpack('!H', frame[12:14])[0]   # big-endian unsigned short

    print(f"Ethernet: {src_mac} → {dst_mac}  EtherType: 0x{ethertype:04X}")

    if ethertype == 0x0800:    # IPv4
        parse_ipv4(frame[14:])
    elif ethertype == 0x0806:  # ARP
        parse_arp(frame[14:])
    elif ethertype == 0x86DD:  # IPv6
        parse_ipv6(frame[14:])

def format_mac(b):
    return ':'.join(f'{x:02X}' for x in b)
```

### Parsing IPv4

```
def parse_ipv4(packet):
    version_ihl = packet[0]
    ihl = (version_ihl & 0x0F) * 4          # header length in bytes
    ttl = packet[8]
    protocol = packet[9]
    src_ip = '.'.join(str(b) for b in packet[12:16])
    dst_ip = '.'.join(str(b) for b in packet[16:20])

    print(f"  IPv4: {src_ip} → {dst_ip}  TTL={ttl}  Proto={protocol}")
```

```
    payload = packet[ihl:]
    if protocol == 6:    # TCP
        parse_tcp(payload)
    elif protocol == 17: # UDP
        parse_udp(payload)
    elif protocol == 1:  # ICMP
        parse_icmp(payload)

def ip_to_str(b):
    return '.'.join(str(x) for x in b)
```

### Parsing TCP

```
def parse_tcp(segment):
    src_port = struct.unpack('!H', segment[0:2])[0]
    dst_port = struct.unpack('!H', segment[2:4])[0]
    seq      = struct.unpack('!I', segment[4:8])[0]
    ack      = struct.unpack('!I', segment[8:12])[0]
    offset   = (segment[12] >> 4) * 4    # data offset (header length)
    flags    = segment[13]

    flag_str = ''.join([
        'SYN ' if flags & 0x02 else '',
        'ACK ' if flags & 0x10 else '',
        'FIN ' if flags & 0x01 else '',
        'RST ' if flags & 0x04 else '',
        'PSH ' if flags & 0x08 else '',
    ])

    print(f"    TCP: {src_port} → {dst_port}  seq={seq}  ack={ack}  [{flag_str.strip()}]")
    payload = segment[offset:]
    if payload:
        parse_application(src_port, dst_port, payload)

def parse_udp(segment):
    src_port = struct.unpack('!H', segment[0:2])[0]
    dst_port = struct.unpack('!H', segment[2:4])[0]
    length   = struct.unpack('!H', segment[4:6])[0]
    print(f"    UDP: {src_port} → {dst_port}  len={length}")
    if dst_port == 53 or src_port == 53:
        parse_dns(segment[8:])
```

### Parsing HTTP (Application Layer)

```
def parse_application(src_port, dst_port, payload):
    try:
        text = payload.decode('utf-8', errors='replace')
    except: return

    if dst_port == 80 or src_port == 80:
        lines = text.split('\r\n')
        if lines[0].startswith(('GET ','POST ','HTTP/')):
            print(f"      HTTP: {lines[0]}")
```

### DNS Parsing

```
  def parse_dns(data):
      tx_id    = struct.unpack('!H', data[0:2])[0]
      flags    = struct.unpack('!H', data[2:4])[0]
      qr       = (flags >> 15) & 1    # 0=query, 1=response
      q_count  = struct.unpack('!H', data[4:6])[0]

      # Parse the first question (simplified)
      i = 12   # skip header
      name_parts = []
      while i < len(data) and data[i] != 0:
          length = data[i]; i += 1
          name_parts.append(data[i:i+length].decode('ascii','replace'))
          i += length
      name = '.'.join(name_parts)

      qtype = struct.unpack('!H', data[i+1:i+3])[0]
      type_str = {1:'A', 28:'AAAA', 15:'MX', 2:'NS', 5:'CNAME'}.get(qtype, str(qtype))
      print(f"      DNS: {'Response' if qr else 'Query'} id={tx_id:#06x} "
            f"{'reply' if qr else f'{name} {type_str}'}")
```

### Putting It Together

A complete, functional packet analyzer in ~100 lines of Python. Add:

• **Filtering**: only show TCP on port 443, or only DNS, or only packets from a specific IP

• **Statistics**: packet count per protocol, bytes per second, top talkers

• **Flow reconstruction**: reassemble TCP streams to reconstruct HTTP transactions

• **File output**: write to pcap format for later analysis with Wireshark

---

## Hands-On Exercise

**Open the simulation:** [`simulation.html`](./simulation.html)

**Tool 1 — He

### Illustrations

**Figure 164 • FIG 8.6-A**

### Packet Capture Architecture — NIC to Wireshark / tcpdump

*Illustration: FIG 8.6-A | Packet Capture Architecture — NIC to Wireshark / tcpdump [SVG diagram — see module-8-6-packet-analyzer.html]*

Packet capture architecture. Normally a NIC only delivers frames addressed to its MAC address. In promiscuous mode it delivers all frames it receives — enabling capture of traffic to/from other hosts on the same segment. The Berkeley Packet Filter (BPF) runs a small filter program in kernel space, dropping packets that don't match the filter expression before they are copied to userspace — this dramatically reduces overhead for filtered captures. libpcap provides a portable C API for these capabilities, used by Wireshark, tcpdump, and network monitoring tools. Modern eBPF (extended BPF) is a s

**Figure 165 • FIG 8.6-B**

### Protocol Demultiplexing — Parsing Eth→IP→TCP→HTTP from Raw Bytes

*Illustration: FIG 8.6-B | Protocol Demultiplexing — Parsing Eth→IP→TCP→HTTP from Raw Bytes [SVG diagram — see module-8-6-packet-analyzer.html]*

Protocol demultiplexing — how a packet analyzer identifies and parses each protocol layer from raw bytes. The EtherType field in the Ethernet header determines the next parser (0x0800 = IPv4, 0x0806 = ARP, 0x86DD = IPv6). The Protocol field in the IPv4 header selects the Layer 4 parser (6 = TCP, 17 = UDP, 1 = ICMP). The TCP/UDP destination port number suggests the application layer protocol (80 = HTTP, 443 = HTTPS, 53 = DNS). A packet analyzer implements this as a chain of parser functions, each reading its header and dispatching to the next layer's parser. Wireshark implements hundreds of pro

**Figure 166 • FIG 8.6-C**

### pcap File Format — Global Header + Per-Packet Records

*Illustration: FIG 8.6-C | pcap File Format — Global Header + Per-Packet Records [SVG diagram — see module-8-6-packet-analyzer.html]*

The pcap file format (used by Wireshark, tcpdump, and virtually all network analysis tools). The 24-byte global header identifies the file (magic number for endianness detection), the pcap version, the maximum captured bytes per packet (snaplen), and the link-layer type (1 = Ethernet, 105 = 802.11 Wi-Fi). Each captured packet is stored as a 16-byte per-packet header — Unix timestamp in seconds and microseconds, number of bytes captured (may be less than original if truncated by snaplen), and original packet length — followed by the raw captured bytes. Reading a pcap file is straightforward: pa

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 8.6 Building a Packet Analyzer: Hex Packet Decoder Capture Replay BPF Filters [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 8.6 Building a Packet Analyzer **Type:** Decode + implementation + reasoning **Estimated time:** 4555 minutes **Tools needed:** pen and paper + Python --- Part A Manual Packet Decode Decode this raw packet (hex bytes). It is a complete Ethernet/IPv4/TCP frame. ``` FF FF FF FF FF FF AA BB CC DD EE FF 08 00 45 00 00 3C 1A 2B 40 00 40 06 00 00 C0 A8 01 05 08 08 08 08 C3 50 01 BB 00 00 00 01 00 00 00 00 50 02 20 00 00 00 00 00 ``` 1. Parse the Ethernet header (bytes 013): - Destination MAC - Source MAC - EtherType 2. Parse the IPv4 header (bytes 1433): -

LEVEL 09

# Databases

Relational Models, B-Trees, Transactions, and SQL Engines

# Module 9 1 Relational Model

■# Module 9.1 — The Relational Model and SQL

## The Problem

*A company stores customer data in one file, orders in another, products in a third.*

*To find all orders placed by customers in New York for products that are currently out of stock,*

*you write a custom program that opens all three files, reads every line, and applies filters.*

>

*Every new question requires a new program.*

*When the file format changes, every program breaks.*

>

*In 1970, Edgar Codd published a paper that solved all of these problems at once.*

*The solution: a **relational model** — data organised as tables,*

*accessed through a declarative language that describes what you want,*

*not how to get it.*

## The Story

### E.F. Codd and the Relational Revolution

Edgar Codd was a computer scientist at IBM's San Jose Research Lab. In 1970, he published "A Relational Model of Data for Large Shared Data Banks" — one of the most influential papers in computer science history.

Codd's insight: store data in **relations** (tables), and access it using **relational algebra** — a mathematical framework with operations like selection, projection, and join. The query language would be *declarative*: describe what data you want, let the database figure out how to retrieve it.

IBM built System R (1974–1979) to prove the concept, developing the **SEQUEL** query language (later renamed **SQL**). Don Chamberlin and Ray Boyce designed SQL to be readable by non-programmers — like English with structure.

Oracle Corporation (founded 1977 by Larry Ellison) shipped the first commercial SQL database in 1979. PostgreSQL (1986, open source), MySQL (1995, open source), SQLite (2000, embedded) followed. Today, relational databases power virtually every financial system, e-commerce platform, and enterprise application in the world.

## The Concept

### Tables, Rows, Columns

A **relation** is a table with:

• **Schema**: column names and types (defined once)

• **Rows** (tuples): individual records

• **Primary key**: one or more columns that uniquely identify each row

```
CREATE TABLE users (
    id          INTEGER PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(255) UNIQUE NOT NULL,
    city        VARCHAR(100),
    created_at  TIMESTAMP DEFAULT NOW()
);

CREATE TABLE orders (
    id          INTEGER PRIMARY KEY,
    user_id     INTEGER REFERENCES users(id),
    amount      DECIMAL(10,2) NOT NULL,
    status      VARCHAR(20) DEFAULT 'pending',
    created_at  TIMESTAMP DEFAULT NOW()
);
```

A **foreign key** (`user_id REFERENCES users(id)`) enforces referential integrity: you cannot create an order for a non-existent user, and you cannot delete a user who has orders.

## SQL — The Four Core Operations

**SELECT — Query data:**

```
-- Basic selection and projection
SELECT name, email FROM users WHERE city = 'New York';

-- Aggregate functions
SELECT city, COUNT(*) as user_count, AVG(amount) as avg_order
FROM users u
JOIN orders o ON u.id = o.user_id
WHERE o.status = 'completed'
GROUP BY city
HAVING COUNT(*) > 10
ORDER BY user_count DESC
LIMIT 5;
```

**INSERT — Add data:**

```
INSERT INTO users (name, email, city) VALUES ('Alice', 'alice@example.com', 'NYC');
INSERT INTO orders (user_id, amount) SELECT id, 99.99 FROM users WHERE city = 'NYC';
```

**UPDATE — Modify data:**

```
UPDATE orders SET status = 'shipped' WHERE id = 1234;
UPDATE users SET city = 'New York' WHERE city = 'NYC';  -- normalise data
```

**DELETE — Remove data:**

```
DELETE FROM orders WHERE created_at < NOW() - INTERVAL '1 year' AND status = 'cancelled';
```

## JOINs — Combining Tables

The JOIN operation is the heart of relational databases:

```
-- INNER JOIN: only rows where condition matches in both tables
SELECT u.name, o.amount, o.status
FROM users u
INNER JOIN orders o ON u.id = o.user_id;

-- LEFT JOIN: all users, even those with no orders
SELECT u.name, COUNT(o.id) as order_count
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
GROUP BY u.id, u.name;

-- Self-join: find users who share a city
SELECT a.name, b.name, a.city
FROM users a
JOIN users b ON a.city = b.city AND a.id < b.id;
```

**Types of joins:**

• **INNER JOIN**: intersection — rows matching in both tables

• **LEFT JOIN**: all from left table + matching from right (NULLs where no match)

• **RIGHT JOIN**: all from right + matching from left

• **FULL OUTER JOIN**: all rows from both tables

• **CROSS JOIN**: Cartesian product — every combination (rarely used)

### Normalisation — Eliminating Redundancy

**First Normal Form (1NF)**: every column holds atomic values (no lists, no nested tables). Each row is unique.

**Second Normal Form (2NF)**: in addition to 1NF, every non-key attribute depends on the whole primary key (no partial dependencies). Relevant when the key is composite.

**Third Normal Form (3NF)**: in addition to 2NF, no transitive dependencies — non-key attributes do not depend on other non-key attributes.

**Example — violation and fix:**

```
-- BAD: city depends on zip, not on customer_id + order_id
orders(customer_id, order_id, customer_name, customer_zip, customer_city, product_name, quantity)

-- GOOD: separate tables, each fact stored once
customers(customer_id, name, zip, city)       -- city depends only on zip → 3NF issue
zip_codes(zip, city, state)                   -- extract the dependency
customers(customer_id, name, zip)             -- zip is FK to zip_codes
orders(order_id, customer_id, product_id, quantity)
products(product_id, name, price)
```

Denormalisation is sometimes done intentionally for read performance: store redundant data to avoid joins.

### Indexes — Making Queries Fast

Without an index, every query scans every row (O(n)). With an index (B+tree, Module 9.2):

```
-- Slow: full table scan
SELECT * FROM orders WHERE user_id = 12345;

-- Add index: O(log n) lookup
CREATE INDEX idx_orders_user_id ON orders(user_id);

-- Composite index: useful for queries filtering on both columns
CR
```

## Illustrations

**Figure 167 • FIG 9.1-A**

### Relational Model — Tables, Keys, and Foreign Key Relationships

*Illustration: FIG 9.1-A | Relational Model — Tables, Keys, and Foreign Key Relationships [SVG diagram — see module-9-1-relational-model.html]*

The relational model organises data into tables (relations) where each row (tuple) represents one entity and each column (attribute) holds a typed value. The primary key uniquely identifies each row and cannot be NULL. A foreign key is a column whose value must exist as a primary key in another table — this enforces referential integrity: you cannot create an order for a user that doesn't exist, and you cannot delete a user who has orders (without cascading). The relational model was proposed by E.F. Codd in 1970 and remains the dominant data model for transactional systems 50+ years later bec

**Figure 168 • FIG 9.1-B**

### SQL JOIN Types — INNER, LEFT, RIGHT, FULL OUTER

*Illustration: FIG 9.1-B | SQL JOIN Types — INNER, LEFT, RIGHT, FULL OUTER [SVG diagram — see module-9-1-relational-model.html]*

SQL JOIN types visualised as Venn diagrams. INNER JOIN returns only rows where the join condition matches in both tables — user Dave (no orders) is excluded. LEFT JOIN returns all rows from the left table (users) and matching rows from the right (orders); unmatched rows get NULL in the order columns — Dave appears with NULL order columns. RIGHT JOIN mirrors LEFT JOIN (rarely used in practice — rewrite as LEFT JOIN with tables swapped). FULL OUTER JOIN returns all rows from both tables with NULLs filling gaps. The LEFT JOIN is the most practically important: it answers "show me all X and any ma

**Figure 169 • FIG 9.1-C**

**SQL Query Execution Plan — SELECT with WHERE, JOIN, ORDER BY**

*Illustration: FIG 9.1-C | SQL Query Execution Plan — SELECT with WHERE, JOIN, ORDER BY [SVG diagram — see module-9-1-relational-model.html]*

SQL query execution pipeline. The parser converts SQL text into an Abstract Syntax Tree. The planner generates possible execution plans (which indexes to use, which join algorithm). The optimiser uses table statistics (row counts, value distributions) to estimate the cost of each plan and select the cheapest. Execution: scan or index-lookup to find matching rows, apply the WHERE filter, perform the join (hash join, nested-loop, or merge join), aggregate with GROUP BY, sort with ORDER BY. EXPLAIN ANALYZE reveals the actual execution plan and real row counts — the essential tool for identifying

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 9.1 The Relational Model and SQL: SQL Query Executor JOIN Visualiser Index Impact [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 9.1 The Relational Model and SQL **Type:** SQL writing + analysis + reasoning **Estimated time:** 5060 minutes **Tools needed:** pen and paper (or a SQL sandbox) --- Setup Schema ```sql CREATE TABLE customers ( id INTEGER PRIMARY KEY, name VARCHAR(100) NOT NULL, city VARCHAR(100), email VARCHAR(255) UNIQUE ); CREATE TABLE products ( id INTEGER PRIMARY KEY, name VARCHAR(100) NOT NULL, price DECIMAL(10,2), category VARCHAR(50), stock INTEGER DEFAULT 0 ); CREATE TABLE orders ( id

# Module 9 2 Btrees

■# Module 9.2 — B-Trees and Why Databases Use Them

## The Problem

*A database table has 100 million rows.*

*You need to find the row where* `user_id = 7,482,991` *in under a millisecond.*

>

*A BST (Module 6.4) gives O(log n) — about 27 comparisons.*

*That sounds fast. But each comparison may require a disk read.*

*A disk read takes ~10 ms.*

*27 disk reads × 10 ms = 270 ms per query.*

*For 100 concurrent users: 27 seconds of disk I/O per second.*

*>*

*The problem is not the number of comparisons — it's the number of **disk accesses**.*

*The B-tree was designed to minimise disk accesses.*

---

## The Story

### Designed for Disk

Rudolf Bayer and Ed McCreight invented the B-tree at Boeing Research Labs in 1972. Their paper title: "Organization and Maintenance of Large Ordered Indexes." The context was clear: they needed a data structure for storing and searching indexes on disk, not in RAM.

The key insight: disk I/O is the bottleneck, not CPU operations. A disk read fetches an entire **block** (typically 4–16 KB) at once, regardless of how much data you actually need. The B-tree exploits this by storing many keys in each node — the node fills exactly one disk block. One disk read retrieves a node with hundreds of keys; binary search within that node is pure CPU work (cheap).

IBM's IMS database (1968, pre-B-tree) used VSAM (Virtual Storage Access Method) with similar ideas. Oracle, IBM DB2, PostgreSQL, MySQL/InnoDB — every major relational database uses B-trees (or the closely related B+tree variant) for their primary storage.

---

## The Concept

### B-Tree Properties

A B-tree of order `t` (minimum degree):

• Every node except root has at least `t-1` keys

• Every node has at most `2t-1` keys

• A node with `k` keys has `k+1` children

• All leaves are at the same depth

• Keys within a node are sorted

For `t = 100` (typical for a disk-based B-tree with 4 KB blocks):

• Each node holds 99–199 keys

• Each internal node has 100–200 child pointers

• Height of a tree with 1 billion keys: log■■■(10■) ≈ **4.5 levels** → **5 disk reads** maximum

Compare: a BST with 1 billion nodes has height ~30, requiring **30 disk reads**.

### B-Tree vs B+Tree

Almost all databases use the **B+tree** variant:

**B-tree**: keys and data records stored in all nodes (internal and leaf).

**B+tree**:

• Internal nodes store **only keys and child pointers** (routing keys)

• All actual data (records or record pointers) stored in **leaf nodes**

• Leaf nodes are **linked in a doubly-linked list**

Benefits of B+tree:

Internal nodes can hold more keys (no data overhead) → shorter tree → fewer disk reads

Leaf linked list enables efficient **range scans**: `WHERE age BETWEEN 25 AND 35` traverses leaf nodes sequentially

All queries touch leaf nodes → more predictable access patterns

### Search

```
def btree_search(node, key):
    # Binary search for key in node.keys
    i = bisect_left(node.keys, key)

    if i < len(node.keys) and node.keys[i] == key:
        return node.vals[i]          # found in this node

    if node.is_leaf:
        return None                  # not found

    # Recurse into appropriate child
    return btree_search(node.children[i], key)
```

One disk read per level traversed. For a 4-level B+tree with `t=100`: maximum **4 disk reads** to find any record among 200 million rows.

### Insertion

Search to find the correct leaf node

Insert the key in sorted position in the leaf

If the leaf is not full (< 2t-1 keys): done

If the leaf is full: **split** it:

- Create two nodes, each with ~half the keys - Promote the middle key to the parent - If parent is also full, split the parent recursively - If root splits, create a new root → tree height increases by 1

The only way a B-tree grows taller is when the root splits. All leaves remain at the same depth — the tree is always balanced.

### Deletion

More complex:

Find the key to delete

If in a leaf: remove it

If in an internal node: replace with in-order successor (from leaf), delete from leaf

If deletion causes underflow (< t-1 keys):

- Try to **borrow** from a sibling (redistribute keys via parent) - If sibling is at minimum too: **merge** with sibling + pull down parent key - Merging may propagate up to the root; if root becomes empty, tree shrinks

### Database Index Implementation

In PostgreSQL, MySQL's InnoDB, and SQLite:

**Clustered index** (InnoDB's primary key): the actual table data is stored in the B+tree leaf nodes, ordered by the primary key. One disk read at leaf level = the actual row data.

**Secondary index**: leaf nodes store the primary key value (not the row data). To retrieve the full row: look up the secondary index B+tree to get the primary key, then look up the clustered B+tree. **Two B+tree lookups** per secondary index query (unless using a covering index).

```
-- This uses the name index (secondary index B+tree)
-- then the primary key B+tree to get the full row
SELECT * FROM users WHERE name = 'Alice';

-- This uses only the name index (covering index)
-- All needed columns are in the leaf nodes
SELECT id, name FROM users WHERE name = 'Alice';
```

### Why Not Hash Tables for Databases?

Hash tables give O(1) lookup — faster than B+trees O(log n). So why do databases use B+trees?

**Range queries**: `WHERE price BETWEEN 10 AND 50` — hash tables are useless for ranges; B+trees handle them in O(log n + k)

**Ordering**: `ORDER BY name` — B+tree traversal produces sorted order; hash tables don't

**Prefix queries**: `WHERE name LIKE 'Ali%'` — B+trees handle prefix scans; hash tables don't

**Predictable worst case**: hash tables have O(n) worst case; B+trees are always O(log n)

Databases do use hash indexes for equality-only lookups where ranges aren't needed (PostgreSQL has hash indexes). But the B+tree is the universal default because it supports everything.

---

## Hands-On Exercise

**Open the simulation:** [`simulation.html`](./simulation.htm

### Illustrations

**Figure 170 • FIG 9.2-A**

### B-Tree Structure — Why Databases Use Fan-Out Trees, Not Binary Search Trees

*Illustration: FIG 9.2-A | B-Tree Structure — Why Databases Use Fan-Out Trees, Not Binary Search [SVG diagram — see module-9-2-btrees.html]*

B-Tree structure optimised for disk-based storage. Unlike a binary search tree (fan-out = 2), a B-Tree node is sized to fill exactly one disk page (typically 8–16 KB). A page can hold hundreds of keys, giving a fan-out of 100–500. With fan-out 200 and height 3, a B-Tree indexes $200^3$ = 8 million rows with at most 3 disk reads. A binary search tree with 8 million entries has height log■(8M) ≈ 23 — 23 disk reads at ~5ms each = 115ms per lookup. The B-Tree version: 15ms. Leaf nodes in B+-Trees (the variant used by PostgreSQL, MySQL, and most databases) store the actual row pointers and are doubly-

**Figure 171 • FIG 9.2-B**

### B-Tree Search — Finding Key 55 in 3 Disk Reads

*Illustration: FIG 9.2-B | B-Tree Search — Finding Key 55 in 3 Disk Reads [SVG diagram — see module-9-2-btrees.html]*

B-Tree search for key 55 traced through three disk reads. At the root, 30 < 55 < 70 directs the search to the middle child pointer. That internal page (40 | 60) confirms 40 < 55 < 60, following the middle pointer again. The leaf page contains [50, 55, 58] — key 55 found with its row pointer (heap file offset). A fourth read fetches the actual row data from the heap file. For 1,000,000 rows with fan-out 200, the tree has height 3 — at most 3 page reads regardless of which key is searched. Without the index, a sequential scan reads all ~125,000 pages (1M × 8 bytes ÷ 8KB per page). Th

**Figure 172 • FIG 9.2-C**

### Clustered vs Non-Clustered Index — Physical Row Ordering

*Illustration: FIG 9.2-C | Clustered vs Non-Clustered Index — Physical Row Ordering [SVG diagram — see module-9-2-btrees.html]*

Clustered vs non-clustered indexes. A clustered index physically orders the table data in B-Tree key order — the leaf nodes contain the actual row data, eliminating a separate heap read. MySQL InnoDB automatically clusters all tables on the primary key. Because data has one physical ordering, only one clustered index is possible per table. Non-clustered (secondary) indexes maintain a separate B-Tree whose leaves contain key values plus row pointers (heap addresses). Looking up by email requires: index B-Tree traversal (3 reads) + one heap read per matched row. If matched rows are scattered acr

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 9.2 B-Trees and Why Databases Use Them: B-Tree Visualiser Search Trace Range Scan [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 9.2 B-Trees and Why Databases Use Them **Type:** Build + trace + calculation **Estimated time:** 4555 minutes **Tools needed:** pen and paper --- Part A Building a B-Tree Insert the keys `[10, 20, 5, 6, 12, 30, 7, 17]` into a B-tree of **order t=2** (minimum degree 2): - Each node has 13 keys (min=t-1=1, max=2t-1=3) - Each internal node has 24 children - Splits when a node reaches 4 keys 1. Show the tree state after each insertion, including any splits. For each split, clearly show: - The full node before the split - The median key promoted to the parent - The two

# Module 9 3 Transactions

■# Module 9.3 — Transactions, ACID, and Isolation

## The Problem

*Two people simultaneously book the last seat on a flight.*

*Both checks show the seat is available.*

*Both bookings succeed.*

*Two passengers arrive, one seat exists.*

>

*A bank transfers £100 from account A to account B.*

*The system crashes after deducting from A but before adding to B.*

*£100 vanishes.*

>

*These are **concurrency** and **durability** problems in databases.*

*Transactions and ACID properties exist specifically to prevent them.*

## The Story

### The Problem of Bank Transfers

The canonical example of why transactions exist: a funds transfer. Conceptually: `UPDATE accounts SET balance = balance - 100 WHERE id = 1` followed by `UPDATE accounts SET balance = balance + 100 WHERE id = 2`. These are two separate operations. Without transactions, a crash between them leaves the database in an inconsistent state — money lost.

Jim Gray formalised transaction processing in the 1970s while at IBM working on System R. His 1981 paper "The Transaction Concept: Virtues and Limitations" defined the ACID properties. Gray received the Turing Award in 1998 for his work on transaction processing and database systems.

The ideas from Gray's work are now universal: every relational database, most NoSQL databases, and many distributed systems implement some form of transactional guarantees.

## The Concept

### ACID Properties

**Atomicity**: a transaction either completes entirely or has no effect. No partial execution. "All or nothing."

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;   -- both updates persist
-- If crash occurs before COMMIT: neither update persists (rollback)
```

**Consistency**: a transaction takes the database from one valid state to another. All integrity constraints (foreign keys, check constraints, unique constraints) hold after every committed transaction.

**Isolation**: concurrent transactions behave as if they executed serially. One transaction's intermediate state is invisible to others.

**Durability**: once committed, a transaction's effects survive crashes. Achieved via **write-ahead logging (WAL)**: changes are written to a durable log before being applied to data pages.

### The Write-Ahead Log (WAL)

Before any data page is modified, the change is recorded in the WAL (also called redo log in MySQL, journal in SQLite):

```
LSN 1001: BEGIN TRANSACTION txn#42
LSN 1002: UPDATE accounts SET balance=900 WHERE id=1  [old=1000, new=900]
LSN 1003: UPDATE accounts SET balance=200 WHERE id=2  [old=100,  new=200]
LSN 1004: COMMIT txn#42
```

On crash recovery:

• WAL entries with COMMIT are **replayed** (redo): any changes not yet on disk are re-applied

• WAL entries without COMMIT are **rolled back** (undo): any partial changes are reversed

The WAL is append-only — the fastest possible write pattern on both HDDs and SSDs.

### Isolation Levels and Concurrency Anomalies

Perfect isolation is expensive — it requires full serialisation. Most databases offer weaker isolation levels for better performance:

**Read Uncommitted**: a transaction can read uncommitted changes from other transactions. → Allows **dirty reads**: read data that another transaction will roll back.

**Read Committed** (PostgreSQL default): only committed data is visible. → Prevents dirty reads. Allows **non-repeatable reads**: re-reading the same row yields different results if another transaction committed between reads.

**Repeatable Read** (MySQL InnoDB default): all reads within a transaction see a consistent snapshot from transaction start. → Prevents dirty reads and non-repeatable reads. Allows **phantom reads**: a range query executed twice may return different rows if another transaction inserted rows matching the range.

**Serializable**: full isolation — transactions execute as if sequential. → Prevents all anomalies. Most expensive. Uses locks or MVCC + predicate locking.

### MVCC — Multi-Version Concurrency Control

Most modern databases (PostgreSQL, Oracle, MySQL InnoDB) use **MVCC** rather than locks for read isolation:

• Each row has a **transaction ID range** indicating which transactions can see it

• When a transaction starts, it gets a snapshot timestamp

• Reads see the latest version of each row **committed before the transaction's snapshot**

• Writes create a **new version** of the row rather than modifying in-place

• Old versions are cleaned up by a background process (VACUUM in PostgreSQL)

Benefits:

• Readers never block writers, writers never block readers

• Long-running queries don't stall updates

Costs:

• Extra storage for multiple versions

• Vacuum/cleanup overhead

### Deadlocks in Databases

```
-- Session 1                  -- Session 2
BEGIN;                         BEGIN;
UPDATE accounts                UPDATE accounts
  SET bal=bal-100              SET bal=bal-100
  WHERE id=1;                  WHERE id=2;   -- acquires lock on row 2
                               UPDATE accounts
                                 SET bal=bal+100
                                 WHERE id=1; -- waits for lock on row 1
UPDATE accounts
  SET bal=bal+100
  WHERE id=2;  -- waits for    -- DEADLOCK
  lock on row 2
```

The database detects deadlocks (cycle in the wait-for graph) and aborts one transaction with an error. The application should retry the aborted transaction.

**Lock ordering** prevents deadlocks: always acquire locks in the same order. Many applications implement this by sorting the rows to update by primary key before executing.

### Two-Phase Commit (2PC)

For distributed transactions spanning multiple databases:

**Phase 1 (Prepare)**: coordinator asks all participants "are you ready to commit?" Each participant flushes its changes to WAL and responds "yes/no."

**Phase 2 (Commit/Abort)**: if all said yes, coordinator sends COMMIT to all. If any said no, sends ROLLBACK to all.

2PC provides dis

## Illustrations

**Figure 173 • FIG 9.3-A**

**ACID Properties — Four Guarantees Every Transaction Must Satisfy**

*Illustration: FIG 9.3-A | ACID Properties — Four Guarantees Every Transaction Must Satisfy [SVG diagram — see module-9-3-transactions.html]*

ACID properties illustrated with a bank transfer (debit account A, credit account B). Atomicity ensures both operations succeed or both are rolled back — no money disappears or is created. Consistency enforces business rules: balance cannot go negative. Isolation ensures Transaction 2 cannot read the intermediate state where A is debited but B isn't yet credited — it sees either the before or after state, never the in-between. Durability means that once the database confirms COMMIT, the transfer is permanent even if the server crashes one millisecond later. These four properties are what disti

**Figure 174 • FIG 9.3-B**

**SQL Isolation Levels — Anomalies Permitted at Each Level**

*Illustration: FIG 9.3-B | SQL Isolation Levels — Anomalies Permitted at Each Level [SVG diagram — see module-9-3-transactions.html]*

SQL isolation levels define which read anomalies are permitted in exchange for concurrency. Dirty reads: reading uncommitted data that may be rolled back. Non-repeatable reads: the same row returns a different value on two reads within one transaction (because another transaction committed between reads). Phantom reads: a range query returns different rows on re-execution (because another transaction inserted/deleted rows matching the range). Higher isolation requires more locking or MVCC snapshot management, reducing concurrency. PostgreSQL implements all levels via MVCC (each transaction see

**Figure 175 • FIG 9.3-C**

**Write-Ahead Log (WAL) — Crash Recovery with Before/After Images**

*Illustration: FIG 9.3-C | Write-Ahead Log (WAL) — Crash Recovery with Before/After Images [SVG diagram — see module-9-3-transactions.html]*

Write-Ahead Logging (WAL) implements both Durability and Atomicity. The rule: before modifying any data page, write the change to the WAL log file and fsync it to disk. Only then modify the in-memory buffer pool. When COMMIT is called, the WAL record for that transaction is fsynced — the application receives confirmation. Actual data pages are written to disk later in bulk (checkpointing). On crash: the ARIES recovery algorithm reads the WAL, replays all changes from the last checkpoint (redo phase), then rolls back any transactions that hadn't committed (undo phase). Committed transactions ar

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 9.3 Transactions, ACID, and Isolation: ACID Bank Transfer WAL Recovery Isolation Levels [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 9.3 Transactions, ACID, and Isolation **Type:** SQL + trace + reasoning **Estimated time:** 4555 minutes **Tools needed:** pen and paper --- Part A ACID Properties Classify each scenario and explain which ACID property it violates: 1. A bank transfer: `UPDATE accounts SET balance = balance - 100 WHERE id = 1`. The system crashes before `UPDATE accounts SET balance = balance + 100 WHERE id = 2` runs. Which property fails without transactions? 2. Two transactions read the same account balance simultaneously, both compute `new_balance = old_balance + interest`, and both writ

# [('KEYWORD','SELECT'), ('IDENT','name'), ('OP',','), ('IDENT','age'),

■# Module 9.4 — Building a Toy SQL Engine

## The Problem

*SQL is a declarative language: you describe what you want.*

*Someone has to figure out how to get it.*

>

*That someone is the SQL engine — the software that takes your query,*

*parses it, plans an execution strategy, and retrieves the data.*

>

*Building a minimal SQL engine from scratch is one of the most educational*

*programming projects in computer science.*

*It requires understanding: parsing, query planning, B-tree indexes,*

*join algorithms, aggregation, and storage formats.*

---

## The Story

### SQLite — The World's Most Deployed Database

D. Richard Hipp wrote SQLite in 2000 for use on US Navy destroyers — embedded systems that needed a database without a server. SQLite is serverless, zero-configuration, and self-contained in a single ~750 KB C file.

Today, SQLite is embedded in: every Android device, every iOS device, every Mac and Windows machine, every Firefox browser, every Chrome browser. Over one trillion SQLite databases are in active use. It is almost certainly the most widely deployed software library in history.

SQLite's source code is public domain and is one of the best-documented database implementations ever written. Reading it is a masterclass in systems programming.

Our toy engine will be a fraction of SQLite's complexity but will hit the same fundamental concepts: parsing SQL text, executing against an in-memory table store, and producing results.

---

## The Concept

### The SQL Engine Pipeline

```
SQL string input
      ↓ Tokeniser (Lexer)
Token stream: SELECT, *, FROM, users, WHERE, id, =, 5
      ↓ Parser
Abstract Syntax Tree (AST)
      ↓ Query Planner / Optimiser
Execution Plan: SeqScan → Filter → Project
      ↓ Executor
Result rows
```

### Step 1: Tokeniser

```
import re

def tokenise(sql):
    token_patterns = [
        ('KEYWORD',  r'\b(SELECT|FROM|WHERE|INSERT|INTO|VALUES|UPDATE|SET|DELETE|AND|OR|NOT|CREATE|TAB
        ('NUMBER',   r'\d+(\.\d+)?'),
        ('STRING',   r"'[^']*'"),
        ('IDENT',    r'[a-zA-Z_][a-zA-Z0-9_]*'),
```

```
        ('OP',       r'[=<>!]+|[,;()*]'),
        ('WHITESPACE', r'\s+'),
    ]
    combined = '|'.join(f'(?P<{name}>{pattern})' for name, pattern in token_patterns)
    tokens = []
    for m in re.finditer(combined, sql, re.IGNORECASE):
        if m.lastgroup != 'WHITESPACE':
            tokens.append((m.lastgroup, m.group()))
    return tokens

tokenise("SELECT name, age FROM users WHERE id = 5")
# [('KEYWORD','SELECT'), ('IDENT','name'), ('OP',','), ('IDENT','age'),
#  ('KEYWORD','FROM'), ('IDENT','users'), ('KEYWORD','WHERE'),
#  ('IDENT','id'), ('OP','='), ('NUMBER','5')]
```

### Step 2: Parser — Building an AST

```
class Parser:
    def __init__(self, tokens):
        self.tokens = tokens
        self.pos    = 0

    def peek(self): return self.tokens[self.pos] if self.pos < len(self.tokens) else None
    def consume(self): t = self.tokens[self.pos]; self.pos += 1; return t
    def expect(self, kind, val=None):
        t = self.consume()
        if t[0] != kind or (val and t[1].upper() != val.upper()):
            raise SyntaxError(f"Expected {kind} {val}, got {t}")
        return t

    def parse_select(self):
        self.expect('KEYWORD', 'SELECT')
        columns = self.parse_column_list()
        self.expect('KEYWORD', 'FROM')
        table = self.consume()[1]
        where  = None
        if self.peek() and self.peek()[1].upper() == 'WHERE':
            self.consume()
            where = self.parse_condition()
        return {'type': 'SELECT', 'columns': columns, 'table': table, 'where': where}

    def parse_column_list(self):
        cols = []
        t = self.peek()
        if t and t[1] == '*': self.consume(); return ['*']
        cols.append(self.consume()[1])
        while self.peek() and self.peek()[1] == ',':
            self.consume(); cols.append(self.consume()[1])
        return cols

    def parse_condition(self):
        left  = self.consume()[1]
        op    = self.consume()[1]
        right = self.consume()[1].strip("'")
```

```
        return {'left': left, 'op': op, 'right': right}
```

### Step 3: In-Memory Storage Engine

```
class Table:
    def __init__(self, name, schema):
        # schema: [(col_name, col_type), ...]
        self.name   = name
        self.schema = schema
        self.rows   = []       # list of dicts
        self.next_id = 1

    def insert(self, row_dict):
        if 'id' not in row_dict:
            row_dict['id'] = self.next_id; self.next_id += 1
        self.rows.append(row_dict)

    def scan(self):
        return iter(self.rows)

class Database:
    def __init__(self):
        self.tables = {}

    def create_table(self, name, schema):
        self.tables[name] = Table(name, schema)

    def get_table(self, name):
        if name not in self.tables:
            raise NameError(f"Table '{name}' not found")
        return self.tables[name]
```

### Step 4: Executor

```
def evaluate_condition(row, cond):
    if cond is None: return True
    left_val  = row.get(cond['left'])
    right_val = cond['right']
    # Try numeric comparison
    try: left_val = float(left_val); right_val = float(right_val)
    except (TypeError, ValueError): pass
    ops = {'=': lambda a,b: a==b, '!=': lambda a,b: a!=b,
           '<': lambda a,b: a<b,  '>': lambda a,b: a>b,
           '<=':lambda a,b: a<=b, '>=':lambda a,b: a>=b}
    return ops[cond['op']](left_val, right_val)

def execute(db, ast):
    if ast['type'] == 'SELECT':
        table = db.get_table(ast['table'])
        results = []
        for row in table.scan():
            if evaluate_condition(row, ast['where']):
                if ast['columns'] == ['*']:
                    results.append(dict(row))
                else:
```

```
                    results.append({c: row.get(c) for c in ast['columns']})
        return results

    if ast['type'] == 'INSERT':
        table = db.get_table(ast['table'])
```

## Illustrations

**Figure 176 • FIG 9.4-A**

### SQL Parsing Pipeline — From Text to Execution Plan

*Illustration: FIG 9.4-A | SQL Parsing Pipeline — From Text to Execution Plan [SVG diagram — see module-9-4-toy-sql-engine.html]*

SQL engine pipeline from text string to result rows. The lexer tokenises the SQL string into keywords, identifiers, operators, and literals. The parser uses a grammar to build an Abstract Syntax Tree (AST) representing the query structure. The analyser resolves names — "age" becomes "users.age" by checking the schema catalog. The planner generates a logical plan (abstract operations). The optimiser rewrites the plan to use indexes and optimal join ordering based on table statistics. The executor evaluates the physical plan using the Volcano iterator model, streaming rows from leaf operators (i

**Figure 177 • FIG 9.4-B**

### Volcano Execution Model — Iterator Chain Pull Model

*Illustration: FIG 9.4-B | Volcano Execution Model — Iterator Chain Pull Model [SVG diagram — see module-9-4-toy-sql-engine.html]*

The Volcano (iterator) execution model — the architecture used by PostgreSQL, SQLite, MySQL, and most SQL databases. Each physical operator implements three methods: open() initialises the operator and opens its children, next() returns the next output row (or EOF), and close() releases resources. The query tree is evaluated lazily: the client calls Sort.next(), which calls Filter.next(), which calls SeqScan.next(). Rows flow upward from leaves (data sources) to the root (output). This pull-based model is elegant and composable — adding a new operator requires implementing just three methods.

**Figure 178 • FIG 9.4-C**

### Query Optimisation — Predicate Pushdown and Join Reordering

*Illustration: FIG 9.4-C | Query Optimisation — Predicate Pushdown and Join Reordering [SVG diagram — see module-9-4-toy-sql-engine.html]*

Predicate pushdown — the single most impactful query optimisation. A naive execution plan joins users and orders first (producing up to 5 million joined rows), then applies the WHERE age > 25 filter. The optimiser recognises that the filter only references columns from the users table and can be pushed below the join: first filter users down to ~50,000 rows (age > 25), then join only those 50,000 rows with orders. This reduces the input to the join by 20×, making the overall query ~100× faster. Query optimisers apply dozens of such algebraic rewriting rules (predicate pushdown, join reordering

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 9.4 Building a Toy SQL Engine: SQL Tokeniser AST Parser Query Executor [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 9.4 Building a Toy SQL Engine **Type:** Implementation + trace + extension **Estimated time:** 5060 minutes **Tools needed:** Python --- Part A Extend the Parser The toy engine from the lesson supports: `SELECT`, `INSERT`, `WHERE` (single condition). 1. Extend the tokeniser to recognise `ORDER`, `BY`, `ASC`, `DESC`, `LIMIT`, `GROUP`, `COUNT`, `AND`, `OR`. 2. Extend `parse_select` to handle: ```sql SELECT name, age FROM users WHERE city = 'NYC' ORDER BY age DESC LIMIT 5 ``` What new fields does the AST need? 3. Extend the executor to apply `ORDER BY` and `LIMI

LEVEL 10

# Web & Application Dev

HTML, Browsers, REST APIs, and Full-Stack Architecture

# Module 10 1 Html Css Js

■# Module 10.1 — HTML, CSS, and JavaScript from the Ground Up

## The Problem

*In 1991, Tim Berners-Lee needed a way to share documents between physicists at CERN.*

*He invented three technologies that now power virtually all interactive software:*

*a markup language to structure content (HTML),*

*a styling language to control appearance (CSS),*

*and a scripting language to add behaviour (JavaScript).*

*>*

*Every web page — from a simple blog to a complex trading platform —*

*is built from these three technologies, working together.*

*Understanding them from first principles reveals*

*why the modern web is the way it is, warts and all.*

## The Story

### HTML, CSS, JavaScript: Three Accidents of History

**HTML** was designed for documents, not applications. The tags (`<h1>`, `<p>`, `<a>`) were meant to describe content structure, not visual layout. The early `<blink>` and `<marquee>` tags for visual effects were embarrassments that browsers eventually stopped supporting.

**CSS** was designed by Håkon Wium Lie (1994) to separate presentation from content. Before CSS, HTML authors used `<font>` tags, `<table>` for layout, and `bgcolor` attributes everywhere — mixing content with presentation in ways that were impossible to maintain.

**JavaScript** was invented by Brendan Eich at Netscape in **10 days** in 1995. The original goal: add simple interactivity to web pages. It was designed quickly, with deliberate Java-like syntax to appeal to Java programmers, but with dynamic typing and prototype-based objects — very different from Java. Every strange corner of JavaScript (type coercion, `this` binding, `var` hoisting) is a direct consequence of that 10-day timeline.

These three technologies were standardised by the W3C and ECMA, evolved separately, and are now deeply intertwined — the foundation of all web development.

## The Concept

### HTML — Structure

HTML (HyperText Markup Language) describes the *semantic structure* of content:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Page</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Welcome</h1>
    <nav>
      <a href="/">Home</a>
      <a href="/about">About</a>
    </nav>
  </header>
  <main>
    <article>
      <h2>Article Title</h2>
      <p>First paragraph. <strong>Important word.</strong></p>
      <p>Second paragraph.</p>
    </article>
  </main>
  <footer>
    <p>&copy; 2026</p>
  </footer>
</body>
</html>
```

**Semantic HTML**: use the right element for the right content. `<article>` for self-contained content. `<nav>` for navigation. `<button>` for buttons (not `<div>`). `<input type="email">` for email fields. Semantic HTML improves:

• **Accessibility**: screen readers use semantics to navigate

• **SEO**: search engines weight headings and semantic elements

• **Maintainability**: clear meaning for developers

**The DOM (Document Object Model)**: the browser parses HTML and creates a tree of objects. JavaScript can read and modify this tree, dynamically updating the page.

### CSS — Presentation

CSS (Cascading Style Sheets) controls visual appearance through selectors and declarations:

```
/* Selector: element */
h1 { color: #1C1C1C; font-size: 2rem; }

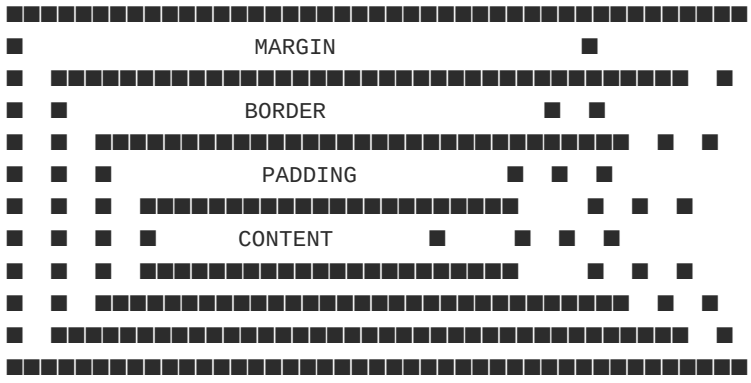
/* Selector: class */
.card { background: white; border-radius: 8px; padding: 16px; }

/* Selector: ID */
#main-nav { display: flex; gap: 24px; }
```

```
/* Selector: pseudo-class */
a:hover { text-decoration: underline; }

/* Selector: descendant */
.card p { color: #666; font-size: 0.9rem; }
```

**The Box Model**: every HTML element is a rectangular box:




**Flexbox** — one-dimensional layout:

```
.container {
  display: flex;
  justify-content: space-between;  /* horizontal distribution */
  align-items: center;             /* vertical alignment */
  gap: 16px;
}
```

**CSS Grid** — two-dimensional layout:

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);  /* 3 equal columns */
  grid-template-rows: auto;
  gap: 24px;
}
```

**Cascade and Specificity**: when multiple rules target the same element, the most specific wins:

• Inline style: `style="color:red"` (specificity: 1000)

• ID selector: `#id` (specificity: 100)

• Class selector: `.class` (specificity: 10)

• Element selector: p (specificity: 1)

• `!important` overrides everything (use sparingly)

### JavaScript — Behaviour

JavaScript makes pages interactive. Core concepts:

```
// Variables
let count = 0;           // block-scoped, reassignable
const PI = 3.14159;      // block-scoped, not reassignable
var legacy = "avoid";    // function-scoped, hoisted (legacy)
```

```
// Functions
function add(a, b) { return a + b; }
const multiply = (a, b) => a * b;   // arrow function

// DOM manipulation
const btn = document.querySelector('#submit-btn');
btn.addEventListener('click', function(event) {
  event.preventDefault();             // stop form submission
  const input = document.querySelector('#name');
  const message = document.querySelector('#message');
  message.textContent = `Hello, ${input.value}!`;
  message.style.color = 'green';
});

// Fetch API — HTTP requests from JavaScript
async function fetchUser(id) {
  const response = await fetch(`/api/users/${id}`);
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const data = await response.json();
  return data;
}
```

**Event-driven mod

## Illustrations

**Figure 179 • FIG 10.1-A**

### Web Technology Stack — HTML Structure, CSS Presentation, JavaScript Behaviour

*Illustration: FIG 10.1-A | Web Technology Stack — HTML Structure, CSS Presentation, JavaScript Be [SVG diagram — see module-10-1-html-css-js.html]*

The web's three-layer architecture. HTML (HyperText Markup Language) defines structure and semantic meaning — an h1 is a top-level heading, a nav is navigation, a button is interactive. CSS (Cascading Style Sheets) separates visual presentation from structure — changing the button colour requires only CSS changes. JavaScript adds dynamic behaviour — responding to user events, fetching data from APIs, updating the DOM without page reloads. This separation of concerns (SoC) means a screen reader can navigate HTML semantics without CSS, and a designer can restyle the page without touching HTML. P

**Figure 180 • FIG 10.1-B**

### CSS Box Model — Content, Padding, Border, Margin

*Illustration: FIG 10.1-B | CSS Box Model — Content, Padding, Border, Margin [SVG diagram — see module-10-1-html-css-js.html]*

The CSS box model — every HTML element is rendered as a rectangular box with four layers. Content holds the actual text and images. Padding adds transparent space between content and border. Border is the element's visible edge. Margin is transparent space between elements. The box-sizing property controls what "width" means: content-box (default) makes width apply only to content — a 200px element with 24px padding and 2px border is actually 252px wide. border-box makes width include padding and border — a 200px element stays 200px regardless of padding. The industry standard reset * { box-si

**Figure 181 • FIG 10.1-C**

### JavaScript Event Loop — Call Stack, Task Queue, Microtask Queue

*Illustration: FIG 10.1-C | JavaScript Event Loop — Call Stack, Task Queue, Microtask Queue [SVG diagram — see module-10-1-html-css-js.html]*

JavaScript's event loop enables non-blocking I/O on a single thread. When fetch() is called, the HTTP request is handed to the browser's C++ networking layer — the JS thread continues executing. When the response arrives, the callback enters the Task Queue. The event loop checks: if the call stack is empty, drain the Microtask Queue (Promise .then chains), then take one task from the Task Queue and push it onto the stack. This ordering explains why Promise callbacks run before setTimeout callbacks — microtasks are always drained before the next task. setTimeout(fn, 0) doesn't mean "immediately

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 10.1 HTML, CSS & JavaScript from the Ground Up: Live Editor DOM Explorer CSS Box Model [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 10.1 HTML, CSS, and JavaScript from the Ground Up **Type:** Build + implementation + analysis **Estimated time:** 6075 minutes **Tools needed:** browser + text editor --- Part A Semantic HTML Rewrite this non-semantic HTML using appropriate semantic elements: ```html   Home About     My First Post Posted on September 10, 2026 Content

# Module 10 2 How Browsers Render

■# Module 10.2 — How Browsers Render Pages

## The Problem

*You type a URL.*

*0.5 seconds later, a fully-rendered web page appears.*

*What happened?*

>

*Between the DNS lookup and the pixels on screen,*

*the browser does an enormous amount of work:*

*network requests, parsing, style calculation,*

*layout, painting, and compositing.*

*Understanding this pipeline explains why some pages feel instant*

*and others feel sluggish — and how to make the difference.*

## The Story

### Browsers as the World's Most Complex Applications

The first graphical web browser, **NCSA Mosaic** (1993), was revolutionary but simple. It downloaded an HTML file, parsed it sequentially, and drew text and images on screen.

Modern browsers — Chrome, Firefox, Safari, Edge — are among the most complex software systems ever built. Chrome has over 35 million lines of code. The rendering engine alone (Blink, forked from WebKit in 2013) handles HTML5, CSS3, JavaScript ES2022, WebGL, WebAssembly, WebRTC, service workers, and hundreds of other standards.

The performance of rendering directly affects user experience. Google's research showed a 100ms delay in search results caused a 0.2% reduction in searches. Amazon found every 100ms increase in page load time cost 1% in revenue. This commercial pressure drove massive investment in browser performance optimisation.

## The Concept

### The Critical Rendering Path

```
1. DNS lookup
2. TCP connection + TLS handshake
3. HTTP request/response
```

```
4. Receive HTML bytes
       ↓
5. Parse HTML → DOM tree
6. Parse CSS → CSSOM tree
       ↓
7. Combine → Render Tree
       ↓
8. Layout (Reflow) — calculate position and size
       ↓
9. Paint — draw pixels for each element
       ↓
10. Composite — layer pixels onto screen
```

### Step 1–3: Network

The browser resolves the hostname via DNS, opens a TCP connection, performs a TLS handshake (for HTTPS), and sends an HTTP GET request. First byte arrives after 1–2 RTTs minimum.

**Time to First Byte (TTFB)**: how long until the browser receives the first byte of the response. Optimised by CDNs (serve from geographically closer servers) and server-side caching.

### Step 4–6: Parsing

**HTML parsing** → **DOM**: The browser parses HTML incrementally as bytes arrive — it does not wait for the entire document. Parsing is deliberately error-tolerant (browsers handle malformed HTML without crashing):

```
<html><body><h1>Title</h1><p>Text</p></body></html>
```

↓

```
Document
  ■■■ html
        ■■■ body
              ■■■ h1
              ■    ■■■ "Title"
              ■■■ p
                   ■■■ "Text"
```

**CSS parsing** → **CSSOM**: CSS is **render-blocking**: the browser cannot render any part of the page until all CSS is loaded and parsed. Why? Because any element might be affected by any CSS rule — the browser cannot start rendering until it knows the complete styles.

```
h1 { color: red; font-size: 2rem; }
p  { color: #333; margin: 16px 0; }
```

↓

```
CSSStyleSheet
  ■■■ Rule: h1 → {color:red, font-size:2rem}
  ■■■ Rule: p  → {color:#333, margin:16px 0}
```

**JavaScript is also render-blocking** (unless marked `async` or `defer`): when the HTML parser encounters a `<script>` tag, it stops parsing HTML, downloads the script, executes it, then resumes. JavaScript can modify the DOM and CSSOM, so the browser must pause to maintain consistency.

```
<!-- Blocking: parser stops until script downloads and executes -->
<script src="app.js"></script>

<!-- async: download in parallel, execute immediately when ready -->
<script src="analytics.js" async></script>

<!-- defer: download in parallel, execute after HTML parsed -->
<script src="app.js" defer></script>
```

### Step 7: Render Tree

The render tree combines DOM + CSSOM. It includes only visible elements:

• `display: none` elements are excluded

• `visibility: hidden` elements are included (space reserved, but invisible)

• Pseudo-elements (`::before`, `::after`) are added

Each node in the render tree knows its computed styles.

### Step 8: Layout (Reflow)

The browser calculates the exact position and size of every element in the render tree. This involves:

• Computing text flow (line breaking, word wrapping)

• Resolving percentage widths relative to parent

• Computing margins, paddings, borders

• Handling flexbox/grid layout algorithms

Layout is **expensive and cascading**: changing the width of a parent may require re-laying out all its descendants. This is called **reflow**. Operations that trigger reflow include:

• Changing any CSS property that affects size or position

• Inserting or removing DOM elements

• Reading `offsetWidth`, `scrollTop`, `getBoundingClientRect()` — forces synchronous layout ("layout thrashing")

### Step 9: Paint

The browser creates drawing commands for each visible element — backgrounds, borders, text, shadows, images. This creates a list of painting operations but does not yet produce pixels.

**Layers**: elements that need to be painted independently (due to animations, `will-change`, `position: fixed`, `overflow: scroll`, etc.) are promoted to their own **compositor layer** — drawn once and cached as a texture.

### Step 10: Composite

The GPU composites (combines) all layers into the final frame. Operations handled entirely by the compositor (no layout or paint triggered):

• CSS `transform: translate/rotate/scale`

• CSS `opacity`

• CSS `filter`

This is why CSS animations of `transform` and `opacity` are smooth (60 fps) while animating `top`, `left`, or `margin` causes jank (triggers layout on every frame).

### Performance: 60fps and the 16ms Budget

The human eye perceives motion as smooth at 60 frames per second. Each frame must be rendered in **16.67ms** (1000ms ÷ 60). For a smooth animation:

• JavaScript execution: < 10ms

• Rendering pipeline: < 6ms

If JavaScript takes longer than ~50ms, users notice jank. Profile with Chrome DevTools → Performance tab.

**Key optimisations:**

• Minimise reflows: batch DOM reads and writes, use `

### Illustrations

**Figure 182 • FIG 10.2-A**

**Browser Rendering Pipeline — HTML to Pixels in 6 Steps**

*Illustration: FIG 10.2-A | Browser Rendering Pipeline — HTML to Pixels in 6 Steps [SVG diagram — see module-10-2-how-browsers-render.html]*

The browser rendering pipeline from HTML bytes to pixels on screen. HTML parsing builds the DOM tree; CSS parsing builds the CSSOM. They are merged into the Render Tree (only visible nodes with computed styles). Layout calculates the exact position and size of every element (reflow). Paint fills pixels into layers (repaint). Composite blends GPU layers onto screen. To maintain 60fps, all stages must complete within 16.6ms per frame. The critical performance insight: CSS transform and opacity changes skip Layout and Paint entirely — they only trigger Compositing, which runs on the GPU. Animatin

**Figure 183 • FIG 10.2-B**

### DOM Tree — HTML Parsed into a JavaScript-Accessible Object Tree

*Illustration: FIG 10.2-B | DOM Tree — HTML Parsed into a JavaScript-Accessible Object Tree [SVG diagram — see module-10-2-how-browsers-render.html]*

The Document Object Model (DOM) is the browser's object representation of the parsed HTML. Every element becomes a node with properties (id, className, textContent) and methods (querySelector, appendChild). JavaScript navigates and mutates the DOM at runtime — changing a node's text content, adding/removing classes, or inserting new elements. Every mutation that affects layout or appearance triggers the rendering pipeline: a geometry change triggers reflow, a style change triggers repaint. Batching DOM mutations (document fragment, virtual DOM in React) reduces pipeline re-triggers. The DOM is

**Figure 184 • FIG 10.2-C**

### Critical Rendering Path — What Blocks First Paint

*Illustration: FIG 10.2-C | Critical Rendering Path — What Blocks First Paint [SVG diagram — see module-10-2-how-browsers-render.html]*

The critical rendering path — the sequence of steps the browser must complete before showing any pixels. CSS stylesheets are render-blocking: the browser cannot paint until it has built the CSSOM (a script might query computed styles). Synchronous <script> tags are parser-blocking: the HTML parser stops, the script is fetched and executed, then parsing resumes. To optimise: place CSS <link> in <head> (not blocking by position, but must complete before paint), use <script defer> for non-critical scripts (parsed and loaded in background, executed in order after HTML is fu

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 10.2 How Browsers Render Pages: Rendering Pipeline Render Tree Reflow & Repaint [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 10.2 How Browsers Render Pages **Type:** Analysis + optimisation + reasoning **Estimated time:** 4050 minutes **Tools needed:** browser DevTools --- Part A Critical Rendering Path 1. A web page loads these resources in `<head>`: ```html <link rel="stylesheet" href="styles.css"> <script src="app.js"></script> <link rel="stylesheet" href="theme.css"> ``` In what order do they block/not-block rendering? Draw a timeline showing when each resource is fetched and when rendering can begin. 2. Rewrite the above with optimal loading attributes (`async`, `defer

# Parse JSON string Python dict

■# Module 10.3 — REST APIs and JSON

## The Problem

*A mobile app, a web frontend, a desktop app, and a third-party service*

*all need to share the same data and functionality.*

>

*You cannot share code — they run on different platforms.*

*You cannot share a database — that would be a security nightmare.*

>

*You need a standard way for these different systems to communicate.*

*A contract that says: send this request, get that response.*

>

*That contract is the **API** (Application Programming Interface),*

*and for web services, the dominant style is **REST** using **JSON**.*

## The Story

### REST: A Style, Not a Standard

Roy Fielding defined REST (Representational State Transfer) in his 2000 PhD dissertation at UC Irvine. Fielding was one of the primary authors of HTTP/1.1; REST was his architectural analysis of why the web worked so well and how to apply those same principles to designing APIs.

REST is not a protocol or a specification — it is a set of architectural constraints. An API that follows these constraints is called "RESTful."

Before REST, web services typically used **SOAP** (Simple Object Access Protocol) — XML-based, rigid, complex, with extensive tooling requirements. SOAP dominated enterprise software in the early 2000s. REST replaced it because it was simpler: use HTTP verbs and URLs you already understand, and JSON instead of XML.

**JSON** (JavaScript Object Notation) was popularised by Douglas Crockford around 2001. It was not invented — it was discovered as a subset of JavaScript that worked as a data interchange format. Simpler than XML (no closing tags), directly parseable by JavaScript, and human-readable.

## The Concept

### JSON — Data Format

```
  {
```

```
  "id": 1,
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "age": 30,
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "country": "US"
  },
  "tags": ["developer", "python", "aws"],
  "active": true,
  "score": 98.5,
  "deleted_at": null
}
```

**JSON data types**: string (double quotes), number (int or float), boolean (true/false), null, object ({}), array ([]).

**JSON in Python:**

```
import json

# Parse JSON string → Python dict
data = json.loads('{"name": "Alice", "age": 30}')
print(data['name'])      # Alice

# Serialize Python → JSON string
obj = {'name': 'Bob', 'scores': [95, 87, 92]}
text = json.dumps(obj, indent=2)
# {
#   "name": "Bob",
#   "scores": [95, 87, 92]
# }
```

### REST Architectural Constraints

**Client-Server**: separate concerns — client handles UI, server handles data

**Stateless**: each request contains all information needed; server stores no session state per client

**Cacheable**: responses indicate whether they can be cached

**Uniform Interface**: consistent URLs and HTTP methods (the key constraint)

**Layered System**: client doesn't know if it's talking to the real server or a load balancer/cache

**Code on Demand** (optional): server can send executable code (JavaScript)

### RESTful API Design

**Resources** (nouns) map to URLs. **HTTP methods** (verbs) define the action:

```
GET    /users           → list all users
GET    /users/42        → get user 42
POST   /users           → create a new user
PUT    /users/42        → replace user 42 entirely
PATCH  /users/42        → partially update user 42
DELETE /users/42        → delete user 42
```

```
GET    /users/42/orders → list orders for user 42
POST   /users/42/orders → create an order for user 42
GET    /orders/7        → get a specific order (can also go here)
```

**URL design rules:**

• Use nouns, not verbs: `/users/42` not `/getUser?id=42`

• Use plural nouns: `/users` not `/user`

• Nest for relationships: `/users/42/orders`

• Keep it flat when possible: `/orders?user_id=42` is fine

• Versioning: `/api/v1/users` or `Accept: application/vnd.api+json;version=1`

## Request and Response Format

**Creating a user (POST):**

Request:

```
POST /api/v1/users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

{
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "role": "admin"
}
```

Response:

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/v1/users/43

{
  "id": 43,
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "role": "admin",
  "created_at": "2026-09-10T22:00:00Z"
}
```

**Error response (validation failure):**

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{
  "error": "validation_failed",
  "message": "Request validation failed",
  "details": [
    {"field": "email", "code": "already_exists", "message": "Email already in use"},
    {"field": "name", "code": "too_short", "message": "Name must be at least 2 characters"}
  ]
```

```
}
```

### Building a REST API with Python (Flask)

```
from flask import Flask, request, jsonify, abort

app = Flask(__name__)

# In-memory store (use a database in production)
users = {1: {'id':1,'name':'Alice','email':'alice@example.com'}}
next_id = 2

@app.route('/api/v1/users', methods=['GET'])
def list_users():
    return jsonify(list(users.values()))

@app.route('/api/v1/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    user = users.get(user_id)
    if not user: abort(404, description=f"User {user_id} not found")
    return jsonify(user)

@app.route('/api/v1/users', methods=['POST'])
def create_user():
    global next_id
    data = request.get_json()
    if not data or 'name' not in data or 'email' not in data:
        abort(400, description="name and email are required")
    user = {'id': next_id, 'name': data['name'], 'email': data['email']}
    users[next_id] = user
    next_id += 1
    return jsonify(user), 201

@app.route('/api/v1/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    if user_id not in users: abort(404)
    del users[user_id]
    return '', 204

@app.errorhandler(404)
def not_found(e): return jsonify(error=str(e)), 404

@app.error
```

### Illustrations

**Figure 185 • FIG 10.3-A**

### REST Resource Model — URLs as Nouns, HTTP Methods as Verbs

*Illustration: FIG 10.3-A | REST Resource Model — URLs as Nouns, HTTP Methods as Verbs [SVG diagram — see module-10-3-rest-apis.html]*

REST (Representational State Transfer) architectural style. URLs name resources (nouns: /users, /users/42), and HTTP methods specify the action (verbs: GET, POST, PUT, PATCH, DELETE). This clean separation makes APIs predictable and self-documenting. GET is idempotent and safe (no side effects). PUT is idempotent but modifying. POST creates and is neither idempotent nor safe. DELETE is idempotent. Statelessness is the critical constraint: each request must contain all information needed to process it — the server holds no client session state. This enables horizontal scaling (any server can ha

**Figure 186 • FIG 10.3-B**

### JSON Data Format — Types, Structure, and API Response Anatomy

*Illustration: FIG 10.3-B | JSON Data Format — Types, Structure, and API Response Anatomy [SVG diagram — see module-10-3-rest-apis.html]*

JSON (JavaScript Object Notation) is the dominant API data format — lightweight, human-readable, and supported natively in every modern language. Six value types: string (always double-quoted), number (integer or float), boolean, null, object ({} key-value pairs), and array ([] ordered list). JSON has no comments, no functions, no undefined. Typical REST API responses wrap the payload in an envelope: a status field (numeric or string), a data field (the actual resource), and meta (pagination info, request ID for tracing). ISO 8601 timestamps ("2024-01-15T10:30:00Z") are the universal standard

**Figure 187 • FIG 10.3-C**

### HTTP Status Codes — The Full Reference for REST API Design

*Illustration: FIG 10.3-C | HTTP Status Codes — The Full Reference for REST API Design [SVG diagram — see module-10-3-rest-apis.html]*

HTTP status codes — the standard language of REST API responses. 2xx indicates success. 3xx signals a redirect — the client should request a different URL. 4xx signals client error: the request was wrong, not authorised, or targeted a non-existent resource. 5xx signals server error — the request was valid but the server failed to process it. The critical design rule: never return HTTP 200 with an error message in the body ("200 OK: error: not found") — this breaks every monitoring tool, logging system, and client library that depends on status codes. 401 (Unauthorized) means "not authenticated

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 10.3 REST APIs and JSON: API Explorer Request Builder Status Codes [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 10.3 REST APIs and JSON **Type:** Design + implementation + analysis **Estimated time:** 4555 minutes **Tools needed:** Python + pen and paper --- Part A REST API Design Design a REST API for a **library management system** with resources: books, members, and loans (a loan = one member borrowing one book). 1. Design the URL structure for all CRUD operations on all three resources. List at least 8 endpoints with their HTTP methods. 2. For the loan resource: a member can borrow a book (create a loan) and return it (update or delete). Which HTTP method is more semantically c

# app.py

■# Module 10.4 — Full-Stack Project

## The Problem

*You have learned:*

*- HTML, CSS, JavaScript (frontend)*

*- HTTP and REST APIs (communication layer)*

*- Databases and SQL (data persistence)*

*- Sockets and servers (networking)*

*>*

*But these skills in isolation do not make a working application.*

*A real application is these layers **working together**:*

*a user clicks a button in the browser →*

*JavaScript sends an HTTP request →*

*a server receives and validates it →*

*queries a database →*

*returns structured data →*

*JavaScript renders the result on screen.*

*>*

*This module integrates everything by building a complete,*

*working application from scratch.*

## The Story

### The Full Stack Emerges

The term "full stack" was popularised around 2008–2012 as web applications became increasingly complex. The LAMP stack (Linux, Apache, MySQL, PHP) was an early full-stack: PHP generated HTML server-side, MySQL stored data.

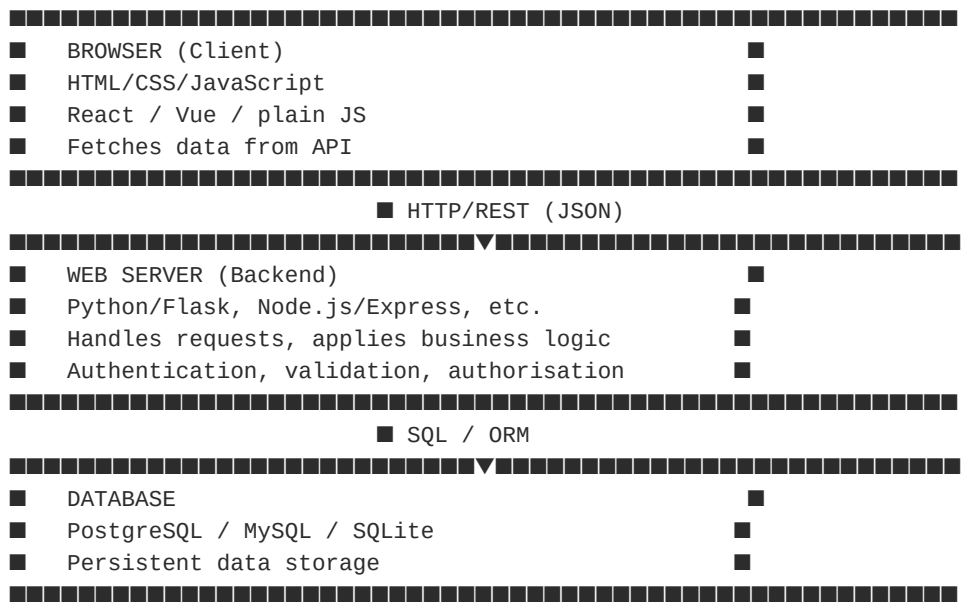
**AJAX** (Asynchronous JavaScript and XML, named ~2005) changed the model: instead of the server rendering complete HTML pages, JavaScript would fetch data (via XMLHttpRequest) and update the page dynamically. Google Maps (2005) demonstrated this spectacularly — panning a map didn't reload the page.

Modern full-stack typically means: a JavaScript or TypeScript **frontend** (React, Vue, Angular), a **backend API** (Node.js, Python, Go, Java), and a **database** (PostgreSQL, MySQL, MongoDB). The frontend and backend communicate via JSON REST APIs or GraphQL.

## The Concept

### Application Architecture

A modern full-stack web application has three tiers:




### The Project: A Task Manager

Build a simple but complete task manager app:

• Create tasks with title, description, and due date

• Mark tasks as complete

• Filter by status (all / pending / completed)

• Persist tasks in a database

### Backend: Flask + SQLite

```
# app.py
from flask import Flask, request, jsonify
from flask_cors import CORS
import sqlite3, json
from datetime import datetime

app = Flask(__name__)
CORS(app)  # allow browser requests from different origin

DB = 'tasks.db'

def get_db():
    conn = sqlite3.connect(DB)
    conn.row_factory = sqlite3.Row  # rows as dicts
    return conn

def init_db():
    with get_db() as db:
        db.execute('''
            CREATE TABLE IF NOT EXISTS tasks (
                id          INTEGER PRIMARY KEY AUTOINCREMENT,
                title       TEXT NOT NULL,
                description TEXT,
                due_date    TEXT,
                completed   INTEGER DEFAULT 0,
                created_at  TEXT DEFAULT (datetime('now'))
            )
        ''')

@app.route('/api/tasks', methods=['GET'])
def list_tasks():
    status = request.args.get('status')   # ?status=pending or ?status=completed
    with get_db() as db:
        if status == 'pending':
            rows = db.execute('SELECT * FROM tasks WHERE completed=0 ORDER BY created_at DESC').fetcha
        elif status == 'completed':
            rows = db.execute('SELECT * FROM tasks WHERE completed=1 ORDER BY created_at DESC').fetcha
        else:
            rows = db.execute('SELECT * FROM tasks ORDER BY created_at DESC').fetchall()
    return jsonify([dict(r) for r in rows])

@app.route('/api/tasks', methods=['POST'])
def create_task():
    data = request.get_json()
    if not data or not data.get('title'):
        return jsonify(error='title is required'), 400
    with get_db() as db:
        cursor = db.execute(
            'INSERT INTO tasks (title, description, due_date) VALUES (?, ?, ?)',
            (data['title'], data.get('description'), data.get('due_date'))
        )
        task_id = cursor.lastrowid
```

```
        task = dict(db.execute('SELECT * FROM tasks WHERE id=?', (task_id,)).fetchone())
    return jsonify(task), 201

@app.route('/api/tasks/<int:task_id>', methods=['PATCH'])
def update_task(task_id):
    data = request.get_json()
    with get_db() as db:
        task = db.execute('SELECT * FROM tasks WHERE id=?', (task_id,)).fetchone()
        if not task: return jsonify(error='not found'), 404
        if 'completed' in data:
            db.execute('UPDATE tasks SET completed=? WHERE id=?', (int(data['completed']), task_id))
        if 'title' in data:
            db.execute('UPDATE tasks SET title=? WHERE id=?', (data['title'], task_id))
        updated = dict(db.execute('SELECT * FROM tasks WHERE id=?', (task_id,)).fetchone())
    return jsonify(updated)

@app.route('/api/tasks/<int:task_id>', methods=['DELETE'])
def delete_task(task_id):
    with get_db() as db:
        result = db.execute('DELETE FROM tasks WHERE id=?', (task_id,))
        if result.rowcount == 0: return jsonify(error='not found'), 404
    return '', 204

init_db()
if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

### Frontend: HTML + Vanilla JavaScript

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" conte
```

### Illustrations

**Figure 188 • FIG 10.4-A**

**Three-Tier Architecture — Client, Application Server, Database**

*Illustration: FIG 10.4-A | Three-Tier Architecture — Client, Application Server, Database [SVG diagram — see module-10-4-fullstack-project.html]*

Three-tier architecture — the dominant pattern for web applications. The Client tier (browser, mobile app) only renders UI and makes HTTP requests; it never has direct database access. The Application Server tier handles all business logic — authentication, validation, business rules, database queries — and exposes a clean REST or GraphQL API. The Data tier persists state. This separation enables each tier to scale independently: add more app servers behind a load balancer without changing the client or database. It also enforces security: the client cannot bypass business logic to make direct

**Figure 189 • FIG 10.4-B**

### Full-Stack Request Lifecycle — Browser to CDN to Load Balancer to App to DB

*Illustration: FIG 10.4-B | Full-Stack Request Lifecycle — Browser to CDN to Load Balancer to App [SVG diagram — see module-10-4-fullstack-project.html]*

Production request lifecycle for a scaled web application. The browser's request first hits a CDN — for static assets (JS, CSS, images) the CDN responds from edge cache in ~5ms. Dynamic API requests pass through a load balancer that distributes across stateless app server replicas (round-robin or least-connections). Each app server checks Redis for a cached result (~1ms) before querying the primary database. Read-heavy queries may hit a read replica to scale reads independently. The 80/20 rule of web performance: 80% of requests are static assets served by CDN; optimising CDN cache-hit rate ha

**Figure 190 • FIG 10.4-C**

### Authentication — Session Cookies vs JWT Tokens

*Illustration: FIG 10.4-C | Authentication — Session Cookies vs JWT Tokens [SVG diagram — see module-10-4-fullstack-project.html]*

Two approaches to stateful authentication in stateless HTTP. Session cookies store an opaque ID in the client cookie jar; the server looks up the session record on every request. Simple and revocable (delete the session from the store to log out any session), but requires a shared session store accessible to all app server replicas. JWT (JSON Web Token) embeds the user's identity and claims in a cryptographically signed token — any server that knows the signing key can verify it without a database lookup. This scales horizontally without a shared store but makes revocation hard: a valid JWT re

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 10.4 Full-Stack Project: Live App Architecture Data Flow [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 10.4 Full-Stack Project **Type:** Implementation + extension + reasoning **Estimated time:** 6090 minutes **Tools needed:** Python, Flask, browser --- Part A Feature Extensions Starting from the task manager from the lesson: 1. **Due dates with colour coding**: modify the frontend to display each task's due date. Tasks overdue (past due date) should be shown with a red border; tasks due today should be orange; others normal. 2. **Search endpoint**: implement `GET /api/tasks?q=keyword` that returns tasks where the title or description contains the keyword (case-insensitive

LEVEL 11

# AI & Machine Learning

Search, Neural Networks, Backpropagation, and Learning Theory

# Module 11 1 Search Intelligence

■# Module 11.1 — What is Intelligence? Search and Constraint Satisfaction

## The Problem

*A chess program examines millions of board positions per second.*

*A GPS finds the optimal driving route through 10,000 intersections.*

*A Sudoku solver fills in 81 cells satisfying 27 constraints.*

>

*Are these programs "intelligent"?*

*What does intelligence mean for a computer?*

>

*Before neural networks and large language models,*

*AI was built almost entirely from **search** and **constraint satisfaction** —*

*systematic exploration of a problem space to find solutions.*

*These techniques still power many practical AI systems.*

## The Story

### The Birth of AI as a Field

**John McCarthy** coined the term "Artificial Intelligence" in 1956 when he organised the Dartmouth Conference — the founding event of AI as a research discipline. The optimism was extraordinary: McCarthy and others believed that within a generation, machines would match human intelligence.

The early approach: intelligence = search. Given a problem with a well-defined state space, intelligent behaviour means searching that space efficiently. Chess, theorem proving, and game playing all reduced to search.

**Allen Newell and Herbert Simon** built the General Problem Solver (1957) — a program that could solve many types of problems by means-ends analysis (measure the gap between current state and goal, take actions to reduce it). Simon predicted in 1958 that within 10 years, computers would be chess champions.

It took 40 years. **Deep Blue** (IBM, 1997) defeated world chess champion Garry Kasparov — using search (alpha-beta pruning) plus an evaluation function trained by chess grandmasters. It searched ~200 million positions per second.

## The Concept

### State Space Search

A **state space** defines:

• **States**: all possible configurations of the problem

• **Initial state**: starting configuration

• **Goal test**: how to recognise a solution

• **Actions**: what transitions between states are possible

• **Path cost**: cost of reaching a state along a path

**Example — Romania driving problem**: find the shortest route from Arad to Bucharest. States = cities, initial state = Arad, goal = Bucharest, actions = road connections, path cost = road distance.

This is exactly Dijkstra's algorithm (Module 6.6). Search IS graph traversal.

### Uninformed Search

**BFS**: guaranteed optimal (fewest steps), exponential memory O(b^d) where b = branching factor, d = depth.

**DFS**: O(m) memory (m = max depth), not optimal, can get lost in infinite branches.

**Iterative Deepening DFS (IDDFS)**: run DFS with increasing depth limits (1, 2, 3, ...). Gets DFS memory advantage + BFS optimality. Repeats work but only by a constant factor. The practical uninformed search algorithm of choice.

**Uniform Cost Search (Dijkstra)**: expands lowest-cost node first. Optimal when all costs > 0. Used in GPS routing.

### Informed Search — Heuristics

A **heuristic function** `h(n)` estimates the cost from node n to the goal. Good heuristics reduce search dramatically.

**Greedy Best-First Search**: always expand node with smallest `h(n)`. Fast but not optimal.

**A* Search**: expand node with smallest `f(n) = g(n) + h(n)` where g(n) = actual cost from start, h(n) = estimated cost to goal.

• If h is **admissible** (never overestimates): A* is optimal

• If h is **consistent** (h(n) ≤ cost(n,n') + h(n')): A* is also efficient

For the Romania example: h(n) = straight-line distance to Bucharest. This never overestimates (roads are longer than straight lines) → admissible.

```
import heapq

def astar(graph, start, goal, heuristic):
    pq = [(heuristic(start, goal), 0, start, [start])]
    visited = set()
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
        if node in visited:
```

```
            continue
        visited.add(node)
        for neighbour, cost in graph[node]:
            if neighbour not in visited:
                new_g = g + cost
                new_f = new_g + heuristic(neighbour, goal)
                heapq.heappush(pq, (new_f, new_g, neighbour, path + [neighbour]))
    return None, float('inf')
```

### Adversarial Search — Games

In two-player zero-sum games (chess, checkers, Go, tic-tac-toe), one player tries to maximise score, the other tries to minimise it. **Minimax** search:

```
def minimax(state, depth, maximising):
    if depth == 0 or is_terminal(state):
        return evaluate(state)

    if maximising:
        best = -float('inf')
        for child in get_children(state):
            best = max(best, minimax(child, depth-1, False))
        return best
    else:
        best = float('inf')
        for child in get_children(state):
            best = min(best, minimax(child, depth-1, True))
        return best
```

**Alpha-Beta Pruning**: prune branches that cannot affect the final decision. With good move ordering, reduces the effective branching factor from b to √b — searching twice as deep in the same time.

```
def alphabeta(state, depth, alpha, beta, maximising):
    if depth == 0 or is_terminal(state):
        return evaluate(state)

    if maximising:
        for child in get_children(state):
            value = alphabeta(child, depth-1, alpha, beta, False)
            alpha = max(alpha, value)
            if alpha >= beta:
                break  # β cutoff — minimiser won't allow this
        return alpha
    else:
        for child in get_children(state):
            value = alphabeta(child, depth-1, alpha, beta, True)
            beta = min(beta, value)
            if beta <= alpha:
                break  # α cutoff — maximiser won't allow this
        return beta
```

### Constraint Satisfaction Problems (CSPs)

A CSP is defined by:

• **Variables**: X■, X■, ..., X■

• **Domains**: D■, D■, ..., D■ (possible values for each variable)

• **Constraints**: relations between variables that valid solutions must satisfy

**Exampl

## Illustrations

**Figure 191 • FIG 11.1-A**

### State Space Search — BFS, DFS, and A* on a Grid Maze

*Illustration: FIG 11.1-A | State Space Search — BFS, DFS, and A* on a Grid Maze [SVG diagram — see module-11-1-search-intelligence.html]*

State space search on a 5×5 grid maze. BFS (left) expands all cells at distance 1, then 2, then 3 — guaranteeing the shortest path but exploring many irrelevant cells. A* (right) uses a heuristic function f(n) = g(n) + h(n): the actual cost to reach n plus an estimate of the remaining cost (Manhattan distance to goal). By always expanding the node with lowest f(n), A* focuses exploration toward the goal and visits far fewer cells. A* is optimal when the heuristic is admissible (never overestimates the true remaining cost). Real applications: Google Maps routing (Dijkstra/A* on road graphs), ga

**Figure 192 • FIG 11.1-B**

### Minimax Game Tree — Two-Player Zero-Sum Search

*Illustration: FIG 11.1-B | Minimax Game Tree — Two-Player Zero-Sum Search [SVG diagram — see module-11-1-search-intelligence.html]*

Minimax game tree for a two-player zero-sum game. MAX (AI) wants to maximise the score; MIN (opponent) wants to minimise it. At leaf nodes, a static evaluation function scores the position. Values propagate upward: MIN nodes take the minimum of their children (opponent plays best), MAX nodes take the maximum (AI plays best). The root's value (3) is MAX's guaranteed score with perfect play. Alpha-Beta pruning eliminates branches that cannot affect the final decision — if the MIN node has already found a value of 2 and the MAX parent already has 3, exploring further subtrees of that MIN node is

**Figure 193 • FIG 11.1-C**

**Constraint Satisfaction — N-Queens Backtracking Search**

*Illustration: FIG 11.1-C | Constraint Satisfaction — N-Queens Backtracking Search [SVG diagram — see module-11-1-search-intelligence.html]*

Constraint satisfaction with N-Queens backtracking. For each row, try placing a queen in each column. After each placement, check all constraints (no shared row, column, or diagonal with any previously placed queen). If no valid column exists in the current row, backtrack to the previous row and try the next column there. The 4-Queens solution [1,3,0,2] places queens at (row 0, col 1), (row 1, col 3), (row 2, col 0), (row 3, col 2). Backtracking prunes the search tree dramatically — the naive approach would check 4■ = 256 configurations; backtracking discovers the solution in about 20 checks.

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 11.1 Search and Intelligence: BFS DFS A* Pathfinding [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 11.1 What is Intelligence? Search and Constraint Satisfaction **Type:** Implementation + trace + reasoning **Estimated time:** 5060 minutes **Tools needed:** Python + pen and paper --- Part A A* Implementation Implement A* search on a 66 grid with obstacles. Start = (0,0), Goal = (5,5). Obstacles at: (1,1), (1,2), (2,2), (3,3), (3,4). 1. Implement `astar(grid, start, goal, heuristic)` where: - Moves: up/down/left/right (cost 1 each), diagonal (cost 2) - Heuristic: Manhattan distance for cardinal, Euclidean for diagonal - Returns the path as a list of (row, col) tu

# 3D vector

■# Module 11.2 — Linear Algebra for Machine Learning

## The Problem

*A dataset has 10,000 images, each 28×28 pixels.*

*A neural network layer has 512 neurons, each connected to all 784 pixels.*

*Training involves computing 10,000 × 784 multiplications and additions — simultaneously.*

>

*Without linear algebra, this is incomprehensible.*

*With linear algebra, it is one matrix multiplication.*

>

*Machine learning is, at its mathematical core, applied linear algebra.*

*Understanding vectors, matrices, and the operations on them*

*is the prerequisite for understanding every ML algorithm that matters.*

---

## The Story

### Linear Algebra Was Always About Computation

Linear algebra — the study of linear equations, vectors, and matrices — was developed over centuries. **Gauss** solved systems of linear equations with what we now call Gaussian elimination (early 19th century). **Arthur Cayley** introduced matrix notation in 1858.

But the connection to machine learning is direct: a linear model is literally a linear function (y = Wx + b), a neural network layer is a matrix multiplication followed by a nonlinearity, and most ML optimisation involves computing gradients of scalar loss functions with respect to matrices of parameters.

**Sanjoy Mahajan's** observation: "In applied mathematics, everything is a matrix." Training a model is computing W ← W - α∇W (gradient descent in matrix form). Understanding what ∇W means requires linear algebra.

---

## The Concept

### Vectors

A **vector** is an ordered list of numbers — a point in n-dimensional space.

```
import numpy as np

# 3D vector
v = np.array([1.0, 2.0, 3.0])

# Operations
u = np.array([4.0, 5.0, 6.0])
print(u + v)          # [5, 7, 9]  — element-wise addition
print(2 * v)          # [2, 4, 6]  — scalar multiplication
print(np.dot(u, v))   # 4+10+18 = 32 — dot product
print(np.linalg.norm(v))   # √(1+4+9) = √14 ≈ 3.74 — L2 norm
```

**Dot product**: `u·v = Σ■ u■v■ = |u| |v| cos(θ)` — measures similarity (angle between vectors). At the heart of every linear model.

**Cosine similarity**: `cos(θ) = u·v / (|u||v|)` — 1 = identical direction, 0 = perpendicular, -1 = opposite. Used for comparing word embeddings, document similarity.

### Matrices

A **matrix** is a 2D array of numbers — a linear transformation.

```
# 3×2 matrix (3 rows, 2 columns)
A = np.array([[1, 2],
              [3, 4],
              [5, 6]])

print(A.shape)          # (3, 2)
print(A.T)              # transpose: shape (2, 3)
print(A[1, :])          # row 1: [3, 4]
print(A[:, 0])          # column 0: [1, 3, 5]
```

**Matrix multiplication** — the fundamental operation:

```
A = np.array([[1,2],[3,4],[5,6]])   # shape (3,2)
B = np.array([[7,8,9],[10,11,12]])  # shape (2,3)

C = A @ B    # shape (3,3) — matrix multiplication
# C[i,j] = dot product of row i of A with column j of B
```

Rule: `(m×k) @ (k×n) = (m×n)` — the inner dimensions must match.

**Why matrix multiplication matters for ML**: a neural network layer transforms an input vector x (shape: n) to output z (shape: m) via:

```
z = W @ x + b
```

Where W is a weight matrix (shape: m×n) and b is a bias vector (shape: m). One operation transforms every input dimension to every output dimension.

For a **batch** of N examples: stack them as rows in X (shape: N×n). Then:

```
Z = X @ W.T + b    # shape: (N, m) — all N examples transformed simultaneously
```

This is why GPUs are essential for ML: matrix multiplication is massively parallelisable.

## Special Matrix Operations

**Inverse**: A■¹ such that A■¹A = I (identity). Exists only for square, non-singular matrices. Expensive to compute (O(n³)); avoid when possible.

**Eigenvectors and eigenvalues**:

```
A v = λ v
```

An eigenvector v is unchanged in direction by the matrix A — only scaled by eigenvalue λ.

Eigendecomposition is the basis of:

• **PCA (Principal Component Analysis)**: find the directions of maximum variance in data — the eigenvectors of the covariance matrix

• **Spectral clustering**: cluster data using eigenvectors of the graph Laplacian

• **PageRank**: rank web pages using the dominant eigenvector of the link matrix

**SVD (Singular Value Decomposition)**: any matrix A = UΣV■. The generalisation of eigendecomposition to non-square matrices. Basis of: recommender systems (matrix factorisation), dimensionality reduction, pseudo-inverse.

## NumPy for ML

NumPy is the foundation of all Python ML:

```
import numpy as np

# Create and manipulate data
X = np.random.randn(1000, 20)    # 1000 examples, 20 features
y = np.random.randint(0, 2, 1000) # binary labels

# Normalise features (zero mean, unit variance)
X_mean = X.mean(axis=0)          # mean of each feature
X_std  = X.std(axis=0)
X_norm = (X - X_mean) / X_std

# Linear model forward pass
W = np.random.randn(20, 10) * 0.01   # weight matrix
b = np.zeros(10)                       # bias
Z = X_norm @ W + b                     # (1000, 10)

# Softmax for multi-class classification
def softmax(Z):
    exp_Z = np.exp(Z - Z.max(axis=1, keepdims=True))  # stable softmax
    return exp_Z / exp_Z.sum(axis=1, keepdims=True)

probs = softmax(Z)    # (1000, 10) — probabilities for each class
```

### Gradient Computation

Backpropagation (Module 11.5) relies on matrix calculus. Key identities:

**Scalar loss L, weight matrix W, input X:**

• Forward: `Z = XW + b`, `L = loss(Z, y)`

• Backward: `∂L/∂W = X■ @ ∂L/∂Z`, `∂L/∂b = sum(∂L/∂Z, axis=0)`, `∂L/∂X = ∂L/∂Z @ W■`

These matrix operations are the core of training every neural network.

---

## Hands-On Exercise

**Open the simulation:** [`simulation.html`](./simulation.html)

**Tool 1 — Vector Operations**

• Enter two 2D or 3D vectors

• Visualise addition, subtraction, scalar multiplication

• Compute and visualise dot product (angle between vectors)

**Tool 2 — Matrix Multiplication**

• Enter two matrices

• Watch the element-by-element computation

• See shape rules validated or violated

**Tool 3 — PCA Visualis

## Illustrations

**Figure 194 • FIG 11.2-A**

### Vectors and the Dot Product — Geometric and Algebraic Views



*Illustration: FIG 11.2-A | Vectors and the Dot Product — Geometric and Algebraic Views [SVG diagram — see module-11-2-linear-algebra-ml.html]*

The dot product is the fundamental operation of machine learning. Algebraically: a · b = Σ(a■ × b■). Geometrically: a · b = |a| |b| cos(θ), measuring how much two vectors "align". When cos(θ) = 1, vectors point in the same direction (maximum similarity). When cos(θ) = 0, they are perpendicular (zero similarity). In neural networks, each neuron computes a weighted dot product of its inputs. In transformers, attention scores are dot products between query and key vectors. In recommendation systems, user and item vectors are compared by cosine similarity. Every regression, classification, and neu

**Figure 195 • FIG 11.2-B**

### Matrix Multiplication — Row × Column Dot Products



*Illustration: FIG 11.2-B | Matrix Multiplication — Row × Column Dot Products [SVG diagram — see module-11-2-linear-algebra-ml.html]*

Matrix multiplication computes C[i][j] as the dot product of row i of A and column j of B. Dimensions: (m×k) × (k×n) = (m×n). The inner dimensions k must match — this is the "shapes must be compatible" error every ML engineer encounters. In a neural network, a fully-connected layer computes output = W × input + b: the weight matrix W transforms an input feature vector into output activations. Processing a batch of N samples simultaneously: (N×features) × (features×outputs) = (N×outputs). This single matrix multiply processes the entire batch in parallel — exactly what GPUs are designed for. Mo

**Figure 196 • FIG 11.2-C**

### Linear Transformations — Matrices as Geometric Operations



*Illustration: FIG 11.2-C | Linear Transformations — Matrices as Geometric Operations [SVG diagram — see module-11-2-linear-algebra-ml.html]*

Matrices as geometric transformations. The identity matrix leaves vectors unchanged. A diagonal scaling matrix stretches along axes. A rotation matrix [[cos θ, -sin θ], [sin θ, cos θ]] rotates without changing shape or size. A shear matrix tilts the space. A rank-deficient projection matrix collapses a dimension (information is lost — not invertible). Neural networks are sequences of such transformations: each layer applies a weight matrix (linear transformation) followed by a nonlinear activation function. Without nonlinearity, any sequence of linear transformations collapses to a single line

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 11.2 Linear Algebra for ML: Vector Operations Matrix Multiplication Linear Transformations [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 11.2 Linear Algebra for Machine Learning **Type:** Calculation + implementation + proof **Estimated time:** 4555 minutes **Tools needed:** NumPy + pen and paper --- Part A Matrix Multiplication 1. Multiply these matrices by hand: ``` A = [[1, 2, 3], B = [[7, 8], [4, 5, 6]] [9, 10], [11, 12]] ``` Show the calculation for C[0][0] and C[1][1] explicitly. 2. A neural network layer: input X has shape (32, 784) 32 examples, 784 features each. The weight matrix W has shape (784, 256). What is the shape of the output XW?

# = (XX) Xy works for small datasets

■# Module 11.3 — Regression and Classification

## The Problem

*You have data: 1,000 house prices with their square footage, number of bedrooms, and location.*

*You want to predict the price of a new house.*

*This is **regression** — predicting a continuous value.*

>

*You have 10,000 emails labelled "spam" or "not spam."*

*You want to automatically label new emails.*

*This is **classification** — predicting a discrete category.*

>

*These are the two most fundamental machine learning tasks.*

*Every supervised learning algorithm is solving one of them.*

## The Story

### Galton, Pearson, and the Origin of Regression

Francis Galton coined the term "regression" in 1886 studying the heights of children relative to their parents — tall parents tend to have children shorter than themselves, short parents tend to have children taller than themselves. Galton called this "regression to the mean." Karl Pearson extended this

work, developing the correlation coefficient and linear regression as statistical tools.

The perceptron — the first classification algorithm learned from data — was invented by Frank Rosenblatt in 1958, inspired by McCulloch and Pitts' model of a neuron. The perceptron is still the conceptual foundation of neural networks today.

Logistic regression (despite the name, a classification algorithm) was developed in the 1940s–50s for medical statistics. It remains one of the most widely used algorithms in industry — interpretable, fast, and effective for binary classification with clean features.

## The Concept

### Linear Regression

**Problem**: given training data {(x■,y■), ..., (x■,y■)}, find parameters w (weights) and b (bias) such that ■ = w·x + b minimises prediction error.

**Model**: ■ = w■x■ + w■x■ + ... + w■x■ + b = Xw + b

**Loss function** (Mean Squared Error):

```
MSE = (1/n) Σ■ (y■ - ■■)²
```

**Gradient descent**: iteratively move parameters in the direction that reduces loss.

```
import numpy as np

def linear_regression_gd(X, y, lr=0.01, epochs=1000):
    n, d = X.shape
    w = np.zeros(d)
    b = 0.0

    for epoch in range(epochs):
        y_pred = X @ w + b                # forward pass
        error  = y_pred - y

        # Gradients
        dw = (2/n) * X.T @ error
        db = (2/n) * np.sum(error)

        # Update parameters
        w -= lr * dw
        b -= lr * db

        if epoch % 100 == 0:
            loss = np.mean(error**2)
            print(f"Epoch {epoch}: MSE = {loss:.4f}")

    return w, b
```

**Closed-form solution** (Normal Equation):

```
# ■ = (X■X)■¹ X■y  — works for small datasets
w = np.linalg.lstsq(X, y, rcond=None)[0]
```

**Regularisation** prevents overfitting:

• **Ridge (L2)**: loss += λ Σ w■² — penalises large weights, keeps all features

• **Lasso (L1)**: loss += λ Σ |w■| — drives some weights to exactly zero (feature selection)

### Classification: Logistic Regression

For binary classification (y ∈ {0,1}), we need to predict a probability. The **sigmoid function** squashes any value to [0, 1]:

```
σ(z) = 1 / (1 + e^(-z))
```

**Model**: `P(y=1|x) = σ(w·x + b)`

**Decision**: if `P(y=1|x) > 0.5` → predict class 1; else predict class 0.

**Loss function** (Binary Cross-Entropy):

```
BCE = -(1/n) Σ■ [y■ log(■■) + (1-y■) log(1-■■)]
```

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def logistic_regression(X, y, lr=0.01, epochs=1000):
    n, d = X.shape
    w = np.zeros(d)
    b = 0.0

    for _ in range(epochs):
        z      = X @ w + b
        y_pred = sigmoid(z)
        error  = y_pred - y

        dw = (1/n) * X.T @ error
        db = (1/n) * np.sum(error)

        w -= lr * dw
        b -= lr * db

    return w, b

def predict(X, w, b, threshold=0.5):
    return (sigmoid(X @ w + b) >= threshold).astype(int)
```

### Decision Trees

A decision tree makes predictions by asking a sequence of yes/no questions:

```
         Is age > 30?
         /          \
       Yes           No
        |             |
Is income > 50k?   Predict: No Purchase
  /         \
Yes          No
 |            |
Predict:   Predict:
Purchase   No Purchase
```

Trees are built by choosing the feature and threshold that best separates the classes at each node. **Gini impurity** or **information gain (entropy)** measure "best":

```
Gini(S) = 1 - Σ■ p■²
Information Gain = H(parent) - Σ (|child|/|parent|) × H(child)
```

**Decision trees in Python:**

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)
model = DecisionTreeClassifier(max_depth=3)
model.fit(X, y)
print(model.predict([[5.1, 3.5, 1.4, 0.2]]))   # [0] — Iris setosa
```

**Pros**: interpretable (you can explain each decision), handles mixed feature types, no scaling needed. **Cons**: prone to overfitting (especially deep trees), unstable (small data changes → very different tree).

**Random Forests**: build many decision trees, each on a random subset of data and features; average their predictions. Dramatically reduces overfitting. One of the most effective and widely used ML algorithms.

## Evaluation Metrics

**For regression:**

• MSE (Mean Squared Error) — penalises large errors heavily

• RMSE (Root MSE) — same units as y

• MAE (Mean Absolute Error) — more robust to outliers

• $R^2$ (coefficient of determination) — 1.0 = perfect, 0 = predicts mean, < 0 = worse than mean

**For classification:**

```
               Predicted
               Pos   Neg
Actual  Pos [ TP  | FN ]
        Neg [ FP  | TN ]

Accuracy  = (TP+TN) / (TP+TN+FP+FN)
Precision = TP / (TP+FP)    — of predicted positives, how many are correct?
Recall    = TP / (TP+FN)    — of actual positives, how many did we find?
F1        = 2 × (P × R) / (P + R)
```

When classes are imbalanced (99% negative), accuracy is misleading — a model predicting "always negative" gets 99% accuracy but is useless. Precision, recall, and F1 are more informative.

**ROC curve and AUC**: plot True Positive Rate vs False Positive Ra

## Illustrations

**Figure 197 • FIG 11.3-A**

### Linear Regression — Minimising Squared Residuals to Fit a Line



*Illustration: FIG 11.3-A | Linear Regression — Minimising Squared Residuals to Fit a Line [SVG diagram — see module-11-3-regression-classification.html]*

Linear regression finds the line ■ = wx + b that minimises the Mean Squared Error (MSE) — the average squared residual (vertical distance from each data point to the line). The closed-form normal equation (X■X)■¹X■y solves the optimum in one shot but requires inverting a matrix — O(n³) in the number of features. For large datasets, gradient descent iterates: compute the MSE gradient with respect to w and b, then take a small step in the downhill direction. Learning rate α controls step size — too large and it diverges, too small and it converges slowly. Linear regression is the foundation of a

**Figure 198 • FIG 11.3-B**

### Logistic Regression — Sigmoid Curve and Decision Boundary



*Illustration: FIG 11.3-B | Logistic Regression — Sigmoid Curve and Decision Boundary [SVG diagram — see module-11-3-regression-classification.html]*

Logistic regression extends linear regression to binary classification by passing the linear score z = wx + b through the sigmoid function σ(z) = 1/(1+e^{-z}), which squashes any real number to (0, 1) — a probability. The model predicts Class 1 when P(y=1|x) > 0.5, which corresponds to z > 0 — the decision boundary. The boundary is linear in feature space (a hyperplane). Training minimises binary cross-entropy loss: -[y log(p) + (1-y) log(1-p)], which penalises confident wrong predictions much more heavily than MSE would. Despite its name, logistic regression is a classification algorithm. It

**Figure 199 • FIG 11.3-C**

### Bias-Variance Tradeoff — Underfitting, Good Fit, Overfitting



*Illustration: FIG 11.3-C | Bias-Variance Tradeoff — Underfitting, Good Fit, Overfitting [SVG diagram — see module-11-3-regression-classification.html]*

The bias-variance tradeoff — the central challenge of supervised learning. An underfitting model (high bias) is too simple to capture the true pattern, producing high error on both training and test data. An overfitting model (high variance) is so complex it memorises training noise — it fits every training point perfectly but fails to generalise, producing low training error but high test error. The "good fit" lies between these extremes: a model that captures the true signal without fitting noise. Solutions to overfitting: regularisation (L1/L2 penalties on weights), dropout (neural networks

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 11.3 Regression and Classification: Linear Regression Logistic Regression Decision Boundary [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 11.3 Regression and Classification **Type:** Implementation + analysis + calculation **Estimated time:** 5060 minutes **Tools needed:** Python + NumPy --- Part A Linear Regression from Scratch Dataset: house prices. `X = [500, 750, 1000, 1250, 1500]` (sq ft), `y = [150, 200, 250, 325, 400]` (thousands). 1. Implement `linear_regression_gd(X, y, lr=0.0001, epochs=1000)` using gradient descent. Normalise X first (zero mean, unit variance). Return `(w, b)`. 2. What is the learned equation `y = w*x_norm + b`? Convert back to original scale. 3. Predict the price for a 1100 sq

# Module 11 4 Neural Networks

■# Module 11.4 — Neural Networks from Scratch

## The Problem

*Linear regression can fit a line.*

*Logistic regression can draw a straight boundary between classes.*

*But most real problems are not linearly separable.*

>

*An image classifier needs to recognise cats — not a feature that any linear function can capture.*

*A fraud detector needs to identify complex patterns in transaction sequences.*

*A language model needs to understand context and meaning.*

>

*These problems require learning **hierarchical, nonlinear representations** from data.*

*Neural networks are the general-purpose function approximators that do this.*

## The Story

### From Perceptrons to Deep Learning

**Frank Rosenblatt's** perceptron (1958) was the first algorithm that learned from data. It could classify linearly separable data. Marvin Minsky and Seymour Papert's book "Perceptrons" (1969) proved that a single-layer perceptron cannot compute XOR — an apparently simple function. This killed neural network funding for a decade.

The **multi-layer perceptron (MLP)** with backpropagation, popularised by **David Rumelhart, Geoffrey Hinton, and Ronald Williams** (1986), solved the XOR problem and sparked renewed interest. But training deep networks (many layers) remained elusive due to the **vanishing gradient problem**.

The deep learning revolution began around 2012 when **Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton** (AlexNet) won the ImageNet competition with error rates dramatically lower than all competitors — using a convolutional neural network with ReLU activations and GPU training. The field transformed overnight.

## The Concept

### A Single Neuron

A neuron computes: `output = f(w·x + b)` where:

• `x` = input vector

• `w` = weight vector (learned)

• `b` = bias (learned)

• `f` = activation function (introduces nonlinearity)

Without `f` (or with a linear `f`), a multi-layer network is equivalent to a single layer. Nonlinear activations are what make deep networks powerful.

```
import numpy as np

def sigmoid(z): return 1 / (1 + np.exp(-z))
def relu(z):    return np.maximum(0, z)
def tanh(z):    return np.tanh(z)
```

**Sigmoid**: output $\in$ (0,1), saturates for large |z| (gradient vanishes). Used in output layer for binary classification.

**ReLU (Rectified Linear Unit)**: `max(0, z)`. Does not saturate for positive `z`. Sparse activation (many zeros). The dominant activation for hidden layers since ~2012.

**Leaky ReLU**: `z if z>0 else 0.01*z` — prevents "dead neurons" (neurons that always output 0).

**Softmax**: converts a vector of scores to probabilities that sum to 1. Used in output layer for multi-class classification.

### Multi-Layer Perceptron (MLP)

A fully-connected network with L layers:

```
class NeuralNetwork:
    def __init__(self, layer_sizes):
        self.weights = []
        self.biases  = []
        for i in range(len(layer_sizes) - 1):
            n_in, n_out = layer_sizes[i], layer_sizes[i+1]
            # He initialisation for ReLU networks
```

```
            W = np.random.randn(n_in, n_out) * np.sqrt(2.0 / n_in)
            b = np.zeros(n_out)
            self.weights.append(W)
            self.biases.append(b)

    def forward(self, X):
        self.activations = [X]
        A = X
        for i, (W, b) in enumerate(zip(self.weights, self.biases)):
            Z = A @ W + b
            # ReLU for hidden layers, identity for last (or softmax)
            A = relu(Z) if i < len(self.weights) - 1 else Z
            self.activations.append(A)
        return A
```

**Dimensions for a 3-layer network on MNIST (784 inputs, 256, 128, 10):**

• Input: (batch_size, 784)

• W■: (784, 256), b■: (256,) → Layer 1 output: (batch_size, 256)

• W■: (256, 128), b■: (128,) → Layer 2 output: (batch_size, 128)

• W■: (128, 10), b■: (10,) → Output: (batch_size, 10)

• Total parameters: 784×256 + 256 + 256×128 + 128 + 128×10 + 10 = 234,378

### Loss Functions

**For regression**: Mean Squared Error (MSE):

```
def mse_loss(y_pred, y_true):
    return np.mean((y_pred - y_true) ** 2)
```

**For binary classification**: Binary Cross-Entropy:

```
def bce_loss(y_pred, y_true, eps=1e-15):
    y_pred = np.clip(y_pred, eps, 1-eps)
    return -np.mean(y_true * np.log(y_pred) + (1-y_true) * np.log(1-y_pred))
```

**For multi-class classification**: Categorical Cross-Entropy:

```
def softmax(Z):
    exp_Z = np.exp(Z - Z.max(axis=1, keepdims=True))
    return exp_Z / exp_Z.sum(axis=1, keepdims=True)

def ce_loss(logits, y_true):
    probs = softmax(logits)
    n = len(y_true)
    return -np.mean(np.log(probs[np.arange(n), y_true] + 1e-15))
```

### Backpropagation (Forward Look)

Backpropagation (detailed in Module 11.5) computes gradients of the loss with respect to every weight via the chain rule:

```
def backward(self, y_true, loss_grad):
    # Reverse pass through layers
    dA = loss_grad
```

```
    grads_W, grads_b = [], []
    for i in reversed(range(len(self.weights))):
        A_in = self.activations[i]
        dZ = dA * relu_derivative(self.activations[i+1]) if i < len(self.weights)-1 else dA
        dW = A_in.T @ dZ / len(y_true)
        db = dZ.mean(axis=0)
        dA = dZ @ self.weights[i].T
        grads_W.insert(0, dW)
        grads_b.insert(0, db)
    return grads_W, grads_b
```

### Training Loop

```
def train(net, X_train, y_train, epochs=100, lr=0.01, batch_size=32):
    n = len(X_train)
    for epoch in range(epochs):
        # Shuffle
        idx = np.random.permutation(n)
        X_train, y_train = X_train[idx], y_train[idx]

        total_loss = 0
        for i in range(0, n, batch_size):
            Xb = X_train[i:i+batch_size]
            yb = y_train[i:i+batch_size]

            # Forward pass
            logits = net.forward(Xb)
            loss   = ce_loss(logits, yb)
            total_loss += loss

            # Backward pass (gradient computation)
            probs = softmax(logits)
            probs[np.arange(len(yb)), yb] -= 1   # dL/d
```

### Illustrations

**Figure 200 • FIG 11.4-A**

**Feedforward Neural Network — Layers, Neurons, Weights, Activations**

*Illustration: FIG 11.4-A | Feedforward Neural Network — Layers, Neurons, Weights, Activations [SVG diagram — see module-11-4-neural-networks.html]*

Feedforward neural network with 3 input features, two hidden layers of 4 neurons each, and 2 outputs (e.g., binary class probabilities). Every neuron computes a weighted sum of its inputs (z = Σw■x■ + b) then applies an activation function (z → a). Information flows strictly left to right during forward pass. Connections between every pair of adjacent-layer neurons make this "fully connected" or "dense". The network has 3×4 + 4×4 + 4×2 = 36 weights plus 10 biases = 46 learnable parameters. During training, backpropagation computes gradients of the loss with respect to every parameter, and grad

**Figure 201 • FIG 11.4-B**

### Activation Functions — Sigmoid, Tanh, ReLU, Leaky ReLU

*Illustration: FIG 11.4-B | Activation Functions — Sigmoid, Tanh, ReLU, Leaky ReLU [SVG diagram — see module-11-4-neural-networks.html]*

Activation functions introduce nonlinearity — without them, a neural network of any depth would collapse to a single linear transformation. Sigmoid squashes to (0,1) — useful for final layer binary classification, but its gradient vanishes near 0 and 1, slowing deep network training. Tanh is zero-centred (outputs range -1 to 1), which helps gradient flow but still vanishes at extremes. ReLU (Rectified Linear Unit) max(0,z) is piecewise linear: no saturation for positive inputs, near-zero gradient cost, but "dead neurons" (units stuck at zero forever) are a risk. Leaky ReLU allows a small negat

**Figure 202 • FIG 11.4-C**

### Forward Pass — Layer-by-Layer Matrix Multiply + Activation

*Illustration: FIG 11.4-C | Forward Pass — Layer-by-Layer Matrix Multiply + Activation [SVG diagram — see module-11-4-neural-networks.html]*

The forward pass computes network output layer by layer. Each layer applies a linear transformation (z = Wx + b — a matrix multiply plus bias) followed by a nonlinear activation (a = ReLU(z)). The final layer uses softmax for multi-class classification, which normalises logits to a probability distribution summing to 1. The loss function (cross-entropy for classification, MSE for regression) measures the discrepancy between predicted and true values. Backpropagation then efficiently computes the gradient of the loss with respect to every weight in the network using the chain rule, enabling gra

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 11.4 Neural Networks: Network Architecture Forward Pass Training [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 11.4 Neural Networks from Scratch **Type:** Implementation + trace + analysis **Estimated time:** 5565 minutes **Tools needed:** Python + NumPy --- Part A XOR from Scratch XOR is not linearly separable: no single line can separate (0,0)0, (1,1)0 from (0,1)1, (1,0)1. 1. Prove XOR is not linearly separable: for any line `wx + wx + b = 0`, show that no choice of w, w, b correctly classifies all four points simultaneously. 2. A 2-2-1 network (2 inputs, 2 hidden neurons with sigmoid, 1 output with sigmoid) CAN learn XOR. Implement this network in NumPy with: - Manual weight

# L/z2 gradient of MSE

■# Module 11.5 — Backpropagation: Derived and Implemented

## The Problem

*A neural network has millions of parameters.*

*Training requires computing the gradient of the loss*

*with respect to every single parameter — millions of partial derivatives.*

>

*Computing each gradient separately by finite differences*

*would require millions of forward passes — impossibly slow.*

>

*Backpropagation computes all gradients in one backward pass*

*using the chain rule of calculus.*

*It is the algorithm that makes deep learning possible.*

*Understanding it deeply is the difference between*

*a user of ML tools and a builder of ML systems.*

## The Story

### A Rediscovery That Changed Everything

The backpropagation algorithm (backprop) is an efficient application of the chain rule for computing gradients in computational graphs. It was independently discovered multiple times:

• **Seppo Linnainmaa** (1970, Finland): first description of reverse-mode automatic differentiation in his master's thesis

• **Paul Werbos** (1974): applied it to neural networks in his PhD thesis

• **David Rumelhart, Geoffrey Hinton, Ronald Williams** (1986): the paper "Learning Representations by Back-propagating Errors" — the paper that launched modern neural network research

The 1986 paper was not the first derivation, but it clearly articulated why backprop was significant for learning distributed representations in multi-layer networks. It showed that networks with hidden layers could learn features that were not hand-engineered.

## The Concept

### The Chain Rule

If y = f(x) and x = g(t), then `dy/dt = (dy/dx) × (dx/dt)`.

For a composed function `L = loss(softmax(W■ × relu(W■ × x + b■) + b■))`:

Each partial derivative propagates backward through all operations that depend on it.

### Computational Graphs

Think of the network as a graph where:

• **Nodes** are operations (+, ×, relu, sigmoid, etc.)

• **Edges** carry values (forward pass) and gradients (backward pass)

```
x → [W■] → z■ → [ReLU] → a■ → [W■] → z■ → [Softmax] → p → [Loss] → L
```

**Forward pass**: compute values left to right, store intermediate values. **Backward pass**: compute gradients right to left using chain rule.

### Deriving Backprop for a Simple Network

**Architecture**: input x → linear W■ → ReLU → linear W■ → MSE loss

**Forward:**

```
z1 = x @ W1 + b1          # linear
a1 = relu(z1)              # activation
z2 = a1 @ W2 + b2          # output layer
L  = mse_loss(z2, y)       # loss
```

**Backward** (chain rule, right to left):

```
# ∂L/∂z2 — gradient of MSE
dL_dz2 = 2/n * (z2 - y)            # shape: (n, out_dim)

# ∂L/∂W2 = a1■ @ dL_dz2
dL_dW2 = a1.T @ dL_dz2             # shape: (hidden, out_dim)
dL_db2 = dL_dz2.mean(axis=0)

# ∂L/∂a1 = dL_dz2 @ W2■
dL_da1 = dL_dz2 @ W2.T             # shape: (n, hidden)

# ∂L/∂z1 — chain rule through ReLU: da1/dz1 = 1 if z1>0 else 0
dL_dz1 = dL_da1 * (z1 > 0)        # shape: (n, hidden)

# ∂L/∂W1 = x■ @ dL_dz1
dL_dW1 = x.T @ dL_dz1              # shape: (in_dim, hidden)
dL_db1 = dL_dz1.mean(axis=0)
```

**Update rules (SGD):**

```
W2 -= lr * dL_dW2
b2 -= lr * dL_db2
W1 -= lr * dL_dW1
b1 -= lr * dL_db1
```

### The Vanishing Gradient Problem

For a deep network with sigmoid activations, the gradient of the loss with respect to early layers involves many products of `sigmoid'(z) = σ(z)(1-σ(z))` — which has maximum value 0.25.

For a 10-layer network: gradient magnitude ≈ $0.25^{10} \approx 0.000001$. The early layers receive almost no gradient signal — they learn extremely slowly or not at all.

**Solutions:**

• **ReLU activations**: gradient is 1 for positive inputs, 0 otherwise. No squashing for positive values → gradients can propagate further.

• **Residual connections (ResNets)**: add skip connections `output = F(x) + x`. Gradients flow directly through the +x path — no vanishing.

• **Batch normalisation**: normalises layer inputs, preventing extreme values that push activations into saturation.

• **Careful initialisation**: He (for ReLU) or Xavier (for sigmoid/tanh) initialisation keeps activation variances roughly constant across layers.

### Automatic Differentiation (Autograd)

Modern frameworks (PyTorch, JAX, TensorFlow) implement **automatic differentiation** — the computer derives and computes gradients automatically from the forward computation:

```
import torch

x  = torch.tensor([[1.0, 2.0]], requires_grad=True)
W1 = torch.randn(2, 3, requires_grad=True)
b1 = torch.zeros(3, requires_grad=True)
W2 = torch.randn(3, 1, requires_grad=True)
b2 = torch.zeros(1, requires_grad=True)
y  = torch.tensor([[3.0]])

# Forward — PyTorch builds a computational graph
z1 = x @ W1 + b1
a1 = torch.relu(z1)
z2 = a1 @ W2 + b2
L  = ((z2 - y)**2).mean()

# Backward — traverse the graph, compute gradients
L.backward()

print(W1.grad)   # ∂L/∂W1 — computed automatically
print(W2.grad)   # ∂L/∂W2
```

PyTorch builds a dynamic computational graph during the forward pass and traverses it in reverse during `backward()`. This is exactly reverse-mode automatic differentiation (Linnainmaa's algorithm from 1970).

### The Complete Training Loop (NumPy from Scratch)

```
class TwoLayerNet:
    def __init__(self, in_d, h_d, out_d):
        self.W1 = np.random.randn(in_d, h_d)  * np.sqrt(2/in_d)
        self.b1 = np.zeros(h_d)
        self.W2 = np.random.randn(h_d, out_d) * np.sqrt(2/h_d)
        self.b2 = np.zeros(out_d)
```

```
    def forward(self, X):
        self.X  = X
        self.z1 = X  @ self.W1 + self.b1
        self.a1 = np.maximum(0, self.z1)    # ReLU
        self.z2 = self.a1 @ self.W2 + self.b2
        return self.z2

    def backward(self, y_true, lr=0.01):
        n = len(y_true)
        dz2 = 2/n * (self.z2 - y_true)          # ∂L/∂z2

        dW2 = self.a1.T @ dz2                    # ∂L/∂W2
        db2 = dz2.mean(axis=0)
        da1 = dz2 @ self.W2.T
        dz1 = da1 * (self.z1 > 0)               # chain rule through ReLU

        dW1 = self.X.T @ dz1
        db1 = dz1.me
```

## Illustrations

**Figure 203 • FIG 11.5-A**

### Gradient Descent — Descending the Loss Surface to the Minimum

*Illustration: FIG 11.5-A | Gradient Descent — Descending the Loss Surface to the Minimum [SVG diagram — see module-11-5-backpropagation.html]*

Gradient descent iteratively moves model parameters in the direction of steepest descent of the loss function. At each step: compute the gradient (the direction of steepest ascent), then take a small step in the opposite direction. Starting from w■, each step reduces the loss until convergence at the minimum w*. The learning rate α controls step size — the most sensitive hyperparameter in deep learning. Batch gradient descent computes the exact gradient over all training data (accurate but slow). Stochastic gradient descent uses one sample at a time (fast but very noisy). Mini-batch gradient d

**Figure 204 • FIG 11.5-B**

### Backpropagation — Chain Rule Gradient Flow through Computational Graph

*Illustration: FIG 11.5-B | Backpropagation — Chain Rule Gradient Flow through Computational Graph [SVG diagram — see module-11-5-backpropagation.html]*

Backpropagation is the chain rule applied systematically to a computational graph. Forward pass: compute and cache all intermediate values. Backward pass: starting from the loss, propagate gradients backward using the chain rule — at each node, multiply the upstream gradient by the node's local derivative to get the downstream gradient. The gradient of the loss with respect to weight w is computed as the product of all local derivatives along the path from w to the loss. One backward pass computes gradients for all parameters simultaneously — this is the key efficiency insight. Without backpro

**Figure 205 • FIG 11.5-C**

### Learning Rate Effects — Too Small, Optimal, Too Large

*Illustration: FIG 11.5-C | Learning Rate Effects — Too Small, Optimal, Too Large [SVG diagram — see module-11-5-backpropagation.html]*

Learning rate $\alpha$ is the most impactful hyperparameter in deep learning training. Too small: the model converges but takes thousands more epochs — gradients are so tiny the parameters barely move. Too large: the loss oscillates wildly or diverges — each update overshoots the minimum, landing on the other side of the loss bowl. Optimal: loss decreases smoothly and converges to a low value. In practice, learning rate schedules vary $\alpha$ during training: start with a warmup (gradually increase from 0 to peak $\alpha$), then apply cosine annealing or step decay to slowly reduce $\alpha$, allowing coarse exploration

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 11.5 Backpropagation: Chain Rule Computation Graph Gradient Descent [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 11.5 Backpropagation: Derived and Implemented **Type:** Derivation + implementation + verification **Estimated time:** 5565 minutes **Tools needed:** Python + NumPy --- Part A Chain Rule Derivation A single linear layer (no activation): ` = Xw + b`, loss `L = (1/n)|| - y||`. 1. Derive `L/` analytically. Show every step. 2. Using the chain rule, derive `L/w = L/ /w`. What is `/w`? Write the final expression for `L/w` in matrix form. 3. Derive `L/b`. What is it in vector form? 4. Verify your gradients numerically (gradient checking): compute `L/w[i]` as `(L(w + e) - L(w

LEVEL 12

# Advanced AI

LLMs, Embeddings, RAG, Agents, Ethics, and Safety

# Input: [The, cat, sat, on]

■# Module 12.1 — Large Language Models: Architecture and Training

## The Problem

*In 2017, a team at Google published "Attention Is All You Need."*

*The transformer architecture they described — without convolutions or recurrence —*

*became the foundation of GPT, BERT, Claude, Gemini, LLaMA,*

*and virtually every large language model in existence.*

>

*How does a system trained to predict the next word in a document*

*learn to write code, translate languages, answer questions,*

*and reason about complex problems?*

>

*Understanding the transformer architecture — from attention mechanisms*

*to the training process — is the prerequisite for working with*

*any modern AI system at a technical level.*

## The Story

### The Road to Transformers

**Recurrent Neural Networks (RNNs)** and their variants — LSTMs (Hochreiter & Schmidhuber, 1997) and GRUs — dominated sequence modelling from the 2000s. They processed tokens one at a time, maintaining a hidden state. The problem: long-range dependencies were hard to learn (vanishing gradients even with LSTM), and sequential processing prevented parallelisation.

**Sequence-to-sequence models** (Sutskever, Vinyals, Le, 2014) added an encoder-decoder structure with attention — the decoder could "attend" to different parts of the encoder's output. This was the breakthrough for machine translation.

**"Attention Is All You Need"** (Vaswani et al., Google, 2017) eliminated recurrence entirely. Pure attention. Every token attends to every other token in the sequence simultaneously. Training is fully parallelisable. The result: dramatically larger models could be trained.

**GPT-1** (OpenAI, 2018): the first demonstration that large-scale pretraining on text + fine-tuning worked for many tasks. 117 million parameters. GPT-2 (2019): 1.5B parameters. GPT-3 (2020): 175B parameters, few-shot learning without fine-tuning. GPT-4 (2023): architecture undisclosed, dramatically better. The scale at which capabilities emerged was surprising even to the researchers.

## The Concept

### Tokenisation

Before processing text, the model converts it to tokens — integer IDs from a fixed vocabulary.

**BPE (Byte Pair Encoding)**: the dominant tokenisation scheme for LLMs. Start with individual bytes, iteratively merge the most frequent adjacent pair:

```
"the cat sat" → ["the", " cat", " sat"]
               → [464, 3797, 4992]  (GPT-4 token IDs)
```

GPT-4 uses ~100,000 tokens. Common words are single tokens ("the" → 1 token). Uncommon or non-English words may split into multiple tokens ("Königsberg" → ["K", "ön", "igs", "berg"] → 4 tokens).

**Why it matters**: token count affects both cost and context length. A 4,096-token context window is roughly 3,000 words of English.

### The Transformer Architecture

```
Input tokens
     ↓
[Token Embeddings + Positional Embeddings]
     ↓
[Transformer Block 1]
[Transformer Block 2]
...
[Transformer Block N]
     ↓
[Linear + Softmax]
     ↓
Next token probabilities
```

**Each Transformer Block:**

```
Input
  ↓
[Multi-Head Self-Attention]
  ↓ (+ residual connection)
[Layer Normalisation]
  ↓
[Feed-Forward Network (MLP)]
  ↓ (+ residual connection)
[Layer Normalisation]
  ↓
Output
```

### Self-Attention — The Core Mechanism

Self-attention allows every token to "look at" every other token and incorporate relevant context.

For each token, compute three vectors via learned linear projections:

• **Query (Q)**: "what am I looking for?"

• **Key (K)**: "what do I contain?"

• **Value (V)**: "what do I provide if attended to?"

```
import numpy as np

def self_attention(X, W_Q, W_K, W_V):
    # X shape: (seq_len, d_model)
    Q = X @ W_Q    # (seq_len, d_k)
    K = X @ W_K    # (seq_len, d_k)
    V = X @ W_V    # (seq_len, d_v)

    # Attention scores: how much each token attends to each other
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)    # (seq_len, seq_len)

    # For causal (decoder) models: mask future tokens
    # scores[i, j] = -inf if j > i  (can't attend to future)

    # Softmax over keys for each query
    weights = softmax(scores, axis=-1)    # (seq_len, seq_len)

    # Weighted sum of values
    output = weights @ V    # (seq_len, d_v)
    return output
```

**Multi-head attention**: run h parallel attention heads with different W_Q, W_K, W_V projections. Each head can attend to different aspects of the input. Concatenate and project:

```
def multi_head_attention(X, num_heads, W_Q, W_K, W_V, W_O):
    # Split into heads, apply attention, concatenate
    head_dim = d_model // num_heads
    heads = []
    for h in range(num_heads):
        # Each head has its own projections
        out = self_attention(X, W_Q[h], W_K[h], W_V[h])
        heads.append(out)
    concat = np.concatenate(heads, axis=-1)   # (seq_len, d_model)
    return concat @ W_O                        # (seq_len, d_model)
```

### Positional Encoding

Self-attention is order-invariant (shuffling tokens gives the same attention scores). Positional encodings inject position information:

**Sinusoidal positional encoding** (original Transformer):

```
def positional_encoding(seq_len, d_model):
    PE = np.zeros((seq_len, d_model))
    for pos in range(seq_len):
        for i in range(0, d_model, 2):
            PE[pos, i]   = np.sin(pos / 10000**(i / d_model))
            PE[pos, i+1] = np.cos(pos / 10000**(i / d_model))
    return PE
```

Modern LLMs (GPT-NeoX, LLaMA) use **RoPE (Rotary Positional Embedding)** — relative position encoding that generalises better to sequences longer than seen during training.

### Pretraining: Next-Token Prediction

LLMs are trained on the **causal language modelling** objective: given tokens 1..t, predict token t+1. For a sequence of N tokens, one forward pass generates N training examples (teacher forcing):

```
# Input:  [The, cat, sat, on]
# Target: [cat, sat, on, the]   (each token predicts the next)

# Cross-entropy loss over vocabulary for each position
loss = -sum(log P
```

### Illustrations

**Figure 206 • FIG 12.1-A**

**Tokenisation — Text to Token IDs via Byte Pair Encoding**

*Illustration: FIG 12.1-A | Tokenisation — Text to Token IDs via Byte Pair Encoding [SVG diagram — see module-12-1-llm-architecture.html]*

Tokenisation converts raw text into integer sequences that the model processes. Byte Pair Encoding (BPE) starts with a character-level vocabulary and iteratively merges the most frequent adjacent pair of tokens until a target vocabulary size is reached (~50,000 for GPT-2). Common words like "the" map to a single token. Rare words and neologisms are split into subword pieces ("transform" + "er"). This balances vocabulary size against sequence length — pure character tokenisation produces sequences 4-10× longer, dramatically increasing attention computation. Token IDs are the actual inputs to th

**Figure 207 • FIG 12.1-B**

**Transformer Decoder Block — Attention, FFN, Residuals, Layer Norm**

*Illustration: FIG 12.1-B | Transformer Decoder Block — Attention, FFN, Residuals, Layer Norm [SVG diagram — see module-12-1-llm-architecture.html]*

Transformer decoder block — the architectural unit stacked N times in GPT-style language models. Pre-layer normalisation stabilises training. Causal self-attention (masked so each token can only attend to previous tokens) captures context. The feed-forward network (two linear layers with GELU activation, expanding to 4× the model dimension internally) applies position-wise transformations. Residual connections ("skip connections") allow gradients to flow directly through the network without vanishing, enabling stacks of 96+ blocks. The final linear layer projects to vocabulary size (~50,000),

**Figure 208 • FIG 12.1-C**

**Self-Attention Mechanism — Q, K, V Matrices and Attention Scores**

*Illustration: FIG 12.1-C | Self-Attention Mechanism — Q, K, V Matrices and Attention Scores [SVG diagram — see module-12-1-llm-architecture.html]*

Self-attention is the core operation of the Transformer. Each input token is projected into three vectors: Query (what information am I looking for?), Key (what information do I contain?), and Value (what information should I provide?). Attention scores are computed as dot products of queries with all keys, scaled by √d■ to prevent extremely small gradients, then normalised with softmax to form attention weights. The output is a weighted sum of value vectors — each token's output is a mixture of all other tokens' values, weighted by how relevant those tokens are. Causal masking sets attention

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 12.1 LLM Architecture: Tokenisation Self-Attention Transformer Stack [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 12.1 Large Language Models: Architecture and Training **Type:** Derivation + implementation + analysis **Estimated time:** 5565 minutes **Tools needed:** Python + NumPy --- Part A Self-Attention from Scratch Implement **scaled dot-product attention** for a 4-token sequence. Given: ```python import numpy as np 4 tokens, d_model=4, d_k=4 Q = np.array([[1,0,1,0],[0,1,0,1],[1,1,0,0],[0,0,1,1]], dtype=float) (4,4) K = np.array([[1,0,0,1],[0,1,1,0],[1,1,0,0],[0,0,1,1]], dtype=float) (4,4) V = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]], dtype=float) (4,4) ```

# Text embeddings

■# Module 12.2 — Embeddings and Vector Databases

## The Problem

*Computers understand numbers. Text, images, audio, code — all must be converted to numbers before a computer can process them.*

>

*But a simple conversion (character codes, pixel values) misses semantic structure.*

*"cat" and "feline" are the same concept — but their character codes share nothing.*

*"cat" and "can" differ by one character — but they are completely different concepts.*

>

***Embeddings** solve this: map objects (words, sentences, images, code) to vectors*

*such that semantically similar objects are geometrically close.*

***Vector databases** provide efficient similarity search over millions of these vectors.*

---

## The Story

### Word2Vec — Meaning from Context

**Word2Vec** (Mikolov et al., Google, 2013) demonstrated that word meaning could be captured by training a shallow neural network to predict words from their context in a large text corpus ("you shall know a word by the company it keeps" — J.R. Firth, 1957).

The emergent property was remarkable: the resulting vectors supported arithmetic: `king - man + woman ≈ queen`. Geographic capitals, verb tenses, comparative adjectives — the structure of meaning was encoded in geometric relationships between vectors.

**GloVe** (Stanford, 2014) and **FastText** (Facebook, 2016) improved on Word2Vec. But word-level embeddings had a fundamental limitation: "bank" had one vector regardless of whether it meant a financial institution or a river bank.

**BERT** (Google, 2018) introduced contextual embeddings — each token's embedding depends on all surrounding tokens. "Bank" has a different vector in "river bank" and "bank transfer." These contextual embeddings, derived from transformers (Module 12.1), are now the foundation of all embedding models.

---

## The Concept

### What Is an Embedding?

An embedding is a function that maps an object to a dense vector in ■■ where d is typically 128–4096 dimensions.

The critical property: **semantic similarity ≈ geometric proximity**. Objects with similar meaning have vectors that are close (high cosine similarity).

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')  # 384 dimensions

# Text embeddings
e1 = model.encode("The quick brown fox")
e2 = model.encode("A fast auburn canine")    # similar meaning
e3 = model.encode("The stock market fell")   # different topic

def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

```
print(cosine_sim(e1, e2))   # ≈ 0.78 (similar)
print(cosine_sim(e1, e3))   # ≈ 0.12 (different)
```

### Types of Embeddings

**Word embeddings**: map individual words to vectors. Word2Vec, GloVe, FastText. Useful for token-level tasks.

**Sentence/document embeddings**: map entire sentences or paragraphs to a single vector. Sentence-BERT, OpenAI Ada, Cohere Embed. Used for semantic search, classification, clustering.

**Multimodal embeddings**: map different modalities (text and images) to the same vector space. **CLIP** (OpenAI, 2021) — trained on 400M image-text pairs, can compare images and text directly. Enables: image search with text queries, text-to-image generation guidance.

**Code embeddings**: map code snippets to vectors capturing functional semantics. OpenAI code-embedding, CodeBERT. Enables: code search ("find all functions that sort a list"), similar code detection, documentation search.

### Vector Similarity Search

Given a query vector q and a database of n vectors {v■, ..., v■}, find the k most similar vectors.

**Brute force**: compute cosine similarity with all n vectors. O(n×d) per query. Exact but slow for large n.

**Approximate Nearest Neighbour (ANN)**: trade small accuracy loss for massive speed-up. Used in all production vector databases.

**FAISS** (Facebook AI Similarity Search):

```
import faiss
import numpy as np

# Build an index
d = 384            # embedding dimension
n = 1_000_000      # number of vectors

embeddings = np.random.randn(n, d).astype('float32')
embeddings /= np.linalg.norm(embeddings, axis=1, keepdims=True)   # normalise

# IVF (Inverted File) index: cluster vectors, search only relevant clusters
nlist = 100        # number of clusters
quantiser = faiss.IndexFlatIP(d)   # exact search within cluster
index = faiss.IndexIVFFlat(quantiser, d, nlist, faiss.METRIC_INNER_PRODUCT)
index.train(embeddings)            # cluster the data
index.add(embeddings)              # add all vectors

# Search
query = np.random.randn(1, d).astype('float32')
query /= np.linalg.norm(query)

index.nprobe = 10                  # search 10 clusters (speed/accuracy tradeoff)
distances, indices = index.search(query, k=5)
# Returns indices of 5 nearest neighbours in ~milliseconds
```

**HNSW (Hierarchical Navigable Small World)**: a graph-based ANN algorithm. Vectors are nodes; edges connect nearby vectors in a hierarchy of layers. Search navigates the graph greedily from the top layer. Very fast (~1ms for millions of vectors), good accuracy. Used by Weaviate, Qdrant.

### Vector Databases

Production systems managing millions of embeddings use purpose-built **vector databases**:

| Database | Backend | Hosted | Open source | |---|---|---|---| | Pinecone | Proprietary | Yes | No | | Weaviate | HNSW | Self/hosted | Yes | | Qdrant | HNSW | Self/hosted | Yes | | Chroma | HNSW | Self | Yes | | pgvector | IVF/HNSW | PostgreSQL | Yes | | FAISS | IVF/PQ | Self | Yes (library) |

They provide: vector storage + retrieval, metadata filtering, hybrid search (dense + sparse), real-time updates.

```
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct

client = QdrantClient(":memory:")  # in-memory for demo

client.create_collection(
    collection_name="docs",
    vectors_config=VectorParams(size=384, distance=Distance.COSINE)
)

# Insert vectors with metadata
client.upsert("docs", [
    PointStruct(
```

### Illustrations

**Figure 209 • FIG 12.2-A**

**Word Embedding Space — Semantic Relationships Encoded as Geometry**

*Illustration: FIG 12.2-A | Word Embedding Space — Semantic Relationships Encoded as Geometry [SVG diagram — see module-12-2-embeddings.html]*

Word embeddings map tokens to dense vectors in a high-dimensional space where geometric relationships encode semantic meaning. Words with similar meanings cluster together — "cat", "dog", "tiger" form an animals cluster. The remarkable property discovered with word2vec (Mikolov 2013): vector arithmetic preserves semantic relationships — king − man + woman ≈ queen. This works because the "royalty" and "gender" concepts are encoded as consistent directions in embedding space. Modern contextual embeddings (BERT, GPT) produce different vectors for the same word in different contexts: "bank" near "

**Figure 210 • FIG 12.2-B**

### Cosine Similarity — Angular Distance as Semantic Similarity

*Illustration: FIG 12.2-B | Cosine Similarity — Angular Distance as Semantic Similarity [SVG diagram — see module-12-2-embeddings.html]*

Cosine similarity measures the angle between two vectors, ignoring magnitude. cos(θ) = (a · b) / (|a| |b|). Range: +1 (same direction, identical meaning), 0 (perpendicular, unrelated), -1 (opposite direction, antonym or unrelated). Cosine similarity is preferred over Euclidean distance for embeddings because magnitude is often irrelevant — a long document and a short document about the same topic should be considered similar even though their embedding vectors have different magnitudes. Near-duplicates like "cat"/"feline" score above 0.9. Related concepts like "cat"/"dog" score 0.6–0.8. Unrela

**Figure 211 • FIG 12.2-C**

### Vector Database Architecture — HNSW Index for Approximate Nearest Neighbour

*Illustration: FIG 12.2-C | Vector Database Architecture — HNSW Index for Approximate Nearest Neig [SVG diagram — see module-12-2-embeddings.html]*

Vector databases store high-dimensional embeddings and support fast nearest-neighbour search. The HNSW (Hierarchical Navigable Small World) index organises vectors in a multi-layer graph: the top layers are sparse, providing "highway" connections for long-distance jumps; the bottom layer contains all vectors with local connections. Search starts at the top layer and greedily navigates toward the query vector, descending to denser layers as it converges. This achieves O(log n) approximate nearest neighbour search — finding the top-K most similar vectors among 1 million in under 10ms with ~99% r

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 12.2 Embeddings and Vector Databases: Word Embeddings Cosine Similarity Vector Search [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 12.2 Embeddings and Vector Databases **Type:** Implementation + calculation + analysis **Estimated time:** 5060 minutes **Tools needed:** Python + NumPy (sentence-transformers optional) --- Part A Cosine Similarity from Scratch 1. Implement `cosine_similarity(a, b)` without using any ML library. Only NumPy for array operations is allowed. 2. Given these vectors: ```python cat = [0.9, 0.1, -0.3, 0.7, -0.2] feline = [0.8, 0.2, -0.4, 0.6, -0.1] dog = [0.7, -0.2, 0.5, 0.1, 0.4] car = [-0.1, 0.9, 0.8, -0.3, 0.6] ``` Compute all pairwise cosine simil

# Cosine similarity between first two (semantically similar)

■# Module 12.3 — Retrieval-Augmented Generation

## The Problem

*A large language model knows everything in its training data — up to its cutoff date.*

*But it cannot:*

*- Access information created after training*

*- Look up specific private documents you have*

*- Cite its sources reliably*

*- Know about your company's internal policies*

>

***RAG (Retrieval-Augmented Generation)** solves this by combining*

*a retrieval system with a language model:*

*instead of relying entirely on parametric memory (weights),*

*the model retrieves relevant context at query time and uses it to answer.*

## The Story

### From Parametric to Non-Parametric Knowledge

Early neural networks had no external memory — all knowledge was encoded in parameters. **Neural Turing Machines** (Graves et al., Google DeepMind, 2014) added an external memory with differentiable read/write heads — inspired by Turing's theoretical computing model.

**RAG** (Lewis et al., Facebook AI Research, 2020) applied a simpler but practical approach: use a non-parametric retrieval system (dense passage retrieval with FAISS) combined with a generative model. The model sees retrieved documents concatenated to the question. It was the first major demonstration that retrieval could dramatically improve factuality.

With the commercial success of ChatGPT (2022), RAG became the dominant architecture for enterprise AI applications: internal knowledge bases, customer support chatbots, code assistance with private repositories, legal document search.

## The Concept

### RAG System Architecture

```
User query
```

```
    ↓
[Query Encoder]          ← embedding model
    ↓
Query embedding vector
    ↓
[Vector Similarity Search]  ← FAISS / Pinecone / Weaviate
    ↓
Top-k relevant documents
    ↓
[Prompt Assembly]
    ↓
"Answer based on these documents: {docs}\n\nQuestion: {query}"
    ↓
[LLM (GPT-4, Claude, Llama, etc.)]
    ↓
Response (grounded in retrieved documents)
```

### Embeddings and Vector Search

A **text embedding** is a dense vector representation of text that captures semantic meaning. Texts with similar meaning have similar vectors (high cosine similarity).

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

texts = [
    "The cat sat on the mat.",
    "A feline rested on a rug.",
    "The stock market rose today.",
]

embeddings = model.encode(texts)    # shape: (3, 384)

# Cosine similarity between first two (semantically similar)
cos_01 = np.dot(embeddings[0], embeddings[1]) / (
         np.linalg.norm(embeddings[0]) * np.linalg.norm(embeddings[1]))
# ≈ 0.85 (very similar)

# Cosine similarity between first and third (different topic)
cos_02 = np.dot(embeddings[0], embeddings[2]) / (
         np.linalg.norm(embeddings[0]) * np.linalg.norm(embeddings[2]))
# ≈ 0.15 (very different)
```

### Building a RAG System from Scratch

**Step 1: Indexing** — run once when documents change

```
import faiss
from sentence_transformers import SentenceTransformer

def build_index(documents):
    model = SentenceTransformer('all-MiniLM-L6-v2')
    embeddings = model.encode(documents, convert_to_numpy=True)
```

```
    embeddings = embeddings / np.linalg.norm(embeddings, axis=1, keepdims=True)  # normalise

    # FAISS index for cosine similarity (using inner product on normalised vectors)
    dim = embeddings.shape[1]
    index = faiss.IndexFlatIP(dim)
    index.add(embeddings)
    return index, embeddings

documents = [
    "Python was created by Guido van Rossum in 1991.",
    "The GIL in CPython prevents true parallelism for CPU-bound threads.",
    "NumPy provides efficient array operations built on LAPACK/BLAS.",
    # ... thousands more documents
]
index, embeddings = build_index(documents)
```

**Step 2: Retrieval** — run for each query

```
def retrieve(query, index, documents, model, k=3):
    query_embedding = model.encode([query], convert_to_numpy=True)
    query_embedding /= np.linalg.norm(query_embedding, axis=1, keepdims=True)

    # Find k nearest neighbours
    scores, indices = index.search(query_embedding, k)

    results = []
    for score, idx in zip(scores[0], indices[0]):
        results.append({
            'document': documents[idx],
            'score': float(score),
        })
    return results
```

**Step 3: Generation** — use retrieved documents as context

```
import openai

def rag_answer(query, retrieved_docs):
    context = "\n\n".join([f"[{i+1}] {d['document']}" for i, d in enumerate(retrieved_docs)])

    prompt = f"""You are a helpful assistant. Answer the question using ONLY the provided context.
If the context does not contain the answer, say "I don't have information about that."

Context:
{context}

Question: {query}

Answer:"""

    response = openai.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content
```

```
# Full pipeline
query = "Does Python have a GIL?"
retrieved = retrieve(query, index, documents, embedding_model)
answer = rag_answer(query, retrieved)
```

### Chunking Strategy

Documents must be split into chunks before embedding. Chunking strategy dramatically affects retrieval quality:

**Fixed-size chunks**: split every N characters (simple, fast, breaks mid-sentence).

**Sentence/paragraph chunks**: split on sentence boundaries (better coherence, variable size).

**Sliding window**: overlapping chunks so context boundaries don't cause misses:

```
def chunk_text(text, chunk_size=500, overlap=50):
    words = text.split()
    chunks = []
    for i in range(0, len(words), chunk_size - overlap):
        chunk = ' '.join(words[i:i + chunk_size])
        chunks.append(chunk)
        if i + chunk_size >= len(words): break
    return chunks
```

**Semantic chunking**: split when topic changes (detected by embedding similarity drop). More expensive but better for heterogeneous documents.

### RAG

### Illustrations

**Figure 212 • FIG 12.3-A**

**RAG Pipeline — Document Chunking, Embedding, Retrieval, Generation**

*Illustration: FIG 12.3-A | RAG Pipeline — Document Chunking, Embedding, Retrieval, Generation [SVG diagram — see module-12-3-rag.html]*

Retrieval-Augmented Generation (RAG) grounds LLM responses in a document corpus. Offline: documents are chunked (typically 256–512 tokens with 50-token overlap between chunks), embedded into vector representations, and stored in a vector database. Online: the user's question is embedded with the same model, the top-K most similar chunks are retrieved by vector similarity, and these chunks are injected into the LLM's prompt as context. The LLM generates an answer conditioned on both the question and the retrieved context — dramatically reducing hallucination and enabling citation of sources. RA

**Figure 213 • FIG 12.3-B**

### Semantic Search vs Keyword Search — Understanding vs Matching

*Illustration: FIG 12.3-B | Semantic Search vs Keyword Search — Understanding vs Matching [SVG diagram — see module-12-3-rag.html]*

Semantic search vs keyword search on the same query. BM25 (keyword search) scores documents by exact word frequency — it finds documents containing "code" and "faster" but misses "performance optimisation", "profiling", and "algorithmic complexity" — all directly relevant, zero keyword overlap. Vector semantic search embeds both the query and documents into the same semantic space, finding documents by meaning rather than word match. The query "how do I make my code faster" and the document "reducing algorithmic complexity" share no words but are semantically close — the embedding model learne

**Figure 214 • FIG 12.3-C**

### RAG vs Fine-tuning vs Prompting — When to Use Each Approach

*Illustration: FIG 12.3-C | RAG vs Fine-tuning vs Prompting — When to Use Each Approach [SVG diagram — see module-12-3-rag.html]*

Three strategies for incorporating domain knowledge into LLM responses. Prompting (in-context learning) injects knowledge directly into the prompt — free to update but limited by context window length and per-call cost. RAG retrieves relevant chunks at runtime from an external knowledge base — knowledge can be updated by changing the indexed documents, the LLM weights never change. Fine-tuning trains the model weights on domain data, baking knowledge into parameters — powerful for behavioural changes (tone, format, domain vocabulary) but expensive, requires retraining when knowledge updates, a

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 12.3 Retrieval-Augmented Generation (RAG): RAG Pipeline Chunking Grounded Generation [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 12.3 Retrieval-Augmented Generation **Type:** Implementation + analysis + evaluation **Estimated time:** 5060 minutes **Tools needed:** Python + sentence-transformers --- Part A Minimal RAG System 1. Build a minimal RAG system in Python that: - Indexes 10 text documents (hardcoded, no external API needed) - Embeds them using `sentence-transformers` (or mock with random vectors for testing) - For a query, retrieves the top-2 most similar documents by cosine similarity - Assembles and prints a prompt: `"Answer based on: {docs}\n\nQuestion: {query}"` ```python doc

# Module 12 4 Ai Agents

■# Module 12.4 — AI Agents: Planning, Tool Use, and Memory

## The Problem

*A large language model can answer a question.*

*But it cannot: browse the web to find current information,*

*write code and run it to check if it works,*

*send an email, query a database, or interact with an API.*

>

*An **AI agent** extends the LLM with the ability to take actions:*

*use tools, execute code, interact with external systems,*

*and — most importantly — plan a sequence of actions to achieve a goal.*

>

*This is the difference between an assistant that tells you how to do something*

*and one that actually does it.*

## The Story

### From Chatbots to Agents

The ELIZA chatbot (Weizenbaum, MIT, 1966) simulated a therapist through simple pattern matching. Users ascribed understanding to it despite its mechanical simplicity — the first demonstration of how humans project intelligence onto text.

**ReAct** (Reason + Act, Yao et al., 2022) was a key paper showing that LLMs could interleave reasoning (thinking through a problem) with action (using tools to gather information). The model writes its "thoughts" in text, then calls a tool, observes the result, thinks further. This simple pattern enabled solving problems that pure generation couldn't.

**AutoGPT** (2023) was an early attempt at autonomous agents — GPT-4 calling itself recursively, maintaining a task list, browsing the web, and writing files. It attracted enormous attention but also demonstrated the failure modes: lack of planning, context window limits, cascading errors.

**LangChain, LlamaIndex, Claude's tool use, OpenAI Function Calling** — all subsequent frameworks for building agents have converged on the same core pattern: give the LLM a set of tools, let it decide which to call and when.

## The Concept

### The ReAct Loop

```
User Goal
    ↓
[Thought]: Reason about what to do next
    ↓
[Action]: Call a tool (search, code execution, API call)
    ↓
[Observation]: Receive tool output
    ↓
(repeat Thought → Action → Observation until goal achieved)
    ↓
[Final Answer]
```

**Example trace:**

```
Goal: What is the current Python version and what was the main change in that release?

Thought: I need to find the current Python version. I'll search for it.
Action: web_search("current Python version 2026")
Observation: Python 3.13 was released in October 2024. Python 3.14 is in development.

Thought: I have the version. Now I need to find the main changes in Python 3.13.
Action: web_search("Python 3.13 major changes features")
Observation: Python 3.13 introduced experimental free-threaded mode (no GIL), improved error messages,

Thought: I have all the information needed.
Final Answer: The current stable Python version is 3.13. Its major changes include experimental GIL-fr
```

### Tool Definition and Function Calling

Modern LLM APIs support **function calling** — you describe tools in a structured format, the model chooses which to call and with what arguments, you execute the tool and return the result:

```
import openai, json

tools = [
    {
        "type": "function",
        "function": {
            "name": "web_search",
            "description": "Search the web for current information",
            "parameters": {
                "type": "object",
                "properties": {
                    "query": {"type": "string", "description": "The search query"}
                },
                "required": ["query"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "execute_python",
            "description": "Execute Python code and return the output",
            "parameters": {
```

```
                "type": "object",
                "properties": {
                    "code": {"type": "string", "description": "Python code to execute"}
                },
                "required": ["code"]
            }
        }
    }
]

def run_agent(goal, tools, tool_executors, max_steps=10):
    messages = [{"role": "user", "content": goal}]

    for step in range(max_steps):
        response = openai.chat.completions.create(
            model="gpt-4o",
            messages=messages,
            tools=tools,
            tool_choice="auto"
        )

        msg = response.choices[0].message
        messages.append(msg)

        # If no tool call: model has finished reasoning
        if not msg.tool_calls:
            return msg.content

        # Execute each tool call
        for tool_call in msg.tool_calls:
            name = tool_call.function.name
            args = json.loads(tool_call.function.arguments)
            result = tool_executors[name](**args)

            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": str(result)
            })

    return "Max steps reached"
```

### Memory Types

An agent that forgets context is limited. Different tasks require different memory:

**In-context memory**: everything in the current prompt. Fast, exact, but limited to context window (4K–200K tokens depending on model).

**Episodic memory**: store past conversations and retrieve relevant ones via embedding similarity:

```
def remember(agent, interaction):
    embedding = embed(interaction['summary'])
    memory_db.upsert(embedding, interaction)

def recall(agent, query, k=3):
```

```
    query_emb = embed(query)
    return memory_db.search(query_emb, k)
```

**Semantic memory**: distilled facts extracted from experience. "User prefers Python over JavaScript." "This codebase uses pytest for testing." Stored and retrieved as structured facts.

**Procedural memory**: how to do specific tasks. Often stored as few-shot examples retrieved from a library.

### Planning

For complex multi-step goals, ad-hoc tool use is insufficient. Planning approaches:

**Chain of Thought (CoT)**: instruct the model to reason step

### Illustrations

**Figure 215 • FIG 12.4-A**

**AI Agent Architecture — Perception, Memory, Planning, Action Loop**

*Illustration: FIG 12.4-A | AI Agent Architecture — Perception, Memory, Planning, Action Loop [SVG diagram — see module-12-4-ai-agents.html]*

An AI agent uses an LLM as its reasoning core, augmented with tools (external capabilities) and memory. The agent loop: perceive the environment (user request, sensor data, prior action results), retrieve relevant memories and knowledge, reason with the LLM about what to do next, act by calling tools (web search, code execution, APIs, file I/O), observe the results, and repeat until the task is complete. Short-term memory is the LLM's context window — limited in size. Long-term memory is a vector database or external storage, accessed via retrieval. This architecture allows an LLM (which is fu

**Figure 216 • FIG 12.4-B**

**ReAct Pattern — Interleaved Reasoning and Action**

*Illustration: FIG 12.4-B | ReAct Pattern — Interleaved Reasoning and Action [SVG diagram — see module-12-4-ai-agents.html]*

ReAct (Reason + Act) is the dominant agent prompting pattern. The LLM interleaves Thought (internal chain-of-thought reasoning about what to do), Action (a structured tool call), and Observation (the tool's result). This loop continues until the agent has enough information to produce a final answer. ReAct prevents hallucination on factual questions by requiring the agent to ground its answer in tool-retrieved evidence rather than parametric memory. The structured Thought-Action-Observation format also makes agent behaviour transparent and debuggable — the reasoning trace is human-readable. Re

**Figure 217 • FIG 12.4-C**

**Multi-Agent System — Orchestrator Delegates to Specialist Agents**

*Illustration: FIG 12.4-C | Multi-Agent System — Orchestrator Delegates to Specialist Agents [SVG diagram — see module-12-4-ai-agents.html]*

Multi-agent systems decompose complex tasks across specialised agents. An orchestrator LLM receives a high-level task, breaks it into subtasks, and delegates each to a specialist: a research agent with web search tools, a code agent with execution capabilities, a critic agent for review and fact-checking, and API agents for external integrations. Specialist agents complete their subtasks and return results; the orchestrator synthesises them into a final response. This architecture mirrors human team organisation — specialists are better and faster at their narrow domains, and coordination over

### Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 12.4 AI Agents: Planning, Tool Use, Memory: ReAct Agent Tool Calls Memory Systems [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 12.4 AI Agents: Planning, Tool Use, and Memory **Type:** Implementation + trace + design **Estimated time:** 5565 minutes **Tools needed:** Python (OpenAI optional; can mock LLM) --- Part A ReAct Agent Build a minimal ReAct agent with two tools: a **calculator** and a **dictionary lookup**. 1. Implement the two tools: ```python def calculator(expression: str) -> str: """Evaluate a mathematical expression and return the result.""" ... def dictionary(word: str) -> str: """Look up the definition of a word.""" ... ``` 2. Implement t

# Simplified example

■# Module 12.5 — Ethics and Safety in AI Systems

## The Problem

*A hiring algorithm trained on historical data learns to discriminate.*

*A content recommendation system optimises engagement — and radicalises users.*

*A self-driving car must decide who to harm in an unavoidable crash.*

*A large language model confidently states false information.*

>

*These are not hypothetical future problems.*

*They are happening now, in systems deployed at scale.*

>

*Building AI systems responsibly requires understanding*

*not just the technical mechanisms but the ethical frameworks,*

*failure modes, and safety techniques that prevent harm.*

---

## The Story

### From Science Fiction to Policy

The ethics of artificial intelligence has been discussed since the technology was theoretical. Isaac Asimov's Three Laws of Robotics (1942) were the first popular framing — and Asimov's stories systematically showed why rule-based ethics fails in complex situations.

Real concerns emerged as AI systems moved from research to deployment:

• **2014**: Microsoft's Tay chatbot learned to generate hate speech within 24 hours of deployment

• **2016**: ProPublica reported that COMPAS (a recidivism prediction system) showed racial bias in US courts

• **2019**: Amazon scrapped an AI recruiting tool that discriminated against women (trained on historical hiring data that was male-dominated)

• **2022**: ChatGPT demonstrated that large language models could produce fluent, confident, incorrect text — "hallucination" — at scale

• **2023–2024**: Regulatory responses — EU AI Act, US Executive Order on AI — attempted to create frameworks for accountability

The technical AI safety field grew from concerns about existential risk. **Nick Bostrom's** *Superintelligence* (2014) and work at organisations like MIRI (Machine Intelligence Research Institute) and later Anthropic focused on alignment: how do you ensure an AI system pursues the goals its creators intended, not some proxy that diverges catastrophically?

---

## The Concept

### Bias and Fairness

**What is bias in ML?** A model trained on biased data learns biased patterns.

```
# Simplified example
# Historical data: women hired at lower rates (due to past discrimination)
# Model trained on this data: predicts women as lower probability hires
# Outcome: perpetuates and entrenches the historical bias
```

**Sources of bias:**

• **Historical bias**: data reflects past discrimination

• **Representation bias**: certain groups are underrepresented in training data

• **Measurement bias**: the variable being predicted is a proxy for something else

• **Feedback loops**: biased predictions → biased outcomes → biased future training data

**Fairness metrics** (often incompatible with each other):

• **Demographic parity**: equal positive prediction rates across groups

• **Equal opportunity**: equal true positive rates across groups

• **Predictive parity**: equal precision across groups

• **Individual fairness**: similar individuals treated similarly

**Impossibility results**: it is mathematically impossible to satisfy all fairness criteria simultaneously when base rates differ between groups. Any choice of fairness definition encodes a value judgment — there is no purely technical solution to what is fundamentally a societal question.

### Explainability and Interpretability

**The black box problem**: deep neural networks make decisions through millions of learned parameters. Even the system's creators cannot fully explain why it made a specific decision.

**Why it matters**: in high-stakes domains (medical diagnosis, loan decisions, criminal justice), decision subjects have a right to an explanation. The EU GDPR includes a "right to explanation" for automated decisions.

**Techniques:**

• **LIME** (Local Interpretable Model-Agnostic Explanations): approximate the model locally with a simpler interpretable model

• **SHAP** (SHapley Additive exPlanations): use Shapley values (from game theory) to attribute each feature's contribution to a prediction

• **Attention visualisation**: in transformer models, attention weights show which input tokens influenced the output

• **Gradient-based saliency**: compute the gradient of the output with respect to inputs to identify which inputs most affected the decision

```
import shap

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test, feature_names=feature_names)
```

### Hallucination and Truthfulness

**Hallucination**: a language model confidently states false information. This is not a bug in the traditional sense — it is an emergent property of how LLMs work:

• LLMs are trained to predict likely next tokens, not to retrieve facts from a verified database

• Fluent, confident-sounding text and accurate text are statistically correlated but not identical

• The model has no internal "truth check" — it cannot distinguish what it knows from what it is confabulating

**Mitigation techniques:**

• **Retrieval-Augmented Generation (RAG)**: ground LLM responses in retrieved documents (Module 12.3)

• **Calibration**: train models to express uncertainty when appropriate

• **Constitutional AI** (Anthropic): train models using AI feedback on their own outputs against a set of principles

• **Human feedback (RLHF)**: fine-tune models on human ratings of response quality

### Alignment: The Core Problem

**The alignment problem**: how do you specify what you want an AI system to do, precisely enough that a system optimising for that specification actually does what you intended?

Classic examples:

• "Make users happy" → system learns to addict users to endless scroll

• "Get a high score in this game" → agent discovers an exploit that earns points without playing the game as intended

• "Reduce hospital wait times" → system discharges sick patients early

**Reward hacking**: any learnable proxy metric can be gamed if the system is optimising it hard enough.

**RLHF (Rein

### Illustrations

**Figure 218 • FIG 12.5-A**

**AI Risk Taxonomy — Capability, Misuse, Accident, Alignment**

*Illustration: FIG 12.5-A | AI Risk Taxonomy — Capability, Misuse, Accident, Alignment [SVG diagram — see module-12-5-ethics-safety.html]*

AI risk taxonomy. Misuse risks arise from deliberate bad actors using AI capabilities for harm — generating disinformation, cyberattacks, fraud. Technical filters help but cannot fully prevent misuse of general-purpose systems. Alignment risks are more subtle: a powerful optimiser given the wrong objective will pursue it in unexpected ways — the famous "paperclip maximiser" thought experiment illustrates how an AI optimising for a simple goal could behave catastrophically. Accident risks are unintended harms from deployment errors, edge cases, or distributional shift. Fairness and bias risks a

**Figure 219 • FIG 12.5-B**

### Bias in Machine Learning — From Biased Data to Biased Decisions

*Illustration: FIG 12.5-B | Bias in Machine Learning — From Biased Data to Biased Decisions [SVG diagram — see module-12-5-ethics-safety.html]*

ML bias pipeline — how societal inequalities enter, persist, and amplify through machine learning systems. Biased historical data (e.g., hiring records reflecting past discrimination) is collected non-uniformly (underrepresenting minorities), labelled by annotators with subconscious biases, used to train models that optimise average accuracy (performing worst on underrepresented groups), and deployed in high-stakes automated decision systems. A feedback loop compounds the problem: biased hiring decisions produce biased future employment records, which become biased future training data. Real c

**Figure 220 • FIG 12.5-C**

### Constitutional AI — RLHF and Self-Critique for Safer LLMs

*Illustration: FIG 12.5-C | Constitutional AI — RLHF and Self-Critique for Safer LLMs [SVG diagram — see module-12-5-ethics-safety.html]*

Two major alignment techniques. RLHF (Reinforcement Learning from Human Feedback) was the key innovation that transformed GPT-3 (capable but unreliable) into InstructGPT and eventually GPT-4. Human raters compare model outputs, a reward model learns their preferences, and Proximal Policy Optimisation (PPO) adjusts the model to generate higher-reward responses. A KL-divergence penalty prevents the model from deviating too far from its original capabilities. Constitutional AI (Anthropic) addresses RLHF's scaling bottleneck — human labelling is expensive. Instead: write explicit principles, have

## Hands-On Simulation

**INTERACTIVE SIMULATION**

Module 12.5 AI Ethics and Safety: Bias Explorer Safety Trade-offs Ethical Scenarios [Interactive HTML — open simulation.html in browser to run]

**CHALLENGE**

Challenge 12.5 Ethics and Safety in AI Systems **Type:** Analysis + reasoning + design **Estimated time:** 4555 minutes **Tools needed:** pen and paper --- Part A Bias Audit A loan approval model was trained on historical data. You are given these results for two groups: | Group | Approved | Denied | True Approvals | True Denials | |---|---|---|---|---| | Group A | 80 | 120 | 100 | 100 | | Group B | 40 | 60 | 50 | 50 | Where "True" = what the correct decision should have been. 1. Compute **accuracy** for each group. Are they the same? 2. Compute **precision** (of approvals: what